# cge-stata

## Reference Manual

Computable general equilibrium modelling in Stata

Version 0.9.7

Mustapha Wasseja

October 2026

**cge-stata** builds, calibrates and solves single-country computable general equilibrium models from a social accounting matrix, in Stata and Mata. It runs on Stata 13 or later.

The package and its source are at `https://github.com/Mustapha-Wasseja/cge-stata`. This manual is built from the LaTeX sources in `docs/manual/tex` with `tools/build_manual.py`.

The social accounting matrices shipped with the package and used in the examples of this manual are SYNTHETIC: they are constructed for testing and describe no country.

# Contents

I Introduction and user's guide 1

1 Introduction 2

1.1 What the package does . . . 2
1.2 Installation . . . 2
1.3 Recent additions . . . 3
1.4 This manual . . . 3
1.5 How the package is tested . . . 4

2 A first model 5

2.1 The SAM . . . 5
2.2 Roles . . . 5
2.3 Calibration and checks . . . 6
2.4 A tariff removal . . . 6
2.5 The same shock under another closure . . . 7
2.6 Next steps . . . 8

3 The workflow 9

3.1 Steps . . . 9
3.2 Where the model lives . . . 9
3.3 Principles . . . 9
3.4 Reporting results . . . 10

4 A worked example: from a raw SAM to results 11

4.1 The raw SAM . . . 11
4.2 Roles, aggregation and balancing . . . 11
4.3 Elasticities and calibration . . . 12
4.4 Three policy simulations . . . 13
4.5 Sensitivity . . . 15
4.6 Dynamics . . . 16
4.7 Poverty and inequality . . . 17
4.8 Saving the model . . . 17

5 Validation, performance and troubleshooting 18

5.1 How results are validated . . . 18
5.2 Performance . . . 18
5.3 Common problems . . . 19

II The model: methods and formulas 20

6 Notation and the social accounting matrix 21

6.1 The social accounting matrix . . . 21
6.2 Sets . . . 21
6.3 Variables . . . 22
6.4 Parameters . . . 23

**7 The equations of the standard model 26**
7.1 Prices . . . 26
7.2 Production . . . 27
7.3 Trade . . . 28
7.4 Institutions . . . 29
7.5 System constraints . . . 31

**8 Closures 32**
8.1 Closure options . . . 32
8.2 Swaps . . . 33
8.3 Checks . . . 33

**9 Calibration 34**
9.1 SAM checks and the flows the model represents . . . 34
9.2 Layouts rewritten before the checks . . . 34
9.3 Normalisation . . . 36
9.4 Taxes and trade . . . 36
9.5 Production . . . 37
9.6 CET and Armington functions . . . 37
9.7 Institutions . . . 38
9.8 Tax accounts . . . 39
9.9 Factor quantities . . . 39
9.10 Benchmark replication . . . 39

**10 The solution method 40**
10.1 Newton's method . . . 40
10.2 The reduced system . . . 41
10.3 The Jacobian . . . 41
10.4 Fallbacks and continuation . . . 42
10.5 Diagnostics . . . 42
10.6 Constraints . . . 42
10.7 Other implementations . . . 42

**11 Validation checks 43**
11.1 Agreement between implementations . . . 44

**12 Reporting measures and welfare 45**
12.1 Measures . . . 45
12.2 Welfare . . . 46
12.3 Changes . . . 46

**13 Fiscal instruments 47**
13.1 Revenue-neutral tax replacement . . . 47
13.2 Value added tax with input credits . . . 47
13.3 Public debt and interest . . . 48

**14 Trade and household extensions 49**
14.1 Home consumption . . . 49
14.2 Export demand . . . 49
14.3 Export-only commodities . . . 49
14.4 Margins by market . . . 50

**15 Labour-market regimes** 51

15.1 Labour force and unemployment . . . 51
15.2 Wage curve . . . 51
15.3 Minimum wage . . . 51
15.4 Results and dynamics . . . 52

**16 Health** 53

16.1 A social health insurance fund . . . 53
16.2 Health services and factor efficiency . . . 53
16.3 Days lost to illness . . . 53
16.4 Mortality . . . 54
16.5 Population and labour in dynamic runs . . . 54
16.6 Reporting . . . 54

**17 Bounds and complementarity** 55

17.1 Rents . . . 55
17.2 Constraints . . . 55
17.3 Solution . . . 56
17.4 Reporting . . . 56
17.5 User inequalities . . . 56

**18 Factor taxes, nests and output aggregation** 57

18.1 Taxes on factor use and on factor income . . . 57
18.2 Nests of value added . . . 57
18.3 Output of a commodity from several activities . . . 58

**19 Recursive dynamics** 59

19.1 Updates between periods . . . 59
19.2 Baselines and scenarios . . . 60
19.3 Lags in user equations . . . 60

**20 The link to household surveys** 61

20.1 Income . . . 61
20.2 Prices and real consumption . . . 61
20.3 Poverty and inequality . . . 61
20.4 Distribution and incidence . . . 62
20.5 Welfare and the channels of change . . . 62
20.6 Fiscal incidence . . . 62
20.7 Graphs . . . 63
20.8 Weights, sampling error and the uncertainty of the model . . . 63

**21 Sensitivity analysis** 64

21.1 Draws . . . 64
21.2 Summary . . . 64
21.3 Keeping the draws . . . 64
21.4 Interpretation . . . 64

**22 Policy optimisation** 65

22.1 Algorithm . . . 65
22.2 Results . . . 65

**23 User variables and equations** **66**

23.1 The language . . . 66
23.2 User sets and parameters . . . 66
23.3 Conditions . . . 67
23.4 Two indexes and sets of pairs . . . 67
23.5 Lags . . . 67
23.6 Functions and macros . . . 68
23.7 Replacing built-in equations . . . 68
23.8 Inequalities and bounds . . . 69
23.9 Variables and equations . . . 69
23.10 Compilation . . . 69

**24 SAM tools** **70**

24.1 Reading a SAM . . . 70
24.2 Writing a SAM . . . 70
24.3 Mapping templates . . . 71
24.4 Aggregation . . . 71
24.5 Balancing . . . 71
24.6 Accounting multipliers . . . 73
24.7 Structural path analysis . . . 74

**25 The GAMS implementation** **75**

25.1 What is written . . . 75
25.2 Solvers and model types . . . 76
25.3 Comparison and import . . . 76

**26 The elasticity library** **78**

26.1 Contents . . . 78
26.2 How the values were read and checked . . . 78
26.3 Using the values . . . 79

**III Command reference** **80**

**Introduction to the command reference** **81**

**`cge` — Computable general equilibrium modelling in Stata** **82**

**`cge_aggregate` — Merge SAM accounts into groups** **86**

**`cge_balance` — Balance a SAM by RAS or cross entropy** **90**

**`cge_calibrate` — Calibrate the model to the SAM** **96**

**`cge_check` — Run model validation checks** **104**

**`cge_closure` — Choose the model closure** **108**

**`cge_constraint` — Limits that bind in some scenarios and not in others** **113**

**`cge_dynamic` — Solve a recursive dynamic run** **120**

**`cge_dynresults` — Results of a dynamic run as a dataset** **129**

**`cge_elasticities` — Published elasticities, with their sources** **133**

`cge_equation` — User equations: new relations among the model’s variables 138

`cge_gams` — Write the model as a GAMS program, and solve it in GAMS 151

`cge_listing` — Equation listing and infeasibility report 159

`cge_macro` — Macros of the language of user variables and equations 164

`cge_map` — Assign SAM accounts to model roles 168

`cge_microsim` — Pass simulations to household survey data 177

`cge_multiplier` — SAM multipliers and their decomposition 196

`cge_optimize` — Choose policy instruments to maximise an outcome or reach targets 202

`cge_parameter` — User parameters for user variables and equations 209

`cge_parameters` — The calibrated parameters as a dataset 214

`cge_report` — Standard report tables for one or more simulations 218

`cge_results` — Report simulation results 223

`cge_sam` — Read a social accounting matrix 227

`cge_samexport` — Write the SAM to a file or to memory 231

`cge_save` — Save the model and its simulations to a file 234

`cge_sensitivity` — Sensitivity of a scenario’s results to the elasticities 236

`cge_set` — User sets of elements, for indexes and sums 241

`cge_shock` — Change a parameter or a fixed variable 245

`cge_solve` — Solve the model 251

`cge_spa` — Structural path analysis of a SAM multiplier 257

`cge_synthetic` — Load the synthetic social accounting matrix 262

`cge_template` — Write a template that maps SAM accounts to roles and groups 266

`cge_use` — Load a model saved with cge_save 271

`cge_variable` — User variables, defined by an expression or determined by user equations 274

References 279

## Part I

# Introduction and user’s guide

# 1 Introduction

## 1.1 What the package does

cge-stata builds, calibrates and solves single-country computable general equilibrium (CGE) models from a social accounting matrix (SAM), within Stata and Mata, without an external modelling system. The model is the standard SAM-based model of the IFPRI and PEP traditions (Lofgren et al., 2002; Decaluwé et al., 2013), stated in full in Part II:

- production with CES or Cobb–Douglas value added and Leontief intermediate inputs, with an optional CES nest between them; activities may produce several commodities;
- trade through a CET function between exports and domestic sales and an Armington CES function between imports and domestic goods;
- households with Cobb–Douglas or linear expenditure system demand, enterprises, the government, savings–investment and the rest of the world, with transfers among them;
- direct, sales, import, export and activity taxes, taxes on factors, and VAT with input credits;
- closures for the external account, the government, savings–investment, the numeraire and each factor.

Extensions cover public debt, home consumption, finite export demand, margins by market, labour-market regimes, a social health insurance fund, factor efficiency linked to health services, health outcomes, nests of value added, the aggregation of output from several activities, and limits that bind in some scenarios and not in others. The model can be solved for one period or as a recursive dynamic sequence. Its results can be passed to household survey data: poverty for several lines, inequality, incidence by decile and the growth incidence curve, each household's welfare, the contribution of each channel to every change, and fiscal incidence as the Commitment to Equity method measures it, with survey weights that follow the model, standard errors from the survey design and ranges across draws of the elasticities. The sensitivity of results to the elasticities can be measured; policy instruments can be chosen to maximise an outcome or reach targets; and users can add their own variables, equations, sets and parameters, with conditions and inequalities paired with variables as in GAMS. Tools read, aggregate, balance and analyse the SAM, and a library gives published elasticities with their sources. The model can also be written as a GAMS program, solved there and brought back.

The package does not do intertemporal optimisation (agents do not look ahead), multi-region modelling, or the estimation of elasticities: its elasticities are inputs, stated by the user.

## 1.2 Installation

The package needs Stata 13 or later and runs on Stata/IC. Install it from GitHub (tested with StataNow 19.5):

```
. net install cge, from("https://raw.githubusercontent.com/Mustapha-Wasseja/cge-stata/main/") replace
```

Stata 13 cannot connect to GitHub over HTTPS, because its TLS version is too old. With Stata 13, download or clone the repository and install from the folder:

```
. net install cge, from("C:/path/to/cge-stata") replace
```

`help cge` lists the commands, and `cge` displays the version. The Mata library is compiled the first time a command runs in a session.

## 1.3 Recent additions

| Version | Additions | Where |
|---|---|---|
| 0.7.0 | An exact Jacobian and Broyden's updates: large models solve three to four times faster | chapter 10, section 5.2 |
| 0.7.1 | User sets and user parameters, for the indexes, sums and coefficients of user equations | **[CGE]** `cge_set`,<br>**[CGE]** `cge_parameter` |
| 0.7.2 | Conditions, as GAMS's dollar operator: sums, terms and domains restricted to the elements that meet a condition | section 23.3 |
| 0.8.0 | The survey link for several simulations at once, with the results as a dataset; several poverty lines, including lines by household; eight more inequality measures; incidence by quantile and the growth incidence curve | **[CGE]** `cge_microsim` |
| 0.8.1 | Each household's equivalent and compensating variations; the Shapley decomposition of every change by channel; growth and redistribution | **[CGE]** `cge_microsim` |
| 0.8.2 | Fiscal incidence: market to final income, concentration and Kakwani indexes, coverage, marginal contributions, the Reynolds–Smolensky index and fiscal impoverishment; in-kind health benefits that follow the model's health spending | **[CGE]** `cge_microsim` |
| 0.8.3 | Survey weights that follow the model's employment; standard errors from the survey design; the draws of a sensitivity analysis kept as simulations, and bands across them | **[CGE]** `cge_microsim`,<br>**[CGE]** `cge_sensitivity` |
| 0.9.0 | Inequalities paired with variables, as in GAMS's mixed complementarity problems; exogenous user variables; bounds on user variables | **[CGE]** `cge_equation`,<br>section 23.8 |
| 0.9.1 | Variables and equations with two indexes; sums over two indexes; sets of pairs, as GAMS's mapping sets, in sums and conditions | **[CGE]** `cge_set`, section 23.4 |
| 0.9.2 | Lags in user equations, the values of earlier periods of a dynamic run, for stocks and partial adjustment | **[CGE]** `cge_equation`,<br>section 23.5 |
| 0.9.3 | User equations that replace elements of the model's own equations | **[CGE]** `cge_equation`,<br>section 23.7 |
| 0.9.4 | Expressions as the objective and targets of the optimiser, and inequality constraints | **[CGE]** `cge_optimize`,<br>chapter 22 |
| 0.9.5 | The equation listing and infeasibility report | **[CGE]** `cge_listing` |
| 0.9.6 | Graphs of the results of the survey link, in the style of journal figures | **[CGE]** `cge_microsim` |
| 0.9.7 | The functions of GAMS in the user language, products and extremes over sets, and macros | **[CGE]** `cge_macro`,<br>section 23.6 |

## 1.4 This manual

Part I introduces the package: a first model (chapter 2), the workflow and its principles (chapter 3), a worked example from a raw SAM to results (chapter 4), and how the results are validated, how fast the solver is and what to do when something fails (chapter 5). Part II states the model: its notation, equations, closures and calibration, the solution method, the checks, the reporting measures, the extensions, and the methods of the commands that are not part of the model itself. Part III is the reference for each command, in the form of Stata's own manuals: syntax, options, examples with their output, stored results and methods.

Commands typed in Stata are shown after a prompt, `. cge_solve`; output follows them as Stata displays it. The examples use the SYNTHETIC SAMs that come with the package: they are constructed for testing and describe no country, so the results of the examples illustrate the commands and are not findings about any economy. Mathematical symbols for the model's variables are written in capitals ($PQ_c$, $QA_a$), and parameters in lower case ($tm_c$) or Greek ($\sigma_a^{va}$); the names are those of the commands' results.

## 1.5 How the package is tested

The equations are implemented three times: in Stata and Mata (the package); in Python, a reference implementation that writes the expected results of the tests; and in GAMS, written from the specification and solved with PATH. The Stata test suite compares every result with the Python reference, and the GAMS implementation reproduces the same cases. The measures of the survey link are checked against Stata's own commands, such as `xtile`, `_pctile` and `svy`, and against their formulas. Chapter 5 gives the details.

# 2 A first model

This chapter builds a complete model on the synthetic SAM that comes with the package, removes its import tariffs under two closures, and reads the results. Every step is one command, and each is described in Part III.

## 2.1 The SAM

**[CGE]** `cge_synthetic` loads the standard synthetic SAM as data: one observation per account, a string variable with the account labels and one numeric variable per account. It has three activities and three commodities (agriculture, manufactures and services), labour and capital, one household, the government with direct, sales and import taxes, savings–investment and the rest of the world. **[CGE]** `cge_sam` reads it into a new model:

```
. cge_synthetic, clear
(16 vars, 15 obs)

Synthetic SAM loaded (standard layout): 15 accounts, total   1743.00
Largest row-column imbalance:  0.00e+00

. list account act_agr act_man act_ser com_agr com_man com_ser in 1/6, noobs abbreviate(8)

  +---------------------------------------------------------------------+
  | account   act_agr   act_man   act_ser   com_agr   com_man   com_ser |
  |---------------------------------------------------------------------|
  | act_agr         0         0         0       100         0         0 |
  | act_man         0         0         0         0       140         0 |
  | act_ser         0         0         0         0         0       150 |
  | com_agr        10        20         5         0         0         0 |
  | com_man         8        30        15         0         0         0 |
  |---------------------------------------------------------------------|
  | com_ser         5        15        20         0         0         0 |
  +---------------------------------------------------------------------+

. cge_sam, labels(account)
SAM read: 15 accounts, total 1743
Largest row-column imbalance:  0.00e+00
```

Cell (r, c) of the SAM is a payment from the column account c to the row account r. The listing shows the make matrix (activities selling their output to commodities) and the intermediate inputs (commodities bought by activities). The SAM balances: every account's receipts equal its payments.

## 2.2 Roles

**[CGE]** `cge_map` assigns each account to a role of the model and checks that every non-zero cell is a flow the model represents (section 9.1). Activities and commodities are paired by position, each activity with its main commodity:

```
. cge_map, activities(act_agr act_man act_ser) commodities(com_agr com_man com_ser) ///
>     factors(lab cap) households(hhd) government(gov) savinv(sav) row(row)       ///
>     dirtax(tax_dir) saltax(tax_sal) imptax(tax_imp)
Accounts mapped: 3 activities, 3 commodities, 2 factors, 1 households, 0 enterprises, 0 margin accou
> nts
```

## 2.3 Calibration and checks

The model's elasticities have no defaults. Here value added has an elasticity of substitution of 0.8 between labour and capital, and the Armington and CET elasticities are 2 for every commodity; **[CGE]** `cge_elasticities` gives published values with their sources. **[CGE]** `cge_calibrate` computes every parameter and the base value of every variable from the SAM and the elasticities (chapter 9), sets the default closure, and checks that the base solves every equation; **[CGE]** `cge_check` runs the validation checks of chapter 11:

```
. cge_calibrate, sigmava(0.8) sigmaq(2) sigmat(2)
Calibrated: 93 equations, 93 free variables
Benchmark replication: largest scaled residual  3.55e-16

. cge_check, homogeneity perturb(0.1)
  [ok]   SAM balance (largest imbalance)                   0.00e+00
  [ok]   benchmark replication (largest scaled residual)   3.55e-16
  [ok]   closure is square (free variables minus equations)  0.00e+00
  [ok]   homogeneity (largest relative deviation)          5.33e-15
  [ok]   perturbed start returns the base (largest deviation)  2.00e-14
All checks passed
```

The model has 93 equations. The largest scaled residual at the base is at the precision of double arithmetic; doubling every fixed nominal value doubles every price and leaves every quantity unchanged; and the solver returns to the base from starting values perturbed by up to 10 per cent.

## 2.4 A tariff removal

**[CGE]** `cge_shock` sets every import tariff to zero, and **[CGE]** `cge_solve` solves the model and stores the solution as the simulation `notariff`:

```
. cge_shock tm, value(0)
Parameter tm changed for 3 element(s)

. cge_solve, name(notariff)
Simulation notariff solved in 7 iteration(s) (newton)
Largest scaled residual  4.49e-14; WALRAS = -1.56e-14
```

**[CGE]** `cge_results` summarises it, as percentage changes from the base:

```
. cge_results, sim(notariff)

Simulation notariff (closure: flexexr flexgsav savings cpi lab:mobile cap:mobile)
----------------------------------------------------
                                       base % change
----------------------------------------------------
Real value added (base prices)          262   -0.000
Exchange rate                             1    4.759
Consumer price index                      1    0.000
Investment volume (IADJ)                  1  -11.861
Imports (world prices)                  100    4.233
Exports (world prices)                   75    5.644
Government savings                        3 -271.374
Price of lab                              1    2.811
Price of cap                              1    2.631
Output of act_agr                       100   -0.409
Output of act_man                       140   -1.399
Output of act_ser                       150    1.240
----------------------------------------------------
Equivalent variation, hhd              6.137 ( 2.67% of consumption)
WALRAS -1.56e-14; GDP identity gap -6.82e-13
```

The default closure has a flexible exchange rate with fixed foreign savings, fixed tax rates with flexible government savings, savings-driven investment, the CPI as numeraire, and both factors fully employed and mobile (chapter 8). Under it, the exchange rate depreciates by 4.8 per cent, and imports and exports both rise. Government savings, 3 in the base, fall by 271 per cent, to a deficit of about 5: the government loses its tariff revenue. Investment, which adjusts to available savings, falls by 11.9 per cent. The household's equivalent variation is 2.67 per cent of its base consumption.

## 2.5 The same shock under another closure

The closure matters as much as the shock. With government savings fixed and direct tax rates adjusting to keep them, the tariff revenue is replaced by a higher direct tax:

```
. cge_closure, government(fixgsav direct)
Closure
  external account     flexexr
  government           fixgsav
  savings-investment   savings
  numeraire            cpi
  factor lab           mobile
  factor cap           mobile

. cge_solve, name(notariff_direct)
Simulation notariff_direct solved in 7 iteration(s) (newton)
Largest scaled residual  7.70e-14; WALRAS = -3.37e-13
```

**[CGE]** `cge_report` shows the two simulations side by side:

```
. cge_report, sims(notariff notariff_direct) tables(macro government welfare)

Macroeconomic aggregates
------------------------------------------------------------
                                  base   notariff notariff~t
gdp_nominal                        300     -0.305     -0.288
gdp_real                           300      0.081      0.139
gdp_deflator                         1     -0.386     -0.427
consumption_real                   230      2.684     -0.401
government_real                     45      0.000      0.000
investment_real                     50    -11.861      2.678
insurance_fund_real                  0          .          .
exports_real                        75      5.644      6.674
imports_real                       100      4.233      5.005
value_added_real                   262     -0.000     -0.000
cpi                                  1      0.000      0.000
dpi                                  1      1.013      0.985
exchange_rate                        1      4.759      4.846
real_exchange_rate                   1      3.709      3.824
private_savings                     32      2.652     -0.431
investment_value                    50    -13.157      1.178
foreign_savings                     15      0.000      0.000
------------------------------------------------------------

Government
------------------------------------------------------------
                                  base   notariff notariff~t
revenue                             63    -11.727      1.191
spending                            60      1.255      1.250
savings                              3   -271.374      0.000
tax_direct                          25      2.652     34.925
tax_sales                           30     -0.171      0.063
tax_import                           8   -100.000   -100.000
tax_export                           0          .          .
tax_activity                         0          .          .
tax_vat                              0          .          .
tax_factor_use                       0          .          .
tax_factor_income                    0          .          .
------------------------------------------------------------

Welfare (per cent of base consumption)
--------------------------------------------------------------
                                           notariff notariff~t
equivalent_variation hhd                      2.668     -0.416
compensating_variation hhd                    2.668     -0.416
--------------------------------------------------------------
Percentage changes from the base (rates: changes in points)
```

Direct tax revenue now rises by 34.9 per cent, government savings are unchanged, investment rises by 2.7 per cent, and real consumption falls by 0.4 per cent; the equivalent variation is $-0.42$ per cent of consumption. Under both closures real GDP changes little and the exchange rate depreciates by about

4.8 per cent. The two simulations differ in who bears the loss of tariff revenue: investment under the first closure, households under the second.

## 2.6 Next steps

**[CGE]** `cge_report` with `saving()` writes every table to Excel, and **[CGE]** `cge_save` stores the model with its simulations. A real application starts from a country's SAM, prepared with the SAM tools of chapter 24, and states the source of every elasticity, the closure and the sensitivity of the main results to the elasticities. Chapter 4 takes an unbalanced SAM through every step.

# 3 The workflow

## 3.1 Steps

A model is built in a fixed order. Each step is a command:

| Step | Commands |
|---|---|
| Read the SAM | [CGE] cge_sam, [CGE] cge_synthetic |
| Prepare it: template, aggregation, balancing, export | [CGE] cge_template, [CGE] cge_aggregate, [CGE] cge_balance, [CGE] cge_samexport |
| SAM multipliers and structural paths | [CGE] cge_multiplier, [CGE] cge_spa |
| Assign accounts to model roles | [CGE] cge_map |
| Choose elasticities | [CGE] cge_elasticities |
| Calibrate and check | [CGE] cge_calibrate, [CGE] cge_check, [CGE] cge_parameters |
| Choose a closure, add limits, apply shocks, solve | [CGE] cge_closure, [CGE] cge_constraint, [CGE] cge_shock, [CGE] cge_solve |
| Add variables, equations, sets and parameters | [CGE] cge_variable, [CGE] cge_equation, [CGE] cge_set, [CGE] cge_parameter |
| Report | [CGE] cge_results, [CGE] cge_report |
| Choose instruments | [CGE] cge_optimize |
| Dynamic runs | [CGE] cge_dynamic, [CGE] cge_dynresults |
| Sensitivity to the elasticities | [CGE] cge_sensitivity |
| Household surveys: poverty, inequality, incidence, welfare, fiscal incidence | [CGE] cge_microsim |
| Solve in GAMS | [CGE] cge_gams |
| Save and reload | [CGE] cge_save, [CGE] cge_use |

## 3.2 Where the model lives

The model lives in Mata, not in the data in memory. [CGE] `cge_sam` starts a new model, and the later commands add to it: the mapping, the calibrated parameters and base solution, the closure, the shocks in force, and every simulation solved. Commands that load data (the `clear` options of the results commands, [CGE] `cge_microsim`) leave the model intact, and [CGE] `cge_save` stores it, with its simulations, in a file that [CGE] `cge_use` loads. Commands that change the SAM ([CGE] `cge_aggregate`, [CGE] `cge_balance`) start a new model from the result.

A simulation records the closure, the shocks, the solution and its residual. Shocks accumulate until `cge_shock, reset`, and a closure stays until it is changed; [CGE] `cge_calibrate` resets both to the defaults.

## 3.3 Principles

- **No defaults for economic assumptions.** Elasticities have no default values. The closure starts from a documented default and is displayed and stored with every simulation.
- **Checks bound to the values used.** Every calibration checks that the base solves every equation and stops if it does not; [CGE] `cge_check` runs the validation checks on the model actually solved; and every solution reports its residual and the slack of Walras's law.
- **Errors, not guesses.** A SAM cell that the model cannot represent, an unbalanced account, or a number that cannot be read stops the command with a report; nothing is set to zero silently.

- **Results as data.** Every result is available as a Stata dataset, in a long layout that keeps the names of the model's variables and measures.

## 3.4 Reporting results

State with every result the SAM, its year and source, and how it was prepared (aggregation, balancing, the totals given); the elasticities and their sources; the closure; and the sensitivity of the main results to the elasticities. A simulation is a counterfactual equilibrium compared with the base: the change once the economy has adjusted under the closure's assumptions, not a forecast. Compare simulations that differ in one respect: the same shock under two closures, or two shocks under one closure.

# 4 A worked example: from a raw SAM to results

This chapter takes an unbalanced SAM through every step of the workflow: a template of roles, aggregation and balancing, mapping, elasticities from the library, calibration and checks, three policy simulations, sensitivity analysis, a dynamic baseline and scenario, the link to a household survey, and saving the model. The SAM and the household survey are SYNTHETIC: they are constructed for testing and describe no country. The elasticities are published values for Togo, applied to the synthetic SAM only to show the commands, so the results illustrate the workflow and are not findings about any country. The commands are those of the do-file `raw_sam_to_results.do` in the `examples` folder of the package's repository.

## 4.1 The raw SAM

```
. cge_synthetic, clear unbalanced
(18 vars, 17 obs)

Synthetic SAM loaded (institutions layout, unbalanced for the balancing tests): 17 accounts, total
>  1860.46
Largest row-column imbalance:  1.21e+01

. cge_sam, labels(account)
SAM read: 17 accounts, total 1860.4569
Largest row-column imbalance:  1.21e+01
```

The SAM has 17 accounts; its largest difference between an account's row and column totals is 12.1. It has two household accounts, households (`hhd`) and non-profit institutions (`npi`), and an enterprise (`ent`).

## 4.2 Roles, aggregation and balancing

**[CGE]** `cge_template` writes a template with one row per account, to be filled in, with a real SAM usually in Excel:

```
. cge_template using roles.csv, replace
Template of 17 accounts
Template saved to roles.csv
```

The do-file fills in the roles and puts `hhd` and `npi` in one household group, `hh`:

```
. list account role group row_total column_total difference, noobs clean

    account          role   group           row_total         column_total           difference
    act_agr    activities                     95.0903    97.30879999999999   -2.218499999999992
    act_man    activities                    138.4698             138.8606   -.3908000000000129
    act_ser    activities                     158.729             146.6431    12.08590000000001
    com_agr   commodities                    125.3554             125.1786    .1768000000000143
    com_man   commodities                     218.935             217.9935    .9414999999999907
    com_ser   commodities                    184.1488             189.5941   -5.445300000000032
        lab       factors                    135.0493             134.4151    .6341999999999928
        cap       factors                    122.9286             130.3917   -7.463099999999983
        hhd    households      hh             247.888             255.1076   -7.219600000000014
        npi    households      hh   9.383199999999999    9.445600000000001     -.062400000000002
        ent   enterprises                    108.8762             105.0257    3.850499999999997
        gov    government                     70.9528    70.67999999999999    .2728000000000037
    tax_dir        dirtax                     28.6243              29.2452   -.6209000000000025
    tax_sal        saltax                     30.3698              30.4754   -.1056000000000026
    tax_imp        imptax                      7.6589    8.395200000000001   -.7363000000000008
        sav        savinv                     51.8279              47.6634    4.164499999999997
        row           row                    126.1696             124.0333    2.136299999999977
```

The last three columns show where the SAM does not balance. **[CGE]** `cge_aggregate` merges the two household accounts, and **[CGE]** `cge_balance` balances the SAM by cross entropy, with the income of the household group taken as known (section 24.5.2):

```
. cge_aggregate using roles.csv
Accounts merged: 2 accounts into 1 group; 15 accounts left as they are
Diagonal cells (flows within an account): 3, total 7.843, kept
Aggregated SAM: 16 accounts, total 1860.4569
Largest row-column imbalance:  1.21e+01

. cge_balance, method(ce) totals(hh:290)
Cross entropy: converged in 4 iterations
Largest row-column imbalance:  1.21e+01 before,  5.68e-14 after
SAM total: 1860.4569 before, 1943.80685 after
Cells changed: 61 of 63 non-zero; largest relative change  3.58e-01

Largest changes
  row                  column                    before         after   change %
  hh                   lab                      134.415       148.579     10.538
  hh                   ent                      66.1137       74.6388     12.895
  com_ser              hh                       84.6701       92.5983      9.364
  lab                  act_ser                   56.475       63.4746     12.394
  com_man              hh                       81.1112        87.931      8.408
  com_agr              hh                       66.3231        72.541      9.375
  act_agr              com_agr                  95.0903       100.203      5.377
  cap                  act_ser                   49.801       54.9053     10.249
  act_man              com_man                   138.47       143.092      3.338
  hh                   gov                      18.9545        22.466     18.526

Balanced SAM: 16 accounts, total 1943.80685
Largest row-column imbalance:  5.68e-14
```

The method converges in four iterations. Every account now balances, 61 of the 63 non-zero cells have changed, and the SAM total has risen from 1,860.5 to 1,943.8. The known household income, 290, is above the row total of the merged household accounts in the raw SAM, about 257, and the largest changes are in the flows that pay and spend it: the labour income of households, for example, rises by 10.5 per cent. These are the changes to report with any result. The same template then maps the balanced SAM; the options it implies are displayed, so that the mapping can also be written as a command:

```
. cge_map using roles.csv
Mapping from the template:
cge_map, activities(act_agr act_man act_ser) commodities(com_agr com_man com_ser) factors(lab cap) h
> ouseholds(hh) enterprises(ent) government(gov) savinv(sav) row(row) dirtax(tax_dir) saltax(tax_sal
> ) imptax(tax_imp)
Accounts mapped: 3 activities, 3 commodities, 2 factors, 1 households, 1 enterprises, 0 margin accou
> nts
```

## 4.3 Elasticities and calibration

**[CGE]** `cge_elasticities` returns values from the library as options for **[CGE]** `cge_calibrate`: the aggregate Armington and CET elasticities estimated for Togo, the capital–labour elasticity estimated for Africa, income elasticities for food (mapped to agriculture) and for other goods and services, and the Frisch parameter (chapter 26):

```
. quietly cge_elasticities sigmaq, country(TGO) values

. local sq `r(values)'

. quietly cge_elasticities sigmat, country(TGO) values

. local st `r(values)'

. quietly cge_elasticities sigmava, country(AFR) values

. local va `r(values)'

. quietly cge_elasticities incelas, country(TGO) values map(com_agr:food) others(other)
. local ie `r(values)'

. quietly cge_elasticities frisch, values

. local fr `r(values)'

. display "sigmava(`va')"
sigmava(0.5897 0.5897 0.5897)

. display "sigmaq(`sq') sigmat(`st')"
sigmaq(0.620 0.620 0.620) sigmat(0.542 0.542 0.542)

. display "incelas(`ie') frisch(`fr')"
incelas(0.804 1.913 1.913) frisch(-1.3661)
```

With income elasticities and a Frisch parameter, household demand is a linear expenditure system:

```
. cge_calibrate, sigmava(`va') sigmaq(`sq') sigmat(`st') incelas(`ie') frisch(`fr')
Household demand: linear expenditure system
Calibrated: 98 equations, 98 free variables
Benchmark replication: largest scaled residual  6.66e-16

. cge_check, homogeneity perturb(0.1)
  [ok]   SAM balance (largest imbalance)                      5.68e-14
  [ok]   benchmark replication (largest scaled residual)      6.66e-16
  [ok]   closure is square (free variables minus equations)   0.00e+00
  [ok]   homogeneity (largest relative deviation)             1.21e-14
  [ok]   perturbed start returns the base (largest deviation)  2.29e-14
All checks passed
```

**[CGE]** `cge_parameters` shows the implied rates: import tariffs of 10.6 per cent on agriculture and 9.8 per cent on manufactures, sales taxes of 5.7 to 7.7 per cent, and direct tax rates of 6.6 per cent for the household group and 9.9 per cent for the enterprise:

```
. cge_parameters, clear

. list parameter element1 value if inlist(parameter, "tm", "tq", "tins0"), noobs clean

    parame~r   element1       value
          tm    com_agr     .105969
          tm    com_man   .09838221
          tm    com_ser           0
          tq    com_agr   .07681605
          tq    com_man   .06786131
          tq    com_ser   .05735944
       tins0         hh   .06567474
       tins0        ent   .09858013
```

## 4.4 Three policy simulations

The do-file removes every import tariff under the default closure and with direct tax rates replacing the lost revenue, and then raises government purchases of services by 20 per cent under the default closure:

```
. cge_shock tm, value(0)
Parameter tm changed for 3 element(s)

. cge_solve, name(notariff)
Simulation notariff solved in 6 iteration(s) (newton)
Largest scaled residual  3.77e-15; WALRAS = -2.74e-13

. cge_closure, government(fixgsav direct)
Closure
  external account     flexexr
  government           fixgsav
  savings-investment   savings
  numeraire            cpi
  factor lab           mobile
  factor cap           mobile
```

```
. cge_solve, name(notariff_direct)
Simulation notariff_direct solved in 6 iteration(s) (newton)
Largest scaled residual  4.47e-14; WALRAS = -1.35e-12

. cge_closure, government(flexgsav)
Closure
  external account     flexexr
  government           flexgsav
  savings-investment   savings
  numeraire            cpi
  factor lab           mobile
  factor cap           mobile

. cge_shock, reset
All shocks removed

. cge_shock qg, elements(com_ser) scale(1.2)
Parameter qg changed for 1 element(s)

. cge_solve, name(gov_services)
Simulation gov_services solved in 6 iteration(s) (newton)
Largest scaled residual  1.55e-15; WALRAS = -3.12e-13

. cge_shock, reset
All shocks removed
```

```
. cge_report, sims(notariff notariff_direct gov_services) tables(macro government welfare)

Macroeconomic aggregates
-----------------------------------------------------------------------
                                  base   notariff notariff~t gov_serv~s
gdp_nominal                      313.5     -0.425     -0.397     -0.104
gdp_real                         313.5      0.033      0.104     -0.150
gdp_deflator                         1     -0.458     -0.501      0.046
consumption_real                 253.1      2.496     -0.351      0.055
government_real                  42.04      0.000      0.000     17.651
investment_real                  48.84    -11.860      4.015    -16.584
insurance_fund_real                  0          .          .          .
exports_real                     72.86      1.342      2.123     -2.052
imports_real                     103.3      1.355      2.218     -1.516
value_added_real                   274     -0.000     -0.000     -0.000
cpi                                  1      0.000      0.000      0.000
dpi                                  1      0.976      0.959      0.162
exchange_rate                        1      4.622      4.678     -0.497
real_exchange_rate                   1      3.610      3.683     -0.657
private_savings                  21.27      2.464     -0.582     -0.137
investment_value                 48.84    -13.080      2.584    -16.626
foreign_savings                  29.62      0.000      0.000      0.000
-----------------------------------------------------------------------

Government
-----------------------------------------------------------------------
                                  base   notariff notariff~t gov_serv~s
revenue                          72.15    -10.455      1.021     -0.720
spending                         74.21      0.994      0.993     10.004
savings                         -2.058   -402.374      0.000   -385.973
tax_direct                       29.89      2.468     29.825     -0.039
tax_sales                        31.19     -0.151      0.182     -0.724
tax_import                       8.234   -100.000   -100.000     -3.422
tax_export                           0          .          .          .
tax_activity                         0          .          .          .
tax_vat                              0          .          .          .
tax_factor_use                       0          .          .          .
tax_factor_income                    0          .          .          .
-----------------------------------------------------------------------

Welfare (per cent of base consumption)
------------------------------------------------------------------------
                                         notariff notariff~t gov_serv~s
equivalent_variation hh                     2.483     -0.363      0.055
compensating_variation hh                   2.484     -0.363      0.055
------------------------------------------------------------------------
Percentage changes from the base (rates: changes in points)
```

The closure matters as much as the shock. With government savings flexible, the lost tariff revenue (8.2, or 11 per cent of revenue) widens the deficit; with savings-driven investment, investment falls by 11.9 per cent and real consumption rises by 2.5 per cent. With direct taxes replacing the revenue, the direct tax take rises by 29.8 per cent, real consumption falls by 0.35 per cent and investment rises by 4.0 per cent. The exchange rate depreciates by 4.6 to 4.7 per cent in both cases, and exports rise. More government purchases of services lower investment by 16.6 per cent under the same closure. [CGE] `cge_report` with `saving()` writes every table to Excel.

## 4.5 Sensitivity

[CGE] `cge_sensitivity` recalibrates the model for 50 draws of the elasticities, each within plus or minus 50 per cent of the central values, sector by sector, and runs the tariff removal:

```
. cge_sensitivity, scenario(tariff_cut) draws(50) spread(0.5) seed(2026)
Sensitivity analysis: 50 draws of the elasticities (uniform within +/-50 per cent)
  10 draws done
  20 draws done
  30 draws done
  40 draws done
  50 draws done

Sensitivity of the scenario to the elasticities: 50 of 50 draws converged
Per cent changes from the base (welfare: per cent of base consumption)

macro
--------------------------------------------------------------------------------
                             central       mean         sd         p5        p95
gdp_nominal                   -0.425     -0.421      0.036     -0.480     -0.372
gdp_real                       0.033      0.032      0.006      0.022      0.043
gdp_deflator                  -0.458     -0.453      0.039     -0.512     -0.398
consumption_real               2.496      2.496      0.004      2.490      2.503
government_real                0.000      0.000      0.000      0.000      0.000
investment_real              -11.860    -11.882      0.213    -12.189    -11.569
insurance_fund_real                .          .          .          .          .
exports_real                   1.342      1.317      0.302      0.854      1.788
imports_real                   1.355      1.328      0.245      0.999      1.713
value_added_real              -0.000     -0.000      0.000     -0.000     -0.000
cpi                            0.000      0.000      0.000      0.000      0.000
dpi                            0.976      0.990      0.125      0.797      1.165
exchange_rate                  4.622      4.577      0.409      3.998      5.195
real_exchange_rate             3.610      3.552      0.534      2.800      4.363
private_savings                2.464      2.467      0.024      2.424      2.506
investment_value             -13.080    -13.103      0.233    -13.435    -12.756
foreign_savings                0.000      0.000      0.000      0.000      0.000
--------------------------------------------------------------------------------

welfare
---------------------------------------------------------------------------------
                              central       mean         sd         p5        p95
equivalent_variation hh         2.483      2.483      0.004      2.477      2.489
compensating_variation hh       2.484      2.484      0.004      2.478      2.490
---------------------------------------------------------------------------------
```

All 50 draws converge. The welfare result barely moves (equivalent variation 2.48 per cent; 5th to 95th percentile 2.48 to 2.49), nor does investment (−12.2 to −11.6 per cent), because both are set mainly by the closure. Trade responds more: exports rise by 0.9 to 1.8 per cent, and the exchange rate depreciates by 4.0 to 5.2 per cent.

## 4.6 Dynamics

A baseline grows real value added by 4 per cent a period, with capital that accumulates from investment and is specific to each activity; the efficiency factors that achieve it fall from 2.4 to 2.0 per cent a period. The scenario removes tariffs from period 1 with the baseline's efficiency path:

```
. cge_closure, factors(cap:specific)
Closure
  external account     flexexr
  government           flexgsav
  savings-investment   savings
  numeraire            cpi
  factor lab           mobile
  factor cap           specific

. local growth capital(cap) depreciation(0.05) rental(0.15) growth(lab:0.025) population(0.025) ///
>     govgrowth(0.03) foreigngrowth(0.02)

. cge_dynamic, name(base) periods(0/5) `growth' gdpgrowth(0.04)
Period 0: solved in 0 iteration(s)
Period 1: solved in 7 iteration(s); efficiency factor 1.0242473340
Period 2: solved in 7 iteration(s); efficiency factor 1.0232215502
Period 3: solved in 8 iteration(s); efficiency factor 1.0222097600
Period 4: solved in 8 iteration(s); efficiency factor 1.0212189589
Period 5: solved in 8 iteration(s); efficiency factor 1.0202552812
Dynamic run base stored as simulations: base0 base1 base2 base3 base4 base5

. cge_dynamic, name(lib) periods(0/5) `growth' baseline(base) shocks(tariff_from1)
Period 0: solved in 0 iteration(s)
Parameter tm changed for 3 element(s)
Period 1: solved in 8 iteration(s); efficiency factor 1.0242473340
Period 2: solved in 8 iteration(s); efficiency factor 1.0232215502
Period 3: solved in 7 iteration(s); efficiency factor 1.0222097600
Period 4: solved in 7 iteration(s); efficiency factor 1.0212189589
Period 5: solved in 7 iteration(s); efficiency factor 1.0202552812
Dynamic run lib stored as simulations: lib0 lib1 lib2 lib3 lib4 lib5
```

**[CGE]** `cge_dynresults` compares the scenario with the baseline:

```
. cge_dynresults, run(lib) baseline(base) clear

. list period element1 value baseline dev_pct if variable == "QA" & period == 5, noobs clean

    period   element1       value    baseline      dev_pct
         5    act_agr   119.26498   120.95393   -1.3963553
         5    act_man   171.86549   175.80722    -2.242074
         5    act_ser   191.19135   192.75872   -.81312575

. cge_closure, factors(cap:mobile)
Closure
  external account     flexexr
  government           flexgsav
  savings-investment   savings
  numeraire            cpi
  factor lab           mobile
  factor cap           mobile
```

Because the static closure lowers investment, capital grows more slowly: by period 5, output is 2.2 per cent below the baseline in manufacturing, 1.4 per cent in agriculture and 0.8 per cent in services. The static gain in consumption has a dynamic cost, which a static run does not show.

## 4.7 Poverty and inequality

The SYNTHETIC survey has 1,000 households with wages, profits and government transfers, and budget shares for food, goods and services. **[CGE]** `cge_microsim` passes the simulation with direct taxes replacing the tariff revenue to it:

```
. cge_microsim [pweight = weight], sim(notariff_direct) group(group) consumption(cons) size(hhsize) ///
>     income(wage:lab profit:cap transfer:gov) budget(food:com_agr goods:com_man services:com_ser)   ///
>     line(`line') generate(nt)

Household survey results of simulation notariff_direct: 1000 households
Real consumption per person (base prices); poverty and inequality over persons
--------------------------------------------------------------------------------------------------
                      base   simulated    % chg      headcount (%)    poverty gap (%)           Gini (%)
--------------------------------------------------------------------------------------------------
all                  184.3       185.5    0.640    23.09 ->   22.97     7.17 ->    7.07    36.32 ->   36.33
hh                   184.3       185.5    0.640    23.09 ->   22.97     7.17 ->    7.07    36.32 ->   36.33
--------------------------------------------------------------------------------------------------
Poverty line 91.223654 per person; severity (FGT2) is in r(table)
```

Real consumption per person rises by 0.64 per cent; the headcount ratio falls from 23.09 to 22.97 per cent, and the Gini coefficient barely changes. The households' gain differs from the group's equivalent variation ($-0.36$ per cent) because they keep their own income composition (here mostly wages, with no transfers from enterprises) and budget shares, and because real consumption is not a welfare measure. A real survey would map every income source of the households.

## 4.8 Saving the model

```
. cge_save using worked.cgem, replace
Model saved to worked.cgem (calibrated, 15 simulation(s))

. cge_use using worked.cgem
Model loaded from worked.cgem (saved by cge-stata 0.7.1; calibrated, 15 simulation(s))
```

**[CGE]** `cge_use` reloads the model with its 15 simulations, and **[CGE]** `cge_report` reproduces the results without solving again.

# 5 Validation, performance and troubleshooting

## 5.1 How results are validated

**Three implementations.** The equations of Part II are implemented in Stata and Mata, in a Python reference model and in GAMS. The Python model runs in continuous integration and writes the expected results of every test case. The Stata test suite, 25 test files, compares every variable of every case with them and requires agreement to $10^{-6}$; in practice they agree to about $10^{-11}$ or better. The GAMS model was written from the specification, calibrates the model from the SAM and the user's settings alone, and is solved with PATH; it reproduces the 186 scenarios of the Python reference, including scenarios with constraints solved as complementarity problems, to $2.9 \times 10^{-11}$ or better. **[CGE]** `cge_gams` writes any model for that GAMS program: 38 exported cases, solved with PATH (17 of them also with CONOPT, KNITRO and IPOPT), reproduce the solutions of Stata to $4 \times 10^{-9}$ or better (section 25.3). The test suite also runs every example do-file.

**The survey link and the user language.** The measures of **[CGE]** `cge_microsim` are checked against Stata's own commands and against their formulas. Quantile groups and percentiles are identical to those of `xtile` and `_pctile`. Standard errors agree with `svy: ratio`, `lincom` and `svy, subpop()` to $10^{-10}$. Poverty, inequality, concentration and the indexes of fiscal incidence agree with direct computations to $10^{-12}$, and the Palma and S80/S20 ratios with closed forms. The welfare of households that follow their group is that of **[CGE]** `cge_report`, under Cobb–Douglas and LES demand. The contributions of the Shapley decomposition add up to the change in every measure, and reweighting meets its targets to $10^{-10}$. User equations with sets, parameters and conditions are written into the GAMS program like the model's own and agree with Stata's solutions (export check cases `user`, `usersets` and `conditions`); user inequalities, solved by GAMS as a mixed complementarity problem, agree too (case `inequalities`), and reproduce the constraints of **[CGE]** `cge_constraint` that they restate, as do variables and equations with two indexes and sets of pairs (case `pairs`). Sums over two indexes and over sets of pairs equal the nested sums and the terms they stand for to $10^{-12}$. Lags take the values of the periods before in every period of a dynamic run, to $10^{-12}$, and GAMS reproduces a run with lags period by period (case `dynlag`). A built-in equation written again in the user's language, in place of the original, leaves the solution unchanged to $10^{-10}$, and GAMS leaves out the elements that user equations replace (case `replace`). An objective written as an expression gives the optimum of the report measure that it equals (to $10^{-10}$), and a binding constraint of **[CGE]** `cge_optimize` gives the instruments that reach it as a target (to $10^{-6}$). The equation listing of **[CGE]** `cge_listing` labels every element as the model's own equations do: with the tariffs removed at the base, it finds the import prices and government revenue off by the tariffs, element by element. The functions `max()`, `min()`, `abs()`, `sqr()`, `log10()` and `ifthen()`, the products and extremes over sets, `card()` and macros give the values of the same formulas computed in Stata from the results, to $10^{-12}$; their exact derivatives agree with central differences away from the kinks, and GAMS solves a model that uses each of them to Stata's solution (case `functions`).

**Checks in every run.** Calibration checks benchmark replication; **[CGE]** `cge_check` adds SAM balance, a square closure, homogeneity, a perturbed start, and, for a simulation, its residual, Walras's law and the GDP identity (chapter 11). Run it after building a model and after changing the closure.

## 5.2 Performance

The solver works on a reduced system with an exact Jacobian and Broyden's updates (chapter 10). On a Windows laptop, synthetic SAMs of increasing size solved a removal of all import tariffs (elasticities 0.8, 2 and 2) in these times, in Stata 13.1/IC and in StataNow/MP 19.5 on four cores with version 0.7.0, and in GAMS with PATH:

| Activities | Accounts | Equations | Solved (reduced) | Stata 13.1/IC (s) | StataNow/MP 19.5 (s) | GAMS (s) |
|---:|---:|---:|---:|---:|---:|---:|
| 40 | 95 | 1,814 | 711 | 0.5 | 0.5 | 1.8 |
| 80 | 180 | 5,446 | 1,407 | 2.6 | 1.8 | 1.8 |

| Activities | Accounts | Equations | Solved (reduced) | Stata 13.1/IC (s) | StataNow/MP 19.5 (s) | GAMS (s) |
|---:|---:|---:|---:|---:|---:|---:|
| 120 | 260 | 10,555 | 2,045 | 3.2 | 3.3 | 2.0 |
| 160 | 350 | 18,483 | 2,757 | 3.9 | 6.0 | 2.5 |
| 200 | 430 | 26,858 | 3,418 | 14.5 | 14.0 | 2.6 |
| 250 | 530 | 40,190 | 4,254 | 27.0 | 29.0 | 4.7 |

The GAMS time is the whole run: GAMS calibrates the model from the SAM, solves the base and then the scenario (version 0.6.3; with version 0.7.0 the GAMS solutions agree with Stata's to $3 \times 10^{-9}$ or better). Times vary from run to run with other work on the machine, by as much as a factor of two; they show orders of magnitude.

Until version 0.7.0 most of Stata's time went into evaluating the model's equations to build the Jacobian by finite differences, about four fifths of the solve of the 200-activity model. The exact Jacobian and Broyden's updates of version 0.7.0 cut the solve of the 250-activity model from 96 seconds to 27–32 in Stata 13.1/IC, and from 72 to 27–29 in StataNow/MP 19.5 (the GAMS times are those of version 0.6.3, for the same models). The dense LU factorisation of the Jacobian is now most of a large solve: in Stata 13.1/IC, 11.4 of the seconds for the 250-activity model, against 3.6 for building four exact Jacobians. GAMS uses a sparse solver, and its time barely grows over this range. For models with several thousand equations in the reduced system, export the scenario to GAMS and bring the solution back with `cge_gams ..., import()` (section 25.3); the solution is checked against the model's equations in Stata. Calibration and mapping are fast compared with solving. Stata/IC holds at most 2,048 variables, but **[CGE]** `cge_sam` reads larger SAMs directly from CSV files or from long data.

## 5.3 Common problems

| Symptom | Likely cause and remedy |
|---|---|
| `cge_map` exits with error 459 and a table of problems | cells the model does not represent, unbalanced accounts, or accounts without roles: map the accounts differently, use a layout option (section 9.2), or correct the SAM |
| "activity output does not equal its sales" | an activity's column total differs from its row of sales: balance the SAM, or check that exports and home consumption are recorded as the model expects |
| "every activity needs positive value added" | an activity without factor payments: merge it with another activity or remove it |
| benchmark replication fails | an error in the SAM or the mapping that the checks did not catch; check negative cells, zero sales and the elasticities (for example `sigmaq(1)` for an imported commodity is not allowed) |
| "closure is not square" | a closure fixes too many or too few variables, for example a factor closure that the model's data cannot support |
| `cge_solve` exits with error 430 | the shock has no solution, or one too far from the base: read the diagnostics (the equations with the largest residuals, and the smallest pivot), try more steps with `steps()`, check the closure, and look at `trace` |
| "the closure is singular" | a swap leaves an equation with no free variable: free a variable of that equation, or undo the swap |
| a shock "is not fixed under the current closure" | the variable is endogenous under the closure: change the closure, or shock a parameter |
| a negative or missing change in a report | percentage changes of negative bases (a deficit) are relative to the absolute value; a zero base gives a missing change |
| Stata 13 cannot install from GitHub | its TLS version is too old: install from a downloaded copy (chapter 1) |
| Stata 13 cannot read a saved model | the model was saved in a later version of Stata, whose Mata file format Stata 13 cannot read: save it in Stata 13 |

# Part II

# The model: methods and formulas

# 6 Notation and the social accounting matrix

This part of the manual states the model that cge-stata solves: its sets, variables, parameters and equations, its closures, how it is calibrated to a social accounting matrix, how it is solved, and how its results are measured. The model is a single-country, static, SAM-calibrated computable general equilibrium model in the tradition of the IFPRI standard model (Lofgren et al., 2002) and the PEP-1-1 model (Decaluwé et al., 2013). Only the structure of the model is taken from that literature; no data from either source are used. The extensions of chapters 13 to 23 and the recursive dynamics of chapter 19 leave the standard model unchanged when they are not used.

The Stata and Mata implementation, a Python reference implementation and an independent GAMS implementation follow the equations of this part; the test suites check that the three agree (chapter 5).

## 6.1 The social accounting matrix

A social accounting matrix (SAM) records every payment in the economy in one square matrix. Cell $SAM(r, c)$ is a payment from the column account $c$ to the row account $r$: the row of an account records its receipts, and its column its payments. A SAM is balanced when every account's row total equals its column total. The model is calibrated to a balanced SAM: base values of the variables, and the parameters, are computed from its cells (chapter 9).

Accounts are given roles by **[CGE]** `cge_map`: activities, commodities, factors, households, enterprises, the government, savings–investment, the rest of the world, tax accounts, margin accounts and, when there is one, a health insurance fund. Activities and commodities are separate accounts: an activity buys inputs and pays factors, and sells its output to commodity accounts, which also receive imports and sell to every user. Each activity is paired with a primary commodity.

Base prices are normalised to one (section 9.3), so the base quantity of a flow equals its value in the SAM. A superscript 0 marks a base value, as in $QA^0_a$.

## 6.2 Sets

| Set | Description |
|---|---|
| $A$ | activities |
| $C$ | commodities; $c(a)$ is the primary commodity of activity $a$. Activities and commodities are paired by position in the mapping; commodities may outnumber activities, and the commodities after the last paired one have no paired activity |
| $CX \subseteq C$ | commodities with domestic output (base output positive); the others, $C \setminus CX$, are imported only |
| $CR \subseteq C$ | commodities whose exports are drawn from composite supply: base exports positive and at least domestic output |
| $CE \subseteq CX \setminus CR$ | commodities exported from domestic output (base exports positive) |
| $CNE = CX \setminus CE$ | produced commodities not exported from domestic output |
| $CM \subseteq C$ | imported commodities (base imports positive) |
| $CNM = C \setminus CM$ | commodities that are not imported |
| $CXE \subseteq CE$ | export-only commodities: all domestic output is exported, with no domestic sales (when chosen; section 14.3) |
| $CED \subseteq CE$ | exported commodities that face a finite elasticity of export demand (section 14.2) |
| $CAC \subseteq CX$ | commodities whose output from several activities is combined by a CES function (section 18.3) |
| $CT \subseteq C$ | margin commodities: the services that margin accounts buy |
| $HC \subseteq C$ | health services whose consumption raises factor efficiency and health outcomes (may be empty; chapter 16) |

| Set | Description |
|---|---|
| $T$ | margin accounts (may be empty); $T^Q$, $T^D$, $T^M$ and $T^E$ are those paid per unit of composite supply (the default), of domestic sales, of imports and of exports (section 14.4) |
| $F$ | factors |
| $FL \subseteq F$ | factors with a labour force: a base unemployment rate, or a minimum wage, is given (chapter 15) |
| $FS \subseteq F$ | factors with days lost to illness (section 16.3) |
| $H$ | households, including non-profit institutions mapped as households |
| $E$ | enterprises (may be empty) |
| $D = H \cup E$ | domestic non-government institutions |
| $I$ | institutions that receive factor income: $D$, the government $gov$, the rest of the world $row$ and, when it is mapped, the health insurance fund $shi$ |

## 6.3 Variables

The variables are listed in three groups. The names are those of the results of **[CGE]** `cge_results`, which writes every variable of a simulation as a dataset.

### 6.3.1 Prices

| Variable | Description |
|---|---|
| $PA_a$ | activity price: gross revenue per unit of activity |
| $PVA_a$ | price of value added |
| $PINTA_a$ | price of aggregate intermediate input |
| $PX_c$ | producer price of commodity output, $c \in CX$ |
| $PD_c$ | price of domestic output sold domestically, $c \in CX$ |
| $PE_c$ | export price received by producers, in local currency, $c \in CE$ |
| $PWE_c$ | world export price, in foreign currency, $c \in CED$; elsewhere the parameter $pwe_c$ |
| $PM_c$ | import price in local currency, $c \in CM$ |
| $PQ_c$ | composite commodity price, including the sales tax |
| $W_f$ | average price of factor $f$ |
| $EXR$ | exchange rate: local currency per unit of foreign currency |
| $CPI$ | consumer price index |
| $DPI$ | producer price index of domestic sales |
| $PNA_{na}$ | price of nest $n$ of value added in activity $a$ (section 18.2) |
| $PXAC_{ac}$ | price of the output of commodity $c$ by activity $a$, $c \in CAC$ (section 18.3) |

### 6.3.2 Quantities

| Variable | Description |
|---|---|
| $QA_a$ | level of activity |
| $QVA_a$ | aggregate value added |
| $QINTA_a$ | aggregate intermediate input |
| $QF_{fa}$ | demand for factor $f$ by activity $a$ |
| $QINT_{ca}$ | intermediate use of commodity $c$ by activity $a$ |
| $QX_c$ | output of commodity $c \in CX$ |
| $QD_c$ | domestic sales of domestic output, $c \in CX$ |
| $QE_c$ | exports, $c \in CE$ |
| $QM_c$ | imports, $c \in CM$ |
| $QQ_c$ | composite supply of commodity $c$ |
| $QH_{ch}$ | consumption of commodity $c$ by household $h$ |
| $QHA_{ah}$ | home consumption by household $h$ of the output of activity $a$ (section 14.1) |

| Variable | Description |
|---|---|
| $QG_c$ | government consumption |
| $QINV_c$ | investment demand, including changes in inventories |
| $QT_c$ | demand for margin services, $c \in CT$ |
| $QER_c$ | exports drawn from composite supply, $c \in CR$ |
| $QSHI_c$ | purchases of commodity $c$ by the health insurance fund |
| $QFS_f$ | supply of factor $f$ |
| $QNA_{na}$ | quantity of nest $n$ of value added in activity $a$ |
| $QXAC_{ac}$ | output of commodity $c$ by activity $a$, $c \in CAC$ |

### 6.3.3 Incomes, rates and adjustment variables

| Variable | Description |
|---|---|
| $YF_f$ | income of factor $f$ |
| $YIF_{if}$ | income of institution $i$ from factor $f$ |
| $YI_d$ | income of domestic institution $d$ |
| $EH_h$ | consumption spending of household $h$ |
| $YG, EG$ | government revenue and spending |
| $GSAV$ | government savings |
| $FSAV$ | foreign savings, in foreign currency |
| $YSHI$ | revenue of the health insurance fund |
| $SHIADJ$ | scaling factor of the fund's purchases |
| $TINS_d$ | direct tax rate of institution $d$ |
| $MPS_d$ | marginal propensity to save of institution $d$ |
| $TINSADJ$ | scaling factor of direct tax rates |
| $TQADJ, TMADJ, TVADJ$ | scaling factors of the sales tax, import tariff and VAT rates; one unless the closure frees one of them (section 13.1) |
| $MPSADJ$ | scaling factor of savings rates |
| $IADJ$ | scaling factor of investment demand |
| $GADJ$ | scaling factor of government consumption |
| $WFDIST_{fa}$ | wage distortion: the price of factor $f$ in activity $a$ relative to its average price |
| $EFF_f$ | efficiency of factor $f$ from health services; one for a factor whose elasticity is zero (section 16.2) |
| $DW_f$ | days worked per worker of factor $f$, relative to the base; one for $f \notin FS$ (section 16.3) |
| $HS$ | volume of health services bought by households, the government and the health insurance fund |
| $LSW_f, LSM_f$ | slacks of the wage curve and of the minimum-wage condition, $f \in FL$; zero, and fixed, under the regime that uses the equation, free otherwise (chapter 15) |
| $RM_c, RE_c, RA_a$ | premiums on imports and exports, per unit of the world price, and rents on activity output, per unit of the activity price; zero and fixed unless a constraint frees them (section 17.1) |
| $WALRAS$ | slack of the savings–investment balance; zero at every solution |

## 6.4 Parameters

The parameters are computed by calibration (chapter 9) unless they are given by the user. Names in brackets are those of **[CGE]** `cge_parameters` and **[CGE]** `cge_shock` where they differ from the symbol.

| Parameter | Description |
|---|---|
| $pwm_c$, $pwe_c$ | world import and export prices, in foreign currency |
| $tm_c$, $te_c$, $tq_c$, $ta_a$ | import, export, sales and activity tax rates |
| $tv_c$, $\kappa_a$ | VAT rate on commodity $c$, and the share of the VAT on its inputs that activity $a$ recovers as a credit (`kappa`); $tv_c = 0$ and $\kappa_a = 1$ without VAT |

| Parameter | Description |
|---|---|
| $tf_{fa}$, $tfy_f$ | tax rates on the use of factor $f$ by activity $a$, and on the income of factor $f$ (section 18.1) |
| $\theta_{ac}$ | yield: output of commodity $c$ per unit of marketed output of activity $a$ (`theta`) |
| $iva_a$, $inta_a$ | value added and aggregate intermediate input per unit of activity, with a Leontief top nest |
| $aa_a$, $\delta_a^a$, $\rho_a^a$ | efficiency, value-added share and exponent of a CES or Cobb–Douglas top nest (`aa`, `dla`, `rha`) |
| $ica_{ca}$ | intermediate input of $c$ per unit of aggregate intermediate input of $a$ |
| $ad_a$, $\delta_{fa}^{va}$, $\rho_a^{va}$ | efficiency, factor shares and exponent of value added (`ad`, `dva`, `rva`) |
| $af_f$ | exogenous efficiency of factor $f$ in every activity; one in the base |
| $\eta_f$, $HS^0$ | elasticity of the efficiency of factor $f$ with respect to health services (`etah`), and the base volume of health services (`hs0`) |
| $at_c$, $\delta_c^t$, $\rho_c^t$ | efficiency, export share and exponent of the CET function (`at`, `dt`, `rt`) |
| $aq_c$, $\delta_c^q$, $\rho_c^q$ | efficiency, import share and exponent of the Armington function (`aq`, `dq`, `rq`) |
| $icm_{c'c}$ | margin services of $c' \in CT$ per unit of composite supply of $c$ |
| $icd_{c'c}$, $ici_{c'c}$, $ice_{c'c}$ | margin services per unit of domestic sales, imports and exports of $c$ |
| $shif_{if}$ | share of the income of factor $f$, after contributions and factor income taxes, received by institution $i$ |
| $tshi_f$ | health insurance contribution rate on the income of factor $f$ |
| $\overline{qshi}_c$, $sshi$ | base purchases and savings rate of the health insurance fund (`qshi`, `sshi`) |
| $\beta_{ch}$, $\gamma_{ch}$ | marginal budget share and subsistence quantity of commodity $c$ for household $h$; $\gamma_{ch} = 0$ with Cobb–Douglas demand (`beta`, `gamma`) |
| $\beta_{ah}^h$, $\gamma_{ah}^h$ | marginal budget share and subsistence quantity of home consumption (`betah`, `gammah`) |
| $\overline{tins}_d$, $\overline{mps}_d$ | base direct tax rate and savings rate of institution $d$ (`tins0`, `mps0`) |
| $shii_{dd'}$ | share of the income of $d'$, after direct taxes and savings, transferred to $d$ |
| $shir_d$ | share of the income of $d$, after direct taxes and savings, transferred to the rest of the world |
| $hc_h$ | share of the income of household $h$, after direct taxes and savings, spent on consumption |
| $trnsfr_{d,gov}$ | government transfers to institution $d$, in real terms (`trdg`) |
| $trnsfr_{gov,gov}$ | transfers within government, in real terms (`trgg`) |
| $trnsfr_{d,row}$ | transfers from the rest of the world to $d$, in foreign currency (`trdr`) |
| $trnsfr_{f,row}$ | factor income from abroad received by factor $f$, in foreign currency (`trfr`) |
| $trnsfr_{gov,row}$, $trnsfr_{row,gov}$ | transfers from the rest of the world to the government, and from the government to the rest of the world, in foreign currency (`trgr`, `trrg`) |
| $trnsfr_{shi,gov}$, $trnsfr_{shi,row}$ | transfers to the health insurance fund from the government (real terms) and from the rest of the world (foreign currency) (`trshi`, `trshir`) |
| $\overline{qg}_c$, $\overline{qinv}_c$ | base government consumption and investment quantities (`qg`, `qinv`) |
| $\overline{qer}_c$ | base exports from composite supply, $c \in CR$ (`qer`) |
| $cwts_c$, $dwts_c$ | weights of the CPI and the DPI |
| $DD$, $DF$, $r^d$, $r^f$ | public debt at the start of the period, held at home (local currency) and abroad (foreign currency), and their interest rates (`ddebt`, `fdebt`, `rd`, `rf`; section 13.3) |
| $hold_d$, $\phi$ | share of domestic interest received by institution $d$, and share of new borrowing from abroad |
| $\eta_c^e$, $\overline{qe}_c$ | elasticity of export demand and base exports, $c \in CED$ (`etae`, `qe0`) |
| $QLF_f$, $u_f^0$, $\epsilon_f^w$, $\overline{w}_f$, $\underline{w}_f$, $\mu$ | labour force, base unemployment rate, wage-curve elasticity, reference real wage, real minimum wage, and the smoothing of the minimum-wage condition, $f \in FL$ (`qlf`, `u0`, `epsw`, `wref`, `wmin`, `mu`) |
| $s_f^0$, $\sigma_f$, $\epsilon_f^s$ | base share of working time lost to illness, its current value and its elasticity, $f \in FS$ (`sick0`, `sick`, `epss`) |
| $m^0$, $\mu^m$, $\epsilon^m$, $POP$ | base crude death rate, its current value, its elasticity, and the population index (`mort0`, `mort`, `epsm`, `pop`) |

### 6.4.1 Elasticities

The elasticities are supplied by the user; the package has no default values for them. They are:

- $\sigma_a^{va}$, the elasticity of substitution between factors in value added, with $\rho_a^{va} = 1/\sigma_a^{va} - 1$; $\sigma_a^{va} = 1$ selects the Cobb–Douglas form;
- $\sigma_c^q$, the Armington elasticity of substitution between imports and domestic goods, with $\rho_c^q = 1/\sigma_c^q - 1$; for imported commodities with domestic output $\sigma_c^q \neq 1$ is required, because the model has no Cobb–Douglas form of the Armington function;
- $\sigma_c^t$, the elasticity of transformation between exports and domestic sales, with $\rho_c^t = 1/\sigma_c^t + 1$; for $c \in CR$ it is instead the price elasticity of exports from composite supply (equation (T9));
- optionally $\sigma_a^a$, the elasticity of substitution between value added and aggregate intermediate input, with $\rho_a^a = 1/\sigma_a^a - 1$; 0, the default, is the Leontief form and 1 the Cobb–Douglas form;
- optionally the income elasticities $\varepsilon_c$ and the Frisch parameter $\phi_h$ of linear expenditure system demand (section 9.7), and the elasticities of the extensions of chapters 13 to 18.

All elasticities of substitution and transformation must be positive and finite. Chapter 26 describes the library of published estimates that **[CGE]** `cge_elasticities` reads.

# 7 The equations of the standard model

The equations are grouped in five blocks: prices (P), production (Q), trade (T), institutions (I) and the system constraints (S). Their labels are those that **[CGE]** `cge_solve` displays in its diagnostics and that the GAMS program written by **[CGE]** `cge_gams` uses, with a prefix `E` (`EP1` is equation (P1)). Each equation holds for every element of the sets shown with it; a term that refers to a variable that does not exist for an element (for example imports of a commodity that is not imported) is zero.

Several equations include terms of the extensions: margins by market, VAT, taxes on factors, rents of constraints, public debt, home consumption and export demand. Without the extension, the terms are zero (or one, for a scaling factor) and the equation reduces to its standard form.

## 7.1 Prices

The import price is the world price, with the tariff and any import premium, converted at the exchange rate, plus the margins paid per unit of imports:

$$PM_c = pwm_c\,(1 + tm_c\,TMADJ + RM_c)\,EXR + \sum_{c'} PQ_{c'}\,ici_{c'c}, \qquad c \in CM \tag{P1}$$

The export price received by producers is the world price, less the export tax and any export premium, less the margins paid per unit of exports:

$$PE_c = PWE_c\,(1 - te_c - RE_c)\,EXR - \sum_{c'} PQ_{c'}\,ice_{c'c}, \qquad c \in CE \tag{P2}$$

with $PWE_c = pwe_c$ for $c \notin CED$. The premiums $RM_c$ and $RE_c$ are zero unless a quota or another constraint frees them (section 17.1).

The value of composite supply, net of the sales tax, is the value of domestic sales at the price domestic buyers pay, of imports, and of the margin services that composite supply carries:

$$PQ_c\,(1 - tq_c\,TQADJ)\,QQ_c = PDD_c\,QD_c + PM_c\,QM_c + \sum_{c' \in CT} PQ_{c'}\,icm_{c'c}\,QQ_c \tag{P3}$$

where

$$PDD_c = PD_c + \sum_{c'} PQ_{c'}\,icd_{c'c}$$

is the price that domestic buyers pay for domestic output, including the margins on domestic sales ($PDD_c = PD_c$ without them). The domestic term applies only for $c \in CX$, the import term only for $c \in CM$, and the margin term only when $c$ pays margins.

The value of commodity output is the value of its domestic sales and exports:

$$PX_c\,QX_c = PD_c\,QD_c + PE_c\,QE_c, \qquad c \in CX \tag{P4}$$

where the export term applies only for $c \in CE$. The activity price is the value of the commodities a unit of activity yields:

$$PA_a = \sum_c \theta_{ac}\,PX_c \tag{P5}$$

(for commodities whose output is aggregated by a CES function, $PX_c$ is replaced by the activity's own price $PXAC_{ac}$; section 18.3). The price of aggregate intermediate input includes the VAT that the activity cannot recover:

$$PINTA_a = \sum_c PQ_c\,\big(1 + tv_c\,TVADJ\,(1 - \kappa_a)\big)\,ica_{ca} \tag{P6}$$

Revenue net of the activity tax and of any rent on output equals the cost of value added and intermediate input:

$$PA_a\,(1 - ta_a - RA_a)\,QA_a = PVA_a\,QVA_a + PINTA_a\,QINTA_a \tag{P7}$$

The rent $RA_a$ is zero unless a capacity limit or another constraint frees it. The price indexes are

$$CPI = \sum_c cwts_c \, PH_c \tag{P8}$$

$$DPI = \sum_c dwts_c \, PD_c \tag{P9}$$

where

$$PH_c = PQ_c \, (1 + tv_c \, TVADJ)$$

is the price that final users pay, including VAT ($PH_c = PQ_c$ without VAT). $PH_c$ is an expression, not a variable.

## 7.2 Production

### 7.2.1 The top nest

An activity combines aggregate value added and aggregate intermediate input. With a Leontief top nest ($\sigma^a_a = 0$, the default), each is a fixed quantity per unit of activity:

$$QVA_a = iva_a \, QA_a \tag{Q1}$$

$$QINTA_a = inta_a \, QA_a \tag{Q2}$$

With a CES top nest ($\sigma^a_a > 0$, $\sigma^a_a \neq 1$), they substitute for each other as their prices change:

$$QA_a = aa_a \left[ \delta^a_a \, QVA_a^{-\rho^a_a} + (1 - \delta^a_a) \, QINTA_a^{-\rho^a_a} \right]^{-1/\rho^a_a} \tag{Q1C}$$

$$\frac{QVA_a}{QINTA_a} = \left[ \frac{PINTA_a}{PVA_a} \cdot \frac{\delta^a_a}{1 - \delta^a_a} \right]^{1/(1+\rho^a_a)} \tag{Q2C}$$

and with a Cobb–Douglas top nest ($\sigma^a_a = 1$),

$$QA_a = aa_a \, QVA_a^{\delta^a_a} \, QINTA_a^{1-\delta^a_a}, \qquad PVA_a \, QVA_a \, (1 - \delta^a_a) = PINTA_a \, QINTA_a \, \delta^a_a \tag{Q1C$'$, Q2C$'$}$$

In every case (P7) sets the activity price. An activity without intermediate inputs keeps the Leontief top nest.

### 7.2.2 Value added

Factors enter value added in efficiency units,

$$\widetilde{QF}_{fa} = af_f \, EFF_f \, DW_f \, QF_{fa},$$

where $af_f$ is the exogenous efficiency of the factor, $EFF_f$ its efficiency from health services (H2) and $DW_f$ the days worked per worker (H3); all three are one in the base. Factor quantities $QF_{fa}$, supplies $QFS_f$ and prices $W_f$ remain in physical units, so a more efficient factor earns more per unit.

With CES value added ($\sigma^{va}_a \neq 1$),

$$QVA_a = ad_a \left[ \sum_f \delta^{va}_{fa} \, \widetilde{QF}_{fa}^{-\rho^{va}_a} \right]^{-1/\rho^{va}_a} \tag{Q3}$$

and the demand for each factor equates its cost to the value of its marginal product:

$$W_f \, WFDIST_{fa} \, (1 + tf_{fa}) = PVA_a \, QVA_a \left[ \sum_{f'} \delta^{va}_{f'a} \, \widetilde{QF}_{f'a}^{-\rho^{va}_a} \right]^{-1} \delta^{va}_{fa} \, \widetilde{QF}_{fa}^{-\rho^{va}_a - 1} \, af_f \, EFF_f \, DW_f \tag{Q4}$$

where $tf_{fa}$ is the tax on the use of the factor (zero without factor taxes; section 18.1). With Cobb–Douglas value added ($\sigma^{va}_a = 1$),

$$QVA_a = ad_a \prod_f \widetilde{QF}_{fa}^{\delta^{va}_{fa}}, \qquad \sum_f \delta^{va}_{fa} = 1 \tag{Q3$'$}$$

$$W_f \, WFDIST_{fa} \, (1 + tf_{fa}) \, QF_{fa} = \delta^{va}_{fa} \, PVA_a \, QVA_a \tag{Q4$'$}$$

Value added may also combine nests of factors and commodities, each with its own elasticity; equations (Q3) and (Q4) then take the form of section 18.2.

### 7.2.3 Health services and factor efficiency

When some factor's efficiency depends on health services (section 16.2), or a health outcome of chapter 16 has a non-zero elasticity, the volume of health services bought by households, the government and the health insurance fund, at base prices, is

$$HS = \sum_{c \in HC} \Big( \sum_h QH_{ch} + QG_c + QSHI_c \Big) \tag{H1}$$

and the efficiency of factor $f$ is

$$EFF_f = \Big( HS / HS^0 \Big)^{\eta_f}, \qquad \eta_f > 0 \tag{H2}$$

with $EFF_f = 1$ otherwise. With a fixed population, $HS/HS^0$ is also the ratio of health services per head to its base value; in dynamic runs $HS^0$ grows with the population (chapter 19).

### 7.2.4 Intermediate input and output

Intermediate inputs are Leontief within the aggregate:

$$QINT_{ca} = ica_{ca}\, QINTA_a \tag{Q5}$$

Output of each commodity is the sum of the yields of the activities that produce it, applied to their marketed output:

$$QX_c = \sum_a \theta_{ac} \Big( QA_a - \sum_h QHA_{ah} \Big), \qquad c \in CX \tag{Q6}$$

Output that households consume at home ($QHA_{ah}$, section 14.1) does not reach the commodity market. The output of a commodity from different activities is a perfect substitute and sells at one producer price $PX_c$, unless it is aggregated by a CES function (section 18.3).

## 7.3 Trade

### 7.3.1 Exports

For commodities exported from domestic output, $c \in CE$, a CET function transforms output into exports and domestic sales:

$$QX_c = at_c \Big[ \delta^t_c\, QE_c^{\rho^t_c} + (1 - \delta^t_c)\, QD_c^{\rho^t_c} \Big]^{1/\rho^t_c} \tag{T1}$$

$$\frac{QE_c}{QD_c} = \left[ \frac{PE_c}{PD_c} \cdot \frac{1 - \delta^t_c}{\delta^t_c} \right]^{1/(\rho^t_c - 1)} \tag{T2}$$

Produced commodities that are not exported, $c \in CNE$, are sold at home:

$$QX_c = QD_c \tag{T3}$$

and export-only commodities, $c \in CXE$, are exported in full, in place of (T1) and (T2):

$$QX_c = QE_c \tag{T3E}$$

Exports that face a finite elasticity of world demand, $c \in CED$, fall as the world price rises:

$$QE_c = \overline{qe}_c \left( \frac{PWE_c}{pwe_c} \right)^{-\eta^e_c} \tag{T10}$$

where $pwe_c$ now shifts the demand curve: at the base world price the world buys $\overline{qe}_c$ (section 14.2).

Some SAMs record exports that domestic output cannot supply: re-exports of imported goods, exports from inventories (with a matching fall in stocks), or exports valued at purchaser prices, including the sales taxes and margins on the exported goods. For such a commodity the CET is not defined, because its domestic sales would be zero or negative. Its exports, $c \in CR$, are instead drawn from composite supply, like any other demand, and sold at the composite price, with elasticity $\sigma_c^t$ with respect to the world price in local currency:

$$QER_c = \overline{qer}_c \left( \frac{PQ_c}{EXR \cdot pwe_c} \right)^{-\sigma_c^t} \tag{T9}$$

The commodity's domestic output, if any, is sold at home (T3).

### 7.3.2 Imports and composite supply

For imported commodities with domestic output, $c \in CM \cap CX$, an Armington function combines imports and domestic sales:

$$QQ_c = aq_c \left[ \delta_c^q \, QM_c^{-\rho_c^q} + (1 - \delta_c^q) \, QD_c^{-\rho_c^q} \right]^{-1/\rho_c^q} \tag{T4}$$

$$\frac{QM_c}{QD_c} = \left[ \frac{PDD_c}{PM_c} \cdot \frac{\delta_c^q}{1 - \delta_c^q} \right]^{1/(1+\rho_c^q)} \tag{T5}$$

For commodities that are not imported, $c \in CNM$ (all of which have domestic output),

$$QQ_c = aq_c \, QD_c \tag{T6}$$

with $aq_c = QQ_c^0 / QD_c^0$. With base prices of one, a sales tax makes composite supply exceed domestic sales in value units, $QQ_c^0 = QD_c^0 + SAM(tax_{sal}, c)$, so the coefficient is not one. For commodities that are imported only, $c \in CM \setminus CX$,

$$QQ_c = aq_c \, QM_c \tag{T7}$$

with $aq_c = QQ_c^0 / QM_c^0$, which exceeds one when the commodity pays a tariff, sales tax or margins.

### 7.3.3 Margins

Margins are a fixed quantity of services per unit of composite supply, and, for margin accounts declared by market, per unit of domestic sales, imports or exports (section 14.4). Demand for margin services, $c \in CT$, is

$$QT_c = \sum_{c'} \left( icm_{cc'} \, QQ_{c'} + icd_{cc'} \, QD_{c'} + ici_{cc'} \, QM_{c'} + ice_{cc'} \, QE_{c'} \right) \tag{T8}$$

and their cost enters the prices through (P1), (P2) and (P3). SAMs often record one margin payment per commodity, without a split between domestic sales, imports and exports; such margins are attributed to composite supply, as sales taxes are.

## 7.4 Institutions

### 7.4.1 Factor income

The income of a factor is its earnings in every activity, factor income from abroad, and any rents paid to it:

$$YF_f = \sum_a W_f \, WFDIST_{fa} \, QF_{fa} + trnsfr_{f,row} \, EXR + R_f \tag{I1}$$

Here and below $R_\varrho$ is the sum of the rents paid to the recipient $\varrho$ (section 17.1); it is zero without constraints. Factor income is distributed, after contributions to the health insurance fund and taxes on factor income, in fixed shares:

$$YIF_{if} = shif_{if} \, (1 - tshi_f - tfy_f) \, YF_f, \quad i \in I \setminus \{shi\}; \qquad YIF_{shi,f} = tshi_f \, YF_f \tag{I2}$$

Without a fund, $tshi_f = 0$; without a tax on factor income, $tfy_f = 0$.

### 7.4.2 Households and enterprises

The income of a domestic institution is its factor income, transfers from other domestic institutions out of their income after direct taxes and savings, transfers from the government and from abroad, interest on the domestic public debt it holds, and any rents:

$$\begin{aligned} YI_d = {} & \sum_f YIF_{df} + \sum_{d' \in D} shii_{dd'}\,(1 - MPS_{d'})(1 - TINS_{d'})\,YI_{d'} \\ & + trnsfr_{d,gov}\,CPI + trnsfr_{d,row}\,EXR + hold_d\,r^d\,DD + R_d \end{aligned} \tag{I3}$$

A household spends a fixed share of its income after direct taxes and savings on consumption:

$$EH_h = hc_h\,(1 - MPS_h)(1 - TINS_h)\,YI_h \tag{I4}$$

Enterprises do not consume: for $e \in E$, $\sum_d shii_{de} + shir_e = 1$, so all their income after direct taxes and savings is transferred. For households, $hc_h = 1 - \sum_d shii_{dh} - shir_h$. A transfer from an institution to itself (a diagonal SAM cell) is allowed; it appears on both sides of its accounts.

Household demand is a linear expenditure system: households buy subsistence quantities and spend the rest of their consumption budget, their supernumerary spending

$$SN_h = EH_h - \sum_c PH_c\,\gamma_{ch} - \sum_a PA_a\,\gamma^h_{ah},$$

in fixed marginal shares:

$$PH_c\,QH_{ch} = PH_c\,\gamma_{ch} + \beta_{ch}\,SN_h \tag{I5}$$

$$PA_a\,QHA_{ah} = PA_a\,\gamma^h_{ah} + \beta^h_{ah}\,SN_h \tag{I5H}$$

With $\gamma_{ch} = 0$ this is Cobb–Douglas demand, the default: fixed budget shares and income elasticities of one. Home consumption (I5H) is valued at the activity price, without taxes or margins; the marginal shares of marketed and home goods together sum to one.

### 7.4.3 Investment and government demand

$$QINV_c = \overline{qinv}_c\,IADJ \tag{I6}$$

$$QG_c = \overline{qg}_c\,GADJ \tag{I7}$$

### 7.4.4 Government

Government revenue is the direct taxes, the taxes on sales, imports, exports, activities and factors, net VAT, the government's factor income, transfers from abroad and within government, and any rents paid to it:

$$\begin{aligned} YG = {} & \sum_d TINS_d\,YI_d + \sum_c tq_c\,TQADJ\,PQ_c\,QQ_c + \sum_{c \in CM} tm_c\,TMADJ\,pwm_c\,QM_c\,EXR \\ & + \sum_{c \in CE} te_c\,PWE_c\,QE_c\,EXR + \sum_a ta_a\,PA_a\,QA_a + VAT \\ & + \sum_{f,a} tf_{fa}\,W_f\,WFDIST_{fa}\,QF_{fa} + \sum_f tfy_f\,YF_f + \sum_f YIF_{gov,f} \\ & + trnsfr_{gov,row}\,EXR + trnsfr_{gov,gov}\,CPI + R_{gov} \end{aligned} \tag{I8}$$

with VAT revenue, net of the credits that activities recover,

$$VAT = TVADJ \sum_c tv_c\,PQ_c \Big( \sum_h QH_{ch} + QG_c + QINV_c + QSHI_c + \sum_a (1 - \kappa_a)\,QINT_{ca} \Big).$$

Government spending is its consumption, its transfers, and the interest on public debt:

$$\begin{aligned} EG = {} & \sum_c PH_c\,QG_c + \sum_d trnsfr_{d,gov}\,CPI + trnsfr_{gov,gov}\,CPI + trnsfr_{shi,gov}\,CPI \\ & + trnsfr_{row,gov}\,EXR + r^d\,DD + r^f\,DF\,EXR \end{aligned} \tag{I9}$$

Direct tax and savings rates are their base values times scaling factors that the closure fixes or frees:

$$TINS_d = \overline{tins}_d\,TINSADJ \tag{I10}$$

$$MPS_d = \overline{mps}_d\,MPSADJ \tag{I11}$$

### 7.4.5 The health insurance fund

When a social health insurance fund is mapped (section 16.1), its revenue is the contributions on factor income and the transfers from the government and from abroad:

$$YSHI = \sum_f YIF_{shi,f} + trnsfr_{shi,gov}\, CPI + trnsfr_{shi,row}\, EXR \tag{I12}$$

It buys a fixed basket of health services, scaled so that it spends its revenue less a fixed share saved:

$$QSHI_c = \overline{qshi}_c\, SHIADJ \tag{I13}$$

$$\sum_c PH_c\, QSHI_c = (1 - sshi)\, YSHI \tag{I14}$$

## 7.5 System constraints

Factor markets clear:

$$\sum_a QF_{fa} = QFS_f \tag{S1}$$

Commodity markets clear: composite supply equals intermediate demand, household, government and investment demand, margin services, exports from composite supply and the purchases of the health insurance fund:

$$QQ_c = \sum_a QINT_{ca} + \sum_h QH_{ch} + QG_c + QINV_c + QT_c + QER_c + QSHI_c \tag{S2}$$

where $QT_c$ applies only for $c \in CT$ and $QER_c$ only for $c \in CR$. The balance of payments, in foreign currency, equates payments abroad (imports, factor income paid abroad, transfers abroad from domestic institutions and the government, interest on foreign debt, and rents paid abroad) to receipts (exports, transfers and factor income from abroad, and foreign savings):

$$\begin{aligned} &\sum_{c\in CM} pwm_c\, QM_c + \sum_f \frac{YIF_{row,f}}{EXR} + \sum_d \frac{shir_d\,(1 - MPS_d)(1 - TINS_d)\, YI_d}{EXR} \\ &\quad + trnsfr_{row,gov} + r^f\, DF + \frac{R_{row}}{EXR} \\ &= \sum_{c\in CE} PWE_c\, QE_c + \sum_{c\in CR} \frac{PQ_c\, QER_c}{EXR} + \sum_d trnsfr_{d,row} + \sum_f trnsfr_{f,row} \\ &\quad + trnsfr_{gov,row} + trnsfr_{shi,row} + FSAV \end{aligned} \tag{S3}$$

The government's budget defines its savings:

$$YG = EG + GSAV \tag{S4}$$

and savings equal investment:

$$\sum_d MPS_d\,(1 - TINS_d)\, YI_d + GSAV + sshi\, YSHI + EXR \cdot FSAV = \sum_c PH_c\, QINV_c + WALRAS \tag{S5}$$

The price level is set by the numeraire:

$$CPI = \overline{CPI} \quad \text{or} \quad DPI = \overline{DPI} \tag{S6}$$

which is imposed by fixing the numeraire variable in the closure (chapter 8); it is not counted as a separate equation.

By Walras's law one equation is redundant: if every other market clears and every institution meets its budget, savings equal investment. The variable *WALRAS* makes the system square, and must be zero, to the solver's tolerance, at every solution. **[CGE]** `cge_results` and **[CGE]** `cge_check` report it.

# 8 Closures

The model has more variables than equations. The closure chooses which variables are fixed at their current values and which the model determines. It expresses assumptions about how the economy adjusts to a shock, and results can depend on it as much as on the shock itself. **[CGE]** `cge_closure` sets the closure, and every simulation stores the closure under which it was solved.

## 8.1 Closure options

A closure option fixes one variable of each pair and frees the other. The default, which **[CGE]** `cge_calibrate` sets, is listed first for each block.

| Block | Option | Fixed | Free |
|---|---|---|---|
| External | `flexexr`: flexible exchange rate | $FSAV$ | $EXR$ |
| | `fixexr`: fixed exchange rate | $EXR$ | $FSAV$ |
| Government | `flexgsav`: flexible government savings | $TINSADJ$, $TQADJ$, $TMADJ$, $TVADJ$, $GADJ$ | $GSAV$ |
| | `fixgsav` or `fixgsav direct`: direct tax rates adjust | $GSAV$, $TQADJ$, $TMADJ$, $TVADJ$, $GADJ$ | $TINSADJ$ |
| | `fixgsav sales`: sales tax rates adjust | $GSAV$, $TINSADJ$, $TMADJ$, $TVADJ$, $GADJ$ | $TQADJ$ |
| | `fixgsav import`: import tariffs adjust | $GSAV$, $TINSADJ$, $TQADJ$, $TVADJ$, $GADJ$ | $TMADJ$ |
| | `fixgsav vat`: VAT rates adjust | $GSAV$, $TINSADJ$, $TQADJ$, $TMADJ$, $GADJ$ | $TVADJ$ |
| Savings–investment | `savings`: savings-driven | $MPSADJ$ | $IADJ$ |
| | `investment`: investment-driven | $IADJ$ | $MPSADJ$ |
| Factor $f$ | `mobile`: fully employed and mobile | $QFS_f$, $WFDIST_{fa}$ | $W_f$, $QF_{fa}$ |
| | `specific`: fixed in each activity | $W_f$, $QF_{fa}$ | $WFDIST_{fa}$, $QFS_f$ |
| | `unemployed`: unemployment at a fixed wage | $W_f$, $WFDIST_{fa}$ | $QFS_f$, $QF_{fa}$ |
| | `wagecurve`, $f \in FL$ | $WFDIST_{fa}$, $LSW_f$ | $W_f$, $QFS_f$, $QF_{fa}$ |
| | `minwage`, $f \in FL$ | $WFDIST_{fa}$, $LSM_f$ | $W_f$, $QFS_f$, $QF_{fa}$ |
| Numeraire | `cpi` | $CPI$ | $DPI$ |
| | `dpi`: producer prices of domestic sales | $DPI$ | $CPI$ |

The tax scaling factors $TQADJ$, $TMADJ$ and $TVADJ$ are fixed at one, except the one that the government closure frees (section 13.1). The rents $RM_c$, $RE_c$ and $RA_a$ are fixed at zero under every closure; a constraint frees one of them, or another variable, and adds an equation, so the system stays square (chapter 17). Variables that do not exist for an element, such as the imports of a commodity that is not imported, are neither fixed nor free: they are left out of the system.

The closures differ in what adjusts. Under `flexexr` foreign savings are fixed in foreign currency and the exchange rate moves the trade balance; under `fixexr` the exchange rate is fixed and foreign savings absorb any change in the current account. Under `flexgsav` tax rates and real government consumption are fixed and government savings absorb changes in revenue; under `fixgsav` government savings are fixed and one tax's rates are scaled in proportion to keep them. Under `savings` the savings rates are fixed and investment adjusts to available savings; under `investment` real investment is fixed and the savings rates of all domestic institutions are scaled to finance it. The closures of each factor are described with its regime: fully employed factors that move between activities at one average price, factors fixed in each activity that earn activity-specific prices, and factors whose supply adjusts at a fixed price, along a wage curve or above a minimum wage (chapter 15).

## 8.2 Swaps

After the options above, a closure may be changed element by element, as GAMS modellers do by fixing and freeing variables. A swap fixes one element of a variable that the closure leaves free and frees one that it fixes, so the number of free variables does not change. For example,

- `swap(GSAV GADJ)` fixes government savings and lets government consumption adjust;
- `swap(EXR CPI)` makes the exchange rate the numeraire;
- `swap(QG(com_hlt) GADJ)` fixes government purchases of health services, which can then be shocked, and scales the other purchases with them.

Variables that the reduced system computes from their definitions, such as household demand, cannot be fixed (section 10.2).

## 8.3 Checks

Every closure is checked when it is set. It must be square, with as many free variables as equations. It must also leave every equation with at least one free variable: a closure that leaves an equation without one makes the Jacobian singular, and **[CGE]** `cge_closure` names the equation. These are structural checks, over the pattern of the variables that each equation contains.

# 9 Calibration

Calibration computes the base value of every variable and every parameter of the model from the SAM and from the elasticities and other settings given by the user, so that the base solves every equation. **[CGE]** `cge_map` assigns the accounts to roles, checks the SAM and rewrites the common national-accounts layouts; **[CGE]** `cge_calibrate` then computes the parameters and checks benchmark replication. In this chapter $SAM(r, c)$ is a cell of the SAM after the rewrites, and a superscript 0 marks a base value.

## 9.1 SAM checks and the flows the model represents

Before calibration the SAM must be square, with identical row and column labels; it must be balanced, each row total equal to its column total within $10^{-8}$ times the largest absolute cell; and every account with a non-zero flow must have a role. Every non-zero cell must be one of the flows of the table below. **[CGE]** `cge_map` lists every other flow, grouped by the roles of the paying and the receiving accounts, before it stops.

| Paid by (column) | Received by (row) |
|---|---|
| commodity | activities (domestic output); rest of the world (imports); import, sales, export and VAT accounts; margin accounts (margins) |
| activity | commodities (intermediate inputs); factors; activity tax account; accounts of taxes on factor use |
| factor | households; enterprises; government; rest of the world; health insurance fund (contributions); account of taxes on factor income |
| household | commodities; activities (home consumption); direct tax account; savings–investment; households and enterprises (transfers); rest of the world (transfers) |
| enterprise | direct tax account; savings–investment; households and enterprises (transfers); rest of the world (transfers) |
| government | commodities; households and enterprises (transfers); government (internal transfers); rest of the world (transfers); savings–investment; health insurance fund |
| tax account | government |
| savings–investment | commodities (investment, including changes in inventories) |
| margin account | commodities (margin services) |
| health insurance fund | commodities (purchases); savings–investment |
| rest of the world | commodities (exports); households, enterprises and government (transfers); factors (factor income from abroad); savings–investment; health insurance fund |

Negative cells are allowed only in tax rows, the savings–investment row, transfer cells (the flows labelled transfers above, and factor income from abroad), payments of tax accounts to the government, VAT cells, investment demand (a fall in inventories) and sales of commodities by activities (a negative entry of the make matrix gives a negative yield; each activity's output and each commodity's domestic output must still be positive). Every activity needs positive output and positive value added, and every commodity needs domestic output or imports. An account with no flows at all needs no role.

## 9.2 Layouts rewritten before the checks

National SAMs often record some flows differently from the layout of the model. **[CGE]** `cge_map` rewrites the common ones before the checks above and reports the amounts it moves; the SAM itself is not changed, and **[CGE]** `cge_samexport` with `mapped` writes the SAM after the rewrites.

### 9.2.1 Exports recorded as sales by activities

A cell $SAM(a, row)$ is treated as exports of the commodity paired with $a$: it is added to $SAM(c(a), row)$ and to $SAM(a, c(a))$, and set to zero. The activity's output and the rest of the world's spending are unchanged; the commodity's supply and demand both rise by the exports.

### 9.2.2 Taxes paid directly to the government

A tax option of the mapping (direct, sales, import, export or activity tax) may name the government account itself. The cells of the government's row in the columns that pay that tax (households and enterprises for the direct tax, commodities for the sales, import and export taxes, activities for the activity tax) are then moved to an internal tax account, whose column pays their total to the government. Government revenue is unchanged. At most one of the sales, import and export tax options may name the government account, because the cells would otherwise be ambiguous.

### 9.2.3 Margins netted out of margin activities

Some SAMs attribute trade and transport margins to the producing activities, as their purchases of trade and transport services, and net the margin revenue out of the output of the trade and transport activities by recording a negative use of their own product, $SAM(c(a), a) < 0$. For each activity named in the option `margins()`, $-SAM(c(a), a)$ is added to its output $SAM(a, c(a))$, and $SAM(c(a), a)$ is set to zero. The activity's output then equals its costs, and the margins enter the model as the producing activities' fixed-coefficient purchases of trade and transport services, as the SAM records them. Naming an activity without a negative use of its own product is an error. Only the net amount is recorded, so after the rewrite the activity's use of its own product is zero.

### 9.2.4 Territorial correction

National accounts move household consumption from the territorial to the residents' concept with a correction account: residents' purchases abroad are recorded as its imports, non-residents' purchases in the country as its exports (sometimes in the row of a matching activity), and the net as a negative household consumption of the correction. For the commodity (and, optionally, the activity) named in `territorial()`, the activity's exports are first moved to the commodity, and the activity must then have no flows. With $IM$ the commodity's imports, $EX$ its exports and $HC_h$ household $h$'s consumption of it, $\sum_h HC_h = IM - EX$, each household's share is $s_h = HC_h / \sum_h HC_h$, and

$$SAM(row, h) \mathrel{+}= s_h\, IM, \qquad SAM(h, row) \mathrel{+}= s_h\, EX :$$

residents' purchases abroad become household payments to the rest of the world, and non-residents' purchases become transfers from the rest of the world to households. The correction's cells are then set to zero.

### 9.2.5 Stock-change accounts

Some SAMs record changes in inventories in a separate account, financed by savings–investment and buying commodities. For an account named in `stocks()`, its payments are added to the savings–investment column, and its row and column are set to zero. Changes in inventories are then part of investment demand, scaled by $IADJ$ (I6).

### 9.2.6 VAT within the sales tax

When the SAM records VAT within the sales tax account, `vatshare()` gives the VAT share of each commodity's sales tax; that share of $SAM(tax_{sal}, c)$ is moved to an internal VAT account before the checks (section 13.2). `vatshare(0)` creates an empty VAT account, so that a VAT can be introduced by a shock.

### 9.2.7 Roles that are not rewrites

Margin accounts, mapped with `marginaccounts()` or, for margins by market, with `domesticmargins()`, `importmargins()` and `exportmargins()`, collect margins from commodities, $SAM(t, c)$, and buy margin services, $SAM(c', t)$; they may have no other flows. A health insurance fund (`insurance()`) receives contributions from factors and transfers from the government and the rest of the world, buys health services and saves. A household's payment to an activity, $SAM(a, h)$, is home consumption of the activity's output (section 14.1).

The rewrites of exports, of taxes paid to the government, of the territorial correction and of stock changes leave every total that enters the model unchanged, except that the territorial correction replaces the correction's imports, exports and negative consumption by transfers. The margin rewrite raises the output of the margin activities, and the supply and demand of their products, by the margins.

## 9.3 Normalisation

All base prices are one:

$$PA_a^0 = PVA_a^0 = PX_c^0 = PD_c^0 = PE_c^0 = PQ_c^0 = 1, \qquad EXR^0 = 1, \qquad pwm_c = 1,$$

and $W_f^0 = WFDIST_{fa}^0 = 1$ unless factor quantities are supplied (section 9.9). The base quantity of every flow is therefore its value in the SAM.

## 9.4 Taxes and trade

- Imports: $QM_c^0 = SAM(row, c)$, $tm_c = SAM(tax_{imp}, c)/QM_c^0$ and $PM_c^0 = 1 + tm_c$.
- Activities: $QA_a^0$ is the column total of activity $a$, and $ta_a = SAM(tax_{act}, a)/QA_a^0$.
- Yields: $\theta_{ac} = SAM(a, c)/QA_a^0$, and commodity output $QX_c^0 = \sum_a SAM(a, c)$; $CX$ is the set of commodities with $QX_c^0 > 0$. With home consumption, marketed output replaces $QA_a^0$ in the yields (section 14.1).
- Exports: a commodity with $SAM(c, row) > 0$ belongs to $CR$ if $SAM(c, row) - SAM(tax_{exp}, c) \geq QX_c^0$, and to $CE$ otherwise.
- For $c \in CE$: $te_c = SAM(tax_{exp}, c)/SAM(c, row)$, $QE_c^0 = SAM(c, row) - SAM(tax_{exp}, c)$, $pwe_c = 1/(1 - te_c)$ and $QD_c^0 = QX_c^0 - QE_c^0$.
- For $c \in CR$: $\overline{qer}_c = QER_c^0 = SAM(c, row)$, $pwe_c = 1$ and $QD_c^0 = QX_c^0$. An export tax on such a commodity is an error.
- For other commodities, $QD_c^0 = QX_c^0$.
- Margins paid by commodity $c$: $MG_c^0 = \sum_{t \in T} SAM(t, c)$.
- Composite supply and the sales tax rate:

  $$QQ_c^0 = QD_c^0 + PM_c^0\, QM_c^0 + SAM(tax_{sal}, c) + MG_c^0, \qquad tq_c = SAM(tax_{sal}, c)/QQ_c^0.$$

  VAT is not part of $QQ_c^0$ (section 13.2).
- Margin services: $QT_{c'}^0 = \sum_{t \in T} SAM(c', t)$ and

  $$icm_{c'c} = \frac{1}{QQ_c^0} \sum_{t \in T} SAM(t, c) \, \frac{SAM(c', t)}{\sum_{c''} SAM(c'', t)},$$

  so that the margins collected by account $t$ are spent on services in the proportions in which $t$ buys them.

Balance of the SAM implies that $QQ_c^0$ equals total demand for $c$ at purchaser prices other than exports from domestic output and VAT: the commodity's row total less $QE_c^0$ and $SAM(vat, c)$. Calibration checks this identity rather than assuming it.

## 9.5 Production

- Intermediate input: $QINTA^0_a = \sum_c SAM(c,a)$ and $ica_{ca} = SAM(c,a)/QINTA^0_a$ (with VAT, the quantities of section 13.2).
- Factors: $QF^0_{fa} = SAM(f,a)/(W^0_f\, WFDIST^0_{fa})$ and $QVA^0_a = \sum_f SAM(f,a)$ (with taxes on factor use, their cost is added; section 18.1).
- Leontief top nest: $iva_a = QVA^0_a/QA^0_a$ and $inta_a = QINTA^0_a/QA^0_a$.
- CES top nest: with $r_a = (PVA^0_a/PINTA^0_a)\,(QVA^0_a/QINTA^0_a)^{1+\rho^a_a}$, $\delta^a_a = r_a/(1+r_a)$ and $1-\delta^a_a = 1/(1+r_a)$ (both stored), and

$$aa_a = QA^0_a\Big/\Big[\delta^a_a\,(QVA^0_a)^{-\rho^a_a} + (1-\delta^a_a)\,(QINTA^0_a)^{-\rho^a_a}\Big]^{-1/\rho^a_a}.$$

- Cobb–Douglas top nest: $\delta^a_a = PVA^0_a\,QVA^0_a/(PVA^0_a\,QVA^0_a + PINTA^0_a\,QINTA^0_a)$ and $aa_a = QA^0_a/\big[(QVA^0_a)^{\delta^a_a}(QINTA^0_a)^{1-\delta^a_a}\big]$.
- CES value added: with $w_{fa} = W^0_f\, WFDIST^0_{fa}$,

$$\delta^{va}_{fa} = \frac{w_{fa}\,(QF^0_{fa})^{1+\rho^{va}_a}}{\sum_{f'} w_{f'a}\,(QF^0_{f'a})^{1+\rho^{va}_a}}, \qquad ad_a = QVA^0_a\Big/\Big[\sum_f \delta^{va}_{fa}\,(QF^0_{fa})^{-\rho^{va}_a}\Big]^{-1/\rho^{va}_a}.$$

- Cobb–Douglas value added: $\delta^{va}_{fa} = SAM(f,a)/QVA^0_a$ and $ad_a = QVA^0_a/\prod_f (QF^0_{fa})^{\delta^{va}_{fa}}$.
- Factor efficiency: $af_f = EFF^0_f = DW^0_f = 1$, so the parameters above are unchanged, and

$$HS^0 = \sum_{c\in HC}\Big[\sum_h SAM(c,h) + SAM(c,gov) + SAM(c,shi)\Big],$$

which must be positive when some health elasticity is positive.

## 9.6 CET and Armington functions

- CET:

$$\delta^t_c = \frac{1}{1+(PD^0_c/PE^0_c)\,(QE^0_c/QD^0_c)^{\rho^t_c-1}},$$
$$at_c = QX^0_c\Big/\Big[\delta^t_c\,(QE^0_c)^{\rho^t_c} + (1-\delta^t_c)\,(QD^0_c)^{\rho^t_c}\Big]^{1/\rho^t_c}.$$

- Armington, for imported commodities with domestic output:

$$\delta^q_c = \frac{(PM^0_c/PD^0_c)\,(QM^0_c/QD^0_c)^{1+\rho^q_c}}{1+(PM^0_c/PD^0_c)\,(QM^0_c/QD^0_c)^{1+\rho^q_c}},$$
$$aq_c = QQ^0_c\Big/\Big[\delta^q_c\,(QM^0_c)^{-\rho^q_c} + (1-\delta^q_c)\,(QD^0_c)^{-\rho^q_c}\Big]^{-1/\rho^q_c};$$

  $aq_c = QQ^0_c/QD^0_c$ for commodities that are not imported, and $aq_c = QQ^0_c/QM^0_c$ for those that are imported only. With margins by market, $PDD^0_c$ and $PM^0_c$ include the margins (section 14.4).

The complements $1-\delta^t_c$ and $1-\delta^q_c$ are stored as separate parameters, computed from the same ratio as the shares (for example $1-\delta^t_c = r/(1+r)$ with $\delta^t_c = 1/(1+r)$), and used in (T1), (T2), (T4) and (T5). Computing $1-\delta$ by subtraction loses most significant digits when a share is close to one, as with a low CET elasticity and a small export share, and can leave a benchmark residual near $10^{-7}$.

## 9.7 Institutions

- Factor income: $YF^0_f$ is the row total of factor $f$, and

$$tshi_f = \frac{SAM(shi,f)}{YF^0_f}, \qquad shif_{if} = \frac{SAM(i,f)}{(1-tshi_f)\,YF^0_f}, \quad i \neq shi,$$

  with $tshi_f = 0$ without a fund; with a tax on factor income the denominator is $(1 - tshi_f - tfy_f)\,YF^0_f$.

- Health insurance fund: $YSHI^0$ is the row total of $shi$, the base purchases are $\overline{qshi}_c = SAM(c, shi)$ and the savings rate is $sshi = SAM(sav, shi)/YSHI^0$; the transfers are $trnsfr_{shi,gov} = SAM(shi, gov)$ and $trnsfr_{shi,row} = SAM(shi, row)$; and $SHIADJ^0 = 1$.

- Domestic institutions: $YI^0_d$ is the row total of $d \in D$; $\overline{tins}_d = SAM(tax_{dir}, d)/YI^0_d$ and $\overline{mps}_d = SAM(sav, d)/\big[(1 - \overline{tins}_d)\,YI^0_d\big]$.

- Income after direct taxes and savings: $DY^0_d = YI^0_d - SAM(tax_{dir}, d) - SAM(sav, d)$; calibration stops with an error if $DY^0_d = 0$ while $d$ makes transfers or consumes. Then $shii_{dd'} = SAM(d, d')/DY^0_{d'}$ and $shir_d = SAM(row, d)/DY^0_d$.

- Consumption: $EH^0_h = \sum_c SAM(c, h)$ (plus home consumption), $hc_h = EH^0_h/DY^0_h$, and budget shares $w_{ch} = SAM(c, h)/EH^0_h$.

- Cobb–Douglas demand, the default: $\beta_{ch} = w_{ch}$ and $\gamma_{ch} = 0$.

- Linear expenditure system: given income elasticities $\varepsilon_c > 0$ and a Frisch parameter $\phi_h < -1$ for each household,

$$\beta_{ch} = \frac{\varepsilon_c\, w_{ch}}{\sum_{c'} \varepsilon_{c'}\, w_{c'h}}, \qquad \gamma_{ch} = QH^0_{ch} + \frac{\beta_{ch}\, EH^0_h}{PH^0_c\, \phi_h},$$

  so that the marginal shares sum to one (the income elasticities used are then $\beta_{ch}/w_{ch}$, proportional to those supplied). Supernumerary spending, $EH^0_h - \sum_c PH^0_c\, \gamma_{ch}$, is then $-EH^0_h/\phi_h$, positive and below total spending because $\phi_h < -1$.

- Transfers: $trnsfr_{d,gov} = SAM(d, gov)$, $trnsfr_{gov,gov} = SAM(gov, gov)$, $trnsfr_{d,row} = SAM(d, row)$, $trnsfr_{f,row} = SAM(f, row)$, $trnsfr_{gov,row} = SAM(gov, row)$ and $trnsfr_{row,gov} = SAM(row, gov)$ (with public debt, the interest is separated from them; section 13.3).

- Final demand: $\overline{qg}_c = SAM(c, gov)$ and $\overline{qinv}_c = SAM(c, sav)$ (divided by $1 + tv_c$ with VAT), $GSAV^0 = SAM(sav, gov)$ and $FSAV^0 = SAM(sav, row)$.

- Scaling factors: $TINSADJ^0 = MPSADJ^0 = IADJ^0 = GADJ^0 = 1$, and the tax scaling factors are one.

- Price index weights:

$$cwts_c = \frac{\sum_h SAM(c,h)}{(1+tv_c)\sum_{c'}\sum_h SAM(c',h)}, \qquad dwts_c = \frac{QD^0_c}{\sum_{c'} QD^0_{c'}},$$

  so that $CPI^0 = DPI^0 = 1$.

Balance of the SAM implies $hc_h = 1 - \sum_d shii_{dh} - shir_h$ for households and $\sum_d shii_{de} + shir_e = 1$ for enterprises; calibration checks both.

## 9.8 Tax accounts

Each tax is recorded in a tax account whose row collects the tax and whose column pays only the government, either in the SAM itself or as an internal account created by a rewrite (section 9.2).

## 9.9 Factor quantities

If the user supplies base factor quantities $QF^0_{fa}$, for example employment, the base prices of the factors are

$$W^0_f = \frac{\sum_a SAM(f,a)}{\sum_a QF^0_{fa}}, \qquad WFDIST^0_{fa} = \frac{SAM(f,a)}{W^0_f\, QF^0_{fa}},$$

so that the average price of a factor is its payment per unit, and the wage distortions are the differences between activities. Otherwise $W^0_f = WFDIST^0_{fa} = 1$.

## 9.10 Benchmark replication

The calibrated base must solve every equation of the model. **[CGE]** `cge_calibrate` evaluates every scaled residual at the base, displays the largest and stops with an error if it exceeds $10^{-9}$. A failure points to a problem in the SAM or the mapping, such as a negative cell where values must be non-negative or an elasticity that the model does not allow, not to the solver. With the default closure, the solver started from the base returns it in zero iterations; chapter 11 lists the further checks.

# 10 The solution method

**[CGE]** `cge_solve` solves the system of equations of chapter 7 under the closure of chapter 8. The variables of all blocks are stacked in one vector. Variables that do not exist for an element (for example the imports of a commodity that is not imported) stay in the vector at zero and are left out of the system, and so are the variables that the closure fixes. The free variables $x$ and the equations $r(x) = 0$ form a square system.

## 10.1 Newton's method

The solver works on scaled variables and scaled residuals,

$$z_j = x_j / s_j, \qquad F_i(z) = r_i(x) / e_i,$$

where $s_j$ and $e_i$ are the base magnitudes of the variable and of the equation (one where the base value is zero), so that the tolerance applies alike to every block. Starting from the base, each Newton step solves

$$J(z)\,\Delta z = -F(z)$$

by the LU factorisation of the Jacobian $J$, and the new point is $z + \lambda\,\Delta z$, with $\lambda$ the largest of $1, \frac{1}{2}, \frac{1}{4}, \ldots$ (down to $10^{-8}$) such that

$$\|F(z + \lambda\,\Delta z)\| \le (1 - 10^{-4}\lambda)\,\|F(z)\|,$$

a backtracking line search on the Euclidean norm of the residuals. Points at which a price or quantity is not positive are rejected, and the step is shortened. The solve has converged when $\max_i |F_i| < \tau$, the tolerance (`1e-10` by default), over every equation of the model, including the definitions left out of the reduced system (section 10.2).

From version 0.7.0, after each Newton step, full or shortened by the line search, further steps use Broyden's updates (Broyden, 1965). With $s$ the step and $y = F(z + s) - F(z)$, Broyden's good method replaces the Jacobian $B$ by

$$B_+ = B + \frac{(y - Bs)\,s'}{s's},$$

which agrees with the change in the residuals along the step. By the Sherman–Morrison formula its inverse $H$ becomes

$$H_+ = (I + a\,s')\,H, \qquad a = \frac{s - Hy}{s'Hy}$$

(Kelley, 1995, chapter 7), so $H$ is kept as the LU factors of the last Jacobian times the factors $(I + a_k s_k')$ of the updates since, and the next direction is $d_+ = -H_+ F(z + s)$. As $HF(z) = -d$ for the direction $d$ of the last step, $Hy = HF(z + s) + d$ needs a single solve with the LU factors: a Broyden step costs one evaluation of the equations, two triangular solves and a few inner products. It is taken with a line search of at most two halvings; a new Jacobian is computed when that fails, after a step that reduces $\|F\|$ by less than a tenth, and after 30 updates. Newton's method with a new Jacobian at each step converges quadratically near a solution, and Broyden's method superlinearly, with far fewer Jacobians and factorisations; far from a solution, where the line search shortens the Newton steps, the updates also save Jacobians. Before version 0.7.0 the factors of a Jacobian were reused without updates while each full step at least halved $\|F\|$, a chord method, which converges only linearly.

Once the tolerance is met, up to three more steps with the same factors are taken while each halves $\|F\|$, toward a thousandth of the tolerance. A solution far inside the tolerance varies smoothly with the shocks, which **[CGE]** `cge_optimize` and **[CGE]** `cge_sensitivity` rely on. **[CGE]** `cge_optimize`, whose solves start from the solution before, takes steps with the factors but without Broyden's updates: from a nearby start a Newton step and such steps reach far below the tolerance.

## 10.2 The reduced system

Some equations are explicit definitions: a variable equals an expression in other variables. Such definitions are left out of the Newton system, and their variables are computed from the other variables before each evaluation of the residuals. They are (P1), (P2) and (P6) (import and export prices and the price of intermediate input), (Q5) (intermediate demand), (Q6) (output by commodity), (I5) (household demand), (T8) (demand for margin services), (I13) (the purchases of the health insurance fund), (H2) (factor efficiency) and (H3) (days worked). With VAT, government revenue (I8) depends on intermediate and household demand and on the fund's purchases, and (P6), (Q5), (I5) and (I13) stay in the system. A definition is left out only when its equation elements are exactly the existing elements of its variable. Variables computed from their definitions cannot be fixed by the closure (section 8.2).

The left-out equations still hold exactly at every point. Intermediate and household demand grow with the square of the number of sectors, so most of a large model's equations are left out: for a SAM with 250 activities and 20 households, 4,254 of 40,190 equations remain. `cge_solve, full` keeps every equation; the test suite checks that both systems give the same solution. The pattern of the reduced system is found by substituting each left-out variable by the variables of its definition.

## 10.3 The Jacobian

From version 0.7.0 the Jacobian is exact. The derivatives of every equation block of chapter 7 are written out from its equation, as entries $(i, j, \partial r_i/\partial x_j)$ of a sparse matrix; an entry that arises more than once, such as the derivative of the value of composite supply (P3) with respect to the commodity's own price, which enters the equation in several terms, is summed. The entries are scaled as the system, $J_{ij} = (\partial r_i/\partial x_j)\, s_j/e_i$, and written into a dense matrix for the LU factorisation.

In the reduced system the variables of the left-out definitions are eliminated by the chain rule. Each left-out definition is an equation $D_k(d_k, y) = 0$ in its variable $d_k$ and other variables $y$, such as $QINT_{ca} - ica_{ca}\, QINTA_a = 0$ or, for household demand (I5), $PH_c\, QH_{ch} - PH_c\, \gamma_{ch} - \beta_{ch}\, SN_h = 0$. Its derivatives give

$$\frac{\partial d_k}{\partial y_j} = -\frac{\partial D_k/\partial y_j}{\partial D_k/\partial d_k},$$

and each entry of another row with respect to $d_k$ is replaced by its products with these derivatives:

$$\frac{dr_i}{dy_j} = \frac{\partial r_i}{\partial y_j} + \sum_k \frac{\partial r_i}{\partial d_k}\,\frac{\partial d_k}{\partial y_j}.$$

No left-out definition depends on another, so one substitution suffices.

User equations (chapter 23) are differentiated by forward differentiation of their compiled postfix form: each value on the stack carries its derivatives with respect to the variables of the equation, which the operators combine by the rules of the derivative, for example $(ab)' = a'b + ab'$, $(a/b)' = a'/b - ab'/b^2$, $(a^b)' = a^b(b\,a'/a + \ln a\, b')$ (or $b\,a^{b-1}a'$ for an exponent without variables) and $(\ln a)' = a'/a$. The rows of constraints (B1) have exact derivatives too.

The test suite checks the exact Jacobian against central differences taken column by column, for the reduced and the full system, at the base, at solutions and at points with every free variable perturbed by up to 3 per cent, in every SAM layout and with every feature of the model; the largest difference, about $10^{-7}$, is the error of the differences themselves. Building the Jacobian takes one pass over its entries, where finite differences needed an evaluation of all the equations for each group of columns (67 groups for the 200-activity model of section 5.2).

Finite differences remain the fallback. Before version 0.7.0 the Jacobian was built by them for the rows with few variables, over groups of columns that share no row (Curtis et al., 1974): the columns are coloured greedily, the most connected first, so that one evaluation of the equations gives the derivatives of every column of a group. For a group $G$, with $e_G$ the vector with ones in the columns of $G$,

$$J_{ij} \approx \frac{F_i(z + h \odot e_G) - F_i(z - h \odot e_G)}{2h_j}, \qquad j \in G, \qquad h_j = 10^{-6}\max(1, |z_j|),$$

central differences, whose error is about $10^{-10}$; version 0.6.3 used one-sided differences, with $h_j = 10^{-7}\max(1, |z_j|)$ and one evaluation for each group. When Newton's method fails with the exact Jacobian, it is tried again from the start with central differences and without Broyden's updates.

## 10.4 Fallbacks and continuation

If Newton's method fails with the exact Jacobian and with central differences, Powell's hybrid method is tried (Mata's `solvenl()`, technique `broydenpowell`), followed by Newton steps to polish the result.

If that fails too, the shock is applied by continuation: the model is solved for $k = 1, \dots, n$ with parameters and fixed values

$$\theta_k = \theta^0 + \frac{k}{n}\,(\theta^1 - \theta^0),$$

between their base values $\theta^0$ and their shocked values $\theta^1$, each step starting from the solution of the step before; by default in $n = 4$ steps and then in $n = 16$. A shock that defeats continuation is often one that the economy cannot absorb.

## 10.5 Diagnostics

A solution of the equations need not be an equilibrium that makes economic sense. From version 0.7.0, after a solve that converges, **[CGE]** `cge_solve` notes any quantity, price, income or scaling factor of demand that is negative or below one millionth of its positive base value, and stores their number in `r(N_collapsed)`. With savings-driven investment, for example, a tenfold rise in government purchases is absorbed by negative investment, with the output of two activities near zero (**[CGE]** `cge_solve`).

After a failure, or on request, **[CGE]** `cge_solve` displays:

- the equations with the largest scaled residuals at the last point, by block and element;
- any equation with no free variable, and any free variable in no equation, which make the Jacobian singular;
- the smallest pivot of the LU factorisation of the Jacobian relative to the largest, and the variable it belongs to: partial pivoting permutes rows only, so column $j$ of $U$ belongs to variable $j$.

## 10.6 Constraints

With constraints (chapter 17) the model is solved in two stages: first with smoothed complementarity conditions, which find whether each constraint binds, and then with the exact conditions in the regimes found, changing the regime of any constraint whose other condition fails, for at most ten rounds (section 17.3).

## 10.7 Other implementations

The Python reference implementation uses the same Newton method with a backtracking line search, and its own pure active-set method for constraints. The GAMS program written by **[CGE]** `cge_gams` is solved by the solvers that GAMS provides, PATH by default (chapter 25). Their solutions agree with those of `cge_solve` to the precision of the solutions (chapter 5).

# 11 Validation checks

Every model built with cge-stata must pass the checks below, on the synthetic SAMs and on any SAM it calibrates. **[CGE]** `cge_calibrate` checks benchmark replication; **[CGE]** `cge_check` runs the others on the model actually solved, and reports each with the value tested. In the formulas $m = \max\big(1, \max_{rc} |SAM(r,c)|\big)$ is the largest SAM cell, so that the checks give the same result whatever the units of the SAM.

1. **SAM balance.** Every row total equals its column total: $\max_k \big|\sum_c SAM(k,c) - \sum_r SAM(r,k)\big| \leq 10^{-8} m$.
2. **Benchmark replication.** At the calibrated parameters and base values, every scaled residual is at most $10^{-9}$, and the solver started at the base returns it.
3. **A square closure.** The closure leaves as many free variables as equations, counting one equation for each constraint. (**[CGE]** `cge_closure` also rejects a closure that leaves an equation without a free variable.)
4. **The solution of a simulation.** Its largest scaled residual is at most $10^{-8}$.
5. **Walras's law.** $|WALRAS| \leq 10^{-8} m$ in every solution.
6. **The GDP identity.** GDP measured by expenditure equals GDP measured by income, to $10^{-8} m$:

   $$\begin{aligned} &\sum_c PH_c \Big(\sum_h QH_{ch} + QG_c + QINV_c + QSHI_c\Big) + \sum_c PQ_c\, QER_c + \sum_{a,h} PA_a\, QHA_{ah} \\ &\quad + EXR \sum_c PWE_c\, QE_c - EXR \sum_c pwm_c\, QM_c - R^{abroad} = \sum_a PVA_a\, QVA_a - N + T + R^{home}, \end{aligned}$$

   where $T$ is the revenue from sales, import, export and activity taxes and VAT; $N$ is the cost of the commodities in nests of value added, which are intermediate inputs (section 18.2); $R^{abroad}$ is the import and export premiums paid to the rest of the world; and $R^{home}$ is the premiums received at home and every rent on activity output (section 17.1).
7. **Homogeneity.** Doubling every fixed nominal value, the numeraire among them, doubles every price and nominal value and leaves every quantity unchanged: the largest deviation, relative to each value with a floor of one, is at most $10^{-8}$.
8. **A robust start.** The solver started from base values perturbed uniformly within $\pm p$ times their values (for example $p = 0.1$) returns the base, to a relative deviation of at most $10^{-7}$.

In the last two checks a variable whose base value is zero ($WALRAS$) is measured relative to $m$, as in the check of Walras's law.

**[CGE]** `cge_listing` (from version 0.9.5) shows every element of the equations at a point, with its residual: an equation listing and infeasibility report, as GAMS gives after a solve. It finds the equations that a shock moves from the base before the model is solved, and, after a solve that fails, where the system is furthest from a solution.

## 11.1 Agreement between implementations

The model is implemented three times: in Stata and Mata, in a Python reference implementation, and in GAMS. The Python model writes the expected results of every test case, and the Stata test suite compares every variable of every case with them. The GAMS model is written from the equations of this part, calibrates the model from the SAM and the user's settings alone, and is solved with PATH. Chapter 5 gives the cases and the agreement found.

# 12 Reporting measures and welfare

**[CGE]** `cge_report` computes standard measures at the base point and at each solution, in tables, and **[CGE]** `cge_results` summarises one simulation. The Python reference implementation computes the same measures, and the test suite compares them. Real measures are valued at base prices $PQ^0_c$ (with VAT, $PH^0_c$), $PVA^0_a$ and $EXR^0$, and trade at base world prices $pwm^0_c$ and $pwe^0_c$; nominal measures use current prices and parameters.

## 12.1 Measures

| Measure | Definition |
|---|---|
| Nominal GDP | $\sum_c PH_c \left(\sum_h QH_{ch} + QG_c + QINV_c + QSHI_c\right) + \sum_c PQ_c\, QER_c + \sum_{a,h} PA_a\, QHA_{ah} + EXR \sum_c PWE_c\, QE_c - EXR \sum_c pwm_c\, QM_c$, less the premiums paid abroad |
| Real GDP | the same at base prices and base world prices |
| GDP deflator | nominal GDP / real GDP |
| Real consumption, government consumption, investment, fund purchases | $\sum_c PH^0_c$ times $\sum_h QH_{ch}$, $QG_c$, $QINV_c$ and $QSHI_c$ |
| Real exports, imports | $EXR^0 \sum_c pwe^0_c\, QE_c + \sum_c PQ^0_c\, QER_c$; $EXR^0 \sum_c pwm^0_c\, QM_c$ |
| Real value added | $\sum_a PVA^0_a\, QVA_a$, without the commodities in nests of value added |
| Real exchange rate | $EXR/DPI$ |
| Private savings | $\sum_d MPS_d\,(1 - TINS_d)\, YI_d$ |
| Revenue by tax | direct $\sum_d TINS_d\, YI_d$; sales $\sum_c tq_c\, TQADJ\, PQ_c\, QQ_c$; import $EXR \sum_c tm_c\, TMADJ\, pwm_c\, QM_c$; export $EXR \sum_c te_c\, PWE_c\, QE_c$; activity $\sum_a ta_a\, PA_a\, QA_a$; VAT; taxes on factor use and on factor income |
| Public debt (when the model has debt) | interest paid at home $r^d\, DD$ and abroad $r^f\, DF\, EXR$; primary balance $GSAV + r^d\, DD + r^f\, DF\, EXR$; debt at the end of the period at home, $DD + (1-\phi)(-GSAV)$, and abroad in local currency, $\left[DF + \phi(-GSAV)/EXR\right] EXR$; their total, and the total in per cent of nominal GDP |
| Real wage | $W_f/CPI$ |
| Labour force, unemployment rate ($f \in FL$) | $QLF_f$; $100\,(1 - QFS_f/QLF_f)$ |
| Exports of a commodity | $QE_c + QER_c$ |
| Health | health services per head $R = HS/HS^0$; the share of working time lost, $100\, S_f$ per cent, and days worked $DW_f$ for $f \in FS$; the death rate per 1,000, $1000\, MR$; the population index $POP$ (chapter 16) |

The other measures are model variables: $CPI$, $DPI$, $EXR$, $FSAV$, $YG$, $EG$, $GSAV$, $YI_d$, $EH_h$, $W_f$, $YF_f$, $QFS_f$, $QA_a$, $PA_a$, $QVA_a$, $QQ_c$, $PQ_c$, $QX_c$, $QD_c$, $QM_c$ and, when the model has them, $HS$, $EFF_f$, $YSHI$ and $SHIADJ$.

The measures are grouped in the tables `macro`, `government`, `debt`, `institutions`, `factors`, `activities`, `commodities`, `health` and `welfare` (**[CGE]** `cge_report`).

## 12.2 Welfare

The welfare of household $h$ is measured with its expenditure function,

$$E_h(PQ, u) = \sum_c PQ_c\,\gamma_{ch} + u \prod_c PQ_c^{\beta_{ch}},$$

which is that of Cobb–Douglas demand when $\gamma = 0$ (with VAT, $PQ$ stands for $PH$, and home goods are valued at $PA_a$). With base spending $EH_h^0$ and new spending $EH_h$, the equivalent variation is the change in spending at base prices that gives the new utility, and the compensating variation the change in spending at new prices that would restore base utility:

$$\begin{aligned} EV_h &= \sum_c PQ_c^0\,\gamma_{ch} + \Big(EH_h - \sum_c PQ_c\,\gamma_{ch}\Big) \prod_c \Big(\frac{PQ_c^0}{PQ_c}\Big)^{\beta_{ch}} - EH_h^0, \\ CV_h &= EH_h - \sum_c PQ_c\,\gamma_{ch} - \Big(EH_h^0 - \sum_c PQ_c^0\,\gamma_{ch}\Big) \prod_c \Big(\frac{PQ_c}{PQ_c^0}\Big)^{\beta_{ch}}. \end{aligned}$$

Both are positive for a gain and are reported in per cent of $EH_h^0$. Neither values public goods, the environment or leisure.

## 12.3 Changes

Reports express a measure $x$ as a change from its base value $x^0$: rates in per cent (public debt in per cent of GDP, unemployment rates, the share of time lost to illness, the death rate) change by $x - x^0$ percentage points; other measures by $100\,(x - x^0)/|x^0|$ per cent, so that a fall is negative even when the base is negative, as for a government deficit. A change from a base of zero is missing.

# 13 Fiscal instruments

## 13.1 Revenue-neutral tax replacement

The sales tax, import tariff and VAT rates are scaled by $TQADJ$, $TMADJ$ and $TVADJ$ (in (P1), (P3), (P6), (P8) and (I8)), as direct tax rates are scaled by $TINSADJ$ (I10). All four are fixed at one, except under a closure that fixes government savings and names one of them as the instrument that adjusts (`government(fixgsav direct)`, `fixgsav sales`, `fixgsav import` or `fixgsav vat`; chapter 8).

The instrument scales every base rate of that tax in proportion, so rates that are zero in the base stay zero; the tax must raise revenue in the base. With $GADJ$ fixed, government consumption is unchanged in real terms, so the closure keeps government savings at their base value: in real terms under the CPI numeraire, in terms of domestic producer prices under the DPI numeraire. It replaces lost revenue, for example from a tariff cut, with whichever tax the user chooses, and the scaling factor shows the rise in rates required. Under these closures a shock to the instrument's own rates, such as removing tariffs while tariffs adjust, is not meaningful: the scaling factor offsets it.

## 13.2 Value added tax with input credits

VAT is charged on sales of each commodity at the rate $tv_c$, on the price $PQ_c$ before VAT. Final users (households, the government, investment and the health insurance fund) pay $PH_c = PQ_c(1 + tv_c)$. An activity recovers the share $\kappa_a$ of the VAT on its inputs as a credit, and so pays $PQ_c\,(1 + tv_c\,(1 - \kappa_a))$ (P6). $\kappa_a = 1$, the default, is a registered activity with full credits; $\kappa_a = 0$ an exempt or unregistered one, which bears VAT on its inputs. Exports, whether from domestic output or from composite supply, and purchases of margin services are zero-rated. Net VAT revenue (I8) is therefore the VAT on final demand plus the VAT that activities cannot recover.

### 13.2.1 The SAM

VAT is recorded in a VAT account whose row collects VAT from the commodity columns, $SAM(vat, c)$, net of credits, and whose column pays the government (**[CGE]** `cge_map`, option `vattax()`). When the SAM records VAT within the sales tax account, the option `vatshare()` gives the VAT share of each commodity's sales tax, which is moved to an internal VAT account before the checks (section 9.2). VAT cells may be negative.

### 13.2.2 Calibration

With $PQ^0_c = 1$, each use $u$ of commodity $c$ recorded in the SAM at the value $V_{cu}$, at purchaser prices, is a quantity $V_{cu}/(1 + tv_c\,k_u)$, where the VAT liability $k_u$ is 1 for final users, $1 - \kappa_a$ for activity $a$ and 0 for exports and margin services. The rate $tv_c$ solves

$$SAM(vat, c) = \sum_u V_{cu}\,\frac{tv_c\,k_u}{1 + tv_c\,k_u},$$

which is increasing in $tv_c$ when the uses are positive. It is solved by Newton's method from $tv_c = SAM(vat, c)/\sum_u k_u V_{cu}$, to a relative change of $10^{-15}$; $tv_c = 0$ when $SAM(vat, c) = 0$. A commodity with VAT but no use that bears it is an error. Then

- $QINT^0_{ca} = SAM(c, a)/(1 + tv_c(1 - \kappa_a))$, $QINTA^0_a = \sum_c QINT^0_{ca}$, $ica_{ca} = QINT^0_{ca}/QINTA^0_a$ and $PINTA^0_a = \sum_c SAM(c, a)/QINTA^0_a$;
- $QH^0_{ch}$, $\overline{qg}_c$, $\overline{qinv}_c$ and $\overline{qshi}_c$ are the SAM values divided by $1 + tv_c$, and $PH^0_c = 1 + tv_c$;
- $QQ^0_c$ (section 9.4) equals the sum of these quantities, margin services and exports from composite supply; calibration checks this.

Spending values, budget shares and the accounts of the government and the fund are unchanged, because they are values at purchaser prices. Without VAT every quantity equals its SAM value. The VAT rates and credit shares can be shocked (`tv`, `kappa`).

## 13.3 Public debt and interest

National SAMs record the interest on public debt within government transfers: to domestic institutions, $SAM(d, gov)$, and to the rest of the world, $SAM(row, gov)$. Given the debt stocks at the start of the base year, held at home, $DD^0$ (local currency), and abroad, $DF^0$ (foreign currency, equal to local currency at $EXR^0 = 1$), and their interest rates $r^d$ and $r^f$, calibration separates the interest from the other transfers:

$$trnsfr_{d,gov} = SAM(d, gov) - hold_d\, r^d\, DD^0, \qquad trnsfr_{row,gov} = SAM(row, gov) - r^f\, DF^0,$$

where $hold_d$ is the share of domestic interest received by institution $d$. By default it is proportional to $SAM(d, gov)$, so that interest is the same share of every institution's transfers; the user may give other shares, which must be non-negative and sum to one. A non-interest transfer that would be negative is an error: the SAM's transfers are then too small for the stated debt and rate. The base equilibrium is unchanged, since at base prices the interest terms of (I3), (I9) and (S3) equal the transfers they replace.

Interest is then paid in nominal terms, on domestic debt in local currency and on foreign debt in foreign currency, while the other transfers keep their indexation. A static simulation can change the interest rates or the stocks, for example a rise in $r^f$ or debt relief that lowers $DF$ (shocks `rd`, `rf`, `ddebt`, `fdebt`).

In a dynamic run the deficit is borrowed, the share $\phi$ abroad (chapter 19); by default $\phi$ is the foreign share of the base debt, $DF^0/(DD^0 + DF^0)$. Net borrowing is modelled, not gross borrowing and amortisation separately: maturing debt is assumed to be refinanced. The change in the stock of foreign debt measured in local currency is net foreign borrowing plus a valuation change, $DF_{t-1}\,(EXR_t - EXR_{t-1})$. Neither default risk nor a link from debt to interest rates is modelled; the rates are exogenous and can be changed by shocks.

# 14 Trade and household extensions

## 14.1 Home consumption

Households in many African economies consume part of what they produce. A SAM records this as a payment from the household's column to the activity's row, $SAM(a, h)$, at producer prices. The model treats it as a separate good:

- the activity's output $QA_a$ is marketed output, which yields commodities through $\theta_{ac}$ (Q6), plus home consumption $\sum_h QHA_{ah}$;
- home consumption is valued at the activity price $PA_a$, so it bears no sales tax, VAT, import competition or margins;
- it enters household demand (I5H) with its own marginal budget share $\beta^h_{ah}$ and, with LES demand, subsistence quantity $\gamma^h_{ah}$; consumption spending $EH_h$ (I4) includes it.

**Calibration.** $QHA^0_{ah} = SAM(a, h)$, with $PA^0_a = 1$; marketed output is $QA^0_a - \sum_h SAM(a, h)$, which must be positive, and $\theta_{ac}$ is $SAM(a, c)$ divided by it. $EH^0_h$ includes home consumption, and $\beta^h_{ah} = SAM(a, h)/EH^0_h$. With LES demand, home consumption of the output of activity $a$ takes the income elasticity of $a$'s paired commodity, and $\gamma^h_{ah} = QHA^0_{ah} + \beta^h_{ah} EH^0_h/(PA^0_a \phi_h)$. The CPI keeps its weights on marketed consumption; welfare measures use the full expenditure function, with $PA_a$ for home goods. GDP includes home consumption at $PA_a$ (real GDP at $PA^0_a$). Population growth in dynamic runs scales $\gamma^h$ as it scales $\gamma$.

## 14.2 Export demand

By default the country is small in world markets: it sells any quantity at the world price $pwe_c$. For commodities given an elasticity of export demand $\eta^e_c > 0$ (**[CGE]** `cge_calibrate`, option `exportdemand()`), the world price is a variable $PWE_c$, and exports fall as it rises (T10). The CET ((T1), (T2)) still determines how much producers offer at the export price $PE_c$ (P2). A larger $\eta^e_c$ approaches the small-country case, and a change in $pwe_c$ shifts world demand. Export revenue (S3), export taxes (I8) and nominal GDP use $PWE_c$; real exports are valued at base world prices.

## 14.3 Export-only commodities

When a commodity's domestic output is all exported, the SAM shows exports equal to domestic output (net of export taxes and export margins), and no domestic sales from domestic output. By default such a commodity falls into $CR$, since its exports are at least its domestic output (section 9.4): its output goes to the domestic market and its exports are drawn from composite supply at $PQ_c$. With `cge_calibrate, exportonly()` its output is exported directly instead: $QD_c$ and $PD_c$ do not exist, the CET is replaced by (T3E), and the export price $PE_c$ is the producer price. If the commodity is also imported, its composite supply is imports alone (T7); otherwise it has no composite supply, and $PQ_c$ and $QQ_c$ do not exist. Calibration lists the commodities whose exports equal domestic output, so that the choice is explicit. An export-only commodity may pay export taxes.

## 14.4 Margins by market

Margins may differ between domestic sales, imports and exports, as in the IFPRI standard model (Lofgren et al., 2002). A margin account can be declared as paid per unit of domestic sales (**[CGE]** `cge_map`, option `domesticmargins()`), of imports (`importmargins()`) or of exports (`exportmargins()`); accounts in `marginaccounts()` keep the treatment per unit of composite supply. With $MG_{tc} = SAM(t,c)$ and the shares in which account $t$ buys margin services, $s_{c't} = SAM(c',t)/\sum_{c''} SAM(c'',t)$:

- $icd_{c'c} = \sum_{t \in T^D} s_{c't}\, MG_{tc}/QD^0_c$: domestic buyers pay $PDD_c$ ((P3), (T5)), and producers receive $PD_c$;
- $ici_{c'c} = \sum_{t \in T^M} s_{c't}\, MG_{tc}/QM^0_c$: the margins are part of the import price $PM_c$ (P1);
- $ice_{c'c} = \sum_{t \in T^E} s_{c't}\, MG_{tc}/QE^0_c$: exporters receive the world price, less export taxes, less margins (P2).

Export margins are part of the export value in the SAM, so

$$QE^0_c = SAM(c,row) - SAM(tax_{exp},c) - \sum_{t \in T^E} MG_{tc}, \qquad pwe_c = SAM(c,row)/QE^0_c,$$

which gives $PE^0_c = 1$. The base prices $PDD^0_c = 1 + \sum_{c'} icd_{c'c}$ and $PM^0_c = 1 + tm_c + \sum_{c'} ici_{c'c}$ enter the Armington shares (section 9.6) in place of $PD^0_c$ and $PM^0_c$. Composite supply $QQ^0_c$ excludes export margins. A commodity that pays domestic, import or export margins must have domestic sales, imports or exports respectively.

# 15 Labour-market regimes

## 15.1 Labour force and unemployment

For a factor $f \in FL$ the user gives the base unemployment rate $u_f^0$ (**[CGE]** `cge_calibrate`, option `unemployment()`), which SAMs do not record, for example from a labour force survey. $QFS_f$ is then employment, and the labour force is

$$QLF_f = QFS_f^0/(1 - u_f^0),$$

exogenous (it grows in dynamic runs at the factor's growth rate), so that the unemployment rate is

$$U_f = 1 - QFS_f/QLF_f.$$

Two regimes link the real wage $W_f/CPI$ to unemployment. Each has an equation for every $f \in FL$, whatever the closure, with a slack variable that the regime's closure fixes at zero; under other closures the slack is free and the equation is inert. The number of equations therefore does not depend on the closure.

## 15.2 Wage curve

Real wages fall as unemployment rises (Blanchflower and Oswald, 1994):

$$\frac{W_f}{CPI} = \overline{w}_f \left(\frac{U_f}{u_f^0}\right)^{-\epsilon_f^w} + LSW_f \tag{L1}$$

with $\overline{w}_f = W_f^0/CPI^0$ and the elasticity $\epsilon_f^w > 0$ given by the user (`wagecurve()`); it needs $u_f^0 > 0$. Under the closure `factors(`$f$`:wagecurve)` both $W_f$ and $QFS_f$ are free and $LSW_f = 0$: a fall in labour demand lowers both employment and the real wage. A small elasticity approaches unemployment at a fixed real wage; a large one, full employment.

## 15.3 Minimum wage

The real wage may not fall below a floor $\underline{w}_f$, and unemployment may not be negative; one of them binds:

$$U_f \geq 0, \qquad \frac{W_f}{CPI\,\underline{w}_f} - 1 \geq 0, \qquad U_f \left(\frac{W_f}{CPI\,\underline{w}_f} - 1\right) = 0.$$

This complementarity condition is solved, with the rest of the model, in the Fischer–Burmeister form (Fischer, 1992), smoothed as in Kanzow (1996):

$$a + b - \sqrt{a^2 + b^2 + 2\mu} = LSM_f, \qquad a = U_f, \quad b = \frac{W_f}{CPI\,\underline{w}_f} - 1 \tag{L2}$$

which with $LSM_f = 0$ holds exactly when $a > 0$, $b > 0$ and $ab = \mu$. With the default $\mu = 10^{-10}$ (`smoothing()`), the wage is at the floor to within $\mu/U_f$ when unemployment is positive, and unemployment is at most $\mu/b$ when the floor does not bind: for example $7 \times 10^{-10}$ with 15 per cent unemployment. The regime is found by the solver, not chosen in advance.

**Calibration.** With unemployment in the base, $u_f^0 > 0$, the floor binds and is the base real wage, $\underline{w}_f = \overline{w}_f/(1 + \mu/u_f^0)$ (`minwage(`$f$`:1)`, the default). With full employment, $u_f^0 = 0$, the floor is given as a share $r < 1$ of the base real wage (`minwage(`$f$`:`$r$`)`), so that $\underline{w}_f = r\,\overline{w}_f$; the base then has the unemployment rate $\mu/(1/r - 1)$, which defines the labour force. Under the closure `factors(`$f$`:minwage)`, $W_f$ and $QFS_f$ are free and $LSM_f = 0$.

The test suite checks the smoothed solutions against the exact regimes: with the floor binding, against unemployment at the fixed wage $\underline{w}_f$; without, against full employment of the labour force.

## 15.4 Results and dynamics

Reports show the labour force and the unemployment rate of each $f \in FL$. In dynamic runs the labour force grows at the factor's growth rate, so that employment is endogenous under both regimes and under unemployment at a fixed wage. The reference wage $\overline{w}_f$ and the floor $\underline{w}_f$ stay constant in real terms; shocks may change them (`wref`, `wmin`), and the labour force (`qlf`).

# 16 Health

This chapter collects the parts of the model that concern health: a social health insurance fund, the efficiency of factors that rises with health services, the days of work lost to illness, and the death rate, which changes population and labour in dynamic runs.

## 16.1 A social health insurance fund

A fund mapped with **[CGE]** `cge_map`, option `insurance()`, is financed by contributions at the rate $tshi_f$ on the income of each factor, taken before the income is distributed (I2), and by transfers from the government (in real terms) and the rest of the world (in foreign currency) (I12). It buys a fixed basket of health services, $QSHI_c = \overline{qshi}_c\, SHIADJ$ (I13), scaled so that it spends its revenue less a fixed share saved (I14). A higher contribution rate or larger transfers therefore raise its purchases. Its savings enter the savings–investment balance (S5). The calibration of the fund is given in section 9.7; its parameters can be shocked (`tshi`, `qshi`, `trshi`, `trshir`). A donor-funded non-profit institution is mapped as a household that receives transfers from the rest of the world.

## 16.2 Health services and factor efficiency

Health outcomes respond to health services per head relative to the base,

$$R = HS/HS^0,$$

where $HS$ is the volume of the health services in $HC$ bought by households, the government and the health insurance fund (H1), and $HS^0$ its base value, which grows with the population in dynamic runs (chapter 19). Each outcome has a constant elasticity with respect to $R$. These elasticities are hypotheses about how services become health, to be taken from studies of the country or region (for example Bokhari et al. 2007); the package supplies none. An outcome whose elasticity is zero is constant unless shocked, and needs no health commodities.

The efficiency of factor $f$ is $EFF_f = R^{\eta_f}$ (H2), with the elasticity $\eta_f$ given by **[CGE]** `cge_calibrate`, options `healthcom()`, `healthfac()` and `healthelas()`. It multiplies the factor's input in value added (section 7.2). The exogenous efficiency $af_f$ can be shocked directly.

## 16.3 Days lost to illness

For a factor $f \in FS$ the user gives the base share of working time lost to illness, $0 \leq s_f^0 < 1$ (`sickdays()`), for example from a labour force or household survey, and its elasticity $\epsilon_f^s$ with respect to $R$ (`sickelas()`; negative when more services mean fewer days lost). The share lost is $S_f = \sigma_f R^{\epsilon_f^s}$, with $\sigma_f = s_f^0$ unless shocked (`sick`, for example an epidemic), and the days worked per worker, relative to the base, are

$$DW_f = \frac{1 - \sigma_f R^{\epsilon_f^s}}{1 - s_f^0}, \qquad f \in FS \tag{H3}$$

and $DW_f = 1$ otherwise. Days worked scale the factor's effective input in production, $\widetilde{QF}_{fa} = af_f\, EFF_f\, DW_f\, QF_{fa}$ ((Q3), (Q4)). Factor quantities $QF_{fa}$ and $QFS_f$, the price $W_f$ and unemployment (chapter 15) stay per worker. In real terms fewer days per worker are the same as less labour measured in days: with $L$ workers and $D$ days each, a wage $w$ per effective unit pays $w\,D\,L$ whether the factor is counted in workers or in days. $EFF_f$ measures output per day worked, and $DW_f$ the days worked. $S_f$ must stay below one.

## 16.4 Mortality

The user may give the base crude death rate $m^0$, deaths per head per period (`mortality()`), and its elasticity $\epsilon^m$ with respect to $R$ (`mortelas()`). The death rate is

$$MR = \mu^m R^{\epsilon^m},$$

with $\mu^m = m^0$ unless shocked (`mort`). It is reported, and does not enter the equations of a period: deaths in period $t$ change the population and labour from period $t+1$ (section 16.5), so each period is solved with its population given.

## 16.5 Population and labour in dynamic runs

The population growth rate $g_P$ given for a run (**[CGE]** `cge_dynamic`, option `population()`) is taken to hold at the base death rate. With a death rate, the population grows from $t-1$ to $t$ at

$$g_{P,t} = g_P + \left(m^0 - MR_{t-1}\right),$$

which scales the subsistence quantities, $HS^0$ and the population index $POP$ (chapter 19). For the factors named in `deathfactors()` (labour, not capital), deaths at working ages change the supply:

$$g_{f,t} = g_f + \frac{\theta^D}{\alpha^W}\left(m^0 - MR_{t-1}\right),$$

where $\theta^D$ is the share of deaths at working ages (`deathshare()`) and $\alpha^W$ the working-age share of the population (`workingage()`): excess deaths per head of the population, $MR - m^0$, remove a share $\theta^D(MR - m^0)/\alpha^W$ of the working-age population, and the factor's workers in the same proportion. The labour force of such a factor (chapter 15) grows at the same rate. $MR_{t-1}$ uses the parameters and health services of period $t-1$, including its shocks.

In a baseline, rising health services per head lower the death rate and raise population growth above $g_P$; scenarios are therefore compared with a baseline run with the same settings.

## 16.6 Reporting

The health table of **[CGE]** `cge_report` shows, when the model has them: health services per head $R$; factor efficiency; the share of working time lost, $100\,S_f$ per cent, and the days worked $DW_f$ for $f \in FS$; the death rate per 1,000, $1000\,MR$; the population index $POP$; and the revenue and purchases of the insurance fund. Shares and rates change in points.

# 17 Bounds and complementarity

A model with only equalities cannot represent a limit that binds in some scenarios and not in others: an import quota, a capacity limit, a ceiling on borrowing. Such limits are complementarity conditions, set with **[CGE]** `cge_constraint` and solved with the rest of the model, so that the solver finds which limits bind.

## 17.1 Rents

Three rent variables give quotas and capacity limits their price effect. Each is zero in the base and fixed by every closure, unless a constraint frees it (section 17.2):

| Variable | Description |
|---|---|
| $RM_c$ | premium on imports of $c \in CM$, per unit of the world price in local currency |
| $RE_c$ | premium on exports of $c \in CE$, per unit of the world price |
| $RA_a$ | rent on the output of activity $a$, per unit of the activity price |

They enter the price equations (P1), (P2) and (P7). A premium is a wedge between the world price and the domestic price, like a tariff, but its revenue is a rent. The rent on each element,

$$R^M_c = RM_c \, pwm_c \, EXR \, QM_c, \qquad R^E_c = RE_c \, pwe_c \, EXR \, QE_c, \qquad R^A_a = RA_a \, PA_a \, QA_a,$$

is paid to one recipient $\varrho$: the government (the default, as when quota licences are auctioned), a household or enterprise, a factor, or the rest of the world (as with a voluntary export restraint, whose rents foreign exporters keep). Rents are added to the incomes in (I1) (factors), (I3) (households and enterprises) and (I8) (government), and rents paid abroad to the payments abroad in (S3), at $R/EXR$. A premium fixed at a positive value (`cge_shock RM`) acts as a tariff whose revenue goes to the recipient.

**GDP.** The income side adds the rents received at home and every capacity rent, which is value added produced at home wherever it is paid. The expenditure side values imports and exports net of the rents paid abroad.

## 17.2 Constraints

A constraint $k$ pairs a limited variable $Y_k$ (one element of a variable, for example $QM_{com\_man}$) and its bound $b_k$ with an adjusting variable $X_k$, which the closure fixes at $\bar{x}_k$ and which may move in one direction only:

$$g_k = \sigma_k \frac{b_k - Y_k}{s^Y_k} \geq 0, \qquad h_k = \delta_k \frac{X_k - \bar{x}_k}{s^X_k} \geq 0, \qquad g_k \, h_k = 0 \tag{B1}$$

with $\sigma_k = 1$ for an upper bound and $-1$ for a lower bound, and $\delta_k = 1$ if $X_k$ may rise and $-1$ if it may fall. The scales $s$ are the base magnitudes of the variables (one where the base value is zero). Either $Y_k$ is at its bound and $X_k$ has moved from $\bar{x}_k$ to keep it there, or $X_k$ is at $\bar{x}_k$ and $Y_k$ is within its bound. Each constraint adds one equation and frees one variable, so the system stays square.

| Constraint | Limited variable | Adjusting variable | Effect when it binds |
|---|---|---|---|
| import quota | $QM_c \leq b$ | $RM_c$, up | the premium rises until imports equal the quota |
| export quota | $QE_c \leq b$ | $RE_c$, up | the premium rises until exports equal the quota |
| capacity | $QA_a \leq b$ | $RA_a$, up | the activity price rises above cost; the rent goes to the owners of the capacity |

| Constraint | Limited variable | Adjusting variable | Effect when it binds |
|---|---|---|---|
| borrowing limit under a peg | $FSAV \le b$ | $EXR$, up | the peg holds until foreign savings reach the limit; the currency then depreciates |
| deficit ceiling | $GSAV \ge b$ | $GADJ$, down | government consumption is cut once the deficit reaches the ceiling |
| deficit ceiling, taxes | $GSAV \ge b$ | $TINSADJ$, up | direct tax rates rise instead |
| nominal wage floor | $W_f \ge b$ | $QFS_f$, down | employment falls once the wage reaches the floor |

$Y_k$ must be free and $X_k$ fixed under the closure. With $X_k$ a rent variable, the constraint and the rent's recipient are given together.

## 17.3 Solution

The conditions are solved in two stages:

1. **Smoothed.** Each condition is written in the smoothed Fischer–Burmeister form of section 15.3,

$$g_k + h_k - \sqrt{g_k^2 + h_k^2 + 2\mu} = 0, \qquad \mu = 10^{-8},$$

   and solved with the rest of the model. The solver finds the regime; none is chosen in advance.

2. **Exact.** Each constraint takes the regime of the smoothed solution: binding, $g_k = 0$, if $h_k > g_k$, and otherwise not binding, $h_k = 0$. The model is solved again with these equations. A constraint whose other condition then fails ($h_k < 0$ when binding, $g_k < 0$ when not) changes regime, and the model is solved again, for at most ten rounds.

The solution therefore satisfies the conditions exactly, to the solver's tolerance. Three implementations check one another, each with its own method: Stata, with the two stages above; the Python reference, with an active-set method that starts from the solution without constraints; and GAMS, as a mixed complementarity problem solved with PATH (section 25.2). The minimum wage of chapter 15 keeps its smoothed condition and its calibration.

## 17.4 Reporting

**[CGE]** `cge_solve` reports each constraint's regime, and the regimes are stored with the simulation. The rent variables appear in the results like other variables. Constraints are kept when the shocks are reset, in dynamic runs and sensitivity analysis, and when the model is saved.

## 17.5 User inequalities

From version 0.9.0, **[CGE]** `cge_equation` writes inequalities among any expressions of the user language, each paired with a variable fixed under the closure (section 23.8). Each element is a constraint of this chapter on a slack variable, the difference between the two sides, bounded at zero and adjusted by the paired variable, and the bounds of user variables are constraints on the variables themselves. They are listed and solved with the constraints of **[CGE]** `cge_constraint`.

# 18 Factor taxes, nests and output aggregation

## 18.1 Taxes on factor use and on factor income

Two taxes of the IFPRI standard model (Lofgren et al., 2002) and of the PEP models (Decaluwé et al., 2013) bear on factors.

**Taxes on factor use**, such as payroll taxes on labour or taxes on capital, are paid by activities to accounts mapped with **[CGE]** `cge_map`, option `factortax(`*account*`:`*factor*`)`. The rate on factor $f$ in activity $a$ is the tax over the factor payment,

$$tf_{fa} = SAM(acc, a)/SAM(f, a).$$

It raises the cost of the factor to the activity: the first-order conditions (Q4) and (Q4′) hold with $W_f \, WFDIST_{fa} \, (1 + tf_{fa})$, and value added at factor cost, $PVA_a \, QVA_a$, includes the tax. The shares of the value-added function are calibrated on the tax-inclusive factor costs.

**Taxes on factor income** are paid by factors to the account mapped with `factorincometax(`*account*`)`. The rate $tfy_f$ is the tax over factor income $YF_f$, taken before the income is distributed, like the contributions to a health insurance fund (I2).

Both taxes are government revenue (I8). Factor income $YF_f$ is net of taxes on use (I1). The shocks `tf` (by factor, in every activity that uses it) and `tfy` change the rates; a value introduces a tax where the SAM has none. Reports add the rows `tax_factor_use` and `tax_factor_income` to the government table.

## 18.2 Nests of value added

Value added may combine nests of inputs, each a CES or Cobb–Douglas aggregate with its own elasticity: labour as a combination of skill types, such as doctors, nurses and other staff, as in the PEP models (Decaluwé et al., 2013), or capital with energy, as in GTAP-E (Burniaux and Truong, 2002). The nests and their members, factors or commodities, are given at calibration:

```
cge_calibrate, ... nests(labour: lab_hi lab_lo, ke: cap com_ene) sigmanest(labour:1.5 ke:0.4)
```

A factor or commodity belongs to at most one nest, and nests do not contain other nests. A nest exists in an activity that uses at least one of its members ($n \in NA_a$). The top of value added in an activity combines its factors in no nest and its nests, with the elasticity $\sigma^{va}_a$.

**Commodities in nests.** A commodity $c$ in nest $n$ bought by activity $a$ leaves intermediate input: $QINT_{ca}$ is set by the nest (N3) instead of (Q5), and is not part of $QINTA_a$ or of the shares $ica$. $PVA_a \, QVA_a$ then includes its cost at the price the activity pays, $p_{ca} = PQ_c \, (1 + tv_c \, TVADJ \, (1 - \kappa_a))$. Value added in reports and the income side of GDP leave it out.

**Equations.** Each member $m$ of a nest has a quantity $x_{ma}$, a factor in efficiency units $z_f \, QF_{fa}$ with $z_f = af_f \, EFF_f \, DW_f$, or a commodity $QINT_{ca}$, and a cost $p_{ma}$: $W_f \, WFDIST_{fa} \, (1 + tf_{fa})$ for a factor, $p_{ca}$ for a commodity. With $\rho_n = 1/\sigma_n - 1$ and the CES sums

$$S_a = \sum_{f \notin N} \delta^{va}_{fa} \, (z_f \, QF_{fa})^{-\rho^{va}_a} + \sum_{n \in NA_a} \delta^{n}_{na} \, QNA_{na}^{-\rho^{va}_a}, \qquad S_{na} = \sum_{m \in n} \delta_{ma} \, x_{ma}^{-\rho_n},$$

where $N$ is the set of the members of all nests, value added and its nests are

$$QVA_a = ad_a \, S_a^{-1/\rho^{va}_a} \tag{Q3}$$

$$QNA_{na} = an_{na} \, S_{na}^{-1/\rho_n} \tag{N1}$$

$$PNA_{na} = \frac{PVA_a \, QVA_a}{S_a} \, \delta^{n}_{na} \, QNA_{na}^{-\rho^{va}_a - 1} \tag{N2}$$

and the demands for the members of a nest are

$$p_{fa} = \frac{PNA_{na}\, QNA_{na}}{S_{na}}\, \delta_{fa}\, (z_f\, QF_{fa})^{-\rho_n - 1}\, z_f \tag{Q4}$$

$$p_{ca} = \frac{PNA_{na}\, QNA_{na}}{S_{na}}\, \delta_{ca}\, QINT_{ca}^{-\rho_n - 1} \tag{N3}$$

Factors in no nest keep (Q4), with $S_a$ in its denominator. With $\sigma = 1$, a nest or the top of value added is Cobb–Douglas: $QNA_{na} = an_{na} \prod_m x_{ma}^{\delta_{ma}}$, and each member's cost is its share of the nest's value, $p_{ma}\, x_{ma} = \delta_{ma}\, PNA_{na}\, QNA_{na}$.

**Calibration.** In the base $PNA^0_{na} = 1$ and $QNA^0_{na}$ is the cost of the nest's members: factor payments with taxes on use, and the SAM's purchases of its commodities. The shares within a nest follow from the members' costs and quantities as in section 9.5, $\delta_{ma} \propto p^0_{ma}\, (x^0_{ma})^{1+\rho_n}$ (or the value shares for Cobb–Douglas), and $an_{na}$ makes (N1) hold. At the top of value added the nests enter at a price of one with quantities $QNA^0_{na}$, and $QVA^0_a$ adds the cost of the commodities in nests.

A nest of factors whose elasticity equals that of value added gives the same solution as no nest: a CES function of CES functions with one elasticity is a CES function. The test suites check this.

**Parameters and results.** The parameters are $\delta^n_{na}$ (`dvn`), $an_{na}$ (`an`), $\delta_{ca}$ (`dce`), $\sigma_n$ (`sgn`) and $\rho_n$ (`rhn`); within a nest, the shares of factors are kept in `dva`. Results list $QNA$ and $PNA$. Sensitivity analysis keeps the nests' elasticities at the values given.

## 18.3 Output of a commodity from several activities

When several activities produce a commodity, (Q6) adds their outputs: the outputs are perfect substitutes, sold at one price $PX_c$. The IFPRI standard model (Lofgren et al., 2002) instead combines them by a CES function, so that the output of different producers, such as public and private health services, can be priced differently. **[CGE]** `cge_calibrate`, option `sigmaac(com_man:2)`, gives the elasticity of substitution for a commodity, or one value for every commodity with more than one producing activity ($c \in CAC$); the others keep (Q6). With $XAC_c$ the activities producing $c$ and $\rho^{ac}_c = 1/\sigma^{ac}_c - 1$:

$$QXAC_{ac} = \theta_{ac} \Big( QA_a - \sum_h QHA_{ah} \Big), \qquad a \in XAC_c \tag{Q6B}$$

$$QX_c = aac_c \Big( \sum_{a \in XAC_c} \delta^{ac}_{ac}\, QXAC_{ac}^{-\rho^{ac}_c} \Big)^{-1/\rho^{ac}_c} \tag{Q6A}$$

$$PXAC_{ac} = \frac{PX_c\, QX_c}{\sum_{a'} \delta^{ac}_{a'c}\, QXAC_{a'c}^{-\rho^{ac}_c}}\, \delta^{ac}_{ac}\, QXAC_{ac}^{-\rho^{ac}_c - 1} \tag{P5A}$$

and each activity is paid its own price in (P5), $PA_a = \sum_c \theta_{ac}\, PXAC_{ac}$ for $c \in CAC$ and $\theta_{ac}\, PX_c$ otherwise. With $\sigma^{ac}_c = 1$ the aggregate is Cobb–Douglas, $QX_c = aac_c \prod_a QXAC_{ac}^{\delta^{ac}_{ac}}$ and $PXAC_{ac}\, QXAC_{ac} = \delta^{ac}_{ac}\, PX_c\, QX_c$. The value of commodity output is the value of the activities' outputs, $PX_c\, QX_c = \sum_a PXAC_{ac}\, QXAC_{ac}$, so GDP and the reports are unchanged.

**Calibration.** $PXAC^0_{ac} = 1$ and $QXAC^0_{ac} = SAM(a, c)$; the shares are $\delta^{ac}_{ac} \propto (QXAC^0_{ac})^{1+\rho^{ac}_c}$ (value shares for Cobb–Douglas), and $aac_c$ makes (Q6A) hold. As $\sigma^{ac}_c$ grows the aggregate tends to the sum of (Q6): with $\sigma^{ac} = 10^8$ the solution agrees with the model without aggregation to $10^{-6}$ in the test suites. The parameters are $\sigma^{ac}_c$ (`sac`), $\rho^{ac}_c$ (`rac`), $aac_c$ (`aac`) and $\delta^{ac}_{ac}$ (`dac`).

# 19 Recursive dynamics

A dynamic run of **[CGE]** `cge_dynamic` solves the static model of chapters 7 to 9 for periods $t = 0, 1, \dots, T$. Period 0 is the calibrated base. Agents do not look ahead: each period is an equilibrium given the stocks and exogenous values of that period, which are updated from the previous period's solution. The updates follow the recursive dynamic extension of the standard model (Thurlow, 2004); the rates are chosen by the user, and none has a default other than zero.

## 19.1 Updates between periods

Before period $t \geq 1$, in this order:

1. **Capital.** For each capital factor $f \in K$, with depreciation rate $\delta_f$ and base gross rental rate $\rho_f$ (rental per unit of capital at base prices, so that the base capital stock is $YF^0_f/\rho_f$), new capital in the factor's units is

   $$N_{f,t} = \rho_f\, \omega_f\, INV_{t-1}, \qquad INV_{t-1} = \sum_c PQ^0_c\, QINV_{c,t-1}, \qquad \omega_f = YF^0_f \Big/ \sum_{f' \in K} YF^0_{f'} :$$

   real investment at base prices, shared among capital factors by their base income. If $f$ is specific to each activity,

   $$QF_{fa,t} = (1 - \delta_f)\, QF_{fa,t-1} + N_{f,t}\, \eta_{fa,t},$$

   with allocation shares that favour activities whose rental rate is above the average,

   $$\eta_{fa,t} \propto \max\Big[0,\ s_{fa}\Big(1 + \beta\Big(\frac{R_{fa}}{\bar{R}_f} - 1\Big)\Big)\Big],$$

   $$s_{fa} = \frac{QF_{fa,t-1}}{\sum_{a'} QF_{fa',t-1}}, \qquad R_{fa} = W_{f,t-1}\, WFDIST_{fa,t-1}, \qquad \bar{R}_f = \sum_a s_{fa}\, R_{fa},$$

   normalised to sum to one, where $\beta \geq 0$ is the sensitivity of investment to rental differences (default 1). If $f$ is mobile, $QFS_{f,t} = (1 - \delta_f)\, QFS_{f,t-1} + N_{f,t}$. A capital factor may not be unemployed.

2. **Other factors.** $QFS_{f,t} = (1 + g_f)\, QFS_{f,t-1}$ for a mobile factor, and $QF_{fa,t} = (1 + g_f)\, QF_{fa,t-1}$ for an activity-specific one. The supply of an unemployed factor is endogenous, so $g_f$ must be zero. The labour force of a factor in $FL$ grows at $g_f$ (chapter 15).

3. **Population.** $\gamma_{ch,t} = (1 + g_P)\, \gamma_{ch,t-1}$ (and $\gamma^h$ alike), $HS^0_t = (1 + g_P)\, HS^0_{t-1}$ and $POP_t = (1 + g_P)\, POP_{t-1}$: subsistence consumption grows with the population, and the health link compares health services per head. With a death rate, $g_P$ becomes $g_{P,t}$, and factors that lose workers to deaths grow at $g_{f,t}$ (section 16.5).

4. **Government and foreign flows.** $\overline{qg}_c$, $trnsfr_{d,gov}$, $trnsfr_{gov,gov}$ and $trnsfr_{shi,gov}$ grow at $g_G$; flows in foreign currency ($trnsfr_{d,row}$, $trnsfr_{f,row}$, $trnsfr_{gov,row}$, $trnsfr_{row,gov}$, $trnsfr_{shi,row}$, and $FSAV$ when it is fixed) grow at $g_W$. Other fixed values, the numeraire among them, are unchanged.

5. **Public debt**, when the model has debt (section 13.3). The deficit of period $t - 1$, $-GSAV_{t-1}$, is borrowed, a share $\phi$ abroad:

   $$DD_t = DD_{t-1} + (1 - \phi)\,(-GSAV_{t-1}), \qquad DF_t = DF_{t-1} + \phi\,(-GSAV_{t-1})/EXR_{t-1}.$$

   A surplus repays debt in the same shares. Interest in period $t$ is paid on these stocks ((I3), (I9), (S3)).

6. **Shocks** for period $t$, if any, are applied by the user's program; they may change the debt stocks and interest rates. A shock stays in force in later periods unless the program changes it.

7. **Efficiency.** $ad_{a,t} = \lambda_{a,t}\, ad_{a,t-1}$, with $\lambda_{a,t} = 1 + g_a^{ad}$; or a uniform $\lambda_t$ taken from a baseline run; or, to calibrate a baseline to growth targets, the uniform $\lambda_t$ for which real value added at base prices, $\sum_a PVA_a^0\, QVA_{a,t}$, equals $(1 + g_t^Y)$ times its value in $t-1$. That $\lambda_t$ is found by secant iterations starting from $\lambda = 1$ and $\lambda = 1 + g_t^Y$, to a relative gap of $10^{-13}$.

Period $t$ is then solved from the solution of $t-1$. Each period is stored as an ordinary simulation, named with the run and the period, and tagged with its run, period and efficiency factor.

## 19.2 Baselines and scenarios

A scenario that changes policy should reuse the baseline's efficiency factors $\lambda_t$, so that efficiency does not respond to the scenario; its results are then compared with the baseline period by period (**[CGE]** `cge_dynresults`). Because the run is recursive, a static effect on investment changes the capital stock of later periods: a policy that raises consumption at the expense of investment can lower output in later periods, which a static run does not show.

## 19.3 Lags in user equations

From version 0.9.2, user equations may refer to the values of variables in earlier periods, `lag(VAR)` for $VAR_{t-1}$ and `lag(VAR, k)` for $VAR_{t-k}$, $k \leq 4$ (section 23.5). Before the first period, the lagged values are those of the base. They extend the updates between periods above with stocks and adjustment rules of the user's own.

# 20 The link to household surveys

The model has a few household groups; poverty and inequality are measured on household surveys. **[CGE]** `cge_microsim` passes the results of a simulation to the survey data in memory, top-down and without feedback (Bourguignon et al., 2008). Each survey household $i$ belongs to a model household group $h(i)$ and keeps its own income composition and, if given, its own budget shares.

## 20.1 Income

For each group $h$ the model gives the ratio of new to base values of each income source $k$; a zero base value is replaced by the ratio of the aggregate shown:

| Source | Ratio $\rho_{hk}$ | If the group's base value is zero |
|---|---|---|
| factor $f$ | $YIF_{hf}/YIF^0_{hf}$ | $YF_f/YF^0_f$ |
| government transfers (`gov`) | $trnsfr_{h,gov}\,CPI/(trnsfr^0_{h,gov}\,CPI^0)$ | $CPI/CPI^0$ |
| transfers from abroad (`row`) | $trnsfr_{h,row}\,EXR/(trnsfr^0_{h,row}\,EXR^0)$ | $EXR/EXR^0$ |
| transfers from other domestic institutions (`dom`) | $\sum_d shii_{hd}\,DY_d / \sum_d shii_{hd}\,DY^0_d$ | $CPI/CPI^0$ |
| indexed to prices (`cpi`) | $CPI/CPI^0$ | |
| unchanged (`fixed`) | 1 | |

Here $DY_d = (1-MPS_d)(1-TINS_d)\,YI_d$ is the income of institution $d$ after direct taxes and savings. With income components $y_{ik}$, the household's nominal consumption becomes

$$C_i^1 = C_i^0 \frac{\sum_k \rho_{h(i)k}\, y_{ik}}{\sum_k y_{ik}}\, \kappa_{h(i)}, \qquad \kappa_h = \frac{(1-MPS_h)(1-TINS_h)}{(1-MPS_h^0)(1-TINS_h^0)},$$

so that it follows its own income and its group's direct tax and savings rates. A household with no income in the components given, or every household when no components are given, follows its group's consumption: $C_i^1 = C_i^0\, EH_h/EH_h^0$.

## 20.2 Prices and real consumption

The household's price index is

$$P_i = \sum_c s_{ic} \frac{PH_c}{PH_c^0},$$

at the prices that households pay, including VAT, with $s_{ic}$ its survey budget shares when budget items are mapped to commodities, and the group's base shares $PH_c^0\, QH_{ch}^0/EH_h^0$ otherwise. Home consumption (section 14.1) enters the group's shares at the activity price $PA_a$. Real consumption is $R_i = C_i^1/P_i$; for households with LES demand this is a first-order (Laspeyres) approximation.

## 20.3 Poverty and inequality

With household size $n_i$ and survey weight $w_i$, poverty measures use consumption per person, $r_i = R_i/n_i$ (base $C_i^0/n_i$), and person weights $w_i\, n_i$. For a poverty line $z$ per person, the Foster–Greer–Thorbecke measures are

$$FGT_\alpha = \frac{\sum_i w_i\, n_i\, \mathbf{1}[r_i < z]\,\big((z - r_i)/z\big)^\alpha}{\sum_i w_i\, n_i}, \qquad \alpha = 0, 1, 2,$$

the headcount ratio, the poverty gap and the squared poverty gap. The Gini coefficient is that of $r_i$ over persons. Both are computed before and after, overall and by group.

## 20.4 Distribution and incidence

From version 0.8.0, a poverty line may differ by household, $z_i$, for regional lines or lines that depend on household composition, and several lines may be compared at once; for each, the number of poor, $\sum_i w_i n_i \mathbf{1}[r_i < z_i]$, is given with the FGT measures. Beside the Gini coefficient, the generalised entropy measures GE(0), the mean log deviation, GE(1), the Theil index, and GE(2), the Atkinson indexes with inequality aversion 0.5, 1 and 2, and the Palma and S80/S20 ratios describe the distribution of consumption per person over persons (**[CGE]** `cge_microsim` gives the formulas).

Incidence follows the same persons: ranked by base consumption per person into quantile groups, deciles by default, each group's mean consumption before and after, its shares of total consumption, and its share of the total change show who gains and who loses. The growth incidence curve of Ravallion and Chen (2003) compares the distributions instead: the growth of each percentile of consumption per person, whoever holds it. Under the top-down link, a household's rank can change only through its own income composition and budget shares, so the two views agree closely unless the shock changes relative incomes strongly.

Several simulations, such as alternative policies or the periods of a dynamic run, can be passed to the survey in one call, with every result in one dataset for tables and graphs.

## 20.5 Welfare and the channels of change

From version 0.8.1, each household's equivalent and compensating variations measure its welfare change in money: with the expenditure function of its group, the group's subsistence quantities scaled to the household, or, when its budget is given, Cobb–Douglas preferences in its own budget shares. A household that follows its group has its group's welfare change, as **[CGE]** `cge_report` reports it, in proportion to its consumption, so the survey and the model agree for the average household of each group.

A change in poverty or inequality has several causes: wages, profits, transfers and remittances change by different amounts, the direct tax and savings rates of the groups change, and prices change. Since a household's consumption is the product of these changes, the contribution of each can be found by switching the channels on one at a time; the order matters, and the Shapley decomposition (Shorrocks, 2013) averages over every order, so that the contributions add up exactly to the change. The change in poverty can also be divided into growth, the change in the mean with an unchanged distribution, and redistribution (Datt and Ravallion, 1992).

## 20.6 Fiscal incidence

From version 0.8.2, the survey's own taxes and benefits can be analysed as the Commitment to Equity method does (Lustig, 2018): market income less direct taxes, plus direct transfers, less indirect taxes, plus subsidies and in-kind benefits, gives net market, disposable, consumable and final income, and each instrument is described by its concentration in market income, its progressivity (Kakwani index), coverage and marginal contribution to poverty and inequality. What the model adds is the simulation: each instrument changes as the model says that its base changes (direct taxes with incomes and tax rates, indirect taxes with spending and the tax share of prices, transfers as the government's transfers, in-kind benefits as the government's or the health insurance fund's spending), so that fiscal incidence can be compared before and after a policy, in a general equilibrium. In-kind health benefits valued at cost, allocated to the households that use the services, then follow the model's health spending: the incidence of a rise in the health budget, from the government's purchases to the households that benefit, is traced in one step.

## 20.7 Graphs

From version 0.9.6, **[CGE]** `cge_microsim` draws the results it computes, in the style of figures in economics journals: the growth incidence curve, incidence by quantile group and by household group, the change in poverty at each line, the channels of the change (`decompose`), inequality by income concept (`market()`) and the range across the draws of the elasticities (`bands()`). The graphs plot the results of the tables above, and can be saved as PDF or EPS files for publication.

## 20.8 Weights, sampling error and the uncertainty of the model

From version 0.8.3, three tools address what the top-down link leaves out. First, a shock that moves workers between sectors changes who works where, which the survey's fixed weights cannot show; reweighting the survey, so that its totals of workers by sector (or any other characteristic) follow the model, gives the households of the expanding sectors more weight (Corral et al., 2022). The new weights are as close as possible to the survey's, in relative entropy. Second, the survey is a sample: with its design declared by `svyset`, the mean and the poverty measures, and their changes, have standard errors. Third, the model's elasticities are uncertain: **[CGE]** `cge_sensitivity` draws them, keeps each draw as a simulation, and the survey link gives the range of poverty and inequality across the draws.

Survey households keep their own income composition and budget shares, so their average change can differ from their group's: a household that lives on wages gains more than its group when wages rise relative to other income. The link is only as good as the mapping of income sources, which should cover every substantial source, and the survey and the SAM should share a base year where possible.

# 21 Sensitivity analysis

Elasticities are rarely estimated for the country and period of a SAM, and results can depend on them. **[CGE]** `cge_sensitivity` shows how much: it recalibrates the model with elasticities drawn around the values used, runs the scenario and solves, draw after draw.

## 21.1 Draws

Each elasticity $\sigma$ of the types chosen (value added, Armington, CET and exports from composite supply, and the top nest) is multiplied, in draw $k$, by a factor $u^{(k)}$ around one:

$$\sigma^{(k)} = u^{(k)}\,\sigma, \qquad u^{(k)} = 1 + s\,v^{(k)},$$

where $s < 1$ is the spread and $v^{(k)}$ is drawn from the uniform distribution on $[-1, 1]$, $v = 2U - 1$, or from the symmetric triangular distribution on the same interval, $v = U_1 + U_2 - 1$, with $U$, $U_1$ and $U_2$ independent and uniform on $[0, 1]$. By default each sector's elasticity has its own factor; with `common` one factor applies to every sector for each type of elasticity. A Leontief top nest (elasticity 0) stays Leontief, and the elasticities of nests of value added keep the values given. Draw 0, with the central elasticities, is always added.

For each draw the model is recalibrated from the SAM with every other calibration setting kept (factor quantities, LES demand, the health link), the closure is restored, the scenario program applies its shocks and the model is solved. The model in memory, with its simulations, is restored at the end.

## 21.2 Summary

For every report measure of chapter 12, the summary gives the result with the central elasticities, and, across the $n$ draws that converged, the mean, the standard deviation and the 5th and 95th percentiles, taken as order statistics of the sorted results $x_{(1)} \le \cdots \le x_{(n)}$:

$$p_5 = x_{(\max(1, \lceil 0.05\,n \rceil))}, \qquad p_{95} = x_{(\max(1, \lceil 0.95\,n \rceil))}.$$

## 21.3 Keeping the draws

From version 0.8.3, `keep(`*name*`)` keeps the solution of every draw that converged as a simulation of the model: *name*`0` with the central elasticities, and *name*`1`, *name*`2`, … for the draws, each with the parameters of its draw. Any command that takes a simulation can then use them. **[CGE]** `cge_microsim` with `bands(`*name*`)` gives the 5th, 50th and 95th percentiles of poverty, inequality and welfare across the draws, by the definition of `_pctile` (chapter 20).

## 21.4 Interpretation

The draws do not estimate uncertainty about the elasticities. They show how the results change over a chosen range around the values used. The range, the distribution and whether sectors move together should be stated with the results. Standard errors and bounds of published estimates (chapter 26) can guide the choice of the range.

# 22 Policy optimisation

**[CGE]** `cge_optimize` chooses the values $u$ of a few instruments to maximise or minimise a report measure $f$ (chapter 12), or to reach targets $g_j(u) = \bar{g}_j$ for report measures, with the model solved at every trial value:

$$\max_u \; f(x(u), u) \quad \text{subject to} \quad g_j(x(u), u) = \bar{g}_j, \qquad \underline{u} \le u \le \overline{u},$$

where $x(u)$ solves the model under the current closure. An instrument is one element of a parameter that **[CGE]** `cge_shock` changes, or of a variable that the closure fixes. From version 0.9.4, $f$ and $g_j$ may also be expressions of the user language (chapter 23), and inequality constraints $h_k(x(u), u) \le \bar{h}_k$ or $\ge \bar{h}_k$ may be added, with $\bar{g}_j$ and $\bar{h}_k$ numbers or expressions: the problem a GAMS modeller would give to an NLP solver, with the model as its equality constraints.

## 22.1 Algorithm

Each trial solves the model by Newton's method from the solution of the trial before. Instruments are measured as $u = u^0 + s \odot z$, with $s$ the scale of each instrument.

**Targets alone**, as many as instruments: the residuals $r_j = (g_j - \bar{g}_j)/|\bar{g}_j|$ are solved by Newton's method, with a Jacobian from central differences (step $10^{-5} \max(1, |z_k|)$ in $z$) and steps halved until the residuals fall.

**An objective**: a projected Newton method maximises $f / \max(1, |f^0|)$. The gradient and Hessian come from central differences, an instrument at a bound whose gradient points outwards is held there, steps are projected on the bounds and halved until the objective rises, and the iterations stop when the gradient of the free instruments is below $10^{-8}$.

**An objective with targets**: the targets are equality constraints, met by the augmented Lagrangian method. The maximiser takes

$$f - \lambda' r - \frac{\rho}{2} r' r,$$

and the multipliers move by $\rho\, r$ after each round, $\lambda \leftarrow \lambda + \rho\, r$, until $\max_j |r_j| < 10^{-10}$.

**Inequality constraints** (from version 0.9.4) enter by the method of Powell, Hestenes and Rockafellar: with the scaled slack $s_k \ge 0$ of each, the maximiser subtracts $\frac{1}{2\rho} \sum_k (\max(0, \mu_k - \rho s_k)^2 - \mu_k^2)$, and $\mu_k \leftarrow \max(0, \mu_k - \rho s_k)$ after each round, until the targets are met and $\max_k |\min(s_k, \mu_k/\rho)| < 10^{-10}$. A constraint that binds at the optimum gives the instruments that the same condition, as a target, would give; one that does not bind leaves the optimum as it is without it. The test suites check both.

## 22.2 Results

The optimum is stored as a simulation and left as the current shocks. The optimum may be local: a flat objective is located only to about $10^{-6}$ in the instruments, and other starting values may lead elsewhere. The Python reference solves the same problems with the SLSQP method of SciPy and with least squares; the test suites compare the instruments (to $10^{-5}$), the objective (to $10^{-9}$) and the solution, and check that a target for government savings reached by the direct tax rate reproduces the closure `fixgsav` (to $10^{-10}$).

# 23 User variables and equations

**[CGE]** `cge_variable` and **[CGE]** `cge_equation` add variables and equations to the calibrated model, written in a small language; **[CGE]** `cge_set` and **[CGE]** `cge_parameter` add the sets and parameters they may use (from version 0.7.1), conditions restrict sums, terms and domains (from version 0.7.2), and **[CGE]** `cge_macro` adds macros (from version 0.9.7).

## 23.1 The language

An expression is made of

- numbers, the operators `+ - * / ^`, parentheses, and the functions `exp()`, `ln()` (or `log()`) and `sqrt()`, and from version 0.9.7 `max()`, `min()`, `abs()`, `sqr()`, `log10()` and `ifthen()` (section 23.6);
- model and user variables, `VAR`, `VAR(e)` or `VAR(e1, e2)`, at an element or at the index of the equation or of a sum;
- parameters that **[CGE]** `cge_shock` changes, `par(e)`, and user parameters, `upar`, `upar(e)` or `upar(e1, e2)`, at their current values;
- `base(VAR(e))`, the base value of a variable;
- `sum(i, expression)`, a sum over the set of index `i`: `a` activities, `c` commodities, `f` factors, `h` households, `d` households and enterprises, `i` institutions that receive factor income, `n` nests, or a user set, by its name; digits may follow, as in `c2` or `goods2`;
- `sum((i,j), expression)`, a sum over every pair of elements of two sets, and `sum(map(i,j), expression)`, a sum over the pairs of a set of pairs (from version 0.9.1);
- `prod()`, `smax()` and `smin()`, the product, the largest and the smallest value over a set, with the domains of `sum()`, and `card(set)`, the number of its elements (from version 0.9.7);
- macros, `name(x, y)`, defined by **[CGE]** `cge_macro` (from version 0.9.7);
- `ord(i)`, the position of the element of index `i` in its set, from 1;
- `lag(VAR)` and `lag(VAR(e), k)`, the value of a variable in the period before or *k* periods before, up to four (from version 0.9.2; section 23.5);
- conditions, `$(condition)`, on sums, terms and domains (section 23.3).

## 23.2 User sets and parameters

A user set (**[CGE]** `cge_set`) is an ordered list of element labels, such as `cge_set goods = c, except(com_ser com_hlt)`. Its name is an index like the letter of a model set: `cge_variable PREL(services) = PQ(services) / CPI` adds a variable for each element of `services`, and `sum(goods, PQ(goods) * QG(goods))` sums over the elements of `goods`. A user set adds no variables or equations of its own.

A user parameter (**[CGE]** `cge_parameter`) has no index, one or two, over model sets or user sets, and values that the user gives: one value for all elements, values by element, or a variable of the data in memory. Its values are part of the model's parameters: every simulation stores them, and **[CGE]** `cge_shock` changes them, as in `cge_shock hgoal, value(1.2)`, so scenarios can be written in terms of the user's own parameters. A parameter's element may also be an instrument of **[CGE]** `cge_optimize`. User sets and parameters are saved with the model, and kept when it is calibrated again, the parameters at their base values.

## 23.3 Conditions

From version 0.7.2, a condition restricts a sum, a term or the domain of a user variable or equation, with the dollar operator of GAMS:

- `sum(c$(tm(c) > 0), PM(c) * QM(c))` sums over the commodities with an import tariff;
- `(PQ(c) * QG(c))$(goods(c))` is the term where `c` belongs to the user set `goods`, and 0 elsewhere;
- a name with a condition, as in `cge_variable TMV(c)$(tm(c) > 0) = tm(c) * PM(c) * QM(c)`, defines a user variable or equation for the elements that meet the condition only.

A condition compares values with `>` `<` `>=` `<=` `==` and `!=`, and combines comparisons with `&`, `|` and `!`; it may test whether an element belongs to a set, `goods(c)`, whether the index stands for a given element, `sameas(c, com_hlt)`, or the position of the element, `ord(c)`. Arithmetic is evaluated first, then comparisons, `!`, `&` and `|`.

A condition is evaluated when the variable or equation is added, and again when the model is calibrated, with the variables and parameters at their base values. It decides which terms and equation elements the model contains; a shock to a parameter that a condition uses changes the value of the terms that remain, not which terms remain. The system therefore has the same size in every simulation, dynamic run and sensitivity draw, and the GAMS program writes out the same terms. GAMS itself would evaluate the condition again at each solve statement, with the current values of the parameters; the program that **[CGE]** `cge_gams` writes contains the terms already chosen, so the two agree. An element of a user variable outside its domain does not exist: it is left out of the results, it is 0 where an expression refers to it, and GAMS fixes it at 0. Because `$`*name* is a global macro on a Stata command line, the condition is always written in parentheses after `$`.

## 23.4 Two indexes and sets of pairs

From version 0.9.1, a user variable or equation may have two indexes, as the model's household demand $QH_{c,h}$ does: `cge_variable X(c,h) = PQ(c) * QH(c, h)` adds a variable for every commodity and household, and a condition may restrict it to some pairs. A set of pairs, `cge_set map(a,c) = act_agr.com_agr act_man.com_man`, records a mapping between two sets, as a GAMS set over two indexes does. A sum runs over its pairs, `sum(map(a,c), PA(a) * QA(a))`, and in a condition, `map(a,c)`, it restricts sums, terms and the domains of variables and equations with two indexes to its pairs. Sums over two indexes and over sets of pairs are written out one index at a time, so they take conditions as other sums do. In the results the second index is the column `element2`; in **[CGE]** `cge_shock` an element is written `e1.e2`; in the GAMS program the variables are declared over two sets of accounts, `(AC, AC)`, with the pairs outside their domains fixed at 0.

## 23.5 Lags

From version 0.9.2, `lag(VAR)` is the value of a variable in the period before, and `lag(VAR(e), k)` its value *k* periods before, for *k* from 1 to 4, as $VAR_{t-k}$ in a model indexed by time. In a period of a dynamic run (chapter 19) the values are those of the periods solved before it, and those of the base before the first period. Within a period they are numbers, as the capital stock is, so a lag adds no variable and its derivative is zero; in a static simulation the period before is the base. A stock that the model does not update itself,

$$S_t = (1 - \delta)S_{t-1} + I_t,$$

is written `cge_equation s: S = (1 - 0.05) * lag(S) + ...`, with `S` a user variable given a value. Each period stores the lagged values it used, and the GAMS program declares them as parameters, `LAG1_U_S` for `lag(S)`, which each period of a dynamic run sets before it is solved.

## 23.6 Functions and macros

From version 0.9.7, the language has the functions of GAMS that a square system solves, as GAMS writes them: `max(x, y, ...)` and `min(x, y, ...)` of two or more arguments, `abs(x)`, `sqr(x)`, `log10(x)` and `ifthen(condition, x, y)`; the products and extremes over sets, `prod()`, `smax()` and `smin()`; and `card()`, the number of elements of a set. The sums of GAMS have the same domains here: one index, two indexes, a set of pairs, each with a condition. A product over no element is 1; a maximum or minimum over none is an error, since GAMS's $\mp\infty$ is no value for a model.

The condition of `ifthen()` uses parameters, lags and numbers, which are given within a simulation; a shock to the parameter switches the branch. GAMS's CNS and MCP models refuse relations between endogenous variables, which make the system discontinuous. A regime switch on endogenous variables is an inequality with its paired variable (section 23.8), the complementarity form in which GAMS solves it too.

The derivatives remain exact. `max()`, `min()` and the branch of `ifthen()` pass on the derivatives of the argument they take, the first of equal arguments at a kink; `abs()` multiplies by the sign of its argument, `sqr()` by $2x$ and `log10()` by $1/(x \ln 10)$. A model whose solution lies at a kink of `max()`, `min()` or `abs()` may need more Newton iterations, in Stata as in GAMS.

A macro (**[CGE]** `cge_macro`), as GAMS's `\$macro`, names an expression with parameters: `cge_macro ces(x, y, d, r) = (d * x^(-r) + (1 - d) * y^(-r))^(-1 / r)` makes `ces(QD(c), QM(c), 0.5, 0.5)` a CES aggregate of domestic goods and imports. Macros are replaced by their expressions, with the arguments in parentheses, before the indexes and sums are written out, so indexes may be arguments and a macro adds nothing to the system. In the GAMS program the expressions are written out, and the functions keep their GAMS names, with `eq`, `ne`, `and`, `or` and `not` in the conditions of `ifthen()`.

## 23.7 Replacing built-in equations

From version 0.9.3, a user equation may take the place of elements of a built-in equation block of chapter 7, with `replace(`*block*`)`:

```
cge_equation qe: QE(com_agr) = base(QE(com_agr)), replace(T2(com_agr))
```

holds the exports of agricultural goods at the base, in place of their export supply. The replaced elements leave the system and the user equation joins it, so the system stays square and the closure is unchanged. The blocks that may be replaced, by domain, are

| Domain | Blocks |
|---|---|
| none | (H1), (I12), (I14), (S4) |
| $c$ | (P1) to (P4), (Q6), (Q6A), (T1) to (T10), (T3E), (I6), (I7), (I13) |
| $a$ | (P5), (P6), (Q1), (Q2), (Q1C), (Q2C), (Q3) |
| $f$ | (H2), (H3), (L1), (L2) |
| $h$ | (I4) |
| $d$ | (I3), (I10), (I11) |
| $(f,a)$ | (Q4) |
| $(n,a)$ | (N1), (N2) |
| $(c,a)$ | (Q5), (N3) |
| $(a,c)$ | (Q6B), (P5A) |
| $(i,f)$ | (I2) |
| $(c,h)$ | (I5) |
| $(a,h)$ | (I5H) |

Within a block written in variants, such as (Q1C) for CES and Cobb–Douglas technologies, the element is replaced whatever its variant. The price indexes, factor incomes, government revenue and market clearing, equations (P7) to (P9), (I1), (I8), (I9), (S1) to (S3) and (S5), have derivatives coded in the package and cannot be replaced; they can be changed through the closure, or with user variables and equations that add to them. A replacement changes the model, and must keep its accounting consistent: the residual of Walras's law, `WALRAS`, which every simulation reports, shows when it does not. In the GAMS program, the parameter `XRP` marks the replaced elements, which the equations of `cge_model.gms` leave out.

## 23.8 Inequalities and bounds

From version 0.9.0, a user equation may be an inequality, $\ell(x) \leq r(x)$ or $\ell(x) \geq r(x)$, written with `=l=` and `=g=` as in GAMS and paired with a variable, as in a mixed complementarity problem:

```
cge_equation quota: QM(com_man) =l= 1.02 * base(QM(com_man)), perp(RM(com_man))
```

The paired variable, fixed under the closure, stays at its level while the inequality has room to spare and moves when it binds. Regime switches that a GAMS model writes as complementarity conditions, such as quotas with their premiums, capacity limits with their rents, floors on spending with an adjusting instrument, or a minimum price, can so be written in the model's own terms, with sets, parameters and conditions. Each inequality becomes a constraint of chapter 17 on its slack variable, and is solved, reported and exported to GAMS in the same way.

A user variable may be exogenous, `cge_variable TAU, value(0) fixed`, or exogenous at a bound from which a paired inequality may move it, `cge_variable TAU, lower(0)`: the analogue of a positive variable in GAMS. A defined variable may be held within bounds, `cge_variable PR = PQ(com_agr) / CPI, lower(0.95)`, which makes it the larger of its expression and the bound.

## 23.9 Variables and equations

`cge_variable NAME(i) = expression` adds the variable $U_i$ and the equation

$$U_i = e_i(x)$$

for every element $i$ of the set; its base value is $e_i(x^0)$. `cge_variable NAME(i), value(v)` adds a free variable without an equation, and `cge_equation NAME(i): lhs = rhs` an equation without a variable; each must be paired with the other, so that the closure stays square.

A defined variable swapped into the closure (`cge_closure, swap(NAME VAR)`) fixes the expression and frees `VAR`. For example, real government savings $GSAV/CPI$ held at their base value, with the direct tax rate adjusting, is under the CPI numeraire the closure `fixgsav`; the test suites check that the solutions agree to $10^{-9}$. Nominal government consumption, $\sum_c PQ_c\, QG_c$, fixed with volumes adjusting is another closure rule that the standard closures do not offer.

## 23.10 Compilation

Each equation is written out for every element of its set that meets its condition, each sum over the elements of its set that meet its condition, and each conditional term as itself or 0, then compiled to postfix order: numbers, variables (positions in the stacked vector of variables), parameters, user parameters (positions in the vector of their values), operators and functions. Stata evaluates the postfix rows at every point. From version 0.7.0 their derivatives are exact, by forward differentiation of the postfix rows (section 10.3): each value on the stack carries its derivatives with respect to the variables that the element uses. Before, they were found by grouped differences. The equations form a block `U` of the system. They are saved with the model and compiled again when it is calibrated again.

**GAMS.** [CGE] `cge_gams` writes the user parameters (`UP_NAME`, with their base values, which the scenario file changes), the user variables (`U_NAME`, with their base levels) and one scalar equation per element (`UE1`, `UE2`, ...) to the file `cge_user.inc`, which the GAMS model includes before its model statement, and the results of the user variables to `cge_user_write.inc`. GAMS then differentiates the same equations itself.

# 24 SAM tools

The SAM tools read, write, aggregate, balance and analyse a SAM before it is mapped and calibrated. They change the SAM only; the model is unchanged. Each command that changes the SAM starts a new model from the result, as **[CGE]** `cge_sam` does: earlier mappings, calibrations and simulations are discarded. In this chapter $\bar{X}$ is the SAM before a command and $X$ the SAM after it, with $x_{ij}$ the payment from column account $j$ to row account $i$.

## 24.1 Reading a SAM

**[CGE]** `cge_sam` reads a SAM in one of three forms:

1. **Wide data**: one observation per account, a string variable with the account labels and one numeric variable per account, from data in memory or from a file read with `use`, `import delimited` or `import excel`.
2. **A wide CSV file read directly**: the file is parsed in Mata and the data in memory are not changed. This is automatic when the SAM has more accounts than Stata can hold as variables (`c(maxvar)`; 2,048 in Stata/IC), and is chosen for any CSV file with the option `direct`.
3. **Long data**: one observation per non-zero cell, with string variables for the row and column accounts and a numeric variable for the value. The number of accounts is limited only by memory. Accounts are ordered as they first appear as row accounts, followed by any that appear only as column accounts, so that a SAM written in long form and read again keeps its order. A cell given twice is an error.

In the direct reader the first line holds the column labels, the first field of each following line the row label, and the remaining fields the values. Fields may be quoted. A blank field is a zero, counted in `r(N_blank)`; a field that is not a number (for example `1,234` with a thousands separator) is an error, never a zero. Row and column labels must agree, in the same order.

**Labels.** Account labels are used as names in **[CGE]** `cge_map`, so they must be valid Stata names. In wide data, Stata renames invalid labels when the file is imported (`s-i` becomes `si`), and the variable names are used. In the direct reader and in long data the same rule is applied: characters other than letters, digits and the underscore are removed, then any leading digits, and the result is cut to 32 characters. If a label cannot be converted (the result is empty, a reserved word, or the same as another label), `cge_sam` stops and names it. Each change is displayed.

## 24.2 Writing a SAM

**[CGE]** `cge_samexport` writes the SAM of the current model, as read, aggregated or balanced, to a `.csv`, `.dta` or `.xlsx` file, or loads it as data. Wide output has one variable per account (`.dta` and `.xlsx` need the accounts to fit as variables; `.csv` is written in Mata and has no such limit). Long output has the variables `row`, `column` and `value`, one observation per non-zero cell. With `mapped`, the SAM after the layout rewrites of section 9.2 is written instead, with the internal tax accounts. Numbers are written in the shortest of `%21.0g` and `%24.17g` that reads back as the same double, so that a SAM written and read again is unchanged.

## 24.3 Mapping templates

**[CGE]** `cge_template` writes a template with one row per account and the columns `account`; `role`, the `cge_map` options that list the account; `pair`, for an activity, the commodity it is paired with; `group`, for aggregation, the account it is merged into; and the account's row and column totals and their difference. An Excel template also has a sheet `roles` that lists the valid entries. When the model in memory is already mapped, `role` and `pair` are filled from the mapping.

A role is any option name of `cge_map`, in either case, with the singular forms (`activity`, `commodity`, `factor`, `household`, `enterprise`, `margin`, `marginaccount`, `stock`) also accepted. An account may have several roles when `cge_map` allows it, for example `activities margins` for a margin activity, or `government saltax` when sales taxes are paid directly to the government. A blank role leaves the account unmapped, which `cge_map` accepts only for accounts without flows.

`cge_map using` reads a template. Each option lists its accounts in the order of the SAM. Activities are paired with commodities by the `pair` column; a blank `pair` pairs the activity with the commodity whose column pays it most (the largest cell $x_{ac}$). A tie, no sales to any commodity, or a commodity paired with two activities is an error. After aggregation with the same template, an account of the current SAM may be a group: it takes the role of its members, which must agree, and is paired with the group of its members' pairs. The options that result are displayed, so that the same mapping can be written as a command.

## 24.4 Aggregation

**[CGE]** `cge_aggregate` merges accounts into groups. With the indicator matrix $G$ of groups by accounts ($G_{gk} = 1$ if account $k$ is in group $g$),

$$X = G\,\bar{X}\,G',$$

so that every cell is the sum of the cells it merges. Totals add up, and an account's balance is the sum of its members' balances. A group takes the position of its first member. Flows among the members of a group become a diagonal cell of the new account; they are kept, as transfers within the group. The model accepts diagonal cells for households, enterprises and the government, and `cge_map` reports any other as a flow the model does not represent. The option `netdiagonal` removes them; totals then fall by the diagonal cell, and balances do not change.

## 24.5 Balancing

**[CGE]** `cge_balance` makes the SAM balanced, every account's row total equal to its column total. Both of its methods keep the sign of every cell, every zero cell, and the cells listed in `fixed()` at their values in $\bar{X}$. Both minimise the same measure of the information added to $\bar{X}$ (Junius and Oosterhaven, 2003; Lenzen et al., 2007), over the cells that may change,

$$D(X,\bar{X}) = \sum_{ij}\Big(|x_{ij}|\,\ln\frac{|x_{ij}|}{|\bar{x}_{ij}|} - |x_{ij}| + |\bar{x}_{ij}|\Big),$$

which for non-negative matrices is the cross entropy that RAS minimises (McDougall, 1999). They differ in what is known about the totals.

### 24.5.1 RAS

Every account $k$ has a target total $t_k$, and the row and column totals of $X$ are set to it. By default $t_k$ is the average of the account's row and column totals in $\bar{X}$; the targets may instead be the row or the column totals, or be given for individual accounts. If an account's row has no cell that may change, its target is its row total (and likewise for its column); if both are fixed and differ, or a given target contradicts them, no balanced SAM exists and `cge_balance` stops.

With $P$ the positive part and $N$ the absolute value of the negative part of the cells that may change, and $u$ and $v$ the targets less the fixed cells of each row and column, generalised RAS (GRAS) finds row factors $r$ and column factors $s$ such that

$$x_{ij} = r_i\, p_{ij}\, s_j - \frac{n_{ij}}{r_i\, s_j}$$

has row totals $u$ and column totals $v$. Starting from $s = 1$, it alternates

$$r_i = \frac{u_i + \sqrt{u_i^2 + 4\, p_i\, n_i}}{2\, p_i}, \qquad p_i = \sum_j p_{ij}\, s_j, \qquad n_i = \sum_j n_{ij}/s_j$$

($r_i = -n_i/u_i$ when $p_i = 0$), and the same for $s$ with the roles of rows and columns exchanged, until every row and column total is within the tolerance of its target. Without negative cells this is RAS. A factor that is not positive and finite means that the targets cannot be met with the signs of the cells, and `cge_balance` stops.

### 24.5.2 Cross entropy

Only some totals are known, or none: the totals of the other accounts are estimated with the cells. $X$ minimises $D(X, \bar{X})$ subject to

$$\sum_j x_{kj} - \sum_i x_{ik} = 0 \quad \text{for every account } k, \qquad \sum_j x_{kj} = t_k \quad \text{for the accounts whose totals are known.}$$

The solution has the form

$$x_{ij} = p_{ij}\, e^{a_{ij}} - n_{ij}\, e^{-a_{ij}}, \qquad a_{ij} = \mu_i - \mu_j + \varrho_i,$$

with a multiplier $\mu_k$ for the balance of each account and $\varrho_k$ for each known total ($\varrho_k = 0$ for the other accounts). The multipliers minimise the convex dual

$$\varphi(\mu, \varrho) = \sum_{ij} p_{ij}\, e^{a_{ij}} + \sum_{ij} n_{ij}\, e^{-a_{ij}} + \sum_k \mu_k\, c_k + \sum_{k \text{ known}} \varrho_k\, (f_k - t_k),$$

where $c_k$ is the fixed cells' row total less their column total and $f_k$ their row total. Its gradient is the vector of balances and of the differences from the known totals, and its Hessian is $H = \sum_{ij} |x_{ij}|\, d_{ij}\, d'_{ij}$ with $d_{ij} = \partial a_{ij}/\partial(\mu, \varrho)$. Newton's method with a backtracking line search minimises $\varphi$ until every balance and known total is within the tolerance. Adding a constant to all $\mu$ in a connected set of accounts changes nothing, so one $\mu$ in each set is fixed at zero; an account without cells that may change must already balance. If the iterations do not converge, no balanced SAM with the same pattern of signs and zeros may exist.

When every account's total is known, the solution is the RAS solution, with $r_i = e^{\mu_i + \varrho_i}$ and $s_j = e^{-\mu_j}$; the test suite uses this identity. This is not the column-coefficient cross entropy of Robinson et al. (2001), which measures the distance between the column shares of $X$ and $\bar{X}$ rather than between the cells; that method is not implemented.

### 24.5.3 Tolerance and results

Convergence is measured against the largest absolute cell of $\bar{X}$: every balance and target is met to `tolerance()` times $\max(1, \max_{ij} |\bar{x}_{ij}|)$, by default $10^{-10}$, well within the $10^{-8}$ of the balance check of `cge_map`. `cge_balance` reports the largest imbalance before and after, the number of cells changed, the largest relative change of a cell, the SAM total before and after, and the cells with the largest changes. The balanced SAM is an estimate: which totals are known, which cells are fixed and which method is used should be stated with any results.

## 24.6 Accounting multipliers

**[CGE]** `cge_multiplier` computes the accounting multipliers of the mapped SAM (Pyatt and Round, 1979): the effects of an injection into one account on the incomes of all the accounts whose spending the SAM explains, with prices fixed. It uses the SAM after the layout rewrites of section 9.2, and needs no calibration.

The accounts are divided into endogenous accounts $n$ and exogenous accounts. By default the endogenous accounts are the activities, commodities and margin accounts (the production block, $b = 1$), the factors ($b = 2$), and the households and enterprises (the institutions block, $b = 3$); the government, savings–investment, the rest of the world, the tax accounts and the health insurance fund are exogenous. With $t_j$ the column total of account $j$, the average expenditure propensities among endogenous accounts are

$$A_{ij} = SAM(i,j)/t_j, \qquad i,j \in n,$$

(zero for an account without payments), and the incomes of the endogenous accounts satisfy $y = A\,y + x$, where $x$ is their income from exogenous accounts. Hence

$$y = M\,x, \qquad M = (I - A)^{-1}.$$

Column $j$ of $M$ is the change in the incomes of every endogenous account for a unit injection into $j$. For each injection the command reports the total multiplier $\sum_i M_{ij}$ and its parts in gross output (the activities), value added (the factors) and household income (the households). $I - A$ is invertible when every chain of payments among the endogenous accounts eventually leaks to an exogenous account (taxes, savings, imports).

### 24.6.1 Decomposition

With the three blocks ordered as above, $A$ has a circular structure: factors are paid by production ($A_{21}$), institutions by factors ($A_{32}$), and production by institutions ($A_{13}$), with transfers within production ($A_{11}$), within factors ($A_{22}$) and within institutions ($A_{33}$). With $\tilde{A}$ the block-diagonal part of $A$ and $A^* = (I - \tilde{A})^{-1}(A - \tilde{A})$, Pyatt and Round (1979) write

$$M = M_3\,M_2\,M_1, \qquad M_1 = (I - \tilde{A})^{-1}, \quad M_2 = I + A^* + A^{*2}, \quad M_3 = (I - A^{*3})^{-1},$$

the transfer, open-loop and closed-loop multipliers, and additively

$$M = I + (M_1 - I) + (M_2 - I)\,M_1 + (M_3 - I)\,M_2\,M_1.$$

The three terms after $I$ are the transfer effects within the injected account's block, the open-loop effects on the other blocks, and the closed-loop effects of the circular flow back to the injected block. The decomposition requires the blocks outside the circle ($A_{12}$, $A_{23}$, $A_{31}$) to be zero; `decompose` stops with an error, listing the largest flows that break the structure, if they are not.

### 24.6.2 Injections and use

`inject()` gives a vector $x$ of injections into endogenous accounts, for example more government purchases of a commodity, and the command reports $M\,x$: the change in gross output by activity, in factor incomes and in household incomes, and their totals.

Accounting multipliers assume fixed prices, unused capacity in every sector and average propensities equal to marginal ones; they are upper bounds on the effects that a CGE model, with its supply constraints and price changes, would find. They complement the CGE model; they do not replace it.

## 24.7 Structural path analysis

**[CGE]** `cge_spa` decomposes one accounting multiplier into the paths along which an injection travels through the SAM (Defourny and Thorbecke, 1984). The accounts are the poles of a graph, with an arc from pole $i$ to pole $j$ when $j$ receives part of $i$'s spending, with intensity $A_{ji}$. An elementary path $p$ from an origin $i$ to a destination $j$ is a sequence of arcs that passes through no pole twice.

- The **direct influence** of $i$ on $j$ along $p$ is the product of the intensities of its arcs, $I_p^D = A_{k_1 i} A_{k_2 k_1} \cdots A_{j k_m}$: the effect along the path with every other account held fixed.
- The **path multiplier** $M_p$ adds the feedback of the loops that start and end at the poles of $p$. With $P$ the set of poles of $p$,

  $$M_p = \frac{\det\bigl((I-A)_{-P,-P}\bigr)}{\det(I-A)} = \det\bigl(M_{P,P}\bigr),$$

  where $(I-A)_{-P,-P}$ leaves out the rows and columns of $P$ and $M_{P,P}$ is the block of $M$ on $P$; the two are equal by Jacobi's identity for complementary minors, and the second needs only a small determinant.
- The **total influence** along $p$ is $I_p^T = I_p^D\, M_p$.
- The **global influence** of $i$ on $j$ is the multiplier $M_{ji}$, and it is the sum of the total influences along all elementary paths from $i$ to $j$ ($i \neq j$).

The number of elementary paths grows quickly with the size of the SAM, so the search stops at paths of `maxlength()` arcs (default 6) and drops a path, and every extension of it, once its direct influence falls below `threshold()` times the global influence in absolute value (default 0.001). With intensities no larger than one, as when a SAM has no negative cells, extensions can only lower the direct influence, so no path above the threshold is missed. The share of the global influence explained by the paths found is reported.

# 25 The GAMS implementation

**[CGE]** `cge_gams` writes the calibrated model and one scenario as a GAMS program, so that anyone with GAMS can solve the model with PATH, CONOPT, KNITRO, IPOPT or another solver; check the results of cge-stata with a second, independent implementation; and extend the equations. The GAMS model is the one that the package's own tests use to check the Stata model.

## 25.1 What is written

GAMS receives what a user supplies, not the parameters that Stata calibrated: the SAM after the layout rewrites of **[CGE]** `cge_map` (section 9.2); the roles of its accounts; and the settings given to **[CGE]** `cge_calibrate`: elasticities, income elasticities and Frisch parameters, VAT credit shares, public debt and interest rates, export-only commodities and export demand, labour-market settings, days lost to illness, and factor quantities. The GAMS program calibrates every parameter itself, solves the base, and then solves the scenario. Its equations are an independent implementation of the equations of this part, written in GAMS rather than translated from Mata.

| File | Content |
|---|---|
| `cge_run.gms` | driver: includes the model, then the scenario |
| `cge_model.gms` | data, calibration, equations and base solve |
| `cge_closure.gms` | bounds, variables that do not exist, and the closure |
| `cge_solve.gms` | the solve statement, for the chosen solver and model type |
| `cge_write.gms` | writes a solution |
| `cge_data.inc` | the SAM, the roles of its accounts and the calibration settings |
| `cge_shock_par.inc` | parameter changes and the closure |
| `cge_shock_var.inc` | changes to fixed variables |
| `cge_constraints.inc` | constraints, as complementarity pairs, and the model `cgecon` |
| `cge_user.inc`,<br>`cge_user_write.inc` | user variables and equations, and their results (chapter 23) |

The scenario is either the shocks and closure in memory (what the next `cge_solve` would solve) or those of a stored simulation. Changed parameters and fixed variables are written as ratios to their base values, for example

```
tm('com_man') = 0 * tm('com_man');
QFS.fx('lab') = 1.05 * QFS.l('lab');
```

so that GAMS applies them to its own calibrated values. A parameter whose base value is zero is set to its new value. Numbers are written with the fewest significant digits (15, 16 or 17) that give back the same double. GAMS writes `cge_base.csv` and `cge_`*name*`.csv`, with one row per variable and element (scenario, variable, element1, element2, value) and the model status in the first row.

The equations of `cge_model.gms` carry the labels of chapter 7 with a prefix for their block: `EP` for prices (`EP1` is (P1)), `EQ` for production, `ET` for trade, `EI` for institutions, `ES` for the system constraints, `EH` for the health links and `EL` for the labour-market regimes. To add an equation, declare a new variable and its equation after the existing ones, fix one variable less or free one more so that the system stays square, and give the new variable a starting level before the base solve.

**Dynamic runs.** Every period of a run of **[CGE]** `cge_dynamic` can be exported. Each period's changes from the period before are written as ratios, to parameters (for example capital accumulation, population growth of subsistence quantities and of the base of the health link, and growth of government and foreign flows) and to fixed variables, in files named after the period's simulation: `cge_dn1_par.inc` and `cge_dn1_var.inc` for the simulation `dn1`. The driver solves the base and then the periods in order, each from the solution of the period before. Runs with constraints, microsimulation and sensitivity analysis are not exported.

## 25.2 Solvers and model types

| `type()` | GAMS model | Solvers |
|---|---|---|
| `cns` (default) | a constrained nonlinear system: the equations, square, with no objective | PATH, CONOPT, KNITRO |
| `nlp` | the same equations and a constant objective, `OBJ =e= 0` | IPOPT, CONOPT, KNITRO, MINOS, SNOPT |
| `mcp` (the default with constraints) | a mixed complementarity problem: the equations, unpaired, and each constraint's bound paired with a slack | PATH |

**Constraints.** A scenario with constraints (chapter 17) can only be written as a mixed complementarity problem. For constraint $k$, `cge_constraints.inc` declares a slack $SLK_k \geq 0$; the bound $BND_k$, $\sigma_k(b_k - Y_k)/s_k^Y \geq 0$, paired with the slack; and the link $LNK_k$, $X_k = \bar{x}_k + \delta_k s_k^X SLK_k$, an unpaired equality that frees the adjusting variable. When the slack is zero, $Y_k$ may lie within its bound and $X_k$ stays at $\bar{x}_k$; when it is positive, $Y_k$ is at its bound. This is the condition (B1), which GAMS passes to PATH exactly, without smoothing. The variables with no pair must be free or fixed, so the bounds that keep prices and quantities positive in the CNS form are left out. The base is solved as the model `cgebase`, the scenario as `cgecon`.

**Tolerances.** Convergence tolerances in GAMS solvers are absolute, so they are scaled to the largest SAM cell, within what double precision allows: $p = \max(10^{-11}, 10^{-13} \max_{rc} |SAM(r,c)|)$. When it runs, the GAMS program writes an option file for four solvers: PATH (`convergence_tolerance` $= p$), CONOPT (`rtnwma`, $p$ relative to the largest cell), IPOPT (`tol` $= 10^{-12}$, `constr_viol_tol` $= p$) and KNITRO (`feastol_abs` $= p$, `feastol` $= 10^{-12}$). Other solvers use their defaults.

**Running GAMS.** In the folder written, `gams cge_run.gms` solves the base and the scenario with PATH; `--cgesolver=conopt` chooses another CNS solver, and `--cgesolver=ipopt --cgetype=nlp` an NLP solver. From interactive Stata, `cge_gams` can start GAMS itself; Stata for Windows cannot start other programs in batch mode. A GAMS licence is needed; GAMS runs solvers outside the licence only for small models. Listing files name the licence holder.

## 25.3 Comparison and import

**Comparison.** `compare` reads the GAMS results and computes, for every variable and element that exists in the model,

$$\frac{|x^{\text{GAMS}} - x^{\text{Stata}}|}{\max(1, |x^{\text{Stata}}|)},$$

and reports the largest value, for the base and for the scenario, with the variable where it occurs.

**The GAMS solution in Stata.** `import()` stores the GAMS solution as a Stata simulation: the free variables take the values of GAMS, the fixed ones those of Stata, and Stata evaluates the model's equations at the result and reports the largest scaled residual. For a model too large for Stata's dense solver (section 5.2) the route is to build, calibrate and shock the model in Stata, export it, solve it with PATH, and import the solution for reports, microsimulation or comparison.

**Agreement.** On 31 exported cases, which together cover every layout, extension and closure of the package, the solutions of GAMS agree with those of Stata to these relative differences:

| Solver | Type | Cases solved | Base | Scenario |
|---|---|---|---|---|
| PATH | CNS | 26 of 26 | $4.3 \times 10^{-16}$ | $1.6 \times 10^{-11}$ |
| PATH | MCP, with constraints | 5 of 5 | $2.8 \times 10^{-17}$ | $4.0 \times 10^{-12}$ |
| CONOPT | CNS | 17 of 17 | $4.3 \times 10^{-16}$ | $3.6 \times 10^{-9}$ |
| KNITRO | CNS | 17 of 17 | $4.3 \times 10^{-16}$ | $1.6 \times 10^{-11}$ |
| IPOPT | NLP | 17 of 17 | $5.6 \times 10^{-15}$ | $1.6 \times 10^{-11}$ |

Stata solved every case with a tolerance of $10^{-12}$ on the scaled residuals, so differences of about $10^{-11}$ are at the precision of the two solutions; CONOPT stops at a looser feasibility tolerance. CONOPT, KNITRO and IPOPT do not solve complementarity problems, so they were not run on the cases with constraints.

# 26 The elasticity library

The model's free parameters, the elasticities of substitution and transformation, the income elasticities, the Frisch parameter, the wage-curve elasticity and the health-outcome elasticities, have no defaults: they are part of the specification and must be stated. The library records published estimates, with the source, the table and page where each is printed, its definition, and the check made when it was read. **[CGE]** `cge_elasticities` lists or loads entries, or returns them as options for **[CGE]** `cge_calibrate`; nothing is applied without a command.

## 26.1 Contents

| Parameter | Entries | Coverage | Source |
|---|---|---|---|
| `sigmaq` | 51 | aggregate Armington elasticity, African countries (series to 2018) | Devarajan et al. (2023), Appendix Table A1 |
| `sigmat` | 51 | aggregate CET elasticity, the same countries | Devarajan et al. (2023), Appendix Table A2 |
| `incelas` | 423 | income elasticities for nine categories of consumption, 47 African countries (2005) | Muhammad et al. (2011), Appendix table 1 |
| `frisch` | 1 | inverse of the income flexibility, 144 countries pooled | Muhammad et al. (2011), Table 3 |
| `sigmava` | 1 | capital–labour substitution, 30 African countries pooled (1970–2017) | Hosni and Ouakil (2024), Tableau 9 |
| `wagecurve` | 2 | international (12 countries) and South Africa (1993) | Blanchflower and Oswald (1995); Kingdon and Knight (2006) |
| `mortelas` | 2 | under-five and maternal mortality with respect to government health expenditure, developing countries | Bokhari et al. (2007) |

Somalia, South Sudan and Sudan are not covered by Devarajan et al. (2023); Muhammad et al. (2011) cover 47 African countries (the 2005 round of the International Comparison Program, without Comoros, which the authors exclude).

Each entry records its identifier (for example `DGR2023-ARM-TGO`); the parameter, named as the option of **[CGE]** `cge_calibrate` that the value is for; the country, as an ISO 3166 alpha-3 code or a group (`AFR` Africa, `WLD` international, `DEV` developing countries), and its name as printed; the category (`aggregate` for the trade elasticities, the consumption category for income elasticities: `food`, `clothing`, `housing`, `furnishings`, `health`, `transport`, `recreation`, `education`, `other`); the value in the package's convention, its standard error and bounds where the source gives them; the value as printed; what was estimated, how, and on what data; the source and where the value is printed; and the check made, with any qualifications. The files are plain ASCII, which Stata 13 displays, so accents are dropped.

The package's conventions differ from some sources in two places: the wage-curve elasticity is the absolute value of the published negative elasticity ((L1)), and the Frisch parameter is the inverse of the published income flexibility ($-1/0.732 = -1.3661$).

## 26.2 How the values were read and checked

The two multi-country tables were read from the source documents, with a check on every value.

- Devarajan et al. (2023) print each estimate with its standard error and $t$-statistic. The $t$-statistic must equal the estimate divided by the standard error, within the rounding of the printed values; 101 of the 102 African entries pass. One fails in the source itself: the CET elasticity of the Democratic Republic of Congo is printed as 0.503 with standard error 0.245 and $t$-statistic 56.713, although $0.503/0.245 = 2.05$. The entry keeps the printed values and a note.
- In Muhammad et al. (2011) each income elasticity is $\eta = \theta/(\theta - \beta)$, with the marginal share $\theta$ in their Appendix table 7 and the coefficient $\beta$ in their Table 3. All 408 African entries that can be checked lie within the range that the rounding of $\theta$ and $\beta$ allows; the 15 that the authors computed from actual budget shares cannot be checked this way and are marked.

The other values were read from the source pages or from the published abstracts. Not included are default parameters of global models (for example GTAP) and values derived from IFPRI SAMs or models; a user who holds such values can add them in a file of the same layout.

## 26.3 Using the values

- **Definitions.** Match the definition, coverage and period before using a value. The mortality elasticities of Bokhari et al. (2007) are with respect to government health expenditure per head, not the death rate with respect to health services per head of section 16.4; they are recorded for reference.
- **Aggregate and sector values.** Aggregate trade elasticities are lower than sector ones, because aggregate imports and exports are composites of many goods (Devarajan et al., 2023). They set a level rather than every sector's value; using one aggregate value for every commodity is a strong simplification.
- **Income elasticities.** They come from a cross-country demand system for 2005, evaluated for each country, not from the country's household survey. They map to model commodities through the nine categories.
- **Uncertainty.** Standard errors and bounds can guide the range of a sensitivity analysis (chapter 21).

# Part III

# Command reference

# Introduction to the command reference

This part describes each command of cge-stata, in alphabetical order. Each entry follows the order of the entries of Stata's own manuals: a description, short examples (Quick start), the syntax and its options, remarks and examples with their output, the results stored in `r()`, the methods and formulas, and related entries. The formulas of the model are stated once, in Part II; each entry gives the formulas of its own command and refers to Part II for the rest.

In a syntax diagram the command is in bold type, arguments are in italics, and the underlined part of an option is its shortest abbreviation; an option with nothing underlined cannot be abbreviated. Options in square brackets are optional.

## The commands by task

| Task | Commands |
|---|---|
| Overview | **[CGE]** `cge` |
| Reading and preparing a SAM | **[CGE]** `cge_sam`, **[CGE]** `cge_synthetic`, **[CGE]** `cge_template`, **[CGE]** `cge_aggregate`, **[CGE]** `cge_balance`, **[CGE]** `cge_samexport` |
| SAM analysis | **[CGE]** `cge_multiplier`, **[CGE]** `cge_spa` |
| Building the model | **[CGE]** `cge_map`, **[CGE]** `cge_elasticities`, **[CGE]** `cge_calibrate`, **[CGE]** `cge_check`, **[CGE]** `cge_listing`, **[CGE]** `cge_parameters` |
| Simulations | **[CGE]** `cge_closure`, **[CGE]** `cge_shock`, **[CGE]** `cge_constraint`, **[CGE]** `cge_solve`, **[CGE]** `cge_variable`, **[CGE]** `cge_equation`, **[CGE]** `cge_set`, **[CGE]** `cge_parameter`, **[CGE]** `cge_macro` |
| Results | **[CGE]** `cge_results`, **[CGE]** `cge_report` |
| Dynamics | **[CGE]** `cge_dynamic`, **[CGE]** `cge_dynresults` |
| Further analysis | **[CGE]** `cge_sensitivity`, **[CGE]** `cge_microsim`, **[CGE]** `cge_optimize` |
| GAMS | **[CGE]** `cge_gams` |
| Saving and loading | **[CGE]** `cge_save`, **[CGE]** `cge_use` |

# `cge` — Computable general equilibrium modelling in Stata

## Description

cge-stata builds, calibrates and solves single-country computable general equilibrium (CGE) models from a social accounting matrix (SAM), in Stata and Mata, without an external modelling system. A model can be solved for one period or as a recursive dynamic sequence, its results can be reported in standard tables, passed to household survey data, or tested for their sensitivity to the elasticities, and the model can be written as a GAMS program. Tools read, aggregate and balance the SAM, and a library gives published elasticities with their sources.

The command `cge` displays the version of the package and its commands.

## Syntax

```
cge
```

## Remarks and examples

### The model

The model is the standard SAM-based model of the IFPRI and PEP traditions (Lofgren, Harris and Robinson 2002; Decaluwé et al. 2013); only the structure of the model is taken from that literature. Its standard form has

- production with CES or Cobb–Douglas value added and Leontief intermediate inputs, with an optional CES or Cobb–Douglas nest between them; activities may produce several commodities, and a commodity may have several producers;
- trade through a CET function between exports and domestic sales and an Armington CES function between imports and domestic goods;
- households with Cobb–Douglas or linear expenditure system demand, enterprises, the government, savings-investment and the rest of the world, with transfers among them;
- direct, sales, import, export and activity taxes;
- closures for the external account, the government, savings-investment, the numeraire and each factor.

Extensions, switched on by the layout of the SAM or by options of **[CGE]** `cge_map` and **[CGE]** `cge_calibrate`, add VAT with input credits, public debt, taxes on factor use and factor income, home consumption, finite export demand, export-only commodities, margins by market, labour-market regimes, a social health insurance fund, factor efficiency linked to health services, health outcomes, nests of value added and the aggregation of output from several activities. Limits that bind in some scenarios and not in others, such as quotas and capacity limits, are complementarity conditions solved with the model. Part II of this manual gives every equation.

The package has no intertemporal optimisation (agents do not look ahead), no multi-region model and no estimation of elasticities: the elasticities are inputs, stated by the user, and have no defaults.

### The commands

| Command | Title |
|---|---|
| *Preparing a SAM* | |
| **[CGE]** `cge_synthetic` | Load the synthetic social accounting matrix |
| **[CGE]** `cge_sam` | Read a social accounting matrix |

| | |
|---|---|
| **[CGE]** `cge_template` | Write a template that maps SAM accounts to roles and groups |
| **[CGE]** `cge_aggregate` | Merge SAM accounts into groups |
| **[CGE]** `cge_balance` | Balance a SAM by RAS or cross entropy |
| **[CGE]** `cge_samexport` | Write the SAM to a file or to memory |
| **[CGE]** `cge_multiplier` | SAM multipliers and their decomposition |
| **[CGE]** `cge_spa` | Structural path analysis of a SAM multiplier |
| *Building the model* | |
| **[CGE]** `cge_map` | Assign SAM accounts to model roles |
| **[CGE]** `cge_elasticities` | Published elasticities, with their sources |
| **[CGE]** `cge_calibrate` | Calibrate the model to the SAM |
| **[CGE]** `cge_check` | Run model validation checks |
| **[CGE]** `cge_parameters` | The calibrated parameters as a dataset |
| *Simulation* | |
| **[CGE]** `cge_closure` | Choose the model closure |
| **[CGE]** `cge_shock` | Change a parameter or a fixed variable |
| **[CGE]** `cge_constraint` | Limits that bind in some scenarios and not in others |
| **[CGE]** `cge_variable` | User variables, defined by an expression or determined by user equations |
| **[CGE]** `cge_equation` | User equations: new relations among the model's variables |
| **[CGE]** `cge_set` | User sets of elements, for indexes and sums |
| **[CGE]** `cge_parameter` | User parameters for user variables and equations |
| **[CGE]** `cge_solve` | Solve the model |
| **[CGE]** `cge_optimize` | Choose policy instruments to maximise an outcome or reach targets |
| **[CGE]** `cge_dynamic` | Solve a recursive dynamic run |
| *Results* | |
| **[CGE]** `cge_results` | Report simulation results |
| **[CGE]** `cge_report` | Standard report tables for one or more simulations |
| **[CGE]** `cge_dynresults` | Results of a dynamic run as a dataset |
| **[CGE]** `cge_sensitivity` | Sensitivity of a scenario's results to the elasticities |
| **[CGE]** `cge_microsim` | Pass simulations to household survey data |
| *Saving the model, and GAMS* | |
| **[CGE]** `cge_save` | Save the model and its simulations to a file |
| **[CGE]** `cge_use` | Load a model saved with cge_save |
| **[CGE]** `cge_gams` | Write the model as a GAMS program, and solve it in GAMS |

### The workflow

A model is built in a fixed order, one command for each step:

1. read the SAM with **[CGE]** `cge_sam`, and prepare it if needed with **[CGE]** `cge_template`, **[CGE]** `cge_aggregate` and **[CGE]** `cge_balance`;
2. assign its accounts to the roles of the model with **[CGE]** `cge_map`, which checks every cell against the flows that the model represents;
3. choose the elasticities, from studies of the country or from the library of **[CGE]** `cge_elasticities`;
4. calibrate with **[CGE]** `cge_calibrate`, which checks that the base solves every equation, and run the checks of **[CGE]** `cge_check`;
5. choose a closure with **[CGE]** `cge_closure`, change parameters or fixed variables with **[CGE]** `cge_shock`, and solve with **[CGE]** `cge_solve`, once for each simulation;
6. report with **[CGE]** `cge_results` and **[CGE]** `cge_report`, and save the model and its simulations with **[CGE]** `cge_save`.

The model is kept in Mata, not in the data in memory. Commands that load data, such as those with a `clear` option, leave the model intact, and **[CGE]** `cge_save` stores it with its simulations. Reading a SAM with **[CGE]** `cge_sam` starts a new model, and mapping or calibrating again replaces the mapping or the calibration.

### Example 1: the package

```
. cge

cge-stata 0.7.1
Computable general equilibrium modelling in Stata

Preparing a SAM
  cge_sam         read a social accounting matrix, of any size, wide or long
  cge_template    write a template for roles and aggregation (Excel or CSV)
  cge_aggregate   merge accounts into groups
  cge_balance     balance the SAM by RAS or cross entropy
  cge_samexport   write the SAM to a file or to memory
  cge_multiplier  SAM multipliers and their decomposition
  cge_spa         structural paths of a SAM multiplier

Workflow
  cge_map         assign SAM accounts to model roles
  cge_calibrate   calibrate the model
  cge_elasticities published elasticities, with sources, as calibration options
  cge_closure     choose the closure
  cge_shock       change parameters or fixed variables
  cge_constraint  limits that may bind: quotas, capacity, borrowing and deficit ceilings
  cge_variable    user variables, defined by an expression or by user equations
  cge_equation    user equations: new relations among the model's variables
  cge_solve       solve and store a simulation
  cge_optimize    choose instruments to maximise an outcome or reach targets
  cge_dynamic     solve a recursive dynamic run, period by period
  cge_results     display or save all results of a simulation
  cge_report      standard report tables, to screen, Stata or Excel
  cge_dynresults  results of a dynamic run, against a baseline
  cge_sensitivity sensitivity of a scenario to the elasticities
  cge_microsim    pass a simulation to household survey data
  cge_check       run validation checks
  cge_parameters  calibrated parameters as a dataset
  cge_save        save the model and its simulations to a file
  cge_use         load a saved model
  cge_gams        write the model as a GAMS program; solve it with PATH, CONOPT or IPOPT

Data
  cge_synthetic   load the synthetic SAM used in tests and examples
```

### Example 2: a complete model

The shortest complete model maps the synthetic SAM, calibrates it, removes every import tariff, solves and displays the main results:

```
. cge_synthetic, clear
(16 vars, 15 obs)

Synthetic SAM loaded (standard layout): 15 accounts, total   1743.00
Largest row-column imbalance:  0.00e+00

. cge_sam, labels(account)
SAM read: 15 accounts, total 1743
Largest row-column imbalance:  0.00e+00

. cge_map, activities(act_agr act_man act_ser) commodities(com_agr com_man com_ser) ///
>     factors(lab cap) households(hhd) government(gov) savinv(sav) row(row)         ///
>     dirtax(tax_dir) saltax(tax_sal) imptax(tax_imp)
Accounts mapped: 3 activities, 3 commodities, 2 factors, 1 households, 0 enterprises, 0 margin accou
> nts

. cge_calibrate, sigmava(0.8) sigmaq(2) sigmat(2)
Calibrated: 93 equations, 93 free variables
Benchmark replication: largest scaled residual  3.55e-16

. cge_shock tm, value(0)
Parameter tm changed for 3 element(s)

. cge_solve, name(notariff)
Simulation notariff solved in 7 iteration(s) (newton)
Largest scaled residual  4.49e-14; WALRAS = -1.56e-14
```

```
. cge_results

Simulation notariff (closure: flexexr flexgsav savings cpi lab:mobile cap:mobile)
----------------------------------------------------
                                       base % change
----------------------------------------------------
Real value added (base prices)          262   -0.000
Exchange rate                             1    4.759
Consumer price index                      1    0.000
Investment volume (IADJ)                  1  -11.861
Imports (world prices)                  100    4.233
Exports (world prices)                   75    5.644
Government savings                        3 -271.374
Price of lab                              1    2.811
Price of cap                              1    2.631
Output of act_agr                       100   -0.409
Output of act_man                       140   -1.399
Output of act_ser                       150    1.240
----------------------------------------------------
Equivalent variation, hhd              6.137 ( 2.67% of consumption)
WALRAS -1.56e-14; GDP identity gap -6.82e-13
```

The synthetic SAMs that come with the package are constructed for testing and describe no country. Chapter 4 follows a SAM from its raw form to policy results.

### The Mata library

The commands of the package share a Mata library, compiled the first time a command runs in a Stata session. cge loads it if it is not in memory and checks that its version is that of the package. If it is not, as after an update of the package within a session, cge stops with error 498; restart Stata to load the new library.

## Stored results

cge stores the following in `r()`:

*Macros*

| | |
|---|---|
| `r(version)` | version of the package |

## References

Decaluwé, B., A. Lemelin, V. Robichaud, and H. Maisonnave. 2013. *PEP-1-1: The PEP Standard Single-Country, Static CGE Model*. Partnership for Economic Policy.

Lofgren, H., R. L. Harris, and S. Robinson. 2002. *A Standard Computable General Equilibrium (CGE) Model in GAMS*. Microcomputers in Policy Research 5. International Food Policy Research Institute.

## Also see

Chapter 1 — Introduction
Chapter 2 — A first model
Chapter 3 — The workflow
Chapter 7 — The equations of the standard model

# `cge_aggregate` — Merge SAM accounts into groups

## Description

`cge_aggregate` merges accounts of the SAM of the current model into groups and starts a new model from the result; any earlier mapping, calibration and simulations are discarded. The groups are given in the `group` column of a template written by **[CGE]** `cge_template`, or in the option `groups()`. Accounts that are not in a group keep their label.

Every cell of the new SAM is the sum of the cells it merges, so totals add up and each group's balance is the sum of its members' balances. Flows among the members of a group become a diagonal cell of the group; they are kept, unless `netdiagonal` is given.

## Quick start

Merge the manufacturing and service activities and commodities

```
cge_aggregate, groups(act_nag: act_man act_ser, com_nag: com_man com_ser)
```

Merge two households and remove every diagonal cell of the new SAM

```
cge_aggregate, groups(hh: hhd npi) netdiagonal
```

Merge the accounts as the `group` column of a template says, and map the result with the same template

```
cge_aggregate using sam_roles.xlsx
cge_map using sam_roles.xlsx
```

## Syntax

Groups from a template

**`cge_aggregate`** `using` *template* [`,` `sheet(`*string*`)` `netdiagonal`]

Groups given as an option

**`cge_aggregate`**`,` `groups(`*new* `:` *old old* ... `,` *new* `:` *old* ...`)` [`netdiagonal`]

*template* ends in `.xlsx`, `.xls`, `.csv` or `.dta`.

| *options* | Description |
|---|---|
| *Main* | |
| `groups(`*spec*`)` | the groups and the accounts they merge |
| `sheet(`*string*`)` | worksheet of an Excel template; default `accounts`, or else the first sheet |
| `netdiagonal` | remove the diagonal cells of the new SAM |

## Options

`groups(`*spec*`)` lists the groups, separated by commas, each as a new label, a colon and the accounts it merges, for example `groups(hh: hhd npi)`. An account may be in only one group. A group label must be a valid Stata name; it may be the label of one of its members, but not of another account. The same rule applies to the groups of a template. `groups()` and a template may not be combined.

`sheet(`*string*`)` names the worksheet of an Excel template. The default is the sheet `accounts`, which **[CGE]** `cge_template` writes, or else the first sheet.

`netdiagonal` removes every diagonal cell of the new SAM, including those of accounts that are not merged. Totals then fall by the diagonal cells, and balances do not change.

## Remarks and examples

Aggregation reduces the number of accounts of a SAM, for example to the sectors of a policy question, or to a size at which every account has flows large enough to calibrate the model. Each group takes the position of its first member in the SAM. The number of accounts merged, the number of groups and any diagonal cells are displayed, followed by the size, total and largest imbalance of the new SAM, as **[CGE]** `cge_sam` displays them.

### Example 1: merging accounts

In the institutions layout of the synthetic SAM, the manufacturing and service activities and commodities are merged, and so are the household and the non-profit institution:

```
. cge_synthetic, clear institutions
(18 vars, 17 obs)

Synthetic SAM loaded (institutions layout): 17 accounts, total   1866.00
Largest row-column imbalance:  0.00e+00

. cge_sam, labels(account)
SAM read: 17 accounts, total 1866
Largest row-column imbalance:  0.00e+00

. cge_aggregate, groups(act_nag: act_man act_ser, com_nag: com_man com_ser, hh: hhd npi)
Accounts merged: 6 accounts into 3 groups; 11 accounts left as they are
Diagonal cells (flows within an account): 3, total 8, kept
Aggregated SAM: 14 accounts, total 1866
Largest row-column imbalance:  0.00e+00
```

The new SAM has 14 accounts and the same total. Three of its diagonal cells are non-zero:

```
. preserve

. cge_samexport, clear
SAM loaded as data: 14 observations

. list account act_agr act_nag com_agr com_nag hh ent gov, noobs sep(0)

  +-------------------------------------------------------------------+
  | account   act_agr   act_nag   com_agr   com_nag    hh   ent   gov |
  |-------------------------------------------------------------------|
  | act_agr         0         0       100         0     0     0     0 |
  | act_nag         0         0         0       290     0     0     0 |
  | com_agr        10        25         0         0    70     0     0 |
  | com_nag        13        80         0         0   160     0    45 |
  |     lab        40        95         0         0     0     0     0 |
  |     cap        37        90         0         0     0     0     0 |
  |      hh         0         0         0         0     3    64    18 |
  |     ent         0         0         0         0     3     2     5 |
  |     gov         0         0         0         0     0     0     3 |
  | tax_dir         0         0         0         0    18    12     0 |
  | tax_sal         0         0         8        22     0     0     0 |
  | tax_imp         0         0         2         6     0     0     0 |
  |     sav         0         0         0         0     5    18    -2 |
  |     row         0         0        20        80     4     8     2 |
  +-------------------------------------------------------------------+

. restore
```

The diagonal cell of `hh`, 3, is the household's transfer to itself in the original SAM, 1, and its transfer to the non-profit institution, 2. The other two, the transfers of the enterprise and of the government to themselves, were in the original SAM. The model accepts diagonal cells for households, enterprises and the government, as transfers within the account, and the new SAM maps:

```
. cge_map, activities(act_agr act_nag) commodities(com_agr com_nag) factors(lab cap) ///
>     households(hh) enterprises(ent) government(gov) savinv(sav) row(row)         ///
>     dirtax(tax_dir) saltax(tax_sal) imptax(tax_imp)
Accounts mapped: 2 activities, 2 commodities, 2 factors, 1 households, 1 enterprises, 0 margin accou
> nts
```

**[CGE]** `cge_map` reports a diagonal cell of any other account as a flow that the model does not support.

### Example 2: removing the flows within groups

With `netdiagonal`, every diagonal cell of the new SAM is removed, and the total falls by their sum:

```
. cge_sam, labels(account)
SAM read: 17 accounts, total 1866
Largest row-column imbalance:  0.00e+00

. cge_aggregate, groups(hh: hhd npi) netdiagonal
Accounts merged: 2 accounts into 1 group; 15 accounts left as they are
Diagonal cells (flows within an account): 3, total 8, removed
Aggregated SAM: 16 accounts, total 1858
Largest row-column imbalance:  0.00e+00

. return list

scalars:
           r(diagonal) =  8
         r(N_diagonal) =  3
            r(N_blank) =  0
            r(maxdiff) =  0
              r(total) =  1858
         r(N_accounts) =  16
```

The SAM stays balanced: a diagonal cell is in both the row and the column of its account.

### Example 3: groups from a template

Example 2 of **[CGE]** `cge_template` fills in the `group` column of a template, aggregates the SAM with it and maps the result with the same template. When the template has roles, the members of each group must have the same role. A template written from a mapping that lists the non-profit institution as a household and `ent` as an enterprise cannot merge them:

```
. cge_sam, labels(account)
SAM read: 17 accounts, total 1866
Largest row-column imbalance:  0.00e+00

. quietly cge_map, activities(act_agr act_man act_ser) commodities(com_agr com_man com_ser) ///
>     factors(lab cap) households(hhd npi) enterprises(ent) government(gov) savinv(sav)      ///
>     row(row) dirtax(tax_dir) saltax(tax_sal) imptax(tax_imp)

. cge_template using inst.csv, replace
Template of 17 accounts, with roles from the current mapping
Template saved to inst.csv

. import delimited using inst.csv, clear stringcols(_all)
(7 vars, 17 obs)

. quietly replace group = "dom" if inlist(account, "hhd", "npi", "ent")

. export delimited account role pair group using inst.csv, replace
file inst.csv saved

. cge_aggregate using inst.csv
accounts hhd and ent of group dom have different roles

r(459);
```

After aggregation, the template's accounts are no longer the accounts of the SAM; `cge_aggregate using` the same template again is an error, while `cge_map using` it maps each group with its members' role.

## Stored results

`cge_aggregate` stores the following in `r()`:

*Scalars*

| | |
|---|---|
| `r(N_accounts)` | number of accounts after aggregation |

| | |
|---|---|
| `r(total)` | sum of all cells |
| `r(maxdiff)` | largest difference between an account's row and column totals |
| `r(N_diagonal)` | number of non-zero diagonal cells, before `netdiagonal` |
| `r(diagonal)` | their total |
| `r(N_blank)` | always 0 |

## Methods and formulas

Let $\bar{X}$ be the SAM of $n$ accounts before aggregation and $G$ the indicator matrix of $m$ groups by $n$ accounts, with $G_{gk} = 1$ if account $k$ is in group $g$ (an account in no group is a group of its own). The aggregated SAM is

$$X = G\bar{X}G', \qquad x_{gh} = \sum_{i\in g}\sum_{j\in h} \bar{x}_{ij},$$

so that each cell is the sum of the cells it merges. The row and column totals of a group are the sums of its members' totals, and so is its balance:

$$\sum_h x_{gh} - \sum_h x_{hg} = \sum_{k\in g}\Big(\sum_j \bar{x}_{kj} - \sum_i \bar{x}_{ik}\Big).$$

The diagonal cell $x_{gg}$ holds the flows among the members of $g$ and any diagonal cells of its members. With `netdiagonal`, the SAM is $X - \operatorname{diag}(x_{11}, \ldots, x_{mm})$; each account's row and column totals both fall by its diagonal cell, so balances do not change. Aggregation is described in section 24.4.

## Also see

**[CGE]** `cge_template` — Write a template that maps SAM accounts to roles and groups
**[CGE]** `cge_balance` — Balance a SAM by RAS or cross entropy
**[CGE]** `cge_map` — Assign SAM accounts to model roles
**[CGE]** `cge_samexport` — Write the SAM to a file or to memory
**Chapter 24** — SAM tools

# `cge_balance` — Balance a SAM by RAS or cross entropy

## Description

`cge_balance` makes the SAM of the current model balanced, so that every account's row total equals its column total, and starts a new model from the result; any earlier mapping, calibration and simulations are discarded. Two methods are available: RAS, which sets every account's total to a target, and cross entropy, which estimates the totals that are not known together with the cells. Both keep the sign of every cell, every zero cell and the cells listed in `fixed()`, and both minimise the same measure of the information added to the SAM.

## Quick start

Balance the SAM by RAS, with each account's target the average of its row and column totals

```
cge_balance
```

As above, with the row totals as targets and two totals given

```
cge_balance, targets(rows) totals(act_agr:97 hhd:290)
```

Balance by cross entropy, with the household's total known, keeping every payment to the rest of the world and every import tariff

```
cge_balance, method(ce) totals(hhd:290) fixed(row:* tax_imp:*)
```

## Syntax

**`cge_balance`** [, *options*]

| *options* | Description |
|---|---|
| *Main* | |
| `method(`*method*`)` | `ras` or `ce`; default `ras` |
| `targets(`*rule*`)` | RAS targets of the accounts not in `totals()`: `average`, `rows` or `columns`; default `average` |
| `totals(`*account* :*value* ...`)` | known account totals |
| `fixed(`*list*`)` | cells kept at their values |
| *Iteration* | |
| `tolerance(#)` | tolerance, relative to the largest absolute cell; default `1e-10` |
| `iterate(#)` | maximum number of iterations; default 10,000 for RAS and 100 for cross entropy |
| *Reporting* | |
| `nodisplay` | suppress the table of the largest changes |

## Options

`method(`*method*`)` chooses the method: `ras`, generalised RAS (the default), or `ce`, cross entropy.

`targets(`*rule*`)` sets the RAS targets of the accounts not given in `totals()`: `average`, the average of the account's row and column totals (the default); `rows`, its row total; or `columns`, its column total. It applies only to RAS: with `method(ce)`, `rows` and `columns` are errors, and known totals are given in `totals()`.

`totals(`*account* :*value* ...`)` gives known account totals, separated by spaces: targets for RAS, constraints for cross entropy. For example, `totals(act_agr:97 hhd:290)` gives the totals of two accounts. An account may be given only once.

`fixed(`*list*`)` lists, separated by spaces, the cells kept at their values: a cell *row*`:`*col*, a whole row *row*`:*`, a whole column `*:`*col*, or an account's row and column, *account*. Every account named must be in the SAM; `*:*` is an error.

`tolerance(#)` sets the convergence tolerance: every balance and every target or known total is met to # times the larger of one and the largest absolute cell of the SAM before balancing. The default is `1e-10`, well within the balance that **[CGE]** `cge_map` requires.

`iterate(#)` sets the maximum number of iterations: RAS iterations, each a row and a column step, or Newton steps of cross entropy. The default is 10,000 for RAS and 100 for cross entropy.

`nodisplay` suppresses the table of the cells with the largest changes.

## Remarks and examples

SAMs assembled from national accounts, supply–use tables, household surveys and government budgets rarely balance. `cge_balance` adjusts the cells so that every account balances, changing the SAM as little as possible in the sense of the measure given in Methods and formulas. Neither method changes the sign of a cell or turns a zero into a flow, so structural zeros stay zero, and cells in `fixed()` keep their values. The methods differ in what is known about the totals:

- RAS needs a target for every account's total. The default target is the average of the account's row and column totals; `targets()` and `totals()` choose others. Negative cells are handled by generalised RAS (GRAS).
- Cross entropy needs no targets. It estimates every total that is not given in `totals()` together with the cells, subject to balance. With every total given, it gives the RAS solution.

`cge_balance` displays the number of iterations, the largest imbalance and the SAM total before and after, the number of cells changed and the largest relative change of a cell, and the ten cells with the largest absolute changes, followed by the size, total and imbalance of the new SAM, as **[CGE]** `cge_sam` displays them.

The balanced SAM is an estimate. Which totals were given, which cells were fixed and which method was used should be stated with any results from it.

### Example 1: RAS

The unbalanced synthetic SAM is out of balance by up to 12. RAS to the average totals balances it:

```
. cge_synthetic, clear unbalanced
(18 vars, 17 obs)

Synthetic SAM loaded (institutions layout, unbalanced for the balancing tests): 17 accounts, total
>  1860.46
Largest row-column imbalance:  1.21e+01

. cge_sam, labels(account)
SAM read: 17 accounts, total 1860.4569
Largest row-column imbalance:  1.21e+01

. cge_balance
RAS: converged in 468 iterations
Largest row-column imbalance:  1.21e+01 before,  1.53e-08 after
SAM total: 1860.4569 before, 1860.4569 after
Cells changed: 68 of 68 non-zero; largest relative change  2.10e-01

Largest changes
  row                  column                       before          after   change %
  act_ser              com_ser                     158.729        152.686     -3.807
  cap                  act_ser                      49.801        52.7177      5.857
  hhd                  ent                          61.883        64.6935      4.542
  row                  com_ser                     21.1036        23.5877     11.771
  ent                  cap                         94.6853        92.6413     -2.159
  com_man              hhd                         81.1112        79.0956     -2.485
  row                  cap                          10.326        8.75133    -15.250
  com_man              sav                         37.8098        39.2907      3.917
  lab                 act_ser                            56.475        57.7965      2.340
  com_ser             act_ser                            19.451        20.5922      5.867

Balanced SAM: 17 accounts, total 1860.4569
Largest row-column imbalance:  1.53e-08
```

The largest imbalance after balancing, `1.53e-08`, is within the tolerance: `1e-10` times the largest cell, 158.729. With the average targets, the SAM total is unchanged, since the targets add up to it. Every non-zero cell may change, and every one does.

### Example 2: cross entropy with known totals and fixed cells

Suppose that the output of agriculture, 97, and household income, 290, are known, and that the payments to the rest of the world and households' labour income are taken as reliable. Cross entropy estimates every other total:

```
. cge_sam, labels(account)
SAM read: 17 accounts, total 1860.4569
Largest row-column imbalance:  1.21e+01

. cge_balance, method(ce) totals(act_agr:97 hhd:290) fixed(row:* hhd:lab)
Cross entropy: converged in 4 iterations
Largest row-column imbalance:  1.21e+01 before,  1.80e-10 after
SAM total: 1860.4569 before, 1954.96573 after
Cells changed: 58 of 68 non-zero; largest relative change  8.92e-01

Largest changes
  row                 column                             before          after   change %
  hhd                 ent                                61.883        80.3948     29.914
  com_ser             hhd                               74.1971        84.9784     14.531
  cap                 act_ser                            49.801        60.3448     21.172
  com_man             hhd                               81.1112        91.3107     12.575
  hhd                 cap                               25.3804        34.7014     36.725
  hhd                 gov                               15.8542        23.5792     48.725
  cap                 act_man                           38.3355        45.4594     18.583
  com_agr             hhd                               66.3231        72.9289      9.960
  hhd                 row                                9.4004         15.103     60.663
  act_man             com_man                            138.47        143.523      3.649

Balanced SAM: 17 accounts, total 1954.96573
Largest row-column imbalance:  1.80e-10
```

The eight fixed cells, the seven non-zero payments to the rest of the world and `hhd:lab`, keep their values, and so do the transfers of the enterprise and of the government to themselves: a diagonal cell is in both the row and the column of its account, and balance alone does not change it. The other 58 non-zero cells change. Household income of 290 is above both the household's row total, 247.9, and its column total, 255.1, in the unbalanced SAM, so its receipts and its payments rise, and the SAM total with them.

The results are stored in `r()`:

```
. return list

scalars:
       r(maxrelchange) =  .8918061647399571
          r(N_changed) =  58
       r(total_before) =  1860.4569
     r(maxdiff_before) =  12.08590000000001
         r(iterations) =  4
            r(N_blank) =  0
            r(maxdiff) =  1.79795733857e-10
              r(total) =  1954.965727111547
         r(N_accounts) =  17

macros:
             r(method) : "ce"
```

### Example 3: a SAM that cannot be balanced

Zeros and fixed cells may make the targets impossible to meet. Fixing the household's row and column leaves its two totals as they are, and they differ:

```
. cge_sam, labels(account)
SAM read: 17 accounts, total 1860.4569
Largest row-column imbalance:  1.21e+01

. cge_balance, fixed(hhd)
hhd: its row and column cannot change and their totals differ

r(459);
```

`cge_balance` checks such contradictions before it starts: an account whose row and column cannot change but differ, or a given total that differs from the total of cells that cannot change. Other contradictions arise from the pattern of zeros and fixed cells. For example, in a SAM whose activities and product-tax accounts are paid only by commodities, fixing every import cell fixes the difference between the commodities' totals and theirs; RAS targets that do not respect it cannot be met, and RAS stops with error 430 when its gap no longer falls. Cross entropy, which needs fewer totals, can often balance such a SAM.

## Stored results

`cge_balance` stores the following in `r()`:

| | |
|---|---|
| *Scalars* | |
| `r(iterations)` | RAS iterations or Newton steps of cross entropy |
| `r(maxdiff_before)` | largest row–column imbalance before balancing |
| `r(maxdiff)` | largest row–column imbalance after balancing |
| `r(total_before)` | sum of all cells before balancing |
| `r(total)` | sum of all cells after balancing |
| `r(N_changed)` | number of cells changed |
| `r(maxrelchange)` | largest relative change of a cell |
| `r(N_accounts)` | number of accounts |
| `r(N_blank)` | always 0 |
| *Macros* | |
| `r(method)` | `ras` or `ce` |

## Methods and formulas

Let $\bar{X}$ be the SAM before balancing and $X$ the SAM after it. A cell may change if it is not zero and not in `fixed()`; let $Z$ be the set of such cells, and write $P$ for the positive cells of $\bar{X}$ in $Z$, $N$ for the absolute values of its negative cells in $Z$, and $F$ for all other cells, which keep their values. Both methods minimise

$$D(X, \bar{X}) = \sum_{(i,j)\in Z} \left( |x_{ij}| \ln \frac{|x_{ij}|}{|\bar{x}_{ij}|} - |x_{ij}| + |\bar{x}_{ij}| \right),$$

a measure of the information added to $\bar{X}$ (Junius and Oosterhaven 2003; Lenzen, Wood and Gallego 2007), which for a SAM without negative cells is the cross entropy that RAS minimises (McDougall 1999). It is not the column-coefficient cross entropy of Robinson, Cattaneo and El-Said (2001), which measures the distance between the column shares of $X$ and $\bar{X}$; that method is not implemented. Convergence is measured against $\sigma = \max(1, \max_{ij} |\bar{x}_{ij}|)$: the iterations stop when every balance, target and known total is met to `tolerance()` $\times\, \sigma$.

*RAS.* Every account $k$ has a target total $t_k$: the average of its row and column totals in $\bar{X}$, one of them as `targets()` says, or the value in `totals()`. If an account's row has no cell that may change, its target is its row total, and likewise for its column; if both are fixed and differ, or a given total contradicts them, there is no balanced SAM and `cge_balance` stops. With $f_i^r = \sum_j f_{ij}$ and $f_j^c = \sum_i f_{ij}$, the row and column totals of the cells that may change must be $u_i = t_i - f_i^r$ and $v_j = t_j - f_j^c$. Generalised RAS finds row factors $r_i$ and column factors $s_j$ such that

$$x_{ij} = r_i\, p_{ij}\, s_j - \frac{n_{ij}}{r_i\, s_j} + f_{ij}$$

has these totals. Starting from $s = 1$, it alternates a row step and a column step. The row step solves $r_i p_i - n_i / r_i = u_i$ for each row, with $p_i = \sum_j p_{ij} s_j$ and $n_i = \sum_j n_{ij} / s_j$:

$$r_i = \frac{u_i + \sqrt{u_i^2 + 4 p_i n_i}}{2 p_i} \quad (u_i \geq 0), \qquad r_i = \frac{2 n_i}{\sqrt{u_i^2 + 4 p_i n_i} - u_i} \quad (u_i < 0),$$

two forms of the same root that avoid cancellation. The column step is the same with the roles of rows and columns exchanged. Without negative cells this is RAS. A factor that is not positive and finite means that the target cannot be met with the signs of the cells, and `cge_balance` stops. After 100 iterations, RAS also stops, with error 430, when the gap has not fallen by a relative $10^{-6}$ over the last 100 iterations.

*Cross entropy.* Let $K$ be the accounts in `totals()`. $X$ minimises $D(X, \bar{X})$ subject to

$$\sum_j x_{kj} - \sum_i x_{ik} = 0 \quad \text{for every account } k, \qquad \sum_j x_{kj} = t_k \quad \text{for } k \in K.$$

The solution has the form

$$x_{ij} = p_{ij}\, e^{a_{ij}} - n_{ij}\, e^{-a_{ij}} + f_{ij}, \qquad a_{ij} = \mu_i - \mu_j + \rho_i,$$

with a multiplier $\mu_k$ for the balance of each account and $\rho_k$ for each known total ($\rho_k = 0$ for $k \notin K$). The multipliers minimise the convex dual

$$\varphi(\mu, \rho) = \sum_{ij} p_{ij}\, e^{a_{ij}} + \sum_{ij} n_{ij}\, e^{-a_{ij}} + \sum_k \mu_k (f_k^r - f_k^c) + \sum_{k \in K} \rho_k (f_k^r - t_k),$$

whose gradient is the vector of the balances and of the differences from the known totals, and whose Hessian is

$$H = \sum_{ij} (p_{ij}\, e^{a_{ij}} + n_{ij}\, e^{-a_{ij}})\, d_{ij} d_{ij}', \qquad d_{ij} = \frac{\partial a_{ij}}{\partial(\mu, \rho)}.$$

Newton's method minimises $\varphi$, with a backtracking line search that halves the step until $\varphi$ falls by at least $10^{-4}$ times the decrease the step predicts. Adding a constant to every $\mu_k$ in a set of accounts connected by cells that may change leaves $X$ unchanged, so one $\mu$ in each such set is fixed at zero; the fixed cells of each set must balance among themselves. A known total whose row or column cannot change must equal the total of its fixed cells, and needs no multiplier. If Newton's method does not converge, no balanced SAM with the same pattern of signs and zeros may exist. When every total is known, the solution is the RAS solution, with $r_i = e^{\mu_i + \rho_i}$ and $s_j = e^{-\mu_j}$.

*Results.* The number of cells changed counts the cells with $x_{ij} \neq \bar{x}_{ij}$, and the largest relative change is $\max_{ij} |x_{ij} - \bar{x}_{ij}| / |\bar{x}_{ij}|$ over the non-zero cells of $\bar{X}$. Balancing is described in section 24.5, RAS in section 24.5.1 and cross entropy in section 24.5.2.

## References

Junius, T., and J. Oosterhaven. 2003. The solution of updating or regionalizing a matrix with both positive and negative entries. *Economic Systems Research* 15: 87–96.

Lenzen, M., R. Wood, and B. Gallego. 2007. Some comments on the GRAS method. *Economic Systems Research* 19: 461–465.

McDougall, R. A. 1999. Entropy theory and RAS are friends. GTAP Working Paper 6, Center for Global Trade Analysis, Purdue University.

Robinson, S., A. Cattaneo, and M. El-Said. 2001. Updating and estimating a social accounting matrix using cross entropy methods. *Economic Systems Research* 13: 47–64.

## Also see

**[CGE]** `cge_sam` — Read a social accounting matrix
**[CGE]** `cge_aggregate` — Merge SAM accounts into groups
**[CGE]** `cge_samexport` — Write the SAM to a file or to memory
**[CGE]** `cge_map` — Assign SAM accounts to model roles
**Chapter 24** — SAM tools

# cge_calibrate — Calibrate the model to the SAM

## Description

`cge_calibrate` computes the base value of every variable and every parameter of the model from the SAM mapped by **[CGE]** `cge_map` and the elasticities given, sets the default closure, and checks that the base solves every equation of the model (benchmark replication). It stops with an error if replication fails.

Elasticities have no defaults: they are part of the specification of the model and must be stated. **[CGE]** `cge_elasticities` gives published values, with their sources, as values for the options of `cge_calibrate`.

## Quick start

Calibrate with one elasticity of each kind for every sector

```
cge_calibrate, sigmava(0.8) sigmaq(2) sigmat(2)
```

As above, with one value of `sigmava()` per activity and of `sigmaq()` and `sigmat()` per commodity, in the order of **[CGE]** `cge_map`

```
cge_calibrate, sigmava(0.5 1 1.5) sigmaq(0.6 3 1.2) sigmat(0.7 4 1)
```

Linear expenditure system demand, with income elasticities by commodity and a Frisch parameter

```
cge_calibrate, sigmava(0.8) sigmaq(2) sigmat(2) incelas(0.6 1 1.4) frisch(-2)
```

A labour force for `lab`, from a base unemployment rate of 15 per cent, with a wage curve

```
cge_calibrate, sigmava(0.8) sigmaq(2) sigmat(2) unemployment(lab:0.15) wagecurve(lab:0.1)
```

Armington elasticities from the elasticity library

```
cge_elasticities sigmaq, country(TGO) values
cge_calibrate, sigmava(0.8) sigmaq(`r(values)') sigmat(2)
```

## Syntax

`cge_calibrate, sigmava(`*numlist*`) sigmaq(`*numlist*`) sigmat(`*numlist*`)` [*options*]

| *options* | Description |
|---|---|
| *Main* | |
| `sigmava(`*numlist*`)` | substitution between factors in value added, one value or one per activity; required |
| `sigmaq(`*numlist*`)` | substitution between imports and domestic goods (Armington), one value or one per commodity; required |
| `sigmat(`*numlist*`)` | transformation between exports and domestic sales (CET), one value or one per commodity; required |
| `fquantities(`*matname*`)` | base factor quantities, one row per factor and one column per activity |
| *Production* | |
| `sigmaa(`*numlist*`)` | substitution between value added and intermediate input, one value or one per activity |
| `nests(`*spec*`)` | nests of value added and their members |
| `sigmanest(`*spec*`)` | substitution within the nests |

| *options* | Description |
|---|---|
| `sigmaac(`*spec*`)` | substitution between the outputs of a commodity from several activities |
| *Household demand* | |
| `incelas(`*numlist*`)` | income elasticities, one value or one per commodity |
| `frisch(`*numlist*`)` | Frisch parameter, one value or one per household |
| *Fiscal instruments* | |
| `vatcredit(`*spec*`)` | share of the VAT on its inputs that each activity recovers |
| `debt(`*spec*`)` | public debt at home and abroad at the start of the base year |
| `interest(`*spec*`)` | interest rates on the debt at home and abroad |
| `debtholders(`*spec*`)` | shares of domestic interest received by households and enterprises |
| `foreignborrowing(#)` | share of the deficit borrowed abroad in dynamic runs |
| *Trade* | |
| `exportonly(`*namelist*`)` | commodities whose domestic output is all exported |
| `exportdemand(`*spec*`)` | elasticities of export demand |
| *Labour market* | |
| `unemployment(`*spec*`)` | base unemployment rates by factor |
| `wagecurve(`*spec*`)` | wage-curve elasticities by factor |
| `minwage(`*spec*`)` | minimum real wages, as shares of the base real wage |
| `smoothing(#)` | smoothing parameter of the minimum-wage condition; default `1e-10` |
| *Health* | |
| `healthcom(`*namelist*`)` | health commodities |
| `healthfac(`*namelist*`)` | factors whose efficiency depends on health services |
| `healthelas(`*numlist*`)` | elasticities of factor efficiency with respect to health services |
| `sickdays(`*spec*`)` | base shares of working time lost to illness by factor |
| `sickelas(`*spec*`)` | elasticities of the share of time lost |
| `mortality(#)` | base crude death rate |
| `mortelas(#)` | elasticity of the death rate |

## Options

### Main

`sigmava(`*numlist*`)` gives the elasticity of substitution between factors in value added: one value for every activity, or one value per activity in the order of `activities()` of **[CGE]** `cge_map`. A value of 1 selects the Cobb–Douglas form. Values must be positive.

`sigmaq(`*numlist*`)` gives the elasticity of substitution between imports and domestic goods in composite supply (the Armington elasticity), one value or one per commodity, in the order of `commodities()`. Values must be positive, and different from 1 for every commodity that has both imports and domestic sales: the model has no Cobb–Douglas form of the Armington function. The values of commodities without imports, or without domestic sales, are not used.

`sigmat(`*numlist*`)` gives the elasticity of transformation between exports and domestic sales (CET), one value or one per commodity. Values must be positive. For a commodity whose exports are at least its domestic output (re-exports, or exports supplied from inventories), exports are drawn from composite supply instead, and its `sigmat()` value is the price elasticity of those exports in equation (T9); calibration lists such commodities. The values of commodities without exports, and of commodities in `exportonly()`, are not used.

`fquantities(`*matname*`)` supplies base factor quantities, for example employment, as a Stata matrix with one row per factor and one column per activity, in the order of **[CGE]** `cge_map`. Quantities must be positive exactly where the SAM has factor payments. The base price of each factor is then its total payments over its total quantity, and the wage distortion factors make each payment equal to price times quantity. Without `fquantities()`, factor prices are 1 in the base and quantities equal payments.

### Production

`sigmaa(`*numlist*`)` gives the elasticity of substitution between value added and aggregate intermediate input, the top nest of production, one value or one per activity: 0 is the Leontief nest, 1 the Cobb–Douglas form, and other positive values a CES nest, under which an activity uses relatively less of whichever input becomes dearer. Without `sigmaa()`, the top nest is Leontief in every activity. Activities without intermediate inputs keep the Leontief nest, and are listed in a note.

`nests(`*spec*`)` defines nests of value added, separated by commas, each a name followed by a colon and its members, factors or commodities, for example `nests(labour: lab_hi lab_lo, ke: cap com_ene)`. Each nest is a CES or Cobb–Douglas aggregate of its members, and value added combines the factors in no nest with the nests, with the elasticity `sigmava()`. A commodity in a nest, such as energy, leaves intermediate input: activities substitute between it and the other members of its nest. A factor or commodity may be in one nest only, nests do not contain other nests, and a nest's name may not be that of a factor, a commodity or another nest. Results list the quantity (`QNA`) and price (`PNA`) of each nest in each activity (section 18.2).

`sigmanest(`*spec*`)` gives the elasticities of substitution within the nests, one positive value for every nest or *nest*`:`*value* pairs for each. A value of 1 selects the Cobb–Douglas form. `nests()` and `sigmanest()` must be given together.

`sigmaac(`*spec*`)` combines the outputs of a commodity from several activities by a CES function, instead of treating them as perfect substitutes, so that each activity is paid its own price for the commodity (`PXAC`; section 18.3). Give *commodity*`:`*value* pairs, for example `sigmaac(com_man:2)`, for commodities produced by more than one activity, or one value for every such commodity. Values must be positive; a value of 1 selects the Cobb–Douglas form.

### Household demand

`incelas(`*numlist*`)` and `frisch(`*numlist*`)` select linear expenditure system (LES) demand instead of Cobb–Douglas demand. `incelas()` gives positive income elasticities, one value or one per commodity; they are scaled so that the marginal budget shares of each household sum to one, which keeps their ratios. `frisch()` gives the Frisch parameter, one value or one per household, below $-1$; supernumerary spending is then $-1/$*frisch* of consumption spending. `frisch()` is required with `incelas()`; given alone, it sets every income elasticity to 1. Household consumption must be non-negative. Home consumption of the output of an activity takes the income elasticity of the activity's paired commodity. Under LES demand, the equivalent variation reported by **[CGE]** `cge_results` uses the LES expenditure function.

### Fiscal instruments

`vatcredit(`*spec*`)` gives, for a model with a VAT account (see `vattax()` and `vatshare()` of **[CGE]** `cge_map`), the share of the VAT on its inputs that each activity recovers as a credit: *activity*`:`*share* pairs or one share for every activity, between 0 and 1. Activities not listed have share 1, full credit, so that VAT falls on final demand only; a share of 0 is an exempt or unregistered activity, which bears VAT on its inputs. The VAT rate of each commodity is calibrated so that the VAT on its uses, net of credits, equals the VAT that the SAM records (section 13.2).

`debt(`*spec*`)` and `interest(`*spec*`)` give public debt at the start of the base year and its interest rates, as `domestic:#` and `foreign:#`; a part not given is zero. Domestic debt is in SAM units, foreign debt in foreign currency (equal to SAM units at the base exchange rate). Debt must be non-negative and interest rates above $-1$. Interest is taken to be recorded within the government's transfers to domestic institutions and abroad; it is separated from them and then paid in nominal terms on the debt, so that the base is unchanged and interest rates and debt stocks can be shocked. A transfer smaller than the interest it must contain is an error. In dynamic runs (**[CGE]** `cge_dynamic`) the deficit adds to the debt (section 13.3).

`debtholders(`*spec*`)` gives the shares of domestic interest received by households and enterprises, as *institution*`:`*share* pairs that sum to one. By default the shares are proportional to the government's transfers to each.

`foreignborrowing(#)` is the share of the deficit borrowed abroad in dynamic runs, between 0 and 1. By default it is the foreign share of the base debt.

### Trade

`exportonly(`*namelist*`)` lists commodities whose domestic output is all exported: their exports, net of export taxes and export margins, must equal their domestic output. Their output is then exported directly, at the export price, without domestic sales, and may pay export taxes (section 14.3). Without this option such a commodity's exports are drawn from composite supply; calibration lists, in a note, the commodities whose exports equal domestic output.

`exportdemand(`*spec*`)` gives elasticities of export demand, as *commodity*:*elasticity* pairs or one value for every commodity exported from domestic output. The world export price of these commodities then falls as more is exported, and a shock to `pwe` shifts world demand (section 14.2). Without `exportdemand()`, the country is small in world markets: it exports any quantity at a fixed world price.

### Labour market

`unemployment(`*spec*`)` gives base unemployment rates, as *factor*:*rate* pairs with rates at least 0 and below 1, for example `unemployment(lab:0.15)`. SAMs record employment, not unemployment, so the rate comes from a labour force survey. The factor's labour force is then its base employment divided by one minus the rate; reports show the labour force and the unemployment rate, and the closures `wagecurve` and `minwage` of **[CGE]** `cge_closure` become available. A factor with unemployment in the base has a minimum wage at its base real wage (chapter 15).

`wagecurve(`*spec*`)` gives positive wage-curve elasticities, as *factor*:*elasticity* pairs, for factors with unemployment in the base: under the closure `wagecurve` the real wage falls by about *elasticity* per cent for each per cent rise in the unemployment rate (Blanchflower and Oswald 1994). A small elasticity approaches a fixed real wage, a large one full employment.

`minwage(`*spec*`)` gives minimum real wages as shares of the base real wage, *factor*:*share*. With unemployment in the base the floor is the base real wage, a share of 1, the only value allowed. With full employment in the base (no rate, or a rate of 0, in `unemployment()`), give a share below 1, for example `minwage(lab:0.95)`: the floor then binds only after the real wage falls by 5 per cent.

`smoothing(#)` sets the smoothing parameter $\mu$ of the minimum-wage condition; the default is `1e-10`, and it must be positive. Under the closure `minwage`, the condition that either unemployment is zero or the real wage is at its floor is solved, with the rest of the model, in the smoothed Fischer–Burmeister form (Fischer 1992; Kanzow 1996), in which the product of the unemployment rate and the proportional excess of the real wage over the floor is $\mu$.

### Health

`healthcom(`*namelist*`)`, `healthfac(`*namelist*`)` and `healthelas(`*numlist*`)` link factor efficiency to health services. `healthcom()` lists the health commodities; `healthfac()` lists the factors whose efficiency depends on them, all factors by default; and `healthelas()` gives the elasticity of each factor's efficiency with respect to the volume of health services consumed by households, the government and the health insurance fund, one value or one per factor of `healthfac()`, zero or positive. With elasticity $e$, the factor's efficiency is $(HS/HS^0)^e$, where $HS$ is that volume and $HS^0$ its base value; the factor then enters value added in efficiency units, while its quantity, supply and wage stay in physical units. Without these options, or with elasticity 0, efficiency changes only through the parameter `af` of **[CGE]** `cge_shock`. `healthelas()` needs `healthcom()`, and `healthfac()` needs `healthelas()`. The health commodities must have some consumption in the base. These options must be typed in full.

`sickdays(`*spec*`)` gives the base share of working time lost to illness, as *factor*:*share* pairs with shares at least 0 and below 1, for example `sickdays(lab:0.05)` from a labour force or household survey. Days worked per worker then scale the factor's effective input in production; employment, wages and unemployment stay per worker (section 16.3).

`sickelas(`*spec*`)` gives the elasticity of the share of time lost with respect to health services per head, one value for every factor in `sickdays()` or *factor*:*elasticity* pairs for factors in `sickdays()`. With elasticity $\epsilon$, the share lost is $s\,R^{\epsilon}$, where $s$ is the share given in `sickdays()` and $R$ is health services per head relative to the base.

`mortality(#)` gives the base crude death rate, deaths per head per period, at least 0 and below 1 (for example 0.008 for 8 per 1,000 a year), and `mortelas(#)` its elasticity with respect to health services per head; `mortelas()` needs `mortality()`. The death rate is reported; it does not change the solution of a period, but in dynamic runs it changes the growth of the population and, with `deathfactors()` of **[CGE]** `cge_dynamic`, of labour from the next period (section 16.4).

Health outcomes with a non-zero elasticity need `healthcom()`. Their elasticities are hypotheses about how health services become health and should come from studies of the country or region; the package supplies none.

## Remarks and examples

Calibration chooses the parameters of the model so that the SAM is a solution of it, the base. `cge_calibrate`

1. sets the base prices of activities, commodities and value added, the exchange rate and world import prices to one, so that base quantities are values of the SAM, and sets factor prices to one unless `fquantities()` gives factor quantities;
2. computes tax rates, shares and the other parameters from the SAM and the elasticities;
3. checks the identities that a balanced SAM implies, for example that composite supply equals the demand for each commodity, and stops with error 459 if one fails;
4. sets the default closure: a flexible exchange rate with fixed foreign savings, flexible government savings, savings-driven investment, the consumer price index as numeraire, and mobile, fully employed factors (**[CGE]** `cge_closure`);
5. checks benchmark replication: every equation of the model must hold at the base, with a largest scaled residual of at most $10^{-9}$, or the command stops with error 9. User equations need not hold at the base, where they act as shocks; from version 0.7.1 replication leaves them out, and a note says when they do not hold.

Calibrating again replaces the calibration: stored simulations, shocks, constraints and closure swaps are discarded and the default closure is set. User variables and equations (**[CGE]** `cge_variable`, **[CGE]** `cge_equation`) are kept and compiled again from the new base, and user sets and parameters (**[CGE]** `cge_set`, **[CGE]** `cge_parameter`) are kept, the parameters at their base values.

Each elasticity option takes one value for every activity or commodity, or one value per activity (`sigmava()`, `sigmaa()`) or commodity (`sigmaq()`, `sigmat()`, `incelas()`) in the order of the lists of **[CGE]** `cge_map`. A failure of benchmark replication points to a problem in the SAM or in the mapping, not in the solver.

### Example 1: one elasticity of each kind

The synthetic SAM is mapped as in Example 1 of **[CGE]** `cge_map` and calibrated:

```
. cge_calibrate, sigmava(0.8) sigmaq(2) sigmat(2)
Calibrated: 93 equations, 93 free variables
Benchmark replication: largest scaled residual  3.55e-16
```

The model has 93 equations and, under the default closure, as many free variables. The largest scaled residual at the base is of the order of the machine precision. The results are stored in `r()`:

```
. return list

scalars:
       r(max_residual) =  3.55271367880e-16
        r(N_equations) =  93
```

### Example 2: elasticities by sector

The elasticities may differ by activity and by commodity; a value of 1 for `sigmava()` gives Cobb–Douglas value added in manufacturing:

```
. cge_calibrate, sigmava(0.5 1 1.5) sigmaq(0.6 3 1.2) sigmat(0.7 4 1)
Calibrated: 93 equations, 93 free variables
Benchmark replication: largest scaled residual  3.55e-16
```

A value of 1 for `sigmaq()` is not allowed for a commodity with imports and domestic sales:

```
. cge_calibrate, sigmava(0.8) sigmaq(1) sigmat(2)
sigmaq must differ from 1 for imported commodities with domestic sales

r(198);
```

### Example 3: linear expenditure system demand

With `incelas()` and `frisch()`, households first buy subsistence quantities and spend the rest of their budget in fixed marginal shares:

```
. cge_calibrate, sigmava(0.8) sigmaq(2) sigmat(2) incelas(0.6 1 1.4) frisch(-2)
Household demand: linear expenditure system
Calibrated: 93 equations, 93 free variables
Benchmark replication: largest scaled residual  3.55e-16
```

The model has the same number of equations; only the parameters of household demand differ.

### Example 4: exports at least as large as domestic output

In the supply-use layout of the synthetic SAM, `com_agr` exports 105 against domestic output of 100, with a fall in its stocks, and `com_fuel`, which is only imported, re-exports part of its imports:

```
. cge_synthetic, clear sut
(20 vars, 19 obs)

Synthetic SAM loaded (supply-use layout): 19 accounts, total   1952.00
Largest row-column imbalance:  0.00e+00

. cge_sam, labels(account)
Account label s-i is not a valid Stata name; using si
SAM read: 19 accounts, total 1952
Largest row-column imbalance:  0.00e+00

. quietly cge_map, activities(act_agr act_man act_ser)                    ///
>     commodities(com_agr com_man com_ser com_fuel) factors(lab cap)      ///
>     households(hhd) government(gov) savinv(si) row(row) dirtax(tax_dir) ///
>     saltax(tax_sal) imptax(tax_imp) marginaccounts(trd trp) stocks(dstk)

. cge_calibrate, sigmava(0.8) sigmaq(2) sigmat(2)
Calibrated: 103 equations, 103 free variables
Benchmark replication: largest scaled residual  3.55e-16
Exports drawn from composite supply (equation T9): com_agr com_fuel
```

The exports of these two commodities are drawn from composite supply, by equation (T9), and the `sigmat()` value is the price elasticity of those exports.

### Example 5: an extension

The options of the extensions add equations and parameters to the standard model. A base unemployment rate gives labour a labour force, and a wage-curve elasticity gives the closure `wagecurve` its slope:

```
. cge_calibrate, sigmava(0.8) sigmaq(2) sigmat(2) unemployment(lab:0.15) wagecurve(lab:0.1)
Labour force of lab: 158.823529; base unemployment rate .15; wage-curve elasticity .1; real minimum
> wage .999999999 (base real wage 1)
Calibrated: 95 equations, 95 free variables
Benchmark replication: largest scaled residual  3.55e-16
```

The labour force is base employment, 135, divided by $1 - 0.15$. The model has two more equations, the wage curve and the minimum-wage condition of `lab`; under the default closure, which keeps labour fully employed, both are inert, and `cge_closure, factors(lab:wagecurve)` or `factors(lab:minwage)` brings one of them into use. The real minimum wage is the base real wage divided by $1 + \mu/u^0$, where $u^0$ is the base unemployment rate, so that the smoothed condition holds at the base.

The options of the extensions, and the chapters of Part II that describe them:

| Extension | Options | Part II |
|---|---|---|
| top nest of production | `sigmaa()` | section 7.2 |
| nests of value added | `nests(), sigmanest()` | section 18.2 |
| output from several activities | `sigmaac()` | section 18.3 |
| VAT and public debt | `vatcredit(), debt(), interest(), debtholders(), foreignborrowing()` | chapter 13 |
| trade | `exportonly(), exportdemand()` | chapter 14 |
| labour market | `unemployment(), wagecurve(), minwage(), smoothing()` | chapter 15 |
| health | `healthcom(), healthfac(), healthelas(), sickdays(), sickelas(), mortality(), mortelas()` | chapter 16 |

## Stored results

`cge_calibrate` stores the following in `r()`:

*Scalars*

| | |
|---|---|
| `r(N_equations)` | number of equations |
| `r(max_residual)` | largest scaled residual at the base |

## Methods and formulas

Let $SAM(r, c)$ be the payment from column account $c$ to row account $r$ of the mapped SAM, after the rewrites of **[CGE]** `cge_map`. Base values carry a superscript 0. The base prices $PA^0_a = PX^0_c = PD^0_c = PE^0_c = PQ^0_c = PVA^0_a = 1$, $EXR^0 = 1$ and $pwm_c = 1$ (section 9.3); without `fquantities()`, $W^0_f = WFDIST^0_{fa} = 1$. Given base factor quantities $QF^0_{fa}$,

$$W^0_f = \frac{\sum_a SAM(f, a)}{\sum_a QF^0_{fa}}, \qquad WFDIST^0_{fa} = \frac{SAM(f, a)}{W^0_f\, QF^0_{fa}}.$$

The tax rates are taken from the tax accounts:

$$tm_c = \frac{SAM(imp, c)}{SAM(row, c)}, \qquad te_c = \frac{SAM(exp, c)}{SAM(c, row)}, \qquad ta_a = \frac{SAM(act, a)}{QA^0_a}, \qquad tq_c = \frac{SAM(sal, c)}{QQ^0_c},$$

where $QA^0_a$ is the column total of activity $a$ and, without margins by market and VAT, $QQ^0_c = QD^0_c + (1 + tm_c)\, QM^0_c + SAM(sal, c) + \sum_t SAM(t, c)$, the last sum over the margin accounts.

The elasticities enter the functional forms through the exponents

$$\rho^{va}_a = \frac{1}{\sigma^{va}_a} - 1, \qquad \rho^q_c = \frac{1}{\sigma^q_c} - 1, \qquad \rho^t_c = \frac{1}{\sigma^t_c} + 1,$$

and the share parameters follow from the first-order conditions at the base. For CES value added, with $w_{fa} = W^0_f\, WFDIST^0_{fa}$,

$$\delta^{va}_{fa} = \frac{w_{fa}\, (QF^0_{fa})^{1+\rho^{va}_a}}{\sum_{f'} w_{f'a}\, (QF^0_{f'a})^{1+\rho^{va}_a}}, \qquad ad_a = QVA^0_a \Big/ \Big[\sum_f \delta^{va}_{fa}\, (QF^0_{fa})^{-\rho^{va}_a}\Big]^{-1/\rho^{va}_a},$$

and for Cobb–Douglas value added $\delta^{va}_{fa} = SAM(f,a)/QVA^0_a$. The Armington and CET shares are

$$\delta^q_c = \frac{r^q_c}{1+r^q_c}, \quad r^q_c = \frac{PM^0_c}{PD^0_c}\left(\frac{QM^0_c}{QD^0_c}\right)^{1+\rho^q_c}; \qquad \delta^t_c = \frac{1}{1+r^t_c}, \quad r^t_c = \frac{PD^0_c}{PE^0_c}\left(\frac{QE^0_c}{QD^0_c}\right)^{\rho^t_c-1},$$

with the complements $1-\delta$ computed from the same ratios, not by subtraction, so that no precision is lost when a share is close to one.

Under LES demand, with budget shares $w_{ch} = SAM(c,h)/EH^0_h$, income elasticities $\varepsilon_c$ and Frisch parameters $\phi_h < -1$,

$$\beta_{ch} = \frac{\varepsilon_c\, w_{ch}}{\sum_{c'} \varepsilon_{c'}\, w_{c'h}}, \qquad \gamma_{ch} = QH^0_{ch} + \frac{\beta_{ch}\, EH^0_h}{PH^0_c\, \phi_h},$$

so that supernumerary spending is $-EH^0_h/\phi_h$; under Cobb–Douglas demand $\beta_{ch} = w_{ch}$ and $\gamma_{ch} = 0$.

Benchmark replication evaluates every equation $i$ of the model at the base, as $L_i(x^0) = R_i(x^0)$, and requires

$$\max_i \frac{|L_i(x^0) - R_i(x^0)|}{e_i} \le 10^{-9}, \qquad e_i = \max\big(|L_i(x^0)|, |R_i(x^0)|\big),$$

with $e_i = 1$ where that maximum is below $10^{-9}$. The same scales $e_i$ are used by **[CGE]** `cge_solve`.

Chapter 9 gives every calibration formula, including those of the top nest, margins, VAT, institutions and the health insurance fund, and the chapters on the extensions give theirs.

## References

Blanchflower, D. G., and A. J. Oswald. 1994. *The Wage Curve*. Cambridge, MA: MIT Press.

Fischer, A. 1992. A special Newton-type optimization method. *Optimization* 24: 269–284.

Kanzow, C. 1996. Some noninterior continuation methods for linear complementarity problems. *SIAM Journal on Matrix Analysis and Applications* 17: 851–868.

## Also see

**[CGE]** `cge_map` — Assign SAM accounts to model roles
**[CGE]** `cge_elasticities` — Published elasticities, with their sources
**[CGE]** `cge_check` — Run model validation checks
**[CGE]** `cge_parameters` — The calibrated parameters as a dataset
**[CGE]** `cge_closure` — Choose the model closure
**[CGE]** `cge_solve` — Solve the model
**Chapter 9** — Calibration

# `cge_check` — Run model validation checks

## Description

`cge_check` runs the validation checks of chapter 11 on the calibrated model and reports each as `[ok]` or `[fail]`, with the value tested. It always checks the balance of the SAM, benchmark replication and the closure; after a simulation it also checks the simulation's solution, Walras's law and the GDP identity; and on request it checks homogeneity and that the solver returns to the base from perturbed starting values. `cge_check` exits with error 9 if any check fails.

## Quick start

Run the checks of the base and of the last simulation stored

```
cge_check
```

As above, and check homogeneity and a start from base values perturbed by up to 10 per cent

```
cge_check, homogeneity perturb(0.1)
```

Check the simulation `notariff`, with another seed for the perturbed start

```
cge_check, sim(notariff) perturb(0.1) seed(2026)
```

## Syntax

`cge_check` [, *options*]

| *options* | Description |
|---|---|
| *Main* | |
| `sim(`*name*`)` | simulation to check; default the last one stored |
| `homogeneity` | add the homogeneity check |
| `perturb(#)` | add the perturbed-start check, with starting values within ±# times the base |
| `seed(#)` | random-number seed for `perturb()`; default 12345 |

## Options

`sim(`*name*`)` selects the simulation whose solution, Walras's law and GDP identity are checked. The default is the last simulation stored; a simulation solved again under its own name keeps its place. The other checks do not depend on the simulation.

`homogeneity` adds the homogeneity check: doubling every fixed nominal value, under the current closure and the base parameters, must double every price and nominal value and leave every quantity unchanged.

`perturb(#)` adds the perturbed-start check: the solver, started from base values each multiplied by $1 + \# u$, with $u$ drawn uniformly between $-1$ and 1, must return to the base. # must be at least 0 and less than 1; the default, 0, leaves the check out. A value of 0.1 perturbs every variable by up to 10 per cent.

`seed(#)` sets the random-number seed of the perturbed start; the default is 12345.

## Remarks and examples

The checks test the model actually calibrated and solved, with its SAM, mapping, elasticities, closure and shocks. They are, in the order displayed:

| Check | Value tested | Passes if at most |
|---|---|---|
| SAM balance | largest difference between an account's row and column totals | $10^{-8}\,m$ |
| benchmark replication | largest scaled residual of the model's equations at the base | $10^{-9}$ |
| closure is square | free variables, plus constraints, minus equations | 0 |
| solution of the simulation | its largest scaled residual | $10^{-8}$ |
| Walras's law | $\lvert WALRAS \rvert$ in the solution | $10^{-8}\,m$ |
| GDP identity | GDP by expenditure minus GDP by income, in absolute value | $10^{-8}\,m$ |
| homogeneity | largest relative deviation from twice the nominal values and the base quantities | $10^{-8}$ |
| perturbed start | largest relative deviation from the base | $10^{-7}$ |

Here $m$ is the largest absolute cell of the mapped SAM, or 1 if that is smaller, so that the checks give the same result whatever the units of the SAM. The SAM checked is the SAM after the rewrites of **[CGE]** `cge_map`. The three checks of a simulation are made only when a simulation has been solved; the homogeneity and perturbed-start checks are made only when requested. A check whose solve does not converge has a missing value and fails.

Run `cge_check` after building a model and after changing the closure. A model that passes the homogeneity check responds only to relative prices under the closure, and one that passes the perturbed-start check returns to the same solution from starting values away from the base.

### Example 1: the base

After calibration, with no simulation, three checks are run:

```
. cge_check
  [ok]   SAM balance (largest imbalance)                        0.00e+00
  [ok]   benchmark replication (largest scaled residual)        3.55e-16
  [ok]   closure is square (free variables minus equations)    0.00e+00
All checks passed
```

### Example 2: a simulation

After a tariff removal, the solution of the simulation is checked as well, with the homogeneity and perturbed-start checks:

```
. quietly cge_shock tm, value(0)

. quietly cge_solve, name(notariff)

. cge_check, homogeneity perturb(0.1)
  [ok]   SAM balance (largest imbalance)                        0.00e+00
  [ok]   benchmark replication (largest scaled residual)        3.55e-16
  [ok]   closure is square (free variables minus equations)    0.00e+00
Simulation notariff:
  [ok]     solution (largest scaled residual)                  4.49e-14
  [ok]     Walras's law (WALRAS)                               1.56e-14
  [ok]     GDP identity (expenditure minus income)             6.82e-13
  [ok]   homogeneity (largest relative deviation)              5.33e-15
  [ok]   perturbed start returns the base (largest deviation)  2.00e-14
All checks passed
```

The solution's largest scaled residual, $4.49 \times 10^{-14}$, is below the default tolerance of **[CGE]** `cge_solve`. Under the default closure the only fixed nominal value is the consumer price index, the numeraire; doubling it doubles every price and nominal value, with a largest relative deviation of $5.33 \times 10^{-15}$.

### Example 3: another closure

The homogeneity and perturbed-start checks use the current closure. With a fixed exchange rate, investment-driven savings and capital specific to each activity, the fixed nominal values are the exchange rate, the consumer price index and the price of capital, `W(cap)`:

```
. cge_closure, external(fixexr) savinv(investment) factors(cap:specific)
Closure
  external account     fixexr
  government           flexgsav
  savings-investment   investment
  numeraire            cpi
  factor lab           mobile
  factor cap           specific

. cge_check, homogeneity perturb(0.2) seed(2026)
  [ok]   SAM balance (largest imbalance)                    0.00e+00
  [ok]   benchmark replication (largest scaled residual)    3.55e-16
  [ok]   closure is square (free variables minus equations)  0.00e+00
Simulation notariff:
  [ok]     solution (largest scaled residual)               4.49e-14
  [ok]     Walras's law (WALRAS)                            1.56e-14
  [ok]     GDP identity (expenditure minus income)          6.82e-13
  [ok]   homogeneity (largest relative deviation)           2.55e-14
  [ok]   perturbed start returns the base (largest deviation)  2.40e-14
All checks passed
```

The checks of the simulation `notariff` give the same values as before, because they use its solution as stored.

## Stored results

`cge_check` stores the following in `r()`:

*Scalars*

| | |
|---|---|
| `r(failures)` | number of failed checks |

## Methods and formulas

Let $x^0$ be the base values, $\bar{x}$ the values of the variables fixed under the current closure, and $n_j = 1$ for a nominal variable and $n_j = 0$ otherwise. The nominal variables are the prices, the price indices, the exchange rate, incomes, spending and savings (*YF*, *YIF*, *YI*, *EH*, *YG*, *EG*, *GSAV*, *YSHI* and *WALRAS*), and user variables declared nominal. Deviations are measured relative to each value with a floor of one, and relative to $m$ for a variable whose base value is zero (*WALRAS*):

$$d_j(x, y) = \frac{|x_j - y_j|}{\max(u_j, |y_j|)}, \qquad u_j = \begin{cases} 1 & \text{if } x_j^0 \neq 0, \\ m & \text{if } x_j^0 = 0. \end{cases}$$

For homogeneity, the model is solved with the base parameters and the fixed values $\bar{x}_j(1 + n_j)$, starting from the base, and the value tested is $\max_j d_j(x, y)$ with $y_j = x_j^0(1 + n_j)$. For the perturbed start, the model is solved with the base parameters and fixed values, starting from

$$x_j^s = x_j^0\,(1 + p\,(2U_j - 1)), \qquad U_j \sim U(0,1),$$

with $p$ the value of `perturb()`, and the value tested is $\max_j d_j(x, x^0)$. Both maxima are taken over the variable elements that exist in the model, and both solves use a tolerance of $10^{-12}$ and at most 100 Newton iterations.

The scaled residuals are those of **[CGE]** `cge_solve`; the GDP identity, in full, is in chapter 11.

## Also see

**[CGE]** `cge_calibrate` — Calibrate the model to the SAM
**[CGE]** `cge_solve` — Solve the model
**[CGE]** `cge_closure` — Choose the model closure
**[CGE]** `cge_results` — Report simulation results
**Chapter 11** — Validation checks

# `cge_closure` — Choose the model closure

## Description

`cge_closure` sets the closure of the calibrated model: which variables are fixed and which the model determines, for the external account, the government, the savings–investment balance, the numeraire and each factor, and, element by element, through swaps. It checks that the closure has as many free variables as equations and that no equation is left without a free variable, and displays the closure. Without options, it displays the current closure.

## Quick start

Display the current closure

```
cge_closure
```

A fixed exchange rate, investment-driven savings and capital fixed in each activity

```
cge_closure, external(fixexr) savinv(investment) factors(cap:specific)
```

Fixed government savings, with sales tax rates adjusting

```
cge_closure, government(fixgsav sales)
```

Fix government savings and let real government consumption adjust

```
cge_closure, swap(GSAV GADJ)
```

Remove the swaps

```
cge_closure, swap(none)
```

## Syntax

`cge_closure` [, *options*]

| *options* | Description |
|---|---|
| *Main* | |
| `external(flexexr|fixexr)` | external account; default `flexexr` |
| `government(`*spec*`)` | government: `flexgsav`, the default, or `fixgsav` with the adjusting tax |
| `savinv(savings|investment)` | savings–investment balance; default `savings` |
| `numeraire(cpi|dpi)` | numeraire; default `cpi` |
| `factors(`*spec*`)` | closure of each factor, as *factor*`:`*type*; default `mobile` |
| *Swaps* | |
| `swap(`*fix free* [*fix free* ...]`)` | fix and free variable elements in pairs; `swap(none)` removes the swaps |

The defaults are those that **[CGE]** `cge_calibrate` sets. An option that is not specified keeps its current setting.

## Options

`external(flexexr|fixexr)` sets the closure of the external account. `flexexr` fixes foreign savings, *FSAV*, in foreign currency, and the exchange rate, *EXR*, adjusts; `fixexr` fixes the exchange rate, and foreign savings adjust.

`government(`*spec*`)` sets the closure of the government. `flexgsav` fixes tax rates and real government consumption, and government savings, *GSAV*, adjust. `fixgsav` fixes government savings and scales the rates of one tax in proportion to keep them: direct tax rates (`fixgsav` or `fixgsav direct`, scaled by *TINSADJ*), sales tax rates (`fixgsav sales`, *TQADJ*), import tariffs (`fixgsav import`, *TMADJ*) or VAT rates (`fixgsav vat`, *TVADJ*). The adjusting tax must raise revenue in the base; rates that are zero in the base stay zero. A colon may replace the space, as in `fixgsav:vat`, which is how the closure is displayed. With real government consumption fixed, `fixgsav` keeps a policy change revenue neutral: the scaling factor shows the rise in the rates of the adjusting tax that replaces the revenue lost.

`savinv(savings|investment)` sets the savings–investment closure. `savings` fixes the savings rates, scaled by *MPSADJ*, and investment adjusts to the savings available, through *IADJ*; `investment` fixes real investment, and the savings rates of the domestic institutions are scaled to finance it.

`numeraire(cpi|dpi)` chooses the numeraire, the price index fixed in the closure: `cpi`, the consumer price index, or `dpi`, the producer price index of domestic sales.

`factors(`*spec*`)` sets the closure of each factor, with *spec* a list of *factor*`:`*type*. Factors that are not listed are `mobile`, whatever their earlier closure. The types are `mobile`, fully employed and mobile across activities, with a fixed supply and one average price; `specific`, fixed in each activity, which pays its own price; `unemployed`, a fixed price, with the supply adjusting; `wagecurve`, under which the real wage falls as the unemployment rate rises; and `minwage`, under which the real wage cannot fall below a floor: either the floor binds and unemployment absorbs the shock, or the wage is above the floor and the labour force is fully employed, and the solver finds which. `wagecurve` and `minwage` need a labour force, from **[CGE]** `cge_calibrate` with `unemployment()` or `minwage()`; `wagecurve` also needs the elasticity of `wagecurve()`. Under both, the wage and employment are free and the labour force is fixed (chapter 15).

`swap(`*fix free* [*fix free* ...]`)` changes the closure element by element, after the options above. Each pair fixes the first element, which the closure leaves free, and frees the second, which it fixes. Elements are written *NAME*, *NAME*`(`*e1*`)` or *NAME*`(`*e1*`,`*e2*`)`, for example `W(lab)`; each element may appear once. Swaps replace any earlier ones and stay until changed; `swap(none)` removes them. Variables that the reduced system computes from their definitions, such as household demand, cannot be fixed.

## Remarks and examples

The model has more variables than equations. The closure chooses which variables are fixed, at their current values, and which the model determines. It states how the economy is assumed to adjust to a shock, and results can depend on it as much as on the shock. **[CGE]** `cge_calibrate` sets the default closure, with no swaps; the closure set by `cge_closure` then stays until it is changed or the model is calibrated again. **[CGE]** `cge_solve` solves under the current closure and stores it with each simulation; it is saved with the model by **[CGE]** `cge_save`, kept in sensitivity analysis and written into the GAMS program by **[CGE]** `cge_gams`. Chapter 8 lists the variables that each option fixes and frees.

In the examples, the synthetic SAM is mapped and calibrated as in **[CGE]** `cge_solve`.

### Example 1: the default closure

Without options, `cge_closure` displays the closure. After calibration it is the default:

```
. cge_closure
Closure
  external account      flexexr
  government            flexgsav
  savings-investment    savings
  numeraire             cpi
  factor lab            mobile
  factor cap           mobile

. return list

scalars:
            r(N_swaps) =  0

macros:
            r(closure) : "flexexr flexgsav savings cpi lab:mobile cap:mobile"
```

`r(closure)` gives the closure in one line, as it is stored with each simulation.

#### Example 2: other closure options

A fixed exchange rate, investment fixed in real terms and capital fixed in each activity:

```
. cge_closure, external(fixexr) savinv(investment) factors(cap:specific)
Closure
  external account     fixexr
  government           flexgsav
  savings-investment   investment
  numeraire            cpi
  factor lab           mobile
  factor cap           specific
```

The government and numeraire closures, which were not specified, keep their settings, as does labour, which `factors()` does not list and which is therefore mobile.

#### Example 3: revenue-neutral tax replacement

The other blocks are first returned to their defaults. With government savings fixed and sales tax rates adjusting, the revenue lost when import tariffs are removed is replaced by higher sales taxes:

```
. cge_closure, external(flexexr) savinv(savings) factors(cap:mobile) government(fixgsav sales)
Closure
  external account     flexexr
  government           fixgsav:sales
  savings-investment   savings
  numeraire            cpi
  factor lab           mobile
  factor cap           mobile

. cge_shock tm, value(0)
Parameter tm changed for 3 element(s)

. cge_solve, name(notariff_sales)
Simulation notariff_sales solved in 7 iteration(s) (newton)
Largest scaled residual  1.63e-14; WALRAS =  4.92e-14
```

Government savings stay at their base value, direct tax rates and real government consumption are unchanged, and the scaling factor of the sales tax rates, `TQADJ`, rises by about 29 per cent:

```
. quietly cge_results, sim(notariff_sales) clear nodisplay

. list variable base value if inlist(variable, "GSAV", "TQADJ", "TINSADJ", "GADJ"), noobs

  +-----------------------------+
  | variable   base       value |
  |-----------------------------|
  |     GSAV      3           3 |
  |  TINSADJ      1           1 |
  |     GADJ      1           1 |
  |    TQADJ      1   1.2896718 |
  +-----------------------------+
```

Under such a closure a shock to the rates of the adjusting tax itself, such as removing tariffs while tariffs adjust, is not meaningful: the scaling factor offsets it.

### Example 4: swaps

Government purchases of services can be fixed, and so shocked, by freeing the scaling factor of government consumption, *GADJ*:

```
. cge_shock, reset
All shocks removed

. cge_closure, government(flexgsav) swap(QG(com_ser) GADJ)
Closure
  external account     flexexr
  government           flexgsav
  savings-investment   savings
  numeraire            cpi
  factor lab           mobile
  factor cap           mobile
  swap                 QG(com_ser) fixed, GADJ free

. cge_shock QG, elements(com_ser) scale(1.1)
Variable QG changed for 1 element(s)

. cge_solve, name(qgser)
Simulation qgser solved in 5 iteration(s) (newton)
Largest scaled residual  1.73e-14; WALRAS =  7.06e-14
```

The swap is displayed with the closure. Government purchases are $QG_c = \overline{qg}_c \, GADJ$ (I7), so the other purchases rise in the same proportion:

```
. quietly cge_results, sim(qgser) clear nodisplay

. list variable element1 base value if inlist(variable, "QG", "GADJ"), noobs

  +------------------------------------+
  | variable   element1   base   value |
  |------------------------------------|
  |       QG    com_agr      0       0 |
  |       QG    com_man      5     5.5 |
  |       QG    com_ser     40      44 |
  |     GADJ                 1     1.1 |
  +------------------------------------+
```

Other common swaps are `swap(GSAV GADJ)`, which fixes government savings and lets real government consumption adjust, and `swap(EXR CPI)`, which makes the exchange rate the numeraire. A user variable defined by **[CGE]** `cge_variable` can also be swapped into the closure, which fixes the expression that defines it.

### Example 5: closures that are rejected

A closure is checked when it is set. Fixing government purchases of services while freeing the direct tax scaling factor leaves the number of free variables unchanged, but the equation of government demand for services then has no free variable:

```
. cge_shock, reset
All shocks removed

. cge_closure, swap(QG(com_ser) TINSADJ)
the closure is singular: equation I7 (government demand) for com_ser has no free variable

r(459);
```

Such a closure makes the Jacobian singular, and `cge_closure` names the equation. A swap must also fix a variable that is free and free one that is fixed:

```
. cge_closure, swap(FSAV EXR)
swap: FSAV is already fixed under the closure

r(459);

. cge_closure, swap(none)
```

```
Closure
  external account     flexexr
  government           flexgsav
  savings-investment   savings
  numeraire            cpi
  factor lab           mobile
  factor cap           mobile
```

After an error the closure in memory is unchanged; here the swap of Example 4 was still in force until `swap(none)` removed it.

#### Closures, constraints and user equations

Each constraint set by **[CGE]** `cge_constraint` adds an equation and solves for a variable that the closure fixes, so the system stays square; the limited variable must be free under the closure and the adjusting variable fixed, and every later closure is checked against the constraints. The rents $RM_c$, $RE_c$ and $RA_a$ are fixed at zero by every closure unless a constraint frees them. A user variable given a value by **[CGE]** `cge_variable` is free and needs a user equation of its own (**[CGE]** `cge_equation`).

## Stored results

`cge_closure` stores the following in `r()`:

| | |
|---|---|
| *Scalars* | |
| `r(N_swaps)` | number of swaps |
| *Macros* | |
| `r(closure)` | closure in one line, with the swaps |

## Methods and formulas

Let $x$ be the vector of every element of every variable that exists in the model, $\mathcal{F}$ the set of elements that the closure fixes, $N$ the number of equation elements and $K$ the number of constraints. The closure options build $\mathcal{F}$ block by block; each swap $(y_k, x_k)$ then requires $y_k \notin \mathcal{F}$ and $x_k \in \mathcal{F}$, and replaces $\mathcal{F}$ by $(\mathcal{F} \cup \{y_k\}) \setminus \{x_k\}$. The closure is accepted only if it is square,

$$\#\{\, j : x_j \text{ exists},\ j \notin \mathcal{F}\,\} + K = N,$$

where the adjusting variable of each constraint, although fixed, is solved for. It must also leave every equation with at least one free variable, over the pattern of the Jacobian (the variables that each equation element contains); the equations with exact derivatives, which contain many variables, are not part of this check. Swaps leave the count of free variables unchanged. A square closure that passes the check can still be singular at a point; **[CGE]** `cge_solve` then reports the smallest pivot of the Jacobian.

Chapter 8 gives the variables that each option fixes and frees (section 8.1) and the rules for swaps (section 8.2).

## Also see

**[CGE]** `cge_shock` — Change a parameter or a fixed variable
**[CGE]** `cge_solve` — Solve the model
**[CGE]** `cge_constraint` — Limits that bind in some scenarios and not in others
**[CGE]** `cge_variable` — User variables, defined by an expression or determined by user equations
**[CGE]** `cge_calibrate` — Calibrate the model to the SAM
**Chapter 8** — Closures
**Chapter 15** — Labour-market regimes

# `cge_constraint` — Limits that bind in some scenarios and not in others

## Description

`cge_constraint` puts an upper or a lower bound on elements of a model variable, such as the imports of a commodity or the output of an activity, and pairs each bound with an adjusting variable, which the closure fixes and which may move in one direction only. In every solution one of two things holds: either the limited variable is within its bound and the adjusting variable is at its fixed level, or the limited variable is at its bound and the adjusting variable has moved to keep it there. **[CGE]** `cge_solve` finds which, for each constraint. `cge_constraint, list` displays the constraints, and `cge_constraint, clear` removes them.

## Quick start

An import quota on manufactures, 2 per cent above base imports, with the premium on imports adjusting

```
cge_constraint QM, elements(com_man) upper(1.02) scale
```

A capacity limit on health services, 3 per cent above base output, with the rent going to households

```
cge_constraint QA, elements(act_hlt) upper(1.03) scale rents(hhd)
```

Under a fixed exchange rate, a limit of 30 on foreign savings, after which the currency depreciates

```
cge_constraint FSAV, upper(30) adjust(EXR) up
```

Government consumption is cut once government savings fall to zero

```
cge_constraint GSAV, lower(0) adjust(GADJ) down
```

Display the constraints, with their regimes in the last solution; remove them

```
cge_constraint, list
cge_constraint, clear
```

## Syntax

Add constraints

**`cge_constraint`** *variable*`, {upper(#) | lower(#)}` [*options*]

Display the constraints

**`cge_constraint`**`, list`

Remove the constraints

**`cge_constraint`**`, clear`

| *options* | Description |
|---|---|
| *Main* | |
| `upper(#)` | upper bound |
| `lower(#)` | lower bound |

| *options* | Description |
|---|---|
| `elements(`*list*`)` | elements of *variable* to bound; default every element |
| `scale` | read the bound as a multiple of each element's base value |
| *Adjusting variable* | |
| `adjust(`*variable* [*elements*]`)` | the variable that adjusts when the bound binds |
| `up` | the adjusting variable may rise |
| `down` | the adjusting variable may fall |
| `rents(`*account*`)` | recipient of the rent; default the government |

*variable* is one model variable, in upper case. Exactly one of `upper()` and `lower()` is required, and at most one of `up` and `down` is allowed.

## Options

`upper(#)` or `lower(#)` gives the bound, as a level or, with `scale`, as a multiple of the base value.

`elements(`*list*`)` names the elements of the limited variable; there is one constraint for each. Without it, every existing element of a variable with one index is bounded. A variable with two indexes takes one element of each, as in `elements(lab act_agr)`, and a variable without index takes none.

`scale` reads the bound as a multiple of each element's base value. An element whose base value is zero needs a bound given as a level.

`adjust(`*variable* [*elements*]`)` names the adjusting variable, which must be fixed under the closure. Without elements it takes the element of the limited variable, when the two variables have the same index; a variable without index, or an adjusting variable whose elements are given, serves one constraint. A bound on `QM`, `QE` or `QA` is paired by default with the rent of the same element: `RM`, `RE` or `RA`. Other limited variables need `adjust()`.

`up` or `down` gives the direction in which the adjusting variable may move from its fixed level. Rents rise by default; for other adjusting variables the direction is required.

`rents(`*account*`)` names the recipient of the rent of `RM`, `RE` or `RA`: a factor, a household or enterprise, the government or the rest of the world, by the label of its account. The default is the government.

`list` displays the constraints, with their regime in the last solution: `binding`, `not binding`, or `not solved` for a constraint added since.

`clear` removes every constraint and sends every rent to the government.

## Remarks and examples

A model with only equalities cannot represent a limit that binds in some scenarios and not in others: an import quota, the capacity of an activity, a ceiling on borrowing or a floor under a wage. `cge_constraint` writes each limit as a complementarity condition between the limited variable and an adjusting variable, and **[CGE]** `cge_solve` solves the conditions with the rest of the model, so that the solver finds which limits bind; none is chosen in advance.

Each constraint adds one equation and frees one variable, the adjusting variable, so the system stays square. The limited variable must be free and the adjusting variable fixed under the closure, and every later closure set by **[CGE]** `cge_closure` is checked against the constraints. A constraint on the same element with the same type of bound replaces the earlier one, and each element of an adjusting variable serves one constraint. Constraints are kept by `cge_shock, reset`, in dynamic runs, in sensitivity analysis and by **[CGE]** `cge_save`; they are written into the GAMS program by **[CGE]** `cge_gams`, which solves them as a mixed complementarity problem. **[CGE]** `cge_calibrate` removes them.

### Rents

Quotas and capacity limits work through rents, which are zero in the base and fixed by every closure unless a constraint frees them:

| Variable | Description |
|---|---|
| $RM_c$ | premium on imports of commodity $c$, per unit of the world price |
| $RE_c$ | premium on exports of commodity $c$, per unit of the world price |
| $RA_a$ | rent on the output of activity $a$, per unit of its price |

A premium is a wedge between the world price and the domestic price, like a tariff, but its revenue is a rent, paid to the recipient given in `rents()`: the government by default, as with auctioned quota licences; a household or enterprise; a factor, such as the owners of capital; or the rest of the world, as with a voluntary export restraint, whose rents foreign exporters keep. A rent set by hand with **[CGE]** `cge_shock`, without a constraint, acts as a tax whose revenue goes to the recipient: for example `cge_shock RM, elements(com_man) value(0.05)` acts as a tariff of 5 per cent.

### Typical constraints

| Limit | Limited | Adjusting | Effect when it binds |
|---|---|---|---|
| import quota | $QM_c \leq b$ | $RM_c$, up | the premium rises until imports equal the quota |
| export quota | $QE_c \leq b$ | $RE_c$, up | the premium rises until exports equal the quota |
| capacity | $QA_a \leq b$ | $RA_a$, up | the activity price rises above cost; the rent goes to the owners of the capacity |
| borrowing limit under a peg | $FSAV \leq b$ | $EXR$, up | the peg holds until foreign savings reach the limit; the currency then depreciates |
| deficit ceiling | $GSAV \geq b$ | $GADJ$, down | government consumption is cut once the deficit reaches the ceiling |
| deficit ceiling, taxes | $GSAV \geq b$ | $TINSADJ$, up | direct tax rates rise instead |
| nominal wage floor | $W_f \geq b$ | $QFS_f$, down | employment falls once the wage reaches the floor |

The commands are, for example,

```
cge_constraint QM, elements(com_man) upper(1.02) scale
cge_constraint QA, elements(act_hlt) upper(1.03) scale rents(hhd)
cge_constraint FSAV, upper(30) adjust(EXR) up
cge_constraint GSAV, lower(0) adjust(GADJ) down
cge_constraint W, elements(lab) lower(1) scale adjust(QFS) down
```

The borrowing limit needs the closure `external(fixexr)`, under which $EXR$ is fixed and $FSAV$ free, and the wage floor a closure under which the wage is free and the supply of labour fixed, such as `mobile`.

### Example 1: a capacity limit

The health layout of **[CGE]** `cge_synthetic` is mapped, with its social health insurance fund, and calibrated as in the examples of **[CGE]** `cge_solve`. Government purchases of health services rise by 20 per cent, first without and then with a limit on what health providers can deliver, 3 per cent above the base:

```
. cge_shock qg, elements(com_hlt) scale(1.2)
Parameter qg changed for 1 element(s)

. quietly cge_solve, name(hlt20)

. cge_constraint QA, elements(act_hlt) upper(1.03) scale rents(hhd)
Constraint QA(act_hlt) <= 30.9, RA(act_hlt) up, rents to hhd: not solved

. cge_solve, name(hlt20_cap)
Simulation hlt20_cap solved in 14 iteration(s) (newton)
Largest scaled residual  8.12e-14; WALRAS =  1.62e-15
Constraints
  QA(act_hlt) <= 30.9, RA(act_hlt) up, rents to hhd: binding
      QA(act_hlt) =         30.9;   RA(act_hlt) =  .0590320938

. cge_constraint, list
Constraints (regime in the last solution)
```

```
  1. QA(act_hlt) <= 30.9, RA(act_hlt) up, rents to hhd: binding

. return list

scalars:
          r(N_binding) =  1
      r(N_constraints) =  1
```

A new constraint is `not solved` until the model is solved. After a solve, **[CGE]** `cge_solve` displays the regime of each constraint and the values of its two variables: the limit binds, output is at the bound of 30.9, and the rent on output is 5.9 per cent of the activity price. The report shows the effect of the limit:

```
. cge_report, sims(hlt20 hlt20_cap) tables(activities)

Activities
------------------------------------------------------------
                                  base      hlt20  hlt20_cap
output act_agr                     100     -0.575     -0.018
price act_agr                        1      0.013     -0.331
value_added act_agr                 77     -0.575     -0.018
output act_man                     140     -1.197     -0.989
price act_man                        1     -0.019     -0.346
value_added act_man                 75     -1.197     -0.989
output act_ser                     120     -0.294      0.041
price act_ser                        1     -0.009     -0.341
value_added act_ser                 86     -0.294      0.041
output act_hlt                      30      6.631      3.000
price act_hlt                        1      0.163      5.999
value_added act_hlt                 24      6.631      3.000
------------------------------------------------------------
Percentage changes from the base (rates: changes in points)
```

Without the limit, the output of health services rises by 6.6 per cent and their price by 0.2 per cent. With it, output rises by exactly 3 per cent and the price by 6 per cent: the rent on output fills the gap between price and cost, and goes to households, the owners of the capacity here.

### Example 2: a ceiling on the deficit

The same spending, with a government that may not let its savings fall below their base value: spending on other goods and services is cut once the limit is reached.

```
. quietly cge_results, sim(hlt20) clear nodisplay

. quietly summarize base if variable == "GSAV"

. local gsav0 = r(mean)

. cge_constraint, clear
All constraints removed

. cge_constraint GSAV, lower(`gsav0') adjust(GADJ) down
Constraint GSAV >= 1, GADJ down: not solved

. quietly cge_solve, name(hlt20_rule)

. cge_constraint, list
Constraints (regime in the last solution)
  1. GSAV >= 1, GADJ down: binding

. quietly cge_results, sim(hlt20_rule) clear nodisplay

. list variable base value if inlist(variable, "GSAV", "GADJ"), noobs

  +-----------------------------+
  | variable   base       value |
  |-----------------------------|
  |     GSAV      1           1 |
  |     GADJ      1   .95424154 |
  +-----------------------------+
```

The command echo shows the local macro as typed; its value, 1, is the base value of government savings. The constraint binds, and the scaling factor of government consumption, *GADJ*, falls by 4.6 per cent.

### Example 3: a quota that binds in one scenario and not in another

An import quota on manufactures, 2 per cent above base imports, does not bind in the base. When import tariffs are removed, demand for imports rises and the quota binds:

```
. cge_constraint, clear
All constraints removed

. cge_shock, reset
All shocks removed

. cge_constraint QM, elements(com_man) upper(1.02) scale
Constraint QM(com_man) <= 61.2, RM(com_man) up, rents to gov: not solved

. quietly cge_solve, name(quota_base)

. cge_constraint, list
Constraints (regime in the last solution)
  1. QM(com_man) <= 61.2, RM(com_man) up, rents to gov: not binding

. cge_shock tm, value(0)
Parameter tm changed for 4 element(s)

. quietly cge_solve, name(quota)

. cge_constraint, list
Constraints (regime in the last solution)
  1. QM(com_man) <= 61.2, RM(com_man) up, rents to gov: binding
```

Imports are at the quota, and the premium on imports is 4.1 per cent of the world price; the rent goes to the government:

```
. quietly cge_results, sim(quota) clear nodisplay

. list variable element1 base value if inlist(variable, "QM", "RM") & element1 == "com_man", noobs

  +----------------------------------------+
  | variable   element1   base       value |
  |----------------------------------------|
  |       QM    com_man     60        61.2 |
  |       RM    com_man      0    .0409356 |
  +----------------------------------------+
```

### Example 4: a borrowing limit under a peg

Foreign savings are fixed under the default closure and cannot be bounded. Under a fixed exchange rate they are free, and the peg can hold until they reach a limit, here 5 per cent above the base, with the tariff removal of Example 3 still in force:

```
. cge_constraint, clear
All constraints removed

. cge_constraint FSAV, upper(1.05) scale adjust(EXR) up
FSAV is fixed under the current closure, so it cannot be bounded

r(459);

. cge_closure, external(fixexr)
Closure
  external account     fixexr
  government           flexgsav
  savings-investment   savings
  numeraire            cpi
  factor lab           mobile
  factor cap           mobile

. cge_constraint FSAV, upper(1.05) scale adjust(EXR) up
Constraint FSAV <= 14.7, EXR up: not solved

. quietly cge_solve, name(peg)
. cge_constraint, list
Constraints (regime in the last solution)
  1. FSAV <= 14.7, EXR up: binding

. quietly cge_results, sim(peg) clear nodisplay

. list variable base value if inlist(variable, "FSAV", "EXR"), noobs

  +----------------------------+
  | variable   base      value |
  |----------------------------|
  |      EXR      1   1.047003 |
  |     FSAV     14       14.7 |
  +----------------------------+
```

The limit binds: foreign savings are at 14.7, and the exchange rate has depreciated by 4.7 per cent. A closure that makes the constraint invalid is itself rejected: with this constraint, `cge_closure, external(flexexr)` fails, since it would fix foreign savings and free the exchange rate.

### The minimum wage

The minimum real wage of the closure `minwage` of **[CGE]** `cge_closure` is a complementarity condition of the same kind, with its own calibration and its own smoothed condition (section 15.3). It is not set with `cge_constraint`.

## Stored results

`cge_constraint` stores the following in `r()`:

*Scalars*

| | |
|---|---|
| `r(N_constraints)` | number of constraints |
| `r(N_binding)` | number binding in the last solution (`list` only) |

**[CGE]** `cge_solve` stores the regimes of a solution in `r(constraints)`, with `r(N_constraints)` and `r(N_binding)`.

## Methods and formulas

A constraint $k$ pairs a limited variable $Y_k$, one element of a model variable, and its bound $b_k$ with an adjusting variable $X_k$, which the closure fixes at $\bar{x}_k$, its current value including any shock. With `scale`, $b_k$ is the number given times the base value $Y_k^0$. The conditions are

$$g_k = \sigma_k \frac{b_k - Y_k}{s_k^Y} \geq 0, \qquad h_k = \delta_k \frac{X_k - \bar{x}_k}{s_k^X} \geq 0, \qquad g_k h_k = 0,$$

with $\sigma_k = 1$ for an upper bound and $-1$ for a lower bound, $\delta_k = 1$ for `up` and $-1$ for `down`, and $s$ the base magnitudes of the variables (one where the base value is zero).

**[CGE]** `cge_solve` solves them in two stages. First each condition is written in the smoothed Fischer–Burmeister form

$$g_k + h_k - \sqrt{g_k^2 + h_k^2 + 2\mu} = 0, \qquad \mu = 10^{-8},$$

and solved with the rest of the model. Then each constraint takes the regime of that solution, binding ($g_k = 0$) if $h_k > g_k$ and otherwise not binding ($h_k = 0$), and the model is solved again with these exact equations. A constraint whose other condition then fails ($h_k < 0$ when binding, $g_k < 0$ when not) changes regime, and the model is solved again, for at most ten rounds.

The rent on each element,

$$R_c^M = RM_c\, pwm_c\, EXR\, QM_c, \qquad R_c^E = RE_c\, PWE_c\, EXR\, QE_c, \qquad R_a^A = RA_a\, PA_a\, QA_a,$$

is income of its recipient, and rents paid abroad are payments to the rest of the world. The premiums enter the import and export prices, (P1) and (P2), and the rent on output the activity price, (P7). Chapter 17 gives the details (section 17.1, section 17.2 and section 17.3).

## Also see

**[CGE]** `cge_solve` — Solve the model
**[CGE]** `cge_closure` — Choose the model closure
**[CGE]** `cge_shock` — Change a parameter or a fixed variable
**[CGE]** `cge_gams` — Write the model as a GAMS program, and solve it in GAMS
**Chapter 17** — Bounds and complementarity

# cge_dynamic — Solve a recursive dynamic run

## Description

`cge_dynamic` solves the calibrated model for a sequence of periods. The first period is the calibrated base; each later period is an equilibrium of the static model given stocks and exogenous values that are updated from the solution of the period before. Agents do not look ahead. Each period is stored as a simulation named after the run and the period, such as `base2025`, so that **[CGE]** `cge_report`, **[CGE]** `cge_results`, **[CGE]** `cge_dynresults`, **[CGE]** `cge_save` and **[CGE]** `cge_gams` work with it. The run uses the current closure and starts from the base; the parameters and fixed values that the model had before the run are restored when it ends. If a period cannot be solved, or the `shocks()` program fails, the run stops and nothing is stored: the model, its shocks and its stored simulations are as they were before the run.

## Quick start

A baseline from 2018 to 2030 in which capital accumulates, labour grows at 3 per cent, the population at 2.5 per cent, and efficiency at the rate that gives 5 per cent growth of real value added

```
cge_dynamic, name(base) periods(2018/2030) capital(cap) depreciation(0.05) rental(0.15) ///
    growth(lab:0.03) population(0.025) gdpgrowth(0.05)
```

A scenario with the same settings, the baseline's efficiency path and the shocks applied by program `policy`

```
cge_dynamic, name(policy) periods(2018/2030) capital(cap) depreciation(0.05) rental(0.15) ///
    growth(lab:0.03) population(0.025) baseline(base) shocks(policy)
```

Ten periods with efficiency growth of 1 per cent in every activity and government flows that grow at 3 per cent

```
cge_dynamic, name(run) periods(0/10) tfp(0.01) govgrowth(0.03)
```

Labour that loses the deaths at working ages, in a model calibrated with a death rate

```
cge_dynamic, name(hb) periods(0/10) growth(lab:0.02) population(0.02) ///
    deathfactors(lab) deathshare(0.3) workingage(0.55)
```

## Syntax

`cge_dynamic, name(`*name*`) periods(`*numlist*`)` [*options*]

| *options* | Description |
|---|---|
| *Main* | |
| `name(`*name*`)` | name of the run; required |
| `periods(`*numlist*`)` | periods, the base first; required |
| `shocks(`*program*`)` | program that applies the shocks of each period |
| *Capital* | |
| `capital(`*factors*`)` | factors that accumulate |
| `depreciation(`*numlist*`)` | depreciation rates of the capital factors |
| `rental(`*numlist*`)` | base gross rental rates of the capital factors |
| `sensitivity(#)` | response of the allocation of new capital to rental rates; default `1` |
| *Growth* | |
| `growth(`*factor*`:`*rate* ...`)` | growth rates of other factors |
| `population(#)` | population growth rate; default `0` |

| *options* | Description |
|---|---|
| `govgrowth(#)` | growth rate of government consumption and transfers; default `0` |
| `foreigngrowth(#)` | growth rate of flows in foreign currency; default `0` |
| *Efficiency* | |
| `tfp(`*numlist*`)` | growth rate of efficiency, one or one per activity |
| `gdpgrowth(`*numlist*`)` | growth targets for real value added, one or one per period after the base |
| `baseline(`*name*`)` | reuse the efficiency path of run *name* |
| *Deaths* | |
| `deathfactors(`*factors*`)` | labour factors that lose workers to deaths |
| `deathshare(#)` | share of deaths at working ages; default `0` |
| `workingage(#)` | working-age share of the population; default `1` |
| *Solver* | |
| `tolerance(#)` | tolerance of each period's solve; default `1e-12` |
| `iterate(#)` | maximum number of Newton iterations of each solve; default `100` |

Growth rates are per period and are zero unless they are given. At most one of `tfp()`, `gdpgrowth()` and `baseline()` may be specified.

## Options

`name(`*name*`)` names the run. Each period is stored as the simulation *name* followed by the period, for example `base2025`, so *name* with each period must form a valid Stata name of at most 32 characters. Simulations of the same name are replaced, and a run solved again replaces every period of the earlier run of that name.

`periods(`*numlist*`)` lists the periods: at least two integers, zero or positive, in ascending order, for example `periods(2018/2030)` or `periods(0/5)`. The first is the base. Each step from one listed period to the next is one period of the rates, whatever the difference between the two numbers.

`shocks(`*program*`)` names a program that is called as *program period* before each period after the first, once stocks and exogenous values have been updated and before efficiency is set. It may apply any shock with **[CGE]** `cge_shock`, for example to cancel debt with `cge_shock fdebt`. Shocks persist in later periods, so a shock with `scale()` should be applied in one period only. If the program fails, the run stops, and nothing is stored.

`capital(`*factors*`)` lists the factors that accumulate capital. Each must be mobile or specific to each activity in the closure, not unemployed or under a wage curve or minimum wage. `capital()` needs `depreciation()` and `rental()`, which need it.

`depreciation(`*numlist*`)` gives the depreciation rates of the capital factors, at least 0 and less than 1: one rate for all, or one for each factor in the order of `capital()`.

`rental(`*numlist*`)` gives the base gross rental rates of the capital factors, the rental per unit of capital at base prices, so that the base capital stock is base capital income divided by the rate. One unit of real investment adds *rental* units of the factor. Rates are positive: one for all, or one for each factor in the order of `capital()`.

`sensitivity(#)` sets how strongly new capital specific to each activity moves to activities whose rental rate is above the average; 0 allocates it in proportion to capital stocks. The default is 1; # must be zero or positive. It has no effect on mobile capital.

`growth(`*factor*`:`*rate* ...`)` sets the growth rates of factors that are not capital; factors that are not listed do not grow. With `growth(lab_f:0.03 lab_i:0.025)`, for example, `lab_f` grows at 3 per cent a period and `lab_i` at 2.5 per cent. The supply of an unemployed factor is endogenous, so its rate must be zero, unless the factor has a labour force (**[CGE]** `cge_calibrate`, `unemployment()`). The labour force of a factor that has one grows at the factor's rate, whatever its closure, so that under `unemployed`, `wagecurve` or `minwage` employment and unemployment follow from labour demand.

`population(#)` sets the population growth rate. Subsistence consumption of LES demand, the base of the health link and the population index grow at this rate. With a death rate (**[CGE]** `cge_calibrate`, `mortality()`), the rate is taken to hold at the base death rate: population then grows at `population()` plus the base death rate less the death rate of the period before.

`govgrowth(#)` sets the growth rate of government consumption and of real transfers from the government to domestic institutions, within the government and to the health insurance fund.

`foreigngrowth(#)` sets the growth rate of flows in foreign currency: transfers from abroad to domestic institutions, factors, the government and the health insurance fund, transfers from the government abroad, and foreign savings when the closure fixes them.

`tfp(`*numlist*`)` sets the growth rate of value-added efficiency: one rate, used for every activity, or one for each activity in the order of the model's activities.

`gdpgrowth(`*numlist*`)` calibrates a baseline to growth targets: in each period, efficiency in every activity grows by the uniform factor for which real value added at base prices grows at the target. Give one target, used in every period, or one for each period after the first. The efficiency factor of each period is displayed and stored with its simulation.

`baseline(`*name*`)` reuses the efficiency factors of run *name*, which must have been solved with `gdpgrowth()` and must include every period of this run, so that a policy scenario does not change efficiency.

`deathfactors(`*factors*`)` lists the labour factors that lose workers to deaths at working ages. They may not be capital factors, nor unemployed factors without a labour force, and the model needs a death rate. Each listed factor, and its labour force, then grows at its `growth()` rate plus `deathshare()/workingage()` times the base death rate less the death rate of the period before.

`deathshare(#)` is the share of deaths at working ages, between 0 and 1; the default is 0. It needs `deathfactors()`.

`workingage(#)` is the working-age share of the population, above 0 and at most 1; the default is 1. It needs `deathfactors()`.

`tolerance(#)` and `iterate(#)` are passed to the solver of each period; the defaults are `1e-12` and 100.

## Remarks and examples

A dynamic run strings static equilibria together. Before each period after the first, `cge_dynamic` updates, in this order, capital, the supply of other factors, the population, government and foreign flows and public debt; it then calls the `shocks()` program and sets efficiency, and solves the period from the solution of the period before. The updates follow the recursive dynamic extension of the standard model (Thurlow 2004); Methods and formulas below gives them, and chapter 19 the details. None of the rates has a default other than zero, and none is estimated by the package: they are the user's assumptions about the economy.

### Example 1: a baseline calibrated to a growth target

The synthetic SAM is calibrated as in **[CGE]** `cge_solve`, and capital is made specific to each activity, so that new capital is allocated among activities:

```
. cge_synthetic, clear
(16 vars, 15 obs)

Synthetic SAM loaded (standard layout): 15 accounts, total   1743.00
Largest row-column imbalance:  0.00e+00

. cge_sam, labels(account)
SAM read: 15 accounts, total 1743
Largest row-column imbalance:  0.00e+00

. cge_map, activities(act_agr act_man act_ser) commodities(com_agr com_man com_ser) ///
>     factors(lab cap) households(hhd) government(gov) savinv(sav) row(row)       ///
>     dirtax(tax_dir) saltax(tax_sal) imptax(tax_imp)
Accounts mapped: 3 activities, 3 commodities, 2 factors, 1 households, 0 enterprises, 0 margin accou
> nts

. cge_calibrate, sigmava(0.8) sigmaq(2) sigmat(2)
```

```
Calibrated: 93 equations, 93 free variables
Benchmark replication: largest scaled residual  3.55e-16

. cge_closure, factors(cap:specific)
Closure
  external account     flexexr
  government           flexgsav
  savings-investment   savings
  numeraire            cpi
  factor lab           mobile
  factor cap           specific
```

A baseline of four periods after the base, in which capital depreciates at 5 per cent a period, labour and the population grow at 2.5 per cent, government flows at 3 per cent and foreign flows at 2 per cent, and efficiency grows at the rate that gives 4 per cent growth of real value added:

```
. cge_dynamic, name(base) periods(0/4) capital(cap) depreciation(0.05) rental(0.15) ///
>     growth(lab:0.025) population(0.025) govgrowth(0.03) foreigngrowth(0.02)      ///
>     gdpgrowth(0.04)
Period 0: solved in 0 iteration(s)
Period 1: solved in 7 iteration(s); efficiency factor 1.0223831984
Period 2: solved in 7 iteration(s); efficiency factor 1.0213821436
Period 3: solved in 7 iteration(s); efficiency factor 1.0204080406
Period 4: solved in 7 iteration(s); efficiency factor 1.0194645412
Dynamic run base stored as simulations: base0 base1 base2 base3 base4
```

Period 0 is the base and needs no iteration. In each later period, the efficiency factor is the uniform growth factor of value-added efficiency that, given the growth of capital and labour, meets the target; it is found by secant iterations, each of which solves the model. The run returns the names of its simulations:

```
. return list

macros:
               r(name) : "base"
               r(sims) : "base0 base1 base2 base3 base4"
```

**[CGE]** `cge_report` with `run()` reports every period as a change from the base:

```
. cge_report, run(base) tables(macro)

Macroeconomic aggregates
---------------------------------------------------------------------------------------------
                                  base      base0      base1      base2      base3      base4
gdp_nominal                        300      0.000      3.968      8.095     12.388     16.853
gdp_real                           300      0.000      3.984      8.126     12.434     16.914
gdp_deflator                         1      0.000     -0.015     -0.029     -0.042     -0.053
consumption_real                   230      0.000      3.861      7.875     12.047     16.384
government_real                     45      0.000      3.000      6.090      9.273     12.551
investment_real                     50      0.000      4.440      9.072     13.904     18.946
insurance_fund_real                  0          .          .          .          .          .
exports_real                        75      0.000      4.259      8.702     13.336     18.170
imports_real                       100      0.000      3.694      7.536     11.532     15.688
value_added_real                   262      0.000      4.000      8.160     12.486     16.986
cpi                                  1      0.000      0.000      0.000      0.000      0.000
dpi                                  1      0.000     -0.051     -0.100     -0.147     -0.192
exchange_rate                        1      0.000      0.155      0.304      0.446      0.581
real_exchange_rate                   1      0.000      0.206      0.404      0.594      0.775
private_savings                     32      0.000      3.861      7.875     12.046     16.382
investment_value                    50      0.000      4.532      9.256     14.180     19.312
foreign_savings                     15      0.000      2.000      4.040      6.121      8.243
---------------------------------------------------------------------------------------------
Percentage changes from the base (rates: changes in points)
```

Real value added grows at 4 per cent a period, 16.986 per cent in four periods ($1.04^4 = 1.16986$). The SAM is synthetic: the run illustrates the command and is not a projection for any economy.

### Example 2: a policy scenario against the baseline

A scenario removes every import tariff from period 2. The program `tariff_from2` receives the period as its argument; the shock is set once and stays in force in later periods. The scenario takes the baseline's settings and, with `baseline(base)`, its efficiency factors:

```
. program define tariff_from2
  1.     args t
  2.     if `t' == 2 cge_shock tm, value(0)
  3. end

. cge_dynamic, name(lib) periods(0/4) capital(cap) depreciation(0.05) rental(0.15) ///
>     growth(lab:0.025) population(0.025) govgrowth(0.03) foreigngrowth(0.02)      ///
>     baseline(base) shocks(tariff_from2)
Period 0: solved in 0 iteration(s)
Period 1: solved in 7 iteration(s); efficiency factor 1.0223831984
Parameter tm changed for 3 element(s)
Period 2: solved in 9 iteration(s); efficiency factor 1.0213821436
Period 3: solved in 7 iteration(s); efficiency factor 1.0204080406
Period 4: solved in 7 iteration(s); efficiency factor 1.0194645412
Dynamic run lib stored as simulations: lib0 lib1 lib2 lib3 lib4
```

The efficiency factors are those of the baseline, period by period, so that efficiency does not respond to the policy. The shock is reported before period 2, the first period that it changes. Periods of the two runs can be compared side by side:

```
. cge_report, sims(base4 lib4) tables(macro welfare)

Macroeconomic aggregates
------------------------------------------------------------
                                  base      base4       lib4
gdp_nominal                        300     16.853     15.714
gdp_real                           300     16.914     16.169
gdp_deflator                         1     -0.053     -0.392
consumption_real                   230     16.384     18.760
government_real                     45     12.551     12.551
investment_real                     50     18.946      3.545
insurance_fund_real                  0          .          .
exports_real                        75     18.170     23.982
imports_real                       100     15.688     20.048
value_added_real                   262     16.986     16.135
cpi                                  1      0.000      0.000
dpi                                  1     -0.192      0.899
exchange_rate                        1      0.581      5.120
real_exchange_rate                   1      0.775      4.184
private_savings                     32     16.382     18.724
investment_value                    50     19.312      2.058
foreign_savings                     15      8.243      8.243
------------------------------------------------------------

Welfare (per cent of base consumption)
--------------------------------------------------------------
                                             base4       lib4
equivalent_variation hhd                    16.383     18.742
compensating_variation hhd                  16.383     18.739
--------------------------------------------------------------
Percentage changes from the base (rates: changes in points)
```

Both columns are changes from the base, period 0. In period 4 real investment is 3.5 per cent above the base in the scenario, against 18.9 per cent in the baseline, and real value added grows less, 16.1 against 17.0 per cent; real consumption and the equivalent variation of the household are higher. New capital is proportional to real investment in the period before, so lower investment means less capital in later periods. The differences from the baseline, period by period and for every variable, are the dataset of **[CGE]** `cge_dynresults`.

### Example 3: population, deaths and labour

With a death rate (**[CGE]** `cge_calibrate`, `mortality()` and `mortelas()`), the deaths of one period change the population and labour of the next. The health layout of the synthetic SAM is calibrated with a base death rate of 10 per 1,000 and an elasticity of $-0.3$ with respect to health services per head. In the run, 30 per cent of deaths are at working ages, and 55 per cent of the population is of working age:

```
. cge_synthetic, clear health
(19 vars, 18 obs)

Synthetic SAM loaded (health layout): 18 accounts, total   1746.00
Largest row-column imbalance:  0.00e+00

. cge_sam, labels(account)
SAM read: 18 accounts, total 1746
Largest row-column imbalance:  0.00e+00

. cge_map, activities(act_agr act_man act_ser act_hlt)                       ///
>     commodities(com_agr com_man com_ser com_hlt) factors(lab cap)          ///
>     households(hhd) government(gov) savinv(sav) row(row) dirtax(tax_dir)   ///
>     saltax(tax_sal) imptax(tax_imp) insurance(shi)
Accounts mapped: 4 activities, 4 commodities, 2 factors, 1 households, 0 enterprises, 0 margin accou
> nts
Health insurance fund: shi

. cge_calibrate, sigmava(0.8) sigmaq(2) sigmat(2) healthcom(com_hlt) ///
>     mortality(0.01) mortelas(-0.3)
Death rate: .01 per head; elasticity -.3
Calibrated: 118 equations, 118 free variables
Benchmark replication: largest scaled residual  3.55e-16

. cge_closure, factors(cap:specific)
Closure
  external account     flexexr
  government           flexgsav
  savings-investment   savings
  numeraire            cpi
  factor lab           mobile
  factor cap           specific

. local settings capital(cap) depreciation(0.05) rental(0.15) growth(lab:0.02) ///
>     population(0.02) govgrowth(0.03) tfp(0.01) deathfactors(lab)           ///
>     deathshare(0.3) workingage(0.55)

. cge_dynamic, name(hbase) periods(0/4) `settings'
Period 0: solved in 0 iteration(s)
Period 1: solved in 6 iteration(s)
Period 2: solved in 6 iteration(s)
Period 3: solved in 6 iteration(s)
Period 4: solved in 6 iteration(s)
Dynamic run hbase stored as simulations: hbase0 hbase1 hbase2 hbase3 hbase4
```

```
. cge_report, run(hbase) tables(factors health)

Factors
---------------------------------------------------------------------------------------------
                                  base     hbase0     hbase1     hbase2     hbase3     hbase4
wage lab                             1      0.000      0.317      0.635      0.952      1.269
real_wage lab                        1      0.000      0.317      0.635      0.952      1.269
income lab                         135      0.000      2.323      4.701      7.135      9.625
supply lab                         135      0.000      2.000      4.041      6.124      8.251
wage cap                             1      0.000      0.000      0.000      0.000      0.000
real_wage cap                        1      0.000      0.000      0.000      0.000      0.000
income cap                         127      0.000      2.595      5.258      7.990     10.794
supply cap                         127      0.000      0.906      1.820      2.742      3.672
---------------------------------------------------------------------------------------------

Health
---------------------------------------------------------------------------------------------
                                  base     hbase0     hbase1     hbase2     hbase3     hbase4
health_services                     30      0.000      2.740      5.557      8.453     11.432
services_per_head                    1      0.000      0.725      1.455      2.191      2.932
mortality_rate                      10      0.000     -0.022     -0.043     -0.065     -0.086
population                           1      0.000      2.000      4.042      6.128      8.257
fund_revenue                         9      0.000      2.229      4.514      6.856      9.257
fund_volume                          1      0.000      2.504      5.076      7.718     10.432
---------------------------------------------------------------------------------------------
Percentage changes from the base (rates: changes in points)
```

Health services per head rise, and the death rate falls by 0.022 points per 1,000 in period 1. Period 0 has the base death rate, so the population grows at `population()`, 2 per cent, in period 1. From period 2, the lower death rate of the period before adds to population growth: the population index is 4.042 per cent above the base in period 2, against 4.040 at a constant 2 per cent ($1.02^2 = 1.0404$). Labour supply grows by 0.3/0.55 times the fall in the death rate in addition to its rate of 2 per cent, and is 4.041 per cent above the base in period 2. Because rising health services per head lower the death rate in a baseline as well, scenarios should be compared with a baseline run with the same settings.

### Capital

Capital factors are listed in `capital()`. New capital comes from the previous period's real investment at base prices, shared among capital factors in proportion to their base income. When capital is mobile, it adds to the factor's supply. When it is specific to each activity (**[CGE]** `cge_closure`, `factors(cap:specific)`), new capital is allocated among the activities that use the factor in proportion to their capital, tilted towards those whose rental rate is above the average by `sensitivity()`; an activity's share cannot fall below zero. The rental rate of `rental()` converts investment, measured at base prices, into the factor's units, whose base quantity is the factor's base income: with a single capital factor and a rate of 0.15, one unit of real investment adds 0.15 units of the factor, and the base stock is base capital income divided by 0.15.

### Labour force and unemployment

A factor with a labour force (**[CGE]** `cge_calibrate`, `unemployment()` or `minwage()`) has its labour force grow at the factor's growth rate. Under unemployment at a fixed wage, a wage curve or a minimum wage, employment is then determined by labour demand, and the unemployment rate changes (chapter 15).

### Public debt

When the model has public debt (**[CGE]** `cge_calibrate`, `debt()`), the deficit of each period is borrowed in the next, the share given by `foreignborrowing()` of **[CGE]** `cge_calibrate` abroad, and interest is paid on the new stocks; a surplus repays debt in the same shares (section 13.3). A program in `shocks()` may change the stocks and the interest rates, for example `cge_shock fdebt, scale(0.5)` in one period for debt relief.

### Shocks and the model in memory

A run starts from the base: shocks set with **[CGE]** `cge_shock` before the run are not used, and a note is displayed when fixed values have been changed. Constraints set by **[CGE]** `cge_constraint` are kept. Other fixed values, including the numeraire, keep their base values unless a program in `shocks()` changes them. When the run ends, or stops with an error, the parameters and fixed values that the model had before it are restored. A period that does not converge stops the run with error 430; the periods solved before it remain stored.

### Lags in user equations

From version 0.9.2, user equations (**[CGE]** `cge_equation`) may refer to the values of variables in the periods before, `lag(GL)` or `lag(QA(act_man), 2)`, for up to four periods; before the first period they are the base values. Stocks and partial adjustment that the run does not update itself, such as a fund that accumulates or spending that follows revenue with a delay, can so be written in the model's own terms. Each period stores the lagged values it used.

### Results

**[CGE]** `cge_report` with `run()` reports every period of a run, and **[CGE]** `cge_dynresults` returns every variable of every period as a dataset, with the differences from a baseline run. A period can be passed to **[CGE]** `cge_microsim` like any other simulation; its changes are measured from the base. **[CGE]** `cge_gams` with `dynamic()` writes a run as a GAMS program that solves the periods in order.

## Stored results

`cge_dynamic` stores the following in `r()`:

*Macros*

| | |
|---|---|
| `r(name)` | name of the run |
| `r(sims)` | simulations of the run, in period order |

## Methods and formulas

Let periods $t = 0, 1, \dots, T$ be the steps of `periods()`, with $t = 0$ the base. Before period $t \geq 1$, from the solution of period $t-1$:

*Capital.* For each capital factor $f \in K$, with depreciation rate $\delta_f$ and base gross rental rate $\rho_f$, new capital is

$$N_{f,t} = \rho_f\,\omega_f\,INV_{t-1}, \qquad INV_{t-1} = \sum_c PQ^0_c\,QINV_{c,t-1}, \qquad \omega_f = \frac{YF^0_f}{\sum_{f'\in K} YF^0_{f'}}.$$

For mobile capital, $QFS_{f,t} = (1-\delta_f)\,QFS_{f,t-1} + N_{f,t}$. For capital specific to each activity,

$$QF_{fa,t} = (1-\delta_f)\,QF_{fa,t-1} + N_{f,t}\,\eta_{fa,t}, \qquad \eta_{fa,t} = \frac{\max\big[0,\ s_{fa}\big(1+\beta(R_{fa}/\bar{R}_f - 1)\big)\big]}{\sum_{a'} \max\big[0,\ s_{fa'}\big(1+\beta(R_{fa'}/\bar{R}_f - 1)\big)\big]},$$

over the activities that use the factor, with

$$s_{fa} = \frac{QF_{fa,t-1}}{\sum_{a'} QF_{fa',t-1}}, \qquad R_{fa} = W_{f,t-1}\,WFDIST_{fa,t-1}, \qquad \bar{R}_f = \sum_a s_{fa}\,R_{fa},$$

and $\beta$ the value of `sensitivity()`.

*Population and other factors.* With a death rate, base rate $m^0$ and elasticity $\epsilon^m$, the death rate of period $t-1$, with the parameters and health services of that period, is

$$MR_{t-1} = \mu^m \left(\frac{HS_{t-1}}{HS^0_{t-1}}\right)^{\epsilon^m},$$

where $\mu^m = m^0$ unless it is shocked, and

$$g_{P,t} = g_P + \left(m^0 - MR_{t-1}\right), \qquad g_{f,t} = g_f + d_f \, \frac{\theta^D}{\alpha^W} \left(m^0 - MR_{t-1}\right),$$

where $g_P$ is `population()`, $g_f$ the rate of factor $f$ in `growth()`, $d_f = 1$ for the factors in `deathfactors()` and 0 otherwise, $\theta^D$ is `deathshare()` and $\alpha^W$ `workingage()`. Without a death rate, $g_{P,t} = g_P$ and $g_{f,t} = g_f$. A factor that is not capital grows as

$$QFS_{f,t} = (1 + g_{f,t})\, QFS_{f,t-1} \quad \text{(mobile)}, \qquad QF_{fa,t} = (1 + g_{f,t})\, QF_{fa,t-1} \quad \text{(specific)},$$

and its labour force, if it has one, as $qlf_{f,t} = (1 + g_{f,t})\, qlf_{f,t-1}$. The subsistence quantities $\gamma_{ch}$ and $\gamma^h_{ah}$, the base of the health link $HS^0$ and the population index grow at $g_{P,t}$.

*Government and foreign flows.* With $g_G$ the rate of `govgrowth()` and $g_W$ that of `foreigngrowth()`,

$$\begin{array}{ll} \overline{qg}_c,\ trnsfr_{d,gov},\ trnsfr_{gov,gov},\ trnsfr_{shi,gov} & \text{grow at } g_G, \\ trnsfr_{d,row},\ trnsfr_{f,row},\ trnsfr_{gov,row},\ trnsfr_{row,gov},\ trnsfr_{shi,row} & \text{grow at } g_W, \end{array}$$

and so does $FSAV$ when the closure fixes it.

*Public debt.* With debt and a share $\phi$ borrowed abroad,

$$DD_t = DD_{t-1} + (1 - \phi)\,(-GSAV_{t-1}), \qquad DF_t = DF_{t-1} + \phi\,(-GSAV_{t-1})/EXR_{t-1}.$$

*Shocks and efficiency.* The program in `shocks()` is called, and efficiency is set to

$$ad_{a,t} = \lambda_{a,t}\, ad_{a,t-1},$$

with $\lambda_{a,t} = 1 + g^{ad}_a$ under `tfp()`, the uniform factor $\lambda_t$ of the baseline's period $t$ under `baseline()`, and 1 otherwise. Under `gdpgrowth()`, $\lambda_t$ is uniform and solves

$$VA_t(\lambda_t) = (1 + g^Y_t)\, VA_{t-1}, \qquad VA_t = \sum_a PVA^0_a\, QVA_{a,t},$$

real value added at base prices, less commodities bought within nests of value added (section 18.2). It is found by secant iterations that start from $\lambda = 1$ and $\lambda = 1 + g^Y_t$, each of which solves the model, until $|VA_t/((1 + g^Y_t)\, VA_{t-1}) - 1| < 10^{-13}$, in at most 50 iterations; otherwise the run stops with error 430.

Period $t$ is then solved from the solution of period $t-1$ by the solver of **[CGE]** `cge_solve`, without continuation. Chapter 19 states the dynamics in full, and section 16.5 the population and labour updates.

## References

Thurlow, J. 2004. *A dynamic computable general equilibrium (CGE) model for South Africa: Extending the static IFPRI model*. Working Paper 1-2004, Trade and Industrial Policy Strategies.

## Also see

**[CGE]** `cge_dynresults` — Results of a dynamic run as a dataset
**[CGE]** `cge_report` — Standard report tables for one or more simulations
**[CGE]** `cge_shock` — Change a parameter or a fixed variable
**[CGE]** `cge_closure` — Choose the model closure
**[CGE]** `cge_calibrate` — Calibrate the model to the SAM
**[CGE]** `cge_gams` — Write the model as a GAMS program, and solve it in GAMS
**Chapter 19** — Recursive dynamics

# `cge_dynresults` — Results of a dynamic run as a dataset

## Description

`cge_dynresults` replaces the data in memory with the results of a run solved by **[CGE]** `cge_dynamic`: one observation for each period and each element of every model variable, with its base value and its value in the period. With `baseline()`, the value of a baseline run in the same period and the per cent difference from it are added. The dataset can then be listed, tabulated, graphed or saved with any Stata command.

## Quick start

Load every variable of every period of run `base`

```
cge_dynresults, run(base) clear
```

Load the periods of run `exit` with their differences from run `base`, and graph the per cent difference in the consumption of household `hh_ru_p`

```
cge_dynresults, run(exit) baseline(base) clear
line dev_pct period if variable == "EH" & element1 == "hh_ru_p"
```

## Syntax

**`cge_dynresults`**`, run(`*name*`) clear [baseline(`*name*`)]`

| *options* | Description |
|---|---|
| *Main* | |
| `run(`*name*`)` | the dynamic run; required |
| `clear` | replace the data in memory; required |
| `baseline(`*name*`)` | the baseline run to compare with |

## Options

`run(`*name*`)` names the run, as given in `name()` of **[CGE]** `cge_dynamic`.

`clear` is required: the data in memory are replaced. They are kept if the run cannot be loaded, for example because no run has the name given.

`baseline(`*name*`)` names a baseline run to compare with. It must have every period of `run()`; it may have more.

## Remarks and examples

The dataset has one observation for each period of the run and each element of every model variable that exists in the model, in period order. A variable is identified by its name, as listed in section 6.3, and by up to two set elements: `QA` has one, the activity; `QF` has two, the factor and the activity; `EXR` has none. The variables of the dataset are

| Variable | Content |
|---|---|
| `period` | the period |
| `variable` | the model variable |
| `element1` | its first set element, or empty |
| `element2` | its second set element, or empty |

| Variable | Content |
|---|---|
| `base` | its base value, in the calibrated SAM |
| `value` | its value in the period |
| `baseline` | with `baseline()`: its value in the baseline run in the same period |
| `dev_pct` | with `baseline()`: 100 (value − baseline) / \|baseline\|, missing where the baseline value is zero |

The data are sorted by `period`, `variable`, `element1` and `element2`. **[CGE]** `cge_report` with `run()` gives the standard report measures of each period as changes from the base; `cge_dynresults` gives every variable, and its difference from a baseline in each period.

### Example 1: a scenario against its baseline

The two runs of Examples 1 and 2 of **[CGE]** `cge_dynamic` are solved first, on the synthetic SAM: a baseline `base` calibrated to 4 per cent growth of real value added, and a scenario `lib` that removes import tariffs from period 2, with the baseline's efficiency path:

```
. cge_closure, factors(cap:specific)
. local settings capital(cap) depreciation(0.05) rental(0.15) growth(lab:0.025) ///
>     population(0.025) govgrowth(0.03) foreigngrowth(0.02)
. cge_dynamic, name(base) periods(0/4) `settings' gdpgrowth(0.04)
. cge_dynamic, name(lib) periods(0/4) `settings' baseline(base) shocks(tariff_from2)
```

The results of the scenario, with the baseline's, replace the data in memory:

```
. cge_dynresults, run(lib) baseline(base) clear

. describe

Contains data
  obs:           590
 vars:             8
 size:        80,240
----------------------------------------------------------------------------------------------------
              storage   display    value
variable name   type    format     label      variable label
----------------------------------------------------------------------------------------------------
period          double  %10.0g                Period
variable        str32   %32s                  Model variable
element1        str32   %32s                  First set element
element2        str32   %32s                  Second set element
base            double  %10.0g                Base value (calibrated SAM)
value           double  %10.0g                Value in the period
baseline        double  %10.0g                Baseline value in the period
dev_pct         double  %10.0g                Per cent difference from the baseline
----------------------------------------------------------------------------------------------------
Sorted by:  period  variable  element1  element2
     Note:  dataset has changed since last saved
```

Capital is specific to each activity. Its stocks are the same in both runs up to period 2, the period of the shock, because capital is set before each period from the investment of the period before; from period 3 they are lower in the scenario:

```
. list period element2 value baseline dev_pct if variable == "QF" & element1 == "cap", ///
>     noobs sepby(period)

  +--------------------------------------------------------+
  | period   element2       value    baseline      dev_pct |
  |--------------------------------------------------------|
  |      0    act_agr          37          37            0 |
  |      0    act_man          40          40            0 |
  |      0    act_ser          50          50            0 |
  |--------------------------------------------------------|
  |      1    act_agr   37.335039   37.335039            0 |
  |      1    act_man   40.362205   40.362205            0 |
  |      1    act_ser   50.452756   50.452756            0 |
  |--------------------------------------------------------|
  |      2    act_agr   37.752887   37.752887           0 |
  |      2    act_man   40.816892   40.816892           0 |
  |      2    act_ser   51.005707   51.005707           0 |
  |-------------------------------------------------------|
  |      3    act_agr   37.961486   38.253736   -.76397862 |
  |      3    act_man   41.023508   41.364416   -.82415861 |
  |      3    act_ser    51.32118   51.658927   -.65380119 |
  |-------------------------------------------------------|
  |      4    act_agr   38.241148   38.837966   -1.5366863 |
  |      4    act_man   41.308012   42.005347    -1.660112 |
  |      4    act_ser   51.724764   52.412704   -1.3125443 |
  +-------------------------------------------------------+
```

Output changes in the period of the shock; output of services rises above the baseline while that of agriculture and manufactures falls:

```
. list period element1 base value baseline dev_pct if variable == "QA", noobs sepby(period)

  +--------------------------------------------------------------+
  | period   element1   base       value    baseline      dev_pct |
  |--------------------------------------------------------------|
  |      0    act_agr    100         100         100            0 |
  |      0    act_man    140         140         140            0 |
  |      0    act_ser    150         150         150            0 |
  |--------------------------------------------------------------|
  |      1    act_agr    100   104.06634   104.06634            0 |
  |      1    act_man    140   145.62879   145.62879            0 |
  |      1    act_ser    150   155.90932   155.90932            0 |
  |--------------------------------------------------------------|
  |      2    act_agr    100   108.04291   108.29973   -.23713595 |
  |      2    act_man    140   150.58639   151.51038   -.60985071 |
  |      2    act_ser    150   162.97496   162.03019    .58308623 |
  |--------------------------------------------------------------|
  |      3    act_agr    100   112.00837   112.70706   -.61991978 |
  |      3    act_man    140   155.92581   157.65668   -1.0978709 |
  |      3    act_ser    150   168.91969   168.36954    .32675392 |
  |--------------------------------------------------------------|
  |      4    act_agr    100     116.111   117.29549   -1.0098316 |
  |      4    act_man    140   161.46254   164.07998   -1.5952217 |
  |      4    act_ser    150   175.05072   174.93468    .06633669 |
  +--------------------------------------------------------------+
```

The SAM is synthetic: these results illustrate the command and are not findings about any economy.

**Example 2: a run on its own**

Without `baseline()`, the dataset has the base value and the value in each period:

```
. cge_dynresults, run(base) clear

. list period variable element1 base value if variable == "EXR", noobs

  +------------------------------------------------+
  | period   variable   element1   base       value |
  |------------------------------------------------|
  |      0        EXR                 1           1 |
  |      1        EXR                 1   1.0015501 |
  |      2        EXR                 1   1.0030353 |
  |      3        EXR                 1   1.0044564 |
  |      4        EXR                 1   1.0058147 |
  +------------------------------------------------+
```

## Stored results

`cge_dynresults` stores the following in `r()`:

*Macros*

| | |
|---|---|
| `r(run)` | name of the run |

## Methods and formulas

For element $i$ of a variable in period $t$, with value $x_{it}$ in the run and $b_{it}$ in the baseline run,

$$\texttt{dev\_pct}_{it} = 100\,\frac{x_{it} - b_{it}}{|b_{it}|}, \qquad b_{it} \neq 0,$$

and missing when $b_{it} = 0$. The base value is the value of the variable in the calibrated SAM, the same in every period. Chapter 19 describes the runs.

## Also see

**[CGE]** `cge_dynamic` — Solve a recursive dynamic run
**[CGE]** `cge_report` — Standard report tables for one or more simulations
**[CGE]** `cge_results` — Report simulation results
**Chapter 19** — Recursive dynamics

# `cge_elasticities` — Published elasticities, with their sources

## Description

`cge_elasticities` lists, loads, or turns into values for the options of **[CGE]** `cge_calibrate`, the entries of the package's elasticity library. Each entry is a published estimate, with its source, the table and page where it is printed, its definition and the check made when it was read from the source. Elasticities have no defaults in **[CGE]** `cge_calibrate`; the library documents where values come from, and nothing is applied without a command. Entries of the user's own, from a file in the library's layout, may be added.

## Quick start

List the Armington elasticities of Togo, Sierra Leone and Botswana

```
cge_elasticities sigmaq, country(TGO SLE BWA)
```

Load the income elasticities of Togo, with their definitions and sources, into memory

```
cge_elasticities incelas, country(TGO) clear
```

After **[CGE]** `cge_map`, one Armington elasticity per commodity, for `sigmaq()`

```
cge_elasticities sigmaq, country(TGO) values
local sq `r(values)'
```

Income elasticities by commodity, with the consumption category of each commodity

```
cge_elasticities incelas, country(TGO) values map(com_agr:food com_hlt:health) others(other)
```

Add the entries of a file to the library and list those of one source, `MY2026`

```
cge_elasticities using myvalues.csv, source(MY2026)
```

## Syntax

**`cge_elasticities`** [*parameter*] [`using` *filename*] [`,` *options*]

*parameter* is the **[CGE]** `cge_calibrate` option that a value is for: `sigmaq`, `sigmat`, `sigmava`, `incelas`, `frisch`, `wagecurve` or `mortelas` in the package's library.

| *options* | Description |
|---|---|
| *Selection* | |
| `country(`*codes*`)` | entries for these countries or groups |
| `category(`*names*`)` | entries in these categories |
| `source(`*ids*`)` | entries from these sources |
| `id(`*ids*`)` | these entries |
| *Output* | |
| `clear` | load the selected entries into memory |
| `values` | return values for **[CGE]** `cge_calibrate` in `r(values)` |
| `map(`*spec*`)` | with `values`, categories of commodities or activities |
| `others(`*category*`)` | with `values`, the category of those not in `map()` |

## Options

`country(`*codes*`)`, `category(`*names*`)`, `source(`*ids*`)` and `id(`*ids*`)` select entries. Each takes one or more values, and an entry is selected when it matches every option given, and *parameter* if it is given. Countries are ISO 3166 alpha-3 codes, for example `TGO`, or the groups `AFR` (Africa), `WLD` (international) and `DEV` (developing countries). The case of the values does not matter. If no entry is selected, the command stops with error 111.

`using` *filename* adds the entries of a CSV file in the library's layout, with at least the columns `id`, `parameter`, `country`, `category`, `value` and `source`, for example values that the user is licensed to use or has estimated. Other columns of the layout are empty when absent. Entry identifiers must be unique across the library and the file, and every entry needs a numeric value.

`clear` loads the selected entries into memory, replacing the data, with every column of the library and the authors, year, title, publication and address of each source. `value`, `se`, `lower` and `upper` are numeric; the other variables are strings. `clear` may not be combined with `values`.

`values` returns values for **[CGE]** `cge_calibrate` in `r(values)` and displays the entries used, with their definitions, sources and notes. For `sigmaq`, `sigmat` and `incelas` it returns one value per commodity, and for `sigmava` one per activity, in the order of **[CGE]** `cge_map`, which must have been run. Each commodity or activity takes the selected entry of its category, given by `map()` or `others()`, or the single selected entry if it has no category; if no entry or more than one matches, the command stops and names the entries. For `frisch`, `wagecurve`, `mortelas` and `sickelas` it returns one number, from the single selected entry; `wagecurve()` of **[CGE]** `cge_calibrate` takes it with the factor, as in `` wagecurve(lab:`r(values)') ``. *parameter* is required.

`map(`*spec*`)` gives categories to commodities or activities, as *element*`:`*category* pairs, for example `map(com_agr:food com_hlt:health)`. It needs `values`.

`others(`*category*`)` gives a category to the commodities or activities not listed in `map()`. It needs `values`.

## Remarks and examples

The library holds 531 entries (chapter 26):

| Parameter | Entries | Coverage | Source |
|---|---|---|---|
| `sigmaq` | 51 | aggregate Armington elasticities, African countries | Devarajan, Go and Robinson (2023) |
| `sigmat` | 51 | aggregate CET elasticities, the same countries | Devarajan, Go and Robinson (2023) |
| `incelas` | 423 | income elasticities for nine consumption categories, 47 African countries | Muhammad et al. (2011) |
| `frisch` | 1 | the Frisch parameter, 144 countries pooled | Muhammad et al. (2011) |
| `sigmava` | 1 | substitution between capital and labour, 30 African countries pooled | Hosni and Ouakil (2024) |
| `wagecurve` | 2 | international, and South Africa | Blanchflower and Oswald (1995); Kingdon and Knight (2006) |
| `mortelas` | 2 | under-five and maternal mortality with respect to government health expenditure | Bokhari, Gai and Gottret (2007) |

The categories of the income elasticities are `food` (with beverages and tobacco), `clothing`, `housing`, `furnishings`, `health`, `transport` (with communication), `recreation`, `education` and `other`; the trade elasticities have the category `aggregate`. The mortality elasticities are for reference: their outcome and driver differ from those of `mortelas()` of **[CGE]** `cge_calibrate`.

Read the definitions before using a value. Aggregate trade elasticities are lower than sector ones, so they set a level rather than every sector's value (Devarajan, Go and Robinson 2023). The income elasticities come from a cross-country demand system for 2005, not from a household survey of the country. Two values follow the package's conventions rather than the sources: the wage-curve elasticity is the absolute value of the published negative elasticity, and the Frisch parameter is the inverse of the published income flexibility.

### Example 1: listing and loading entries

Without `clear` or `values`, the selected entries are listed, up to 60, with the location of each value in its source:

```
. cge_elasticities sigmaq, country(TGO SLE BWA)

Entry           Parameter Ctry Category        Value    S.e.  Source, location
-------------------------------------------------------------------------------------------------------------
DGR2023-ARM-BWA  sigmaq    BWA  aggregate       0.905   0.353  DGR2023, Appendix Table A1, p. 25 (PDF p. 27)
DGR2023-ARM-SLE  sigmaq    SLE  aggregate       0.430   0.171  DGR2023, Appendix Table A1, p. 29 (PDF p. 31)
DGR2023-ARM-TGO  sigmaq    TGO  aggregate       0.620   0.061  DGR2023, Appendix Table A1, p. 29 (PDF p. 31)
-------------------------------------------------------------------------------------------------------------
clear loads the entries with their definitions, notes and sources
```

With `clear`, the entries are loaded with all their details:

```
. cge_elasticities incelas, country(TGO) clear
9 entries of the elasticity library loaded

. list id category value se, noobs

  +----------------------------------------------------------+
  |                           id      category   value   se |
  |----------------------------------------------------------|
  |        MSMR2011-INC-TGO-food          food    .804    . |
  |    MSMR2011-INC-TGO-clothing      clothing    .968    . |
  |     MSMR2011-INC-TGO-housing       housing   1.076    . |
  | MSMR2011-INC-TGO-furnishings   furnishings   1.055    . |
  |      MSMR2011-INC-TGO-health        health   1.886    . |
  |----------------------------------------------------------|
  |   MSMR2011-INC-TGO-transport     transport   1.228    . |
  |  MSMR2011-INC-TGO-recreation    recreation   3.733    . |
  |   MSMR2011-INC-TGO-education     education    .932    . |
  |       MSMR2011-INC-TGO-other         other   1.913    . |
  +----------------------------------------------------------+
```

The library has no standard errors for these income elasticities.

### Example 2: values for calibration

The values are given for the commodities and activities of a mapped model, here the synthetic SAM:

```
. cge_synthetic, clear
(16 vars, 15 obs)

Synthetic SAM loaded (standard layout): 15 accounts, total   1743.00
Largest row-column imbalance:  0.00e+00

. cge_sam, labels(account)
SAM read: 15 accounts, total 1743
Largest row-column imbalance:  0.00e+00

. quietly cge_map, activities(act_agr act_man act_ser) commodities(com_agr com_man com_ser) ///
>     factors(lab cap) households(hhd) government(gov) savinv(sav) row(row)                ///
>     dirtax(tax_dir) saltax(tax_sal) imptax(tax_imp)
```

```
. cge_elasticities sigmaq, country(TGO) values
sigmaq: 0.620 0.620 0.620
  DGR2023-ARM-TGO (TGO): aggregate Armington elasticity between imports and domestic goods, estimate
> d from national accounts series
    DGR2023 (Devarajan, S.; Go, D. S.; Robinson, S. 2023), Appendix Table A1, p. 29 (PDF p. 31); dat
> a period from Table A3
    Note: aggregate elasticities are lower than sector ones (the source, PDF p. 20); set the level,
> not every sector

. local sq `r(values)'

. cge_elasticities incelas, country(TGO) values map(com_agr:food) others(other)
incelas: 0.804 1.913 1.913
  MSMR2011-INC-TGO-food (TGO): income (total expenditure) elasticity of demand for food, beverages a
> nd tobacco, at geometric mean prices
```

```
    MSMR2011 (Muhammad, A.; Seale, J. L., Jr.; Meade, B.; Regmi, A. 2011), Appendix table 1, p. 22 (
> PDF p. 28)
  MSMR2011-INC-TGO-other (TGO): income (total expenditure) elasticity of demand for other goods and
> services, at geometric mean prices
    MSMR2011 (Muhammad, A.; Seale, J. L., Jr.; Meade, B.; Regmi, A. 2011), Appendix table 1, p. 22 (
> PDF p. 28)

. local ie `r(values)'

. cge_elasticities frisch, values
frisch: -1.3661
  MSMR2011-FRISCH (WLD): Frisch parameter, the inverse of the income flexibility phi (the inverse of
>  the income elasticity of the marginal utility of income)
    MSMR2011 (Muhammad, A.; Seale, J. L., Jr.; Meade, B.; Regmi, A. 2011), Table 3, p. 12 (PDF p. 18
> )
    Note: phi is assumed constant across countries in the source; the source system is not an LES

. local fr `r(values)'
```

The one Armington elasticity of Togo is given to every commodity. The income elasticities need a category for each commodity: agriculture takes the elasticity of food, and the other commodities that of other goods and services. The values are then options of **[CGE]** `cge_calibrate`:

```
. cge_calibrate, sigmava(0.8) sigmaq(`sq') sigmat(2) incelas(`ie') frisch(`fr')
Household demand: linear expenditure system
Calibrated: 93 equations, 93 free variables
Benchmark replication: largest scaled residual  3.55e-16
```

With two countries selected, each commodity matches two entries, and the command stops:

```
. cge_elasticities sigmaq, country(TGO SLE) values
com_agr: 2 entries match (DGR2023-ARM-SLE DGR2023-ARM-TGO); narrow the choice with country(), catego
> ry(), map() or others()

r(459);
```

The results name the entry used for each element:

```
. cge_elasticities sigmat, country(SLE) values
sigmat: 0.571 0.571 0.571
  DGR2023-CET-SLE (SLE): aggregate CET elasticity between exports and domestic sales, estimated from
>  national accounts series
    DGR2023 (Devarajan, S.; Go, D. S.; Robinson, S. 2023), Appendix Table A2, p. 35 (PDF p. 37); dat
> a period from Table A3
    Note: aggregate elasticities are lower than sector ones (the source, PDF p. 20); set the level,
> not every sector

. return list

scalars:
                  r(N) =  3

macros:
          r(parameter) : "sigmat"
                r(ids) : "DGR2023-CET-SLE DGR2023-CET-SLE DGR2023-CET-SLE"
             r(values) : "0.571 0.571 0.571"
```

### Example 3: a file of the user's own entries

A file of additional entries has one row per entry and at least the columns `id`, `parameter`, `country`, `category`, `value` and `source`. The library's other columns, `country_name`, `se`, `lower`, `upper`, `published`, `definition`, `method`, `observations`, `period`, `location`, `check` and `notes`, may be added. Here a file gives two sector Armington elasticities, with their own categories:

```
. type myvalues.csv
id,parameter,country,category,value,source,definition
MY2026-ARM-AGR,sigmaq,TGO,agriculture,1.4,MY2026,sector Armington elasticity
MY2026-ARM-MAN,sigmaq,TGO,manufactures,2.1,MY2026,sector Armington elasticity
```

```
. cge_elasticities sigmaq using myvalues.csv, source(MY2026) values ///
>     map(com_agr:agriculture) others(manufactures)
sigmaq: 1.4 2.1 2.1
  MY2026-ARM-AGR (TGO): sector Armington elasticity
    MY2026,
  MY2026-ARM-MAN (TGO): sector Armington elasticity
    MY2026,
```

The file has no `location` column, so none is displayed after the source.

## Stored results

`cge_elasticities` stores the following in `r()`:

| | |
|---|---|
| *Scalars* | |
| `r(N)` | number of entries selected, or of values returned |
| *Macros* | |
| `r(ids)` | entries selected, or the entry used for each value |
| `r(values)` | values for **[CGE]** `cge_calibrate` (with `values`) |
| `r(parameter)` | parameter (with `values`) |

## References

Blanchflower, D. G., and A. J. Oswald. 1995. An introduction to the wage curve. *Journal of Economic Perspectives* 9(3): 153–167.

Bokhari, F. A. S., Y. Gai, and P. Gottret. 2007. Government health expenditures and health outcomes. *Health Economics* 16(3): 257–273.

Devarajan, S., D. S. Go, and S. Robinson. 2023. Trade elasticities in aggregate models: Estimates for 191 countries. World Bank Policy Research Working Paper 10490.

Hosni, M., and H. Ouakil. 2024. Estimation de l'élasticité de substitution entre le capital et le travail de l'économie africaine : une analyse en données de panel. *Revue AME* 6(3): 247–269.

Kingdon, G. G., and J. Knight. 2006. How flexible are wages in response to local unemployment in South Africa? *ILR Review* 59(3): 471–495.

Muhammad, A., J. L. Seale Jr., B. Meade, and A. Regmi. 2011. *International Evidence on Food Consumption Patterns: An Update Using 2005 International Comparison Program Data*. Technical Bulletin TB-1929, Economic Research Service, U.S. Department of Agriculture.

## Also see

**[CGE]** `cge_calibrate` — Calibrate the model to the SAM
**[CGE]** `cge_sensitivity` — Sensitivity of a scenario's results to the elasticities
**Chapter 26** — The elasticity library

# `cge_equation` — User equations: new relations among the model's variables

## Description

`cge_equation` adds an equation to the calibrated model, written in a small language of the model's variables and parameters, for one element or for every element of a set. With **[CGE]** `cge_variable`, which adds variables, it extends the model without changing its code: Stata compiles the equations for its solver, and **[CGE]** `cge_gams` writes them into the GAMS program, so that GAMS solves the same model. `cge_equation, list` lists the user variables and equations, and `cge_equation, clear` removes them.

## Quick start

Government purchases of services at `LAM` times their base level, with `LAM` a user variable

```
cge_variable LAM, value(1)
cge_equation lam: QG(com_ser) = LAM * base(QG(com_ser))
```

Fix `LAM` and let the scaling factor of government consumption adjust; then raise it by 10 per cent

```
cge_closure, swap(LAM GADJ)
cge_shock LAM, value(1.1)
```

An equation for every commodity

```
cge_variable PIX(c), value(1)
cge_equation pix(c): ln(PIX(c)) = ln(PQ(c)) - ln(CPI)
```

An equation for the commodities with an import tariff only

```
cge_variable LQM(c)$(tm(c) > 0), value(1)
cge_equation lqm(c)$(tm(c) > 0): QM(c) = LQM(c) * base(QM(c))
```

List the user variables and equations; remove them

```
cge_equation, list
cge_equation, clear
```

## Syntax

Add an equation

**`cge_equation`** *name*[`(`*i*[`,`*j*]`)`][`$(`*condition*`)`]]`:` *expression* = *expression* [`,` `replace(`*block*`)`]

Add an inequality paired with a variable (from version 0.9.0)

**`cge_equation`** *name*[`(`*i*[`,`*j*]`)`][`$(`*condition*`)`]]`:` *expression* `=l=`|`=g=` *expression*`,` `perp(`*variable*[`,` *up* | *down* ]`)`

List the user variables and equations

**`cge_equation`**`, list`

Remove the user variables and equations

**`cge_equation`**`, clear`

*name* has at most 24 characters, letters, digits and underscores, and may not be the name of another user equation; a variable defined by **[CGE]** `cge_variable` gives its name to its definition. *i* is an index, the letter of a set or the name of a user set (**[CGE]** `cge_set`), possibly followed by digits. A condition, `$(`*condition*`)`, restricts the equation to the elements that meet it (from version 0.7.2).

## Options

`perp(`*variable*`[, `*up*|*down*`])` (from version 0.9.0) pairs an inequality with a variable fixed under the closure, which stays at its level while the inequality has room to spare and moves, up (the default) or down, when it binds. *variable* is a variable element, such as `RA(act_man)` or `GADJ`, or, for an equation with an index, the variable at that index, as in `RA(a)`. See *Inequalities* below.

`replace(`*block*`)` (from version 0.9.3) makes the equation take the place of the elements of the built-in equation block *block*, named as in chapter 7: the elements with the labels of its own, or, for an equation without an index, the block without an index or the element given, as in `replace(T2(com_agr))`. See *Replacing built-in equations* below.

`list` lists the user variables, with their definitions or values, and the user equations. It is the default when only a comma is given.

`clear` removes every user variable and equation. Swaps and constraints that involve user variables must be removed first, with `cge_closure, swap(none)` and `cge_constraint, clear`.

## Remarks and examples

### The language

Expressions are built from

| Term | Meaning |
|---|---|
| numbers | such as `2`, `0.5` or `1e-3` |
| `+ - * /` and `^` | arithmetic; `^` is the power |
| `( )` | grouping |
| `exp()`, `ln()` or `log()`, `sqrt()` | functions |
| `max()`, `min()`, `abs()`, `sqr()`, `log10()` | functions of GAMS: the largest and smallest of two or more arguments, the absolute value, the square and the logarithm to base 10 (from version 0.9.7) |
| `ifthen(`*condition*`, `*x*`, `*y*`)` | *x* where the condition holds, else *y*; the condition uses parameters, lags and numbers (from version 0.9.7) |
| *VAR*, *VAR*`(`*e*`)`, *VAR*`(`*e1*`, `*e2*`)` | a model or user variable at an element, given by its label, such as `com_hlt`, or by the index of the equation or of a sum |
| *par*, *par*`(`*e*`)` | a parameter, at its current value |
| *upar*, *upar*`(`*e*`[, `*e2*`])` | a user parameter (**[CGE]** `cge_parameter`), at its current value |
| `base(`*VAR*`[(`*e*`)])` | the base value of a variable |
| `sum(`*i*`, `*expression*`)` | the sum over the elements of the set of index *i* |
| `sum((`*i*`,`*j*`), `*expression*`)` | the sum over every pair of elements of the sets of *i* and *j* (from version 0.9.1) |
| `sum(`*map*`(`*i*`,`*j*`), `*expression*`)` | the sum over the pairs of a set of pairs (**[CGE]** `cge_set`; from version 0.9.1) |
| `prod(`*i*`, `*expression*`)` | the product over the set of index *i*, with the domains of `sum()` (from version 0.9.7) |
| `smax(`*i*`, `*expression*`)`, `smin()` | the largest or smallest value over the set, with the domains of `sum()` (from version 0.9.7) |
| `card(`*set*`)` | the number of elements of a set, or of pairs of a set of pairs (from version 0.9.7) |
| *macro*`(`*x*`, ...)` | a macro defined by **[CGE]** `cge_macro`, written out (from version 0.9.7) |
| `ord(`*i*`)` | the position of the element of index *i* in its set, from 1 |
| `lag(`*VAR*`[(`*e*`)][, `*k*`])` | the value of a variable *k* periods before, *k* from 1 to 4, 1 if omitted (from version 0.9.2) |
| `sum(`*i*`$(`*condition*`), `*expression*`)` | the sum over the elements for which the condition holds |
| *term*`$(`*condition*`)` | the term where the condition holds, else 0 |

Variables are written in upper case, with as many elements as they have indexes, and parameters in lower case. The parameters are `tm`, `te`, `tq`, `ta`, `tv`, `kappa`, `pwm`, `pwe`, `ad`, `af`, `qg`, `qinv`, `trdg`, `trdr`, `trfr`, `trgr`, `trrg`, `tshi`, `qshi`, `trshi`, `trshir`, `rd`, `rf`, `ddebt`, `fdebt`, `qlf`, `wmin`, `wref`, `sick` and `tfy`, as in **[CGE]** `cge_shock`, and `trgg`, transfers within government, and `hs0`, the base volume of health services; `tf`, which has two indexes, and `mort` cannot be used. A parameter takes its value in the simulation being solved, with any shock.

The indexes are `a` activities, `c` commodities, `f` factors, `h` households, `d` households and enterprises, `i` institutions that receive factor income and `n` nests of value added, and the names of user sets (**[CGE]** `cge_set`), each possibly followed by digits, as in `c2`. An equation *name*`(`*i*`)` is written for every element of the set of *i*; an index that is neither that of the equation nor that of an enclosing sum is an error. Sums may contain sums. From version 0.9.1, an equation or a variable may have two different indexes, *name*`(`*i*`,`*j*`)`, and is then written for every pair of elements of their sets (Example 7).

`^` binds most tightly and groups from the right; then come `*` and `/`, and then `+` and `-`, which group from the left. A leading minus sign applies after any power: `-X^2` is $-(X^2)$.

### Conditions

From version 0.7.2, a condition written `$(`*condition*`)`, as in GAMS, restricts a sum, a term, or the domain of an equation or a variable, *name*`(`*i*`)$(`*condition*`)`. A term is a number, a reference such as `PQ(c)`, a function or an expression in parentheses. Besides the terms of expressions, a condition may use

| Term | Meaning |
|---|---|
| `> < >= <= == !=` | comparisons, 1 if true and 0 if not |
| `&`, `\|`, `!` | and, or, not; any value other than 0 is true |
| *set*`(`*e*`)` | 1 if element *e* belongs to a user set or a model set, as in `goods(c)` |
| *map*`(`*i*`,`*j*`)` | 1 if the pair belongs to a set of pairs (from version 0.9.1) |
| `sameas(`*i*`,` *label*`)` | 1 if the element of index *i* is *label* |
| `ord(`*i*`)` | the position of the element in its set |

Arithmetic is evaluated first, then comparisons, `!`, `&` and `|`. Comparisons, `sameas()` and set membership are allowed only in conditions.

Conditions are evaluated with the variables and parameters at their base values, when the equation is added and when the model is calibrated again; they choose the terms and the equation elements, which stay the same in every scenario. A condition that is missing at the base, such as `1/tm(c) > 0` where a tariff is 0, and a domain that no element meets are errors. Stata reads `$`*name* on a command line as a global macro, so the condition is always written in parentheses.

### Inequalities

From version 0.9.0, a user equation may be an inequality, written as in GAMS: *lhs* `=l=` *rhs*, or `<=`, for at most, and *lhs* `=g=` *rhs*, or `>=`, for at least. An inequality is paired with a variable, `perp(`*variable*`)`, as an equation is paired with a variable in a mixed complementarity problem: the variable stays at its level while the inequality holds with room to spare, and moves, in the direction given, when it binds, by as much as keeps it binding. The paired variable must be fixed under the closure: a premium or rent of **[CGE]** `cge_constraint` (`RM`, `RE`, `RA`, which go to the government), a scaling factor such as `GADJ`, a tax adjuster that the closure fixes, or an exogenous user variable of **[CGE]** `cge_variable`. Each element of an inequality needs its own paired element.

Inequalities extend **[CGE]** `cge_constraint`, which bounds one model variable, to any expression of the user language, with sets, parameters and conditions. They are solved in the same way, in a smoothed form and then exactly in the regimes found; `cge_constraint, list` shows them, and **[CGE]** `cge_gams` writes them into a mixed complementarity problem. They are kept when the model is saved, calibrated again or given another closure; one whose paired variable a new closure frees is left out, with a note, until a closure fixes it again. `cge_constraint, clear` keeps them; `cge_equation, clear` removes them with the other user equations.

### Lags

From version 0.9.2, `lag(`*VAR*`)` is the value of a variable in the period before, and `lag(`*VAR*`(`*e*`),` *k*`)` its value *k* periods before, for *k* from 1 to 4, as *VAR*`(t-1)` and *VAR*`(t-`*k*`)` in a GAMS model indexed by time. In a period of a dynamic run (**[CGE]** `cge_dynamic`), the values are those of the periods solved before it, and of the base before the first period; within the period they are given, as the capital stock is, so a lag adds no variable to the system. In a static simulation the period before is the base, and `lag(`*VAR*`)` is the base value of *VAR*. Lags carry stocks and partial adjustment that the model does not update itself from one period to the next (Example 8). A defined variable may not refer to its own lag, since its base value would depend on itself: it is given `value()` and an equation instead.

### Replacing built-in equations

From version 0.9.3, `replace(`*block*`)` makes an equation take the place of elements of a built-in equation block, as a GAMS modeller would edit an equation of the model. An equation with an index replaces the elements of the block with the labels of its own; its indexes run over the sets of the block, or over user sets within them, in the same order. An equation without an index replaces a block without an index, or the element given, `replace(`*block*`(`*e*`))`. The replaced elements leave the system, which therefore stays square, and the closure is unchanged. Section 23.7 lists the blocks that may be replaced. Those with derivatives coded in the package cannot be: equations (P7), (P8), (P9), (I1), (I8), (I9), (S1), (S2), (S3) and (S5). A replacement must keep the accounting of the model consistent: Walras's law, whose residual `WALRAS` every simulation reports, shows when it does not (Example 9).

### Functions and macros

From version 0.9.7, expressions may use the functions of GAMS that a square system solves, `max()`, `min()`, `abs()`, `sqr()`, `log10()` and `ifthen()`, the products and extremes over sets, `prod()`, `smax()` and `smin()`, the size of a set, `card()`, and macros (**[CGE]** `cge_macro`), as GAMS's `\$macro`. `prod()`, `smax()` and `smin()` take the domains of `sum()`: one index, two indexes, a set of pairs, and a condition. A product over a set that no element meets is 1, as an empty sum is 0; a maximum or minimum over one is an error (459), since GAMS's would be $-\infty$ or $+\infty$, no number a model can use. Over two indexes or a set of pairs, the elements of the first index without a pair are left out.

The condition of `ifthen()` is a condition as above, evaluated at the current values of the parameters and lags, so that a shock to a parameter switches it. It may not use variables: a switch on endogenous variables makes the system discontinuous, and GAMS's CNS and MCP models refuse it. Such a switch is an inequality paired with a variable (*Inequalities* above), which GAMS solves as a complementarity problem.

`max()`, `min()` and `abs()` are continuous with a kink where two arguments are equal, or the argument of `abs()` is zero. The solver uses the derivative of the first of the equal arguments there, and 0 for `abs()`; a model whose solution lies on a kink may need more iterations, as it may in GAMS. Example 10 uses each function and a macro.

### Equations and the closure

Each equation element needs a free variable, so that the closure stays square: a variable added by `cge_variable` with `value()`, or a variable that a swap frees. In the example below, the equation determines the new variable `LAM`; the swap then fixes `LAM` and frees the scaling factor of government consumption, so that `GADJ` adjusts to satisfy the equation. The base need not satisfy a new equation: if it does not, `cge_equation` says so, and the equation then acts as a shock, moving the solution from the base.

User equations are saved with the model, compiled again when the model is calibrated again, and solved in every simulation, dynamic run and sensitivity draw.

### Example 1: government purchases of services 10 per cent above the base

The synthetic SAM is mapped and calibrated as in **[CGE]** `cge_solve`:

```
. cge_variable LAM, value(1)
User variable LAM: 1 element(s), base value 1
Add the equations that determine it with cge_equation
```

```
. cge_equation lam: QG(com_ser) = LAM * base(QG(com_ser))
User equation lam: 1 element(s)

. cge_closure, swap(LAM GADJ)
Closure
  external account     flexexr
  government           flexgsav
  savings-investment   savings
  numeraire            cpi
  factor lab           mobile
  factor cap           mobile
  swap                 LAM fixed, GADJ free

. cge_shock LAM, value(1.1)
Variable LAM changed for 1 element(s)

. cge_solve, name(qgser)
Simulation qgser solved in 5 iteration(s) (newton)
Largest scaled residual  2.50e-14; WALRAS = -2.57e-13
```

Government purchases of services are 10 per cent above the base, and, with *GADJ* adjusting, so are the other purchases:

```
. quietly cge_results, sim(qgser) clear nodisplay

. list variable element1 base value if inlist(variable, "LAM", "GADJ", "QG"), noobs

  +------------------------------------+
  | variable   element1   base   value |
  |------------------------------------|
  |       QG    com_agr      0       0 |
  |       QG    com_man      5     5.5 |
  |       QG    com_ser     40      44 |
  |     GADJ                 1     1.1 |
  |      LAM                 1     1.1 |
  +------------------------------------+
```

### Example 2: indexed equations and sums

An equation for every commodity defines a price index relative to the CPI in implicit form, and an equation with sums gives a volume index of household consumption, the value of consumption over its value at base quantities:

```
. cge_variable PIX(c), value(1)
User variable PIX(c): 3 element(s), base values 1 1 1
Add the equations that determine it with cge_equation

. cge_equation pix(c): ln(PIX(c)) = ln(PQ(c)) - ln(CPI)
User equation pix: 3 element(s)

. cge_variable CVOL, value(1)
User variable CVOL: 1 element(s), base value 1
Add the equations that determine it with cge_equation

. cge_equation cvol: sum(c, PQ(c) * QH(c, hhd)) = CVOL * sum(c, PQ(c) * base(QH(c, hhd)))
User equation cvol: 1 element(s)

. quietly cge_solve, name(qgser)

. quietly cge_results, sim(qgser) clear nodisplay

. list variable element1 base value if inlist(variable, "PIX", "CVOL"), noobs

  +----------------------------------------+
  | variable   element1   base       value |
  |----------------------------------------|
  |      PIX    com_agr      1    1.000056 |
  |      PIX    com_man      1   .99985438 |
  |      PIX    com_ser      1   1.0000966 |
  |     CVOL                 1   1.0000516 |
  +----------------------------------------+
```

The variables could equally be defined by **[CGE]** `cge_variable`, with an expression; the equations show the functions, an index and a sum with `base()`.

### Example 3: listing

```
. cge_equation, list
variable LAM: value 1
variable PIX(c): value 1
variable CVOL: value 1
equation lam: QG(com_ser) = LAM * base(QG(com_ser))
equation pix(c): ln(PIX(c)) = ln(PQ(c)) - ln(CPI)
equation cvol: sum(c, PQ(c) * QH(c, hhd)) = CVOL * sum(c, PQ(c) * base(QH(c, hhd)))

. return list

scalars:
        r(N_equations) =  5
        r(N_variables) =  3
```

`r(N_equations)` counts equation elements: one for `lam`, three for `pix` and one for `cvol`.

### Example 4: errors, and removing the user equations

A reference to an element that does not exist is an error, and the user equations cannot be removed while a swap involves a user variable:

```
. cge_equation bad: QG(com_xxx) = 1
QG(com_xxx) is not an element of QG

r(111);

. cge_equation, clear
swaps involve user variables: remove them first with cge_closure, swap(none)

r(459);

. cge_closure, swap(none)
Closure
  external account     flexexr
  government           flexgsav
  savings-investment   savings
  numeraire            cpi
  factor lab           mobile
  factor cap           mobile

. cge_equation, clear
User variables and equations removed
```

### Example 5: conditions

Import spending on the commodities that carry a tariff, the tariff revenue of each, a variable and an equation over the same domain, and government purchases of goods, with goods a user set:

```
. cge_set goods = c, except(com_ser)
User set goods: 2 element(s): com_agr com_man

. cge_variable MT = sum(c$(tm(c) > 0), PM(c) * QM(c)), nominal
User variable MT: 1 element(s), base value 88
and its definition, equation MT

. cge_variable TMV(c)$(tm(c) > 0) = tm(c) * PM(c) * QM(c)
User variable TMV(c)$(tm(c) > 0): 2 element(s), base values 2.2 6.6
and its definition, equation TMV

. cge_variable LQM(c)$(tm(c) > 0), value(1)
User variable LQM(c)$(tm(c) > 0): 2 element(s), base values 1 1
Add the equations that determine it with cge_equation

. cge_equation lqm(c)$(tm(c) > 0): QM(c) = LQM(c) * base(QM(c))
User equation lqm: 2 element(s)
```

```
. cge_variable GGOODS = sum(c, (PQ(c) * QG(c))$(goods(c)))
User variable GGOODS: 1 element(s), base value 5
and its definition, equation GGOODS

. cge_shock tm, value(0)
Parameter tm changed for 3 element(s)

. cge_solve, name(notariff)
Simulation notariff solved in 7 iteration(s) (newton)
Largest scaled residual  3.68e-14; WALRAS = -3.83e-14

. quietly cge_results, sim(notariff) clear nodisplay

. list variable element1 base value if inlist(variable, "MT", "TMV", "LQM", "GGOODS"), noobs

  +-----------------------------------------+
  | variable   element1   base        value |
  |-----------------------------------------|
  |       MT                88    89.296578 |
  |      TMV    com_agr    2.2            0 |
  |      TMV    com_man    6.6            0 |
  |      LQM    com_agr      1    1.1118433 |
  |      LQM    com_man      1    1.0500471 |
  |-----------------------------------------|
  |   GGOODS                 5    4.8950264 |
  +-----------------------------------------+
```

`TMV` and `LQM` exist for the commodities with a tariff only. With the tariffs removed, the tariff revenue is 0, while the domain stays as it was in the base. A condition that cannot be evaluated at the base, or that no element meets, is an error:

```
. cge_variable X1 = sum(c$(1 / tm(c) > 0), PQ(c))
a condition is missing at the base: ( ( 1 / tm ( com_ser ) > 0 ) )

r(459);

. cge_variable X2(c)$(tm(c) > 1) = PQ(c)
user variable X2: no element meets the condition

r(459);

. cge_variable X3 = sum(c, PQ(c) * (tm(c) > 0))
comparisons and & | are allowed only in conditions ($)

r(198);
```

### Example 6: inequalities

An import quota on manufactures, paired with its premium; government revenue at least at its base, with the sales tax rates adjusting; and a relative price held within narrow bounds. The tariffs are removed:

```
. cge_shock, reset
All shocks removed

. cge_equation quota: QM(com_man) =l= 1.02 * base(QM(com_man)), perp(RM(com_man))
User inequality quota: 1 element(s), QM(com_man) <= 1.02 * base(QM(com_man)), paired with RM(com_man), which may move up

. cge_equation rev: YG >= base(YG), perp(TQADJ)
User inequality rev: 1 element(s), YG >= base(YG), paired with TQADJ, which may move up

. cge_variable PRA = PQ(com_agr) / CPI, lower(0.9995) upper(1.0005)
User variable PRA: 1 element(s), base value 1
and its definition, equation PRA, held within its bounds

. cge_shock tm, value(0)
Parameter tm changed for 3 element(s)

. cge_solve, name(ineq)
Simulation ineq solved in 17 iteration(s) (newton)
Largest scaled residual  8.88e-16; WALRAS = -2.43e-14
Constraints
  inequality quota, RM(com_man) up, rents to gov: binding
```

```
      _S_quota =            0;  RM(com_man) =  .0557255432
  inequality rev, TQADJ up: binding
      _S_rev =            0;  TQADJ =  1.147889005
  PRA >= .9995, bound of a user variable: binding
      PRA =        .9995;  _L_PRA =  .0087603288
  PRA <= 1.0005, bound of a user variable: not binding
      PRA =        .9995;  _U_PRA =            0

. quietly cge_results, sim(ineq) clear nodisplay

. list variable element1 base value if inlist(variable, "QM", "RM", "YG", "TQADJ", "PRA"), noobs sepby(variable)

  +----------------------------------------+
  | variable   element1   base       value |
  |----------------------------------------|
  |       QM    com_agr     20   22.512137 |
  |       QM    com_man     60        61.2 |
  |       QM    com_ser     20   19.256339 |
  |----------------------------------------|
  |       YG                63          63 |
  |----------------------------------------|
  |    TQADJ                 1    1.147889 |
  |----------------------------------------|
  |       RM    com_agr      0           0 |
  |       RM    com_man      0   .05572554 |
  |       RM    com_ser      0           0 |
  |----------------------------------------|
  |      PRA                 1       .9995 |
  +----------------------------------------+
```

Imports of manufactures stop at the quota, with a premium that keeps them there. The revenue lost with the tariffs is made up by higher sales tax rates. The relative price of agricultural goods would fall below its lower bound, and is held at it.

### Example 7: two indexes and sets of pairs

From version 0.9.1, with two households: household spending by commodity and household, for the pairs with a flow in the base, and its total; a set of pairs that maps activities to the commodities they make, the value of their output and a price ratio for the pairs only. The tariffs are removed:

```
. quietly cge_synthetic, clear institutions

. quietly cge_sam, labels(account)

. quietly cge_map, activities(act_agr act_man act_ser) commodities(com_agr com_man com_ser) ///
>     factors(lab cap) households(hhd npi) enterprises(ent) government(gov)               ///
>     savinv(sav) row(row) dirtax(tax_dir) saltax(tax_sal) imptax(tax_imp)

. quietly cge_calibrate, sigmava(0.8) sigmaq(2) sigmat(2)

. cge_variable XH(c,h)$(base(QH(c, h)) > 0) = PQ(c) * QH(c, h), nominal
User variable XH(c,h)$(base(QH(c, h)) > 0): 4 element(s), base values 70 80 70 10
and its definition, equation XH

. cge_variable XS = sum((c,h), XH(c,h)), nominal
User variable XS: 1 element(s), base value 230
and its definition, equation XS

. cge_set map(a,c) = act_agr.com_agr act_man.com_man
User set map(a,c): 2 pair(s): act_agr.com_agr act_man.com_man

. cge_variable VMAP = sum(map(a,c), PA(a) * QA(a)), nominal
User variable VMAP: 1 element(s), base value 240
and its definition, equation VMAP

. cge_variable PR(a,c)$(map(a,c)) = PA(a) / PQ(c)
User variable PR(a,c)$(map(a,c)): 2 element(s), base values 1 1
and its definition, equation PR

. cge_shock tm, value(0)
Parameter tm changed for 3 element(s)

. cge_solve, name(pairs)
Simulation pairs solved in 7 iteration(s) (newton)
Largest scaled residual  5.15e-14; WALRAS = -1.24e-14
```

```
. quietly cge_results, sim(pairs) clear nodisplay

. list variable element1 element2 base value if inlist(variable, "XH", "XS", "VMAP", "PR"), ///
>     noobs sepby(variable)

  +---------------------------------------------------+
  | variable   element1   element2   base       value |
  |---------------------------------------------------|
  |       XH    com_agr        hhd     70   71.886258 |
  |       XH    com_man        hhd     80   82.155723 |
  |       XH    com_ser        hhd     70   71.886258 |
  |       XH    com_ser        npi     10   10.175226 |
  |---------------------------------------------------|
  |       XS                            230   236.10347 |
  |---------------------------------------------------|
  |     VMAP                            240   241.46729 |
  |---------------------------------------------------|
  |       PR    act_agr    com_agr      1   1.0208603 |
  |       PR    act_man    com_man      1   1.0340888 |
  +---------------------------------------------------+
```

`XH` and `PR` exist for the pairs of their domains only. An element of a variable with two indexes is written `e1.e2` in **[CGE]** `cge_shock`, and an inequality with two indexes is paired with a variable over the same indexes, as in `perp(FL(c,h))`.

#### Example 8: lags in a dynamic run

From version 0.9.2, government purchases that move halfway towards revenue each period: `GL` is a partial adjustment of the revenue index, with `GADJ` following it, and `DQA` is the change in the output of manufactures from the period before. The run lasts five periods, with capital accumulating and labour growing:

```
. quietly cge_synthetic, clear

. quietly cge_sam, labels(account)

. quietly cge_map, activities(act_agr act_man act_ser) commodities(com_agr com_man com_ser) ///
>     factors(lab cap) households(hhd) government(gov) savinv(sav) row(row)              ///
>     dirtax(tax_dir) saltax(tax_sal) imptax(tax_imp)

. quietly cge_calibrate, sigmava(0.8) sigmaq(2) sigmat(2)

. cge_variable GL, value(1)
User variable GL: 1 element(s), base value 1
Add the equations that determine it with cge_equation

. cge_equation gl: GL = 0.5 * lag(GL) + 0.5 * YG / base(YG)
User equation gl: 1 element(s)

. cge_variable LAMG, value(1)
User variable LAMG: 1 element(s), base value 1
Add the equations that determine it with cge_equation

. cge_equation lamg: GADJ = LAMG * GL
User equation lamg: 1 element(s)

. quietly cge_closure, swap(LAMG GADJ)

. cge_variable DQA = QA(act_man) - lag(QA(act_man))
User variable DQA: 1 element(s), base value 0
and its definition, equation DQA

. cge_dynamic, name(lagged) periods(0/4) capital(cap) depreciation(0.05) rental(0.15) ///
>     growth(lab:0.02) tfp(0.01)
Period 0: solved in 0 iteration(s)
Period 1: solved in 6 iteration(s)
Period 2: solved in 7 iteration(s)
Period 3: solved in 7 iteration(s)
Period 4: solved in 7 iteration(s)
Dynamic run lagged stored as simulations: lagged0 lagged1 lagged2 lagged3 lagged4

. quietly cge_dynresults, run(lagged) clear
. quietly keep if inlist(variable, "YG", "GL", "GADJ", "DQA")

. keep period variable value

. quietly reshape wide value, i(period) j(variable) string

. rename value* *

. list period YG GL GADJ DQA, noobs

  +-------------------------------------------------------+
  | period          YG          GL        GADJ         DQA |
  |-------------------------------------------------------|
  |      0          63           1           1           0 |
  |      1   64.470947   1.0116742   1.0116742   3.8830865 |
  |      2   66.007748   1.0297081   1.0297081   3.9296494 |
  |      3   67.610613   1.0514462   1.0514462   4.0380824 |
  |      4   69.280843   1.0755711   1.0755711   4.1807903 |
  +-------------------------------------------------------+
```

In period 1, `GL` is $0.5 \times 1 + 0.5 \times YG_1/63$, and in each later period half its value before plus half the revenue index; government purchases therefore lag behind revenue. Before period 0, the lagged values are those of the base, so `DQA` is 0 in period 0.

### Example 9: replacing built-in equations

From version 0.9.3, exports of agricultural goods held at the base, in place of the export supply of (T2), and government purchases of services 10 per cent above the base, in place of (I7); the tariffs are removed:

```
. quietly cge_synthetic, clear

. quietly cge_sam, labels(account)

. quietly cge_map, activities(act_agr act_man act_ser) commodities(com_agr com_man com_ser) ///
>     factors(lab cap) households(hhd) government(gov) savinv(sav) row(row)             ///
>     dirtax(tax_dir) saltax(tax_sal) imptax(tax_imp)

. quietly cge_calibrate, sigmava(0.8) sigmaq(2) sigmat(2)

. cge_equation qe: QE(com_agr) = base(QE(com_agr)), replace(T2(com_agr))
User equation qe: 1 element(s), replacing T2(com_agr)

. cge_equation gs: QG(com_ser) = 1.1 * base(QG(com_ser)), replace(I7(com_ser))
User equation gs: 1 element(s), replacing I7(com_ser)
Note: the base does not satisfy it (largest relative gap  9.09e-02); it acts as a shock

. cge_equation, list
equation qe: QE(com_agr) = base(QE(com_agr)), replace(T2(com_agr))
equation gs: QG(com_ser) = 1.1 * base(QG(com_ser)), replace(I7(com_ser))

. cge_shock tm, value(0)
Parameter tm changed for 3 element(s)

. cge_solve, name(replaced)
Simulation replaced solved in 8 iteration(s) (newton)
Largest scaled residual  1.02e-15; WALRAS =  7.71e-14

. quietly cge_results, sim(replaced) clear nodisplay

. list variable element1 base value if inlist(variable, "QE", "QG", "WALRAS"), noobs sepby(variable)

  +---------------------------------------+
  | variable   element1   base      value |
  |---------------------------------------|
  |       QE    com_agr     20         20 |
  |       QE    com_man     40  41.561651 |
  |       QE    com_ser     15  16.398136 |
  |---------------------------------------|
  |       QG    com_agr      0          0 |
  |       QG    com_man      5          5 |
  |       QG    com_ser     40         44 |
  |---------------------------------------|
  |   WALRAS                0   7.707e-14 |
  +----------------------------------------+
```

Exports of agricultural goods stay at the base while those of manufactures respond to the tariff cut, and Walras's law holds. Replacing the demand for one intermediate input of (Q5) alone, by contrast, would leave the cost of intermediates in (P6) and (P7) at the old coefficients, and `WALRAS` would show the gap. Written in the user's language, the equation of a block itself leaves the solution as it was:

```
. cge_shock, reset
All shocks removed

. cge_equation, clear
User variables and equations removed

. cge_shock tm, value(0)
Parameter tm changed for 3 element(s)

. quietly cge_solve, name(original)

. cge_equation q5(c,a)$(sameas(a, act_man) & base(QINT(c, a)) > 0): ///
>     QINT(c, a) = base(QINT(c, a)) / base(QINTA(a)) * QINTA(a), replace(Q5)
User equation q5: 3 element(s), replacing Q5(com_agr,act_man) Q5(com_man,act_man) Q5(com_ser,act_man
> )

. quietly cge_solve, name(rewritten)

. foreach s in original rewritten {
  2.     quietly cge_results, sim(`s') clear nodisplay
  3.     quietly summarize value if variable == "QA" & element1 == "act_man"
  4.     display "`s'" _col(12) "QA(act_man) = " %14.10f r(mean)
  5. }
original   QA(act_man) = 138.0414156631
rewritten  QA(act_man) = 138.0414156631
```

#### Example 10: functions and macros

From version 0.9.7, the largest export, the smallest output, the product of the composite prices, the imports of manufactures while they carry a tariff, the number of goods, and a CES aggregate of domestic goods and imports defined by a macro, after the tariffs are removed:

```
. cge_shock, reset
All shocks removed

. cge_equation, clear
User variables and equations removed

. cge_set goods = c, except(com_ser)
User set goods: 2 element(s): com_agr com_man

. cge_variable QEMAX = smax(c, QE(c))
User variable QEMAX: 1 element(s), base value 40
and its definition, equation QEMAX

. cge_variable QAMIN = min(QA(act_agr), QA(act_man), QA(act_ser))
User variable QAMIN: 1 element(s), base value 100
and its definition, equation QAMIN

. cge_variable PPROD = prod(c, PQ(c))
User variable PPROD: 1 element(s), base value 1
and its definition, equation PPROD

. cge_variable MSW = ifthen(tm(com_man) > 0, QM(com_man), 0)
User variable MSW: 1 element(s), base value 60
and its definition, equation MSW

. cge_variable NG = card(goods)
User variable NG: 1 element(s), base value 2
and its definition, equation NG
```

```
. cge_macro ces(x, y, d, r) = (d * x^(-r) + (1 - d) * y^(-r))^(-1 / r)
Macro ces(x, y, d, r) = (d * x^(-r) + (1 - d) * y^(-r))^(-1 / r)

. cge_variable CQ(goods) = ces(QD(goods), QM(goods), 0.5, 0.5)
User variable CQ(goods): 2 element(s), base values 35.5556 76.21
and its definition, equation CQ

. cge_shock tm, value(0)
Parameter tm changed for 3 element(s)

. cge_solve, name(functions)
Simulation functions solved in 7 iteration(s) (newton)
Largest scaled residual  3.75e-14; WALRAS = -2.03e-13

. quietly cge_results, sim(functions) clear nodisplay

. list variable element1 base value if inlist(variable, "QEMAX", "QAMIN", "PPROD", ///
>     "MSW", "NG", "CQ"), noobs sepby(variable)

  +---------------------------------------------+
  | variable   element1        base       value |
  |---------------------------------------------|
  |    QEMAX                     40   42.241051 |
  |---------------------------------------------|
  |    QAMIN                    100   99.590677 |
  |---------------------------------------------|
  |    PPROD                      1   .99948476 |
  |---------------------------------------------|
  |      MSW                     60           0 |
  |---------------------------------------------|
  |       NG                      2           2 |
  |---------------------------------------------|
  |       CQ    com_agr   35.555556   37.899037 |
  |       CQ    com_man   76.209992   76.817918 |
  +---------------------------------------------+
```

`MSW` is the imports of manufactures at the base, where the tariff is positive, and 0 once the tariff is removed: the condition switches with the parameter. `CQ` is $\left(0.5\,QD^{-0.5} + 0.5\,QM^{-0.5}\right)^{-2}$ for each good.

## Stored results

`cge_equation` stores the following in `r()`:

| | |
|---|---|
| *Scalars* | |
| `r(N_elements)` | number of elements of the equation, those that meet its condition |
| `r(N_constraints)` | with an inequality: number of constraints of the model |
| `r(N_variables)` | number of user variables (`list`) |
| `r(N_equations)` | number of equation elements (`list`) |
| *Macros* | |
| `r(name)` | name of the equation |

## Methods and formulas

An equation $\ell(x) = r(x)$ with index $i$ is written out for every element $e$ of the set of $i$, with $i$ replaced by $e$ where it is an argument of a variable or a parameter; each `sum(`$j$`, ·)` is written out in the same way over the elements of the set of $j$, inner sums first, so that

$$\texttt{sum(}j\texttt{, }g\texttt{)} = \sum_{k \in J} g\,[\,j = k\,],$$

where $J$ is the set of index $j$ and $g\,[\,j = k\,]$ is $g$ with $j$ replaced by $k$. With a condition $q$ (from version 0.7.2),

$$\texttt{sum(}j\texttt{\$(}q\texttt{), }g\texttt{)} = \sum_{k \in J:\ q[\,j=k\,](x^0,\theta^0) \neq 0} g\,[\,j = k\,],$$

with $q$ evaluated at the base point $x^0$ and the base parameters $\theta^0$; a conditional term $g$`$(`$q$`)` is $g$ where $q(x^0, \theta^0) \neq 0$ and 0 elsewhere, and an equation with a condition is written out only for the elements $e$

with $q[i=e](x^0,\theta^0) \neq 0$. Sums are written out from the outside in, each element substituted before the sums within, so that an inner condition may use the index of an enclosing sum. From version 0.9.1, a sum over two indexes and a sum over a set of pairs $P$ are sums over one index at a time,

$$\texttt{sum((}j,k\texttt{)\$(}q\texttt{),}\ g\texttt{)} = \texttt{sum(}j\texttt{,}\ \texttt{sum(}k\texttt{\$(}q\texttt{),}\ g\texttt{)),} \qquad \texttt{sum(}P(j,k)\texttt{,}\ g\texttt{)} = \texttt{sum(}j\texttt{,}\ \texttt{sum(}k\texttt{\$(}P(j,k)\texttt{)),}\ g\texttt{)),}$$

and an equation with two indexes is written out for every pair of elements that meets its condition. Each side is then compiled to postfix order, with numbers, variables (as positions in the stacked vector of variables), parameters, operators and functions, and `base()` replaced by the number it denotes. The solver evaluates both sides at each point, with the parameters of the simulation, and the equation element is

$$\ell_e(x) - r_e(x) = 0,$$

a row of the block `U` of the system. Its derivatives are exact, from version 0.7.0: the postfix form of each side is differentiated in forward mode, with respect to the variables that the equation element contains (section 10.3). An inequality $\ell(x) \leq r(x)$ paired with $v$, fixed at $\bar{v}$ and moving up, adds a slack variable $s = r(x) - \ell(x)$, defined by a user equation and named `_S_`*name*, and the constraint

$$s \geq 0, \qquad v - \bar{v} \geq 0, \qquad s\,(v - \bar{v}) = 0,$$

solved as the constraints of **[CGE]** `cge_constraint` are (chapter 17); with =g=, $s = \ell(x) - r(x)$, and moving down, $\bar{v} - v \geq 0$. A defined user variable $u = e(x)$ with a lower bound $b$ becomes $u = e(x) + a$, with an adjuster $a$ fixed at zero paired with the constraint $u \geq b$, so that $u = \max(b, e(x))$; an upper bound subtracts a second adjuster.

A lagged value `lag(`$v$`,` $k$`)` (from version 0.9.2) is a number within a period: $v_{t-k}$, the solution of the period $k$ before in a dynamic run, with $v_{t-k} = v^0$ for $t-k$ before the first period, and $v^0$ in a static simulation. Its derivative is zero.

An equation that replaces the element $e$ of a block $B$ (from version 0.9.3) removes the row $B_e$ from the system, $\ell_B(x)_e - r_B(x)_e = 0$, and adds its own; in the GAMS program, the parameter `XRP(`$B$`,` $e$`)` is 1, and the equations of block $B$ leave out the element.

A product, maximum or minimum over a set (from version 0.9.7) is written out as the sum is, with $*$ or as the arguments of `max()` or `min()`,

$$\texttt{prod(}j\texttt{,}\ g\texttt{)} = \prod_{k \in J} g\,[\,j = k\,], \qquad \texttt{smax(}j\texttt{,}\ g\texttt{)} = \max_{k \in J} g\,[\,j = k\,],$$

and `card(`$J$`)` is the number $|J|$. A function $\phi$ of the arguments $a_1, \ldots, a_m$ has the derivative

$$\frac{\partial \phi}{\partial x} = \sum_l \frac{\partial \phi}{\partial a_l} \frac{\partial a_l}{\partial x},$$

with $\partial \max / \partial a_l$ equal to 1 for the first argument at the maximum and 0 for the others (likewise for `min()` and the branch that `ifthen()` takes), $\partial |a| / \partial a = \operatorname{sign}(a)$, $\partial a^2 / \partial a = 2a$ and $\partial \log_{10} a / \partial a = 1/(a \ln 10)$. A macro is replaced by its expression, each argument substituted for its parameter, before the indexes are written out, so a macro adds nothing to the system; **[CGE]** `cge_macro` gives the rules.

Chapter 23 describes the compilation and the GAMS program, in which GAMS differentiates the same equations itself.

## Also see

**[CGE]** `cge_variable` — User variables, defined by an expression or determined by user equations
**[CGE]** `cge_macro` — Macros of the language of user variables and equations
**[CGE]** `cge_closure` — Choose the model closure
**[CGE]** `cge_shock` — Change a parameter or a fixed variable
**[CGE]** `cge_gams` — Write the model as a GAMS program, and solve it in GAMS
**Chapter 23** — User variables and equations

# `cge_gams` — Write the model as a GAMS program, and solve it in GAMS

## Description

`cge_gams` writes the calibrated model and one scenario as a GAMS program in a folder. Anyone with GAMS can then solve the model with PATH, CONOPT, KNITRO, IPOPT or another solver, check the results of cge-stata with a second implementation, or extend the equations. GAMS receives what a user supplies, not the parameters that Stata calibrated: the SAM after the layout rewrites of **[CGE]** `cge_map`, the roles of its accounts and the settings given to **[CGE]** `cge_calibrate`. The GAMS program calibrates every parameter itself, solves the base and then the scenario. Its equations are an implementation of the model written in GAMS, not translated from Mata.

`cge_gams` can also start GAMS from interactive Stata, compare the GAMS results with the Stata solution, export every period of a dynamic run, and import a GAMS solution as a Stata simulation.

## Quick start

Write the model and the scenario of simulation `notariff` to folder `gams_notariff`, to solve with `gams cge_run.gms` there

```
cge_gams using gams_notariff, sim(notariff)
```

Write it again, solve it in GAMS with PATH and compare the results with Stata's

```
cge_gams using gams_notariff, sim(notariff) replace run compare gams(C:/GAMS/54/gams.exe)
```

Solve the files already written with CONOPT, and compare

```
cge_gams using gams_notariff, sim(notariff) noexport run compare solver(conopt) ///
    gams(C:/GAMS/54/gams.exe)
```

Compare results that GAMS wrote earlier, for example after a batch run

```
cge_gams using gams_notariff, sim(notariff) noexport compare
```

Export the current shocks, and after solving in GAMS store the solution as simulation `big_path`

```
cge_gams using big, replace
cge_gams using big, noexport import(big_path)
```

Export every period of dynamic run `base`

```
cge_gams using gams_base, dynamic(base)
```

## Syntax

**`cge_gams`** `using` *folder* [ , *options*]

| *options* | Description |
|---|---|
| *Scenario* | |
| `sim(`*name*`)` | export the shocks and closure of a stored simulation |
| `dynamic(`*name*`)` | export every period of a dynamic run |
| `replace` | replace the files of an earlier export |
| *GAMS* | |

| *options* | Description |
|---|---|
| `run` | solve the exported model in GAMS |
| `gams(`*path*`)` | the GAMS executable; default `gams` |
| `solver(`*name*`)` | the solver; default `path` |
| `type(`*name*`)` | GAMS model type: `cns`, `nlp` or `mcp` |
| *Results* | |
| `compare` | compare the GAMS results with the Stata solution |
| `import(`*name*`)` | store the GAMS solution as simulation *name* |
| `noexport` | use the files already in *folder* |

`sim()` and `dynamic()` may not be combined, nor `import()` and `dynamic()`. `compare` needs `sim()` or `dynamic()`, and `noexport` needs `run`, `compare` or `import()`.

## Options

`sim(`*name*`)` exports the shocks and closure of a stored simulation. Without `sim()` or `dynamic()`, the shocks and closure in memory are exported, the scenario that the next **[CGE]** `cge_solve` would solve, under the name `current`. A period of a dynamic run is exported with `dynamic()`.

`dynamic(`*name*`)` exports every period of a run of **[CGE]** `cge_dynamic`. The changes of each period from the period before, to parameters and to fixed variables, are written to `cge_`*sim*`_par.inc` and `cge_`*sim*`_var.inc`, where *sim* is the period's simulation, such as `base2025`, and the driver solves the periods in order, each from the solution of the one before. The periods must have the same closure, and runs with constraints are not exported.

`replace` allows the files of an earlier export in *folder* to be replaced. The folder, and any parent folder, is created if it does not exist.

`run` solves the exported model in GAMS. Stata deletes any earlier result files, writes a script in the folder, `cge_run.cmd` on Windows and `cge_run.sh` elsewhere, and runs it with `shell`; the script can be run again by hand. It runs `gams cge_run.gms lo=2 lf=cge_run.log` with the solver and model type chosen. Stata for Windows cannot start other programs in batch mode: there, `run` is an error, and `cge_run.gms` is run with GAMS separately and the results compared with `noexport compare`.

`gams(`*path*`)` gives the GAMS executable, for example `gams(C:/GAMS/54/gams.exe)`. By default, `gams` must be on the system path.

`solver(`*name*`)` chooses the solver; the default is `path`. When GAMS runs, it writes an option file with tolerances scaled to the size of the SAM for PATH, CONOPT, IPOPT and KNITRO; other solvers use their defaults.

`type(`*name*`)` chooses the GAMS model type: `cns`, a constrained nonlinear system (PATH, CONOPT, KNITRO); `nlp`, the same equations with a constant objective, for NLP solvers; or `mcp`, a mixed complementarity problem (PATH), which a scenario with constraints needs. The default is `mcp` for a static scenario with constraints, `nlp` for the solvers `ipopt`, `ipopth`, `minos` and `snopt`, and `cns` otherwise; a scenario with constraints cannot be exported as another type. Both the solver and the type can also be set when GAMS is run, as in `gams cge_run.gms --cgesolver=ipopt --cgetype=nlp`.

`compare` compares the GAMS results with the Stata base and with simulation `sim()`, or with every period of the run of `dynamic()`. It reports the GAMS model status and, for the base and for the scenario or each period, the number of values compared and the largest relative difference, with the variable where it occurs.

`import(`*name*`)` stores the GAMS solution of a static scenario as simulation *name*: the free variables take the values of GAMS, the fixed variables those of Stata, and the model's equations are evaluated in Stata at the result, whose largest scaled residual is reported. A model too large for Stata's solver can thus be solved by PATH and its results used in Stata with **[CGE]** `cge_report`, **[CGE]** `cge_results` or **[CGE]** `cge_microsim`. With `noexport`, the results already in *folder* are imported.

`noexport` uses the files already in *folder*, which must hold `cge_run.gms`: with `run` it solves them again, for example after they have been edited, and with `compare` or `import()` alone it uses results that GAMS wrote earlier.

## Remarks and examples

The GAMS program is a second implementation of the model of Part II. GAMS calibrates the parameters from the SAM and the settings and solves the base and the scenario itself, so a GAMS solution that matches Stata's confirms the calibration as well as the solution. The program can also be given to a modeller who works in GAMS, or extended with equations that the package does not have. Chapter 25 describes the GAMS implementation.

### Example 1: exporting a scenario

The synthetic SAM is calibrated as in **[CGE]** `cge_solve`, and every import tariff is removed, with the sales tax rates replacing the revenue:

```
. cge_closure, government(fixgsav sales)
Closure
  external account     flexexr
  government           fixgsav:sales
  savings-investment   savings
  numeraire            cpi
  factor lab           mobile
  factor cap           mobile

. cge_shock tm, value(0)
Parameter tm changed for 3 element(s)

. cge_solve, name(notariff)
Simulation notariff solved in 7 iteration(s) (newton)
Largest scaled residual  1.63e-14; WALRAS =  4.92e-14
```

`cge_gams` writes the model and the scenario to folder `gams_notariff`, which it creates:

```
. cge_gams using gams_notariff, sim(notariff)
Wrote the model (15 accounts) and scenario notariff (2 change(s) from the base) to gams_notariff
Files: cge_model.gms cge_closure.gms cge_write.gms cge_solve.gms cge_data.inc cge_shock_par.inc cge_
> shock_var.inc cge_constraints.inc cge_user.inc cge_user_write.inc cge_run.gms
To solve it: gams cge_run.gms in that folder

. return list

scalars:
         r(N_accounts) =  15
          r(N_changes) =  2
      r(N_constraints) =  0

macros:
            r(closure) : "flexexr fixgsav:sales savings cpi lab:mobile cap:mobile"
               r(name) : "notariff"
```

Two parameters change from the base: the tariffs on agriculture and manufactures (the base tariff on services is zero). The changes are written as ratios to the base values, so that GAMS applies them to its own calibrated values, and the closure follows as codes for the GAMS program:

```
. type gams_notariff/cge_shock_par.inc
* Parameter changes and the closure, written by cge_gams
tm('com_agr') = 0 * tm('com_agr');
tm('com_man') = 0 * tm('com_man');
clext = 1;
clgov = 3;
clsi = 1;
clnum = 1;
fclo('lab') = 1;
fclo('cap') = 1;
```

The driver, `cge_run.gms`, includes the model, then the scenario, and states how to run it:

```
. type gams_notariff/cge_run.gms
$title cge-stata model exported by cge_gams: notariff
$ontext
Written by cge_gams (cge-stata 0.7.1). GAMS calibrates the model from the
SAM and settings in cge_data.inc, solves the base and then scenario notariff,
whose parameter changes and closure are in cge_shock_par.inc and whose
changes to fixed variables are in cge_shock_var.inc. The equations are in
cge_model.gms (docs/model_spec.md). To run it:
  gams cge_run.gms                                    path, as a constrained nonlinear system
  gams cge_run.gms --cgesolver=conopt                 CONOPT (or KNITRO, PATH)
  gams cge_run.gms --cgesolver=ipopt --cgetype=nlp    an NLP solver
Results: cge_base.csv and cge_notariff.csv, as scenario, variable, element1,
element2, value; the first row holds the model status.
$offtext
$if not set cgesolver $setglobal cgesolver path
$if not set cgetype $setglobal cgetype cns
$setglobal data cge_data.inc
$setglobal out cge_base.csv
$include cge_model.gms

* Scenario notariff: parameter changes and the closure, then fixed variables
$include cge_shock_par.inc
$include cge_closure.gms
$include cge_shock_var.inc
$include cge_constraints.inc
$batinclude cge_solve.gms cgecon
file rescen / 'cge_notariff.csv' /;
$batinclude cge_write.gms rescen notariff
```

An earlier export is not overwritten without `replace`:

```
. cge_gams using gams_notariff, sim(notariff)
gams_notariff/cge_model.gms already exists; specify replace

r(602);
```

### Example 2: solving in GAMS

In interactive Stata, `run` starts GAMS and `compare` reads its results, for example

```
. cge_gams using gams_notariff, sim(notariff) replace run compare gams(C:/GAMS/54/gams.exe)
```

Stata for Windows cannot start GAMS in batch mode, so a do-file run in batch mode stops:

```
. cge_gams using gams_notariff, sim(notariff) noexport run
Stata in batch mode cannot start GAMS: run cge_run.gms in the folder with GAMS, then compare with cg
> e_gams ..., sim() compare noexport (or dynamic())

r(198);
```

In batch mode, `gams cge_run.gms` is run in the folder outside Stata, and

```
. cge_gams using gams_notariff, sim(notariff) noexport compare
```

then compares the results. A GAMS licence covering the solver is needed to solve a model of realistic size; GAMS runs solvers outside the licence only for small models. GAMS listing files (`.lst`) name the holder of the licence.

### Example 3: a scenario with constraints

A constraint set by **[CGE]** `cge_constraint` makes the scenario a mixed complementarity problem. Here imports of manufactures may rise by at most 2 per cent when tariffs are removed, with the premium on the quota going to the government:

```
. cge_shock tm, value(0)
Parameter tm changed for 3 element(s)
```

```
. cge_constraint QM, elements(com_man) upper(1.02) scale
Constraint QM(com_man) <= 61.2, RM(com_man) up, rents to gov: not solved

. cge_solve, name(quota)
Simulation quota solved in 11 iteration(s) (newton)
Largest scaled residual  4.97e-14; WALRAS =  1.55e-14
Constraints
  QM(com_man) <= 61.2, RM(com_man) up, rents to gov: binding
      QM(com_man) =         61.2;  RM(com_man) =  .0570352246

. cge_gams using gams_quota, sim(quota)
Wrote the model (15 accounts) and scenario quota (3 change(s) from the base) to gams_quota
Files: cge_model.gms cge_closure.gms cge_write.gms cge_solve.gms cge_data.inc cge_shock_par.inc cge_
> shock_var.inc cge_constraints.inc cge_user.inc cge_user_write.inc cge_run.gms
To solve it: gams cge_run.gms in that folder

. type gams_quota/cge_constraints.inc
* Constraints (model_spec.md, Section 16), written by cge_gams
* QM(com_man) <= 61.2, RM(com_man) up
positive variable SLK1;
equations BND1, LNK1;
scalar xbar1;
xbar1 = RM.l('com_man');
BND1..  (1) * ((61.2) - QM('com_man')) / 60 =g= 0;
LNK1..  RM('com_man') =e= xbar1 + (1) * 1 * SLK1;
RM.lo('com_man') = -inf;
RM.up('com_man') = inf;
rmto(AC, 'com_man') = 0;
rmto('gov', 'com_man') = 1;
model cgecon / cge, LNK1, BND1.SLK1 /;
cgecon.optfile = cge.optfile;
```

The constraint becomes a slack variable `SLK1`, zero or positive, paired with the bound `BND1`, and the adjusting variable, the import premium `RM(com_man)`, is freed and linked to the slack by `LNK1`. The model type is `mcp` by default, for PATH; CONOPT, KNITRO and IPOPT do not solve complementarity problems.

#### Example 4: a dynamic run

With `dynamic()`, every period of a run is exported. A run of two periods after the base, with capital specific to each activity:

```
. cge_constraint, clear
All constraints removed

. cge_shock, reset
All shocks removed

. cge_closure, government(flexgsav) factors(cap:specific)
Closure
  external account     flexexr
  government           flexgsav
  savings-investment   savings
  numeraire            cpi
  factor lab           mobile
  factor cap           specific

. cge_dynamic, name(dn) periods(0/2) capital(cap) depreciation(0.05) rental(0.15) ///
>     growth(lab:0.02) population(0.02) govgrowth(0.03)
Period 0: solved in 0 iteration(s)
Period 1: solved in 7 iteration(s)
Period 2: solved in 6 iteration(s)
Dynamic run dn stored as simulations: dn0 dn1 dn2

. cge_gams using gams_dn, dynamic(dn)
Wrote the model (15 accounts) and the 3 periods of dynamic run dn (16 change(s) in all) to gams_dn
Files: cge_model.gms cge_closure.gms cge_write.gms cge_solve.gms cge_data.inc cge_constraints.inc cg
> e_user.inc cge_user_write.inc cge_run.gms and the scenario files of each period
To solve it: gams cge_run.gms in that folder
```

Each period has its own pair of scenario files, with its changes from the period before: the growth of government flows and of the base of the health link in the parameter file, and the stocks of capital and labour, which are fixed variables, in the variable file:

```
. return list

scalars:
         r(N_accounts) =  15
          r(N_periods) =  3
          r(N_changes) =  16
      r(N_constraints) =  0

macros:
            r(closure) : "flexexr flexgsav savings cpi lab:mobile cap:specific"
               r(sims) : "dn0 dn1 dn2"
               r(name) : "dn"

. type gams_dn/cge_dn1_par.inc
* Parameter changes and the closure, written by cge_gams
qgbar('com_man') = 1.03 * qgbar('com_man');
qgbar('com_ser') = 1.03 * qgbar('com_ser');
trdg('hhd') = 1.03 * trdg('hhd');
hs0 = 1.02 * hs0;
clext = 1;
clgov = 1;
clsi = 1;
clnum = 1;
fclo('lab') = 1;
fclo('cap') = 2;

. type gams_dn/cge_dn1_var.inc
* Changes to fixed variables, written by cge_gams
QF.fx('cap', 'act_agr') = 1.0090551181102363 * QF.l('cap', 'act_agr');
QF.fx('cap', 'act_man') = 1.0090551181102363 * QF.l('cap', 'act_man');
QF.fx('cap', 'act_ser') = 1.009055118110236 * QF.l('cap', 'act_ser');
QFS.fx('lab') = 1.02 * QFS.l('lab');
```

### The files

| File | Content |
|---|---|
| `cge_run.gms` | the driver: run `gams cge_run.gms` in the folder |
| `cge_model.gms` | data, calibration, equations and the base solve |
| `cge_closure.gms` | bounds, variables that do not exist, and the closure |
| `cge_solve.gms` | the solve statement, for the solver and model type |
| `cge_write.gms` | writes a solution |
| `cge_data.inc` | the SAM, the roles of its accounts and the calibration settings |
| `cge_shock_par.inc` | parameter changes and the closure |
| `cge_shock_var.inc` | closure swaps and changes to fixed variables |
| `cge_constraints.inc` | constraints, as complementarity pairs, and the scenario model `cgecon` |
| `cge_user.inc` | user variables and equations (**[CGE]** `cge_variable`, **[CGE]** `cge_equation`) |
| `cge_user_write.inc` | the results of the user variables |

The four `.gms` files of the model are copied from the package. With `dynamic()`, the files `cge_`*sim*`_par.inc` and `cge_`*sim*`_var.inc` of each period replace the two scenario files. GAMS writes `cge_base.csv` and `cge_`*name*`.csv`, with *name* the value of `sim()` or `current`, or `cge_`*sim*`.csv` for each period of a dynamic run: one row per variable and element (scenario, variable, element1, element2, value), with the model status in the first row. The files are described in detail in section 25.1.

### What is not exported

Microsimulation and sensitivity analysis stay in Stata. The death rate is not a GAMS parameter: it enters dynamic runs only through the population and labour supply of the next period, which are exported; a static scenario that changes it is exported with a note.

## Stored results

`cge_gams` stores the following in `r()`:

*Scalars*

| | |
|---|---|
| `r(N_accounts)` | accounts in the exported SAM |
| `r(N_changes)` | parameter and fixed-variable changes, from the base or, with `dynamic()`, from the period before, in all |
| `r(N_constraints)` | constraints exported |
| `r(N_periods)` | periods exported or compared (`dynamic()`) |
| `r(modelstat)` | GAMS model status of the scenario (`compare`, `import()`); 1, 2, 15 and 16 mean solved |
| `r(base_modelstat)` | GAMS model status of the base (`compare`) |
| `r(maxdiff)` | largest relative difference from the simulation, or from any period (`compare`) |
| `r(base_maxdiff)` | largest relative difference from the Stata base (`compare`) |
| `r(N_compared)` | values compared, over the scenario or every period (`compare`) |
| `r(walras)` | value of `WALRAS` in GAMS, largest over periods with `dynamic()` (`compare`) |
| `r(N_imported)` | values imported (`import()`) |
| `r(max_residual)` | largest scaled residual of the imported solution in Stata (`import()`) |
| `r(converged)` | 1 if the imported solution solves the model in Stata (`import()`) |

*Macros*

| | |
|---|---|
| `r(name)` | name of the exported scenario or run; with `import()`, of the simulation imported |
| `r(closure)` | closure of the exported scenario |
| `r(sims)` | simulations of the periods of the run (`dynamic()`) |
| `r(compared_with)` | the simulation or dynamic run compared (`compare`) |
| `r(worst_period)` | the period with the largest difference (`dynamic()` and `compare`) |
| `r(script)` | the script that ran GAMS (`run`) |

## Methods and formulas

*Scenario files.* A parameter or fixed variable whose scenario value $\theta^1$ differs from its base value $\theta^0$ is written as

$$\theta \leftarrow \frac{\theta^1}{\theta^0}\,\theta \quad (\theta^0 \neq 0), \qquad \theta \leftarrow \theta^1 \quad (\theta^0 = 0),$$

so that GAMS applies the change to its own calibrated value; for a fixed variable the right-hand side uses its level, as in `QFS.fx(’lab’) = 1.02 * QFS.l(’lab’);`. With `dynamic()`, $\theta^0$ is the value of the period before. Numbers are written with the fewest significant digits, 15, 16 or 17, that give back the same double.

*Model types.* Under `cns` the model is the square system of equations; under `nlp` the same equations with the constant objective $\mathrm{OBJ} = 0$; under `mcp` the equations, unpaired, and for each constraint $k$ on variable $Y$ with bound $b_k$ and adjusting variable $X$,

$$\mathrm{SLK}_k \geq 0 \;\perp\; \sigma_k \,\frac{b_k - Y}{s^Y} \geq 0, \qquad X = \bar{x}_k + \delta_k\, s^X\, \mathrm{SLK}_k,$$

with $\sigma_k = 1$ for an upper and $-1$ for a lower bound, $\delta_k = 1$ when $X$ rises and $-1$ when it falls as the constraint binds, $s^Y$ and $s^X$ the scales of the variables, and $\bar{x}_k$ the fixed level of $X$ (section 17.2, section 25.2). The base is solved as model `cgebase` and a scenario with constraints as `cgecon`.

*Tolerances.* The option files that GAMS writes scale the solvers' absolute tolerances to the largest SAM cell, with $p = \max(10^{-11}, 10^{-13} \max_{rc} |SAM(r,c)|)$: PATH `convergence_tolerance` $p$; CONOPT `rtnwma` $p/\max(1, \max_{rc} |SAM(r,c)|)$; IPOPT `tol` $10^{-12}$ and `constr_viol_tol` $p$; KNITRO `feastol_abs` $p$ and `feastol` $10^{-12}$.

*Comparison.* For the values $g_i$ of GAMS and $x_i$ of Stata, over the variable elements that exist in the model, except `WALRAS`,

$$d = \max_i \frac{|g_i - x_i|}{\max(1, |x_i|)}.$$

*Import.* Every free variable that exists in the model, and the adjusting variable of each constraint, takes its GAMS value; a free variable missing from the GAMS results is an error. The imported solution is counted as converged when the GAMS model status is 1, 2, 15 or 16 and the largest scaled residual of the model's equations in Stata is below $10^{-8}$ (section 25.3).

## Also see

**[CGE]** `cge_solve` — Solve the model
**[CGE]** `cge_constraint` — Limits that bind in some scenarios and not in others
**[CGE]** `cge_dynamic` — Solve a recursive dynamic run
**[CGE]** `cge_results` — Report simulation results
**Chapter 25** — The GAMS implementation

# `cge_listing` — Equation listing and infeasibility report

## Description

`cge_listing` (from version 0.9.5) lists the equations of the model at a point, as the equation listing and the infeasibility report of a GAMS run do. For every block of equations of chapter 7, and for the user equations, it shows the elements in the system, the largest scaled residual and where it occurs, and the number of elements that are infeasible there, whose scaled residual exceeds the tolerance. It then lists the infeasible elements, largest first, with their left and right sides, and, on request, every element of the blocks named. The point is a stored simulation, the base, or the current point from which the next solve starts.

## Quick start

The equations at the last simulation stored

```
cge_listing
```

The equations that a shock moves, before the model is solved, with every element of the import prices and of government revenue

```
cge_shock tm, value(0)
cge_listing, current blocks(P1 I8)
```

Every element of every equation at the simulation `notariff`, as a dataset

```
cge_listing, sim(notariff) clear
```

## Syntax

**`cge_listing`** [, *options*]

| *options* | Description |
|---|---|
| *Point* | |
| `sim(`*name*`)` | a stored simulation; default the last one stored |
| `base` | the base |
| `current` | the current point: base values with the current shocks and fixed values |
| *Listing* | |
| `blocks(`*list*`)` | list every element of these blocks |
| `all` | list every element of every block |
| `top(#)` | number of infeasible elements listed; default 20 |
| `tolerance(#)` | scaled residual above which an element is infeasible; default `1e-8` |
| `clear` | replace the data in memory with every element |

## Options

`sim(`*name*`)`, `base`, `current` choose the point; at most one may be given. The default is the last simulation stored, or the base if none is. The current point is the base with the shocks and fixed values that the next **[CGE]** `cge_solve` applies, from which it starts: its listing shows the equations that the shocks move away from the base.

`blocks(`*list*`)` lists every element of the blocks named, such as `blocks(P1 I8)`, with its left and right sides and its scaled residual. `all` lists every block.

`top(#)` sets how many infeasible elements are listed, largest first; the default is 20.

`tolerance(#)` sets the scaled residual above which an element is infeasible; the default, $10^{-8}$, is the tolerance of the solution check of **[CGE]** `cge_check`.

`clear` replaces the data in memory with one observation per element of the model's equations, with the variables `block`, `element`, `description`, `lhs`, `rhs`, `residual` (the left less the right side), `scaled`, and the indicators `insystem`, `replaced` (by a user equation) and `infeasible`.

## Remarks and examples

The SAM is mapped and calibrated:

```
. cge_synthetic, clear
(16 vars, 15 obs)

Synthetic SAM loaded (standard layout): 15 accounts, total   1743.00
Largest row-column imbalance:  0.00e+00

. cge_sam, labels(account)
SAM read: 15 accounts, total 1743
Largest row-column imbalance:  0.00e+00

. quietly cge_map, activities(act_agr act_man act_ser) commodities(com_agr com_man com_ser) ///
>     factors(lab cap) households(hhd) government(gov) savinv(sav) row(row)                 ///
>     dirtax(tax_dir) saltax(tax_sal) imptax(tax_imp)

. quietly cge_calibrate, sigmava(0.8) sigmaq(2) sigmat(2)
```

### Example 1: the base

At the base every equation holds, to rounding:

```
. cge_listing, base
Equations at the base: 93 in the system

Block                                       Elements      Largest  at                   Infeasible
--------------------------------------------------------------------------------------------------
P1    import price                                 3    0.000e+00  com_agr                       0
P2    export price                                 3    0.000e+00  com_man                       0
P3    composite supply price                       3    0.000e+00  com_man                       0
P4    producer price of output                     3    0.000e+00  com_agr                       0
P5    activity price                               3    0.000e+00  act_man                       0
P6    price of intermediate input                  3    0.000e+00  act_agr                       0
P7    zero profit                                  3    0.000e+00  act_agr                       0
P8    consumer price index                         1    0.000e+00                                0
P9    producer price index                         1    0.000e+00                                0
Q1    value added (Leontief)                       3    1.292e-16  act_ser                       0
Q2    intermediate input (Leontief)                3    0.000e+00  act_ser                       0
Q3    value added                                  3    0.000e+00  act_man                       0
Q4    factor demand                                6    2.220e-16  cap,act_ser                   0
Q5    intermediate demand                          9    0.000e+00  com_man,act_ser               0
Q6    commodity output                             3    0.000e+00  com_agr                       0
T1    CET transformation                           3    0.000e+00  com_man                       0
T2    export supply                                3    3.553e-16  com_man                       0
T4    Armington aggregation                        3    1.597e-16  com_man                       0
T5    import demand                                3    1.776e-16  com_ser                       0
I1    factor income                                2    0.000e+00  lab                           0
I2    factor income of institutions                6    0.000e+00  row,cap                       0
I3    income of households and enterprises         1    0.000e+00  hhd                           0
I4    consumption spending                         1    1.236e-16  hhd                           0
I5    household demand                             3    0.000e+00  com_man,hhd                   0
I6    investment demand                            3    0.000e+00  com_agr                       0
I7    government demand                            3    0.000e+00  com_man                       0
I8    government revenue                           1    0.000e+00                                0
I9    government spending                          1    0.000e+00                                0
I10   direct tax rate                              1    0.000e+00  hhd                           0
I11   savings rate                                 1    0.000e+00  hhd                           0
S1    factor market                                2    0.000e+00  cap                           0
S2    commodity market                             3    0.000e+00  com_ser                       0
S3    balance of payments                          1    0.000e+00                                0
S4    government budget                            1    0.000e+00                                0
```

```
S5    savings-investment balance                   1    0.000e+00                                0
--------------------------------------------------------------------------------------------------

No equation element is infeasible: every scaled residual is at most  1.00e-08
```

### Example 2: the equations a shock moves

With the tariffs removed, the current point is the base under the new tariffs. The import prices of the two goods that carry a tariff and government revenue are off; revenue, on the left, exceeds the taxes that the base point now yields by the base tariff revenue, 8:

```
. cge_shock tm, value(0)
Parameter tm changed for 3 element(s)

. cge_listing, current blocks(P1 I8)
Equations at the current point, before it is solved: 93 in the system

Block                                       Elements      Largest  at                   Infeasible
--------------------------------------------------------------------------------------------------
P1    import price                                 3    9.091e-02  com_agr                       2
P2    export price                                 3    0.000e+00  com_agr                       0
P3    composite supply price                       3    0.000e+00  com_agr                       0
P4    producer price of output                     3    0.000e+00  com_ser                       0
P5    activity price                               3    0.000e+00  act_ser                       0
P6    price of intermediate input                  3    0.000e+00  act_ser                       0
P7    zero profit                                  3    0.000e+00  act_ser                       0
P8    consumer price index                         1    0.000e+00                                0
P9    producer price index                         1    0.000e+00                                0
Q1    value added (Leontief)                       3    1.292e-16  act_ser                       0
Q2    intermediate input (Leontief)                3    0.000e+00  act_agr                       0
Q3    value added                                  3    0.000e+00  act_ser                       0
Q4    factor demand                                6    2.220e-16  cap,act_agr                   0
Q5    intermediate demand                          9    0.000e+00  com_agr,act_man               0
Q6    commodity output                             3    0.000e+00  com_agr                       0
T1    CET transformation                           3    0.000e+00  com_agr                       0
T2    export supply                                3    3.553e-16  com_man                       0
T4    Armington aggregation                        3    1.597e-16  com_man                       0
T5    import demand                                3    1.776e-16  com_ser                       0
I1    factor income                                2    0.000e+00  lab                           0
I2    factor income of institutions                6    0.000e+00  gov,lab                       0
I3    income of households and enterprises         1    0.000e+00  hhd                           0
I4    consumption spending                         1    1.236e-16  hhd                           0
I5    household demand                             3    0.000e+00  com_man,hhd                   0
I6    investment demand                            3    0.000e+00  com_man                       0
I7    government demand                            3    0.000e+00  com_man                       0
I8    government revenue                           1    1.270e-01                                1
I9    government spending                          1    0.000e+00                                0
I10   direct tax rate                              1    0.000e+00  hhd                           0
I11   savings rate                                 1    0.000e+00  hhd                           0
S1    factor market                                2    0.000e+00  lab                           0
S2    commodity market                             3    0.000e+00  com_man                       0
S3    balance of payments                          1    0.000e+00                                0
S4    government budget                            1    0.000e+00                                0
S5    savings-investment balance                   1    0.000e+00                                0
--------------------------------------------------------------------------------------------------

Infeasible equation elements, with a scaled residual above  1.00e-08: 3
  Block Element                         Left side       Right side      Scaled
  I8                                           63               55    1.270e-01 INFES
  P1    com_agr                               1.1                1    9.091e-02 INFES
  P1    com_man                               1.1                1    9.091e-02 INFES

Block P1: import price
  Block Element                         Left side       Right side      Scaled
  P1    com_agr                               1.1                1    9.091e-02 INFES
  P1    com_man                               1.1                1    9.091e-02 INFES
  P1    com_ser                                 1                1    0.000e+00

Block I8: government revenue
  Block Element                         Left side       Right side      Scaled
  I8                                           63               55    1.270e-01 INFES
```

### Example 3: a solve that does not converge

A solve stopped after one iteration, with a tolerance it cannot meet, ends with error 430, and its point is stored as a simulation. Its listing shows where the system is furthest from a solution; the largest scaled residual is the one **[CGE]** `cge_solve` reports:

```
. cge_solve, name(stopped) iterate(1) steps(1) tolerance(1e-30)
Simulation stopped did not converge (largest scaled residual  1.19e-02)
Largest scaled residual  1.19e-02; WALRAS = -2.30e-09
Diagnostics
  Largest scaled residuals
    T5   import demand                          com_man                -1.189e-02
    T5   import demand                          com_agr                -5.819e-03
    T5   import demand                          com_ser                -2.861e-03
    S5   savings-investment balance                                    -1.744e-03
    T2   export supply                          com_man                 1.558e-03
  (cge_listing, sim() lists every equation element at this point)
  Smallest pivot of the Jacobian, relative to the largest:  1.30e-03 (WALRAS)

. cge_listing, sim(stopped) top(8)
Equations at simulation stopped: 93 in the system

Block                                       Elements      Largest  at                   Infeasible
--------------------------------------------------------------------------------------------------
P1    import price                                 3    0.000e+00  com_agr                       0
P2    export price                                 3    0.000e+00  com_man                       0
P3    composite supply price                       3    8.387e-04  com_agr                       3
P4    producer price of output                     3    9.186e-04  com_man                       3
P5    activity price                               3    0.000e+00  act_man                       0
P6    price of intermediate input                  3    0.000e+00  act_man                       0
P7    zero profit                                  3    2.700e-14  act_man                       0
P8    consumer price index                         1    0.000e+00                                0
P9    producer price index                         1    0.000e+00                                0
Q1    value added (Leontief)                       3    1.219e-12  act_man                       0
Q2    intermediate input (Leontief)                3    3.355e-13  act_agr                       0
Q3    value added                                  3    2.665e-07  act_man                       3
Q4    factor demand                                6    3.675e-05  lab,act_man                   6
Q5    intermediate demand                          9    0.000e+00  com_man,act_ser               0
Q6    commodity output                             3    0.000e+00  com_man                       0
T1    CET transformation                           3    4.621e-04  com_man                       3
T2    export supply                                3    1.558e-03  com_man                       3
T4    Armington aggregation                        3    6.735e-04  com_agr                       3
T5    import demand                                3    1.189e-02  com_man                       3
I1    factor income                                2    0.000e+00  lab                           0
I2    factor income of institutions                6    0.000e+00  hhd,cap                       0
I3    income of households and enterprises         1    7.817e-13  hhd                           0
I4    consumption spending                         1    1.698e-12  hhd                           0
I5    household demand                             3    0.000e+00  com_ser,hhd                   0
I6    investment demand                            3    0.000e+00  com_ser                       0
I7    government demand                            3    0.000e+00  com_ser                       0
I8    government revenue                           1    1.749e-05                                1
I9    government spending                          1    0.000e+00                                0
I10   direct tax rate                              1    0.000e+00  hhd                           0
I11   savings rate                                 1    0.000e+00  hhd                           0
S1    factor market                                2    0.000e+00  lab                           0
S2    commodity market                             3    4.817e-04  com_man                       3
S3    balance of payments                          1    1.421e-16                                0
S4    government budget                            1    2.195e-11                                0
S5    savings-investment balance                   1    1.744e-03                                1
--------------------------------------------------------------------------------------------------

Infeasible equation elements, with a scaled residual above  1.00e-08: 32 (the largest 8 shown)
  Block Element                         Left side       Right side       Scaled
  T5    com_man                         62.809335          63.5227   -1.189e-02 INFES
  T5    com_agr                         22.239357        22.355746   -5.819e-03 INFES
  T5    com_ser                         19.011569        19.068791   -2.861e-03 INFES
  S5                                    43.445285        43.532471   -1.744e-03 INFES
  T2    com_man                         42.196044        42.133718    1.558e-03 INFES
  T2    com_agr                         20.935418        20.914809    1.030e-03 INFES
  P4    com_man                         139.96367        140.09227   -9.186e-04 INFES
  P3    com_agr                         102.97849        103.06404   -8.387e-04 INFES
```

### Example 4: the listing as a dataset

```
. quietly cge_solve, name(notariff)

. quietly cge_listing, sim(notariff) clear

. describe, simple
block         description  rhs          scaled       replaced
element       lhs          residual     insystem     infeasible

. list block element lhs rhs scaled if inlist(block, "Q4", "I8"), noobs

  +---------------------------------------------------------+
  | block      element         lhs         rhs       scaled |
  |---------------------------------------------------------|
  |    Q4   lab,act_agr   1.0281053   1.0281053   -6.661e-16 |
  |    Q4   lab,act_man   1.0281053   1.0281053   -6.661e-16 |
  |    Q4   lab,act_ser   1.0281053   1.0281053   -2.220e-16 |
  |    Q4   cap,act_agr    1.026315    1.026315            0 |
  |    Q4   cap,act_man    1.026315    1.026315    8.882e-16 |
  |---------------------------------------------------------|
  |    Q4   cap,act_ser    1.026315    1.026315    2.220e-16 |
  |    I8                 55.611776   55.611776            0 |
  +---------------------------------------------------------+
```

Elements with two indexes are labelled by both, in the order of their sets.

## Stored results

`cge_listing` stores the following in `r()`:

*Scalars*

| | |
|---|---|
| `r(N_equations)` | number of elements in the system |
| `r(N_replaced)` | number of elements replaced by user equations |
| `r(N_infeasible)` | number of elements in the system above the tolerance |
| `r(N_listed)` | number of elements listed by `blocks()` or `all` |
| `r(max_residual)` | the largest scaled residual in the system |
| `r(tolerance)` | the tolerance |

*Macros*

| | |
|---|---|
| `r(block)` | the block of the largest scaled residual |
| `r(element)` | its element |
| `r(point)` | the point listed |

## Methods and formulas

At a point $(x, \theta)$, each element $e$ of block $B$ has the left and right sides $\ell_{B,e}(x, \theta)$ and $r_{B,e}(x, \theta)$ of chapter 7, the residual $\ell_{B,e} - r_{B,e}$, and the scaled residual

$$\frac{\ell_{B,e}(x,\theta) - r_{B,e}(x,\theta)}{\sigma_{B,e}}, \qquad \sigma_{B,e} = \max\bigl(|\ell_{B,e}(x^0,\theta^0)|, |r_{B,e}(x^0,\theta^0)|\bigr),$$

or $\sigma_{B,e} = 1$ when that is at most $10^{-9}$: the residual that **[CGE]** `cge_solve` brings below its tolerance. An element is infeasible when it is in the system and its scaled residual exceeds the tolerance in absolute value. Elements outside the model, such as the import price of a commodity that is not imported, are not listed; elements replaced by user equations are listed and counted apart.

## Also see

**[CGE]** `cge_solve` — Solve the model
**[CGE]** `cge_check` — Run model validation checks
**[CGE]** `cge_equation` — User equations: new relations among the model's variables
**Chapter 7** — The equations

# cge_macro — Macros of the language of user variables and equations

## Description

cge_macro (from version 0.9.7) defines a macro of the language of **[CGE]** cge_variable and **[CGE]** cge_equation, as GAMS's \$macro does: a name with parameters that stands for an expression. A CES aggregate, a price deflated by the consumer price index or a constant written once can then be used wherever an expression may be, with the arguments in place of the parameters. cge_macro, list displays the macros, and cge_macro, clear removes them.

## Quick start

A CES aggregate of two quantities

```
cge_macro ces(x, y, d, r) = (d * x^(-r) + (1 - d) * y^(-r))^(-1 / r)
```

Use it in a user variable over the commodities that are imported

```
cge_variable CQ(c)$(base(QM(c)) > 0) = ces(QD(c), QM(c), 0.5, 0.5)
```

A macro without parameters, and one that uses others

```
cge_macro half = 0.5
cge_macro cesq(c) = ces(QD(c), QM(c), half, half)
```

Display the macros; remove them

```
cge_macro, list
cge_macro, clear
```

## Syntax

Define a macro

**cge_macro** *name*[(*a*[, *b* ...])] = *expression*

Display or remove the macros

**cge_macro** , list
**cge_macro** , clear

*name* is a new name: not a variable, a parameter, an element, a set, a user equation, a function of the language or another macro. The parameters *a*, *b*, ... are names, each given once. *expression* is an expression of the language of **[CGE]** cge_equation, in which the parameters stand for the arguments.

| *options* | Description |
|---|---|
| *Main* | |
| list | display the macros |
| clear | remove the macros |

## Options

`list` displays the macros with their parameters and expressions.

`clear` removes every macro. A macro that a user variable or equation uses cannot be removed until they are.

## Remarks and examples

In an expression, *name*(*x*, *y*) is replaced by the macro's expression, in parentheses, with *x* in place of *a* and *y* in place of *b*. An argument that is more than a name or a number is itself put in parentheses, so that with `sq(x) = x * x`, `sq(QA(a) + 1)` is `((QA(a) + 1) * (QA(a) + 1))`, not `QA(a) + 1 * QA(a) + 1`. A macro without parameters stands for its expression. The expression may use variables, parameters, user sets and parameters, sums, conditions, the functions of the language and other macros, which are replaced in turn; a macro may not use itself, directly or through others. Arguments may be indexes: in the definition of a variable over `c`, `cesq(c)` is written out for every commodity.

Macros are replaced when a variable or equation is added, before its indexes are written out, and again each time the model is calibrated. They are saved with the model (**[CGE]** `cge_save`) and kept when it is calibrated again. In the GAMS program, **[CGE]** `cge_gams` writes the expressions out, so it needs no `\$macro`.

The SYNTHETIC SAM is mapped and calibrated:

```
. cge_synthetic, clear
(16 vars, 15 obs)

Synthetic SAM loaded (standard layout): 15 accounts, total   1743.00
Largest row-column imbalance:  0.00e+00

. cge_sam, labels(account)
SAM read: 15 accounts, total 1743
Largest row-column imbalance:  0.00e+00

. cge_map, activities(act_agr act_man act_ser) commodities(com_agr com_man com_ser) ///
>     factors(lab cap) households(hhd) government(gov) savinv(sav) row(row)          ///
>     dirtax(tax_dir) saltax(tax_sal) imptax(tax_imp)
Accounts mapped: 3 activities, 3 commodities, 2 factors, 1 households, 0 enterprises, 0 margin accou
> nts

. cge_calibrate, sigmava(0.8) sigmaq(2) sigmat(2)
Calibrated: 93 equations, 93 free variables
Benchmark replication: largest scaled residual  3.55e-16
```

### Example 1: macros and their use

A CES aggregate, a price relative to the consumer price index, a constant, and a macro that uses the others; then variables defined with them, after the tariffs are removed:

```
. cge_macro ces(x, y, d, r) = (d * x^(-r) + (1 - d) * y^(-r))^(-1 / r)
Macro ces(x, y, d, r) = (d * x^(-r) + (1 - d) * y^(-r))^(-1 / r)

. cge_macro rel(v) = v / CPI
Macro rel(v) = v / CPI

. cge_macro half = 0.5
Macro half = 0.5

. cge_macro cesq(c) = ces(QD(c), QM(c), half, half)
Macro cesq(c) = ces(QD(c), QM(c), half, half)

. cge_macro, list
macro ces(x, y, d, r) = (d * x^(-r) + (1 - d) * y^(-r))^(-1 / r)
macro rel(v) = v / CPI
macro half = 0.5
macro cesq(c) = ces(QD(c), QM(c), half, half)

. cge_variable CQ(c)$(base(QM(c)) > 0) = cesq(c)
```

```
User variable CQ(c)$(base(QM(c)) > 0): 3 element(s), base values 35.5556 76.21 41.7112
and its definition, equation CQ

. cge_variable RP(c) = rel(PQ(c))
User variable RP(c): 3 element(s), base values 1 1 1
and its definition, equation RP

. cge_shock tm, value(0)
Parameter tm changed for 3 element(s)

. cge_solve, name(notariff)
Simulation notariff solved in 7 iteration(s) (newton)
Largest scaled residual  3.96e-14; WALRAS = -2.72e-13

. quietly cge_results, sim(notariff) clear nodisplay

. list variable element1 base value if inlist(variable, "QD", "QM", "CQ", "RP"), ///
>     noobs sepby(variable)

  +--------------------------------------------+
  | variable   element1        base      value |
  |--------------------------------------------|
  |       QD    com_agr          80   78.575158 |
  |       QD    com_man         100   95.730222 |
  |       QD    com_ser         135   135.85348 |
  |--------------------------------------------|
  |       QM    com_agr          20   22.236866 |
  |       QM    com_man          60   63.002825 |
  |       QM    com_ser          20   18.993677 |
  |--------------------------------------------|
  |       CQ    com_agr   35.555556   37.899037 |
  |       CQ    com_man   76.209992   76.817918 |
  |       CQ    com_ser   41.711235   40.248652 |
  |--------------------------------------------|
  |       RP    com_agr           1   .99948068 |
  |       RP    com_man           1   .97900528 |
  |       RP    com_ser           1   1.0214491 |
  +--------------------------------------------+
```

`CQ` is $\left(0.5\,QD^{-0.5} + 0.5\,QM^{-0.5}\right)^{-2}$ for each commodity with imports at the base, and `RP` the composite price deflated by the consumer price index.

### Example 2: errors

A macro needs its arguments, as many as its parameters; a macro may not use itself; and a macro that variables use cannot be removed:

```
. cge_variable X = ces(QD(com_man), QM(com_man))
macro ces takes 4 argument(s); 2 given

r(198);

. cge_variable X = rel
macro rel needs its arguments, as rel(v)

r(198);

. cge_macro loop(x) = loop(x) + 1
cge_macro: macro loop uses itself

r(198);

. cge_macro, clear
macro rel is used by user variables or equations: remove them first

r(459);
```

## Stored results

`cge_macro` stores the following in `r()`:

| | |
|---|---|
| *Scalars* | |
| `r(N_macros)` | number of macros |
| *Macros* | |
| `r(name)` | name of the macro defined |

## Methods and formulas

The tokens of an expression are scanned for the names of macros. Each use, *name*`(`$x_1$`, ... ,` $x_m$`)`, is replaced by the tokens of the macro's expression, with each parameter replaced by its argument, in parentheses unless the argument is a single token, and the whole in parentheses. The scan is repeated while macros remain, up to 30 times; a chain of macros that does not end within that is an error. The macros are replaced before `card()` and before the indexes and sums are written out (**[CGE]** `cge_equation`), so that the indexes in a macro's arguments are written out with the others. A macro therefore adds no variables, equations or derivatives of its own.

## Also see

**[CGE]** `cge_equation` — User equations: new relations among the model's variables
**[CGE]** `cge_variable` — User variables, defined by an expression or determined by user equations
**[CGE]** `cge_set` — User sets of elements, for indexes and sums
**Chapter 23** — User variables and equations

# cge_map — Assign SAM accounts to model roles

## Description

`cge_map` assigns the accounts of the SAM read by **[CGE]** `cge_sam` to the roles of the model: activities, commodities, factors, institutions, tax accounts, margin accounts and the other accounts described below. It then checks every cell of the SAM against the flows that the model represents and reports, in one table, every account that is not balanced, every account with flows but no role, every flow that the model does not represent and every negative cell where the model needs a non-negative value. If there is any such problem, `cge_map` stops with error 459.

Before the checks, layouts that are common in national SAMs are rewritten into the layout of the model, and the amounts moved are displayed. The data in memory are not changed. With `using`, the roles are read from a template written by **[CGE]** `cge_template`.

## Quick start

Map the synthetic SAM, with direct, sales and import tax accounts

```
cge_map, activities(act_agr act_man act_ser) commodities(com_agr com_man com_ser) ///
    factors(lab cap) households(hhd) government(gov) savinv(sav) row(row)        ///
    dirtax(tax_dir) saltax(tax_sal) imptax(tax_imp)
```

As above, for a SAM that records sales and direct taxes as payments to the government account

```
cge_map, activities(act_agr act_man act_ser) commodities(com_agr com_man com_ser) ///
    factors(lab cap) households(hhd) government(gov) savinv(sav) row(row)        ///
    dirtax(gov) saltax(gov) imptax(tax_imp)
```

Save every cell that the checks report to `problems.csv`

```
cge_map, ... report(problems.csv) replace
```

Map the SAM from a template filled in Excel

```
cge_map using roles.xlsx
```

## Syntax

Assign the roles with options

**cge_map,** `activities(`*namelist*`)` `commodities(`*namelist*`)` `factors(`*namelist*`)` `households(`*namelist*`)` `government(`*name*`)` `savinv(`*name*`)` `row(`*name*`)` [*options*]

Assign the roles from a template

**cge_map using** *template* [, *template_options*]

| *options* | Description |
|---|---|
| *Main* | |
| `activities(`*namelist*`)` | activity accounts, each paired with the commodity in the same position of `commodities()`; required |
| `commodities(`*namelist*`)` | commodity accounts: the paired ones first, then any without a producing activity; required |
| `factors(`*namelist*`)` | factor accounts; required |
| `households(`*namelist*`)` | household accounts; required |
| `government(`*name*`)` | government account; required |

| *options* | Description |
|---|---|
| `savinv(`*name*`)` | savings-investment account; required |
| `row(`*name*`)` | rest-of-world account; required |
| *Institutions* | |
| `enterprises(`*namelist*`)` | enterprise accounts |
| `insurance(`*name*`)` | social health insurance fund |
| *Taxes* | |
| `dirtax(`*name*`)` | direct tax account |
| `saltax(`*name*`)` | sales tax account |
| `imptax(`*name*`)` | import tariff account |
| `exptax(`*name*`)` | export tax account |
| `acttax(`*name*`)` | activity tax account |
| `vattax(`*name*`)` | value added tax (VAT) account |
| `vatshare(`*spec*`)` | share of each commodity's sales tax that is VAT |
| `factortax(`*account :factor …*`)` | accounts of taxes on the use of factors, each with the factor it taxes |
| `factorincometax(`*name*`)` | account of taxes on factor income |
| *Margins, inventories and the territorial correction* | |
| `marginaccounts(`*namelist*`)` | margin accounts paid per unit of composite supply |
| `domesticmargins(`*namelist*`)` | margin accounts paid per unit of domestic sales |
| `importmargins(`*namelist*`)` | margin accounts paid per unit of imports |
| `exportmargins(`*namelist*`)` | margin accounts paid per unit of exports |
| `margins(`*namelist*`)` | activities whose margins the SAM nets out of their output |
| `stocks(`*namelist*`)` | stock-change accounts, merged into savings-investment |
| `territorial(`*commodity [activity]*`)` | territorial-correction commodity and, optionally, its activity |
| *Reporting* | |
| `report(`*filename*`)` | save every cell that the checks report to a CSV file |
| `replace` | allow `report()` to overwrite an existing file |

| *template_options* | Description |
|---|---|
| `vatshare(`*spec*`)` | share of each commodity's sales tax that is VAT |
| `sheet(`*sheetname*`)` | worksheet of an Excel template; default `accounts`, or else the first sheet |
| `report(`*filename*`)` | save every cell that the checks report to a CSV file |
| `replace` | allow `report()` to overwrite an existing file |

All options other than `vatshare()`, `domesticmargins()`, `importmargins()`, `exportmargins()`, `factortax()` and `factorincometax()` must be typed in full.

## Options

`activities(`*namelist*`)` and `commodities(`*namelist*`)` list the activity and commodity accounts. They are paired by position: the $k$th commodity is the primary commodity of the $k$th activity, used when layouts are rewritten. An activity may also produce other commodities, in the proportions that the SAM records. Each activity needs a paired commodity; commodities without a producing activity, such as commodities that are only imported, are listed after the paired ones. A payment from a household to an activity, $SAM(activity, household)$, is consumption of own produce at home (section 14.1).

`factors(`*namelist*`)` and `households(`*namelist*`)` list the factor and household accounts. Non-profit institutions serving households can be listed as households.

`government(`*name*`)`, `savinv(`*name*`)` and `row(`*name*`)` name the government, savings-investment and rest-of-world accounts.

`enterprises(`*namelist*`)` lists enterprise accounts (corporations). Enterprises receive factor income and transfers, pay direct taxes and save; the rest of their income is transferred to other institutions or abroad. They do not consume, so a purchase of commodities by an enterprise is reported as a flow that the model does not represent.

`insurance(`*name*`)` names a social health insurance fund. The fund may receive contributions from factors (its row, the factors' columns), transfers from the government and from the rest of the world, and may buy commodities and save; it may have no other flows. Contributions enter the model as a fixed rate on each factor's income, taken before that income is distributed to institutions, and the fund buys a fixed basket of commodities with its revenue less a fixed share saved, as in equations (I2) and (I12) to (I14). A non-profit institution financed by donors that buys health services is mapped as a household instead.

`dirtax(`*name*`)`, `saltax(`*name*`)`, `imptax(`*name*`)`, `exptax(`*name*`)` and `acttax(`*name*`)` name the accounts that collect direct taxes, sales taxes, import tariffs, export taxes and activity taxes. The rates of a tax that is not mapped are zero. If the SAM records a tax as a payment to the government account itself, name the government account in the option, for example `saltax(gov)`: the government row's cells in the columns that pay the tax (households and enterprises for `dirtax()`, commodities for `saltax()`, `imptax()` and `exptax()`, activities for `acttax()`) are then routed through an internal tax account, `_dirtax`, `_saltax`, `_imptax`, `_exptax` or `_acttax`. At most one of `saltax()`, `imptax()`, `exptax()` and `vattax()` may name the government account, because their cells would otherwise be ambiguous.

`vattax(`*name*`)` names the account that collects VAT from the commodities, net of input credits, and pays it to the government. VAT is then a tax on final demand at a rate by commodity, with credits for the VAT on inputs in the shares given by `vatcredit()` of **[CGE]** `cge_calibrate`; exports and margin services are zero-rated (section 13.2). The government account may be named, as for the other taxes; the VAT is then routed through the internal account `_vattax`.

`vatshare(`*spec*`)` is for SAMs that record VAT within the sales tax account. It moves the given share of each commodity's sales tax to an internal VAT account, `_vatshare`, and the same amount of the sales tax account's payment to the government to the payment of `_vatshare`. *spec* is *commodity*:*share* pairs, or one share for every commodity; shares lie between 0 and 1, and commodities not listed have share 0. `vatshare(0)` creates an empty VAT account, so that a VAT can be introduced by a shock to `tv`. `vatshare()` needs `saltax()` and may not be combined with `vattax()`.

`factortax(`*account* :*factor* ...`)` names accounts of taxes on the use of factors, such as payroll taxes on labour or taxes on capital, which activities pay and which pay the government. Each account is followed by the factor whose use it taxes. The rate on each factor in each activity is the tax over the factor payment; it raises the cost of the factor to the activity, and value added at factor cost includes it (section 18.1).

`factorincometax(`*name*`)` names the account of taxes on factor income, which factors pay and which pays the government. The rate on each factor's income is taken before the income is distributed to institutions, as the contributions to a health insurance fund are.

`marginaccounts(`*namelist*`)` lists margin accounts: accounts that collect trade or transport margins from commodities (the margin account's row, the commodities' columns) and spend them on margin services (the commodities' rows, the margin account's column). A margin account may have no other flows. Margins enter the model as a fixed quantity of margin services per unit of each commodity's composite supply, and their cost enters the commodity's price, as in equations (P3) and (T8).

`domesticmargins(`*namelist*`)`, `importmargins(`*namelist*`)` and `exportmargins(`*namelist*`)` list margin accounts whose margins are paid per unit of domestic sales, of imports and of exports, for SAMs that separate them (section 14.4). Domestic buyers then pay the producer price plus domestic margins, the import price includes import margins, and exporters receive the world price less export taxes and export margins. A commodity that pays such margins must have domestic sales, imports or exports respectively; **[CGE]** `cge_calibrate` checks this.

`margins(`*namelist*`)` lists activities, typically trade and transport, whose margin revenue the SAM nets out of their output by recording a negative use of their own product. That negative cell is moved to the activity's output, so that the margins enter the model as the purchases of trade and transport services by the activities that pay them, as the SAM records them. The activity's own use of its product cannot be separated from the netted margins and becomes zero. An activity named here must be one of the activities, and must have a negative use of its paired commodity.

`stocks(`*namelist*`)` lists stock-change accounts: accounts financed by savings-investment that buy commodities, with negative cells for a fall in inventories. Their purchases are moved to the savings-investment account, so that changes in inventories become part of investment demand. A stock-change account may receive payments only from savings-investment and may have no other role.

`territorial(`*commodity [activity]*`)` names the territorial-correction commodity and, optionally, its activity. The correction's imports (purchases abroad by residents) become household payments to the rest of the world, and its exports (purchases in the country by non-residents) become transfers from the rest of the world to households, both divided among households in proportion to their consumption of the correction. The correction may have only household consumption, imports and exports, and the activity only exports, which are first moved to the commodity. Both accounts then have no flows, and may have no other role.

`report(`*filename*`)` saves every cell that the checks report to a CSV file with the columns `problem`, `paid_by`, `paid_by_role`, `received_by`, `received_by_role` and `value`. The file is written whether or not problems are found.

`replace` allows `report()` to overwrite an existing file.

`sheet(`*sheetname*`)` names the worksheet of an Excel template. The default is the sheet `accounts`, which **[CGE]** `cge_template` writes, or else the first sheet. It may be given only with `using`.

## Remarks and examples

The mapping tells the model what each account of the SAM is. Each account has one role, given by the option that lists it, except that the government account may also be named in tax options and a margin activity is listed in both `activities()` and `margins()`. An account with no flows needs no role. The mapping replaces any earlier mapping, and the model must be calibrated again with **[CGE]** `cge_calibrate`.

### The checks

After the layouts are rewritten, `cge_map` checks the SAM and collects four kinds of problem:

- an account that is not balanced: its row total differs from its column total;
- an account with flows but no role;
- a non-zero cell that is not one of the flows of the model, listed below;
- a negative cell where the model needs a non-negative value.

The problems are summarised in one table, grouped by problem and by the roles of the paying and receiving accounts, the largest group first, with the number of cells, their sum in SAM units and their share of the sum of the absolute values of all cells. For an account with no role, the value is its row total, and its flows are also listed by role. Unknown account names (error 111) and an account given two roles (error 198) stop the command before the checks.

The model represents the following payments, from the column account to the row account:

| Paid by (column) | Received by (row) |
|---|---|
| commodity | activities (domestic output); rest of the world (imports); sales, import and export tax accounts; VAT account; margin accounts |
| activity | commodities (intermediate inputs); factors; activity tax account; accounts of taxes on factor use |
| factor | households; enterprises; government; rest of the world; health insurance fund; account of taxes on factor income |
| household | commodities; activities (home consumption); direct tax account; savings-investment; households and enterprises; rest of the world |
| enterprise | direct tax account; savings-investment; households and enterprises; rest of the world |

| | |
|---|---|
| government | commodities; households and enterprises; government; rest of the world; savings-investment; health insurance fund |
| tax account | government |
| savings-investment | commodities (investment, including changes in inventories) |
| margin account | commodities (margin services) |
| health insurance fund | commodities; savings-investment |
| rest of the world | commodities (exports); households, enterprises and government; factors (factor income from abroad); savings-investment; health insurance fund |

Negative cells are allowed in the rows of the tax accounts and of savings-investment, in transfers (the payments among households, enterprises, the government and the rest of the world, and factor income from abroad), in the payments of tax accounts to the government, in investment demand (a fall in inventories) and in sales of commodities by activities. A negative sale gives a negative yield; it is displayed as a note and counted in `r(N_negative_sales)`. Section 9.1 gives the rules in full.

### Layouts rewritten before the checks

National SAMs often record flows differently from the model. `cge_map` rewrites these layouts, in this order, and displays the amounts moved:

| The SAM records | Option | Rewrite |
|---|---|---|
| exports as sales by activities to the rest of the world | (always) | moved to the commodity paired with the activity |
| margins netted out of margin activities, as a negative use of their own product | `margins()` | moved to the activity's output |
| a territorial correction | `territorial()` | household payments to, and transfers from, the rest of the world |
| changes in inventories in a stock-change account | `stocks()` | merged into savings-investment |
| taxes paid to the government account | tax option naming the government | routed through an internal tax account |
| VAT within the sales tax | `vatshare()` | moved to the internal account `_vatshare` |

The rewrites leave unchanged every total that enters the model, except that a margin rewrite raises the output of the margin activity, and the supply and demand of its product, by the margins, and a territorial rewrite replaces the correction's imports, exports and consumption by transfers (section 9.2). **[CGE]** `cge_samexport` with `mapped` writes the SAM after the rewrites.

### Example 1: the standard layout

The synthetic SAM has separate tax accounts and records exports as sales of commodities:

```
. cge_synthetic, clear
(16 vars, 15 obs)

Synthetic SAM loaded (standard layout): 15 accounts, total   1743.00
Largest row-column imbalance:  0.00e+00

. cge_sam, labels(account)
SAM read: 15 accounts, total 1743
Largest row-column imbalance:  0.00e+00

. cge_map, activities(act_agr act_man act_ser) commodities(com_agr com_man com_ser) ///
>     factors(lab cap) households(hhd) government(gov) savinv(sav) row(row)         ///
>     dirtax(tax_dir) saltax(tax_sal) imptax(tax_imp)
Accounts mapped: 3 activities, 3 commodities, 2 factors, 1 households, 0 enterprises, 0 margin accou
> nts
```

### Example 2: a problem, and a layout rewrite

The national-accounts layout of the synthetic SAM describes the same economy, but records exports as sales by activities, and sales and direct taxes as payments to the government account. Mapped with an import tax account only, it gives two groups of problems:

```
. cge_synthetic, clear national
(14 vars, 13 obs)

Synthetic SAM loaded (national-accounts layout): 13 accounts, total   1613.00
Largest row-column imbalance:  0.00e+00

. cge_sam, labels(account)
SAM read: 13 accounts, total 1613
Largest row-column imbalance:  0.00e+00

. cge_map, activities(act_agr act_man act_ser)                              ///
>     commodities(com_agr com_man com_ser) factors(lab cap) households(hhd) ///
>     government(gov) savinv(sav) row(row) imptax(tax_imp) report(problems.csv)
Exports recorded as sales by activities, moved to the paired commodities: 75
Mapping report saved to problems.csv (4 row(s))
The SAM contains flows that the model does not represent.
4 problem(s) in 2 group(s); values are SAM units, shares are of the sum of absolute cells.
For an account with no role the value is its total; its flows are also listed by role.
--------------------------------------------------------------------------------------------------------
Problem                        Paid by (column)     Received by (row)     Cells            Value   Share
--------------------------------------------------------------------------------------------------------
flow not represented           commodity            government                3               30   1.78%
flow not represented           household            government                1               25   1.48%
--------------------------------------------------------------------------------------------------------
Use the report() option to save every cell to a file.

r(459);
```

The exports of the activities, 75 in all, were moved to their paired commodities before the checks. The cells left are payments by the commodities and by the household to the government, which the model represents only through tax accounts. With `report()`, every cell of the table was saved:

```
. import delimited using problems.csv, clear
(6 vars, 4 obs)

. list, noobs abbreviate(16)

  +----------------------------------------------------------------------------------------+
  |              problem   paid_by   paid_by_role   received_by   received_by_role   value |
  |----------------------------------------------------------------------------------------|
  | flow not represented   com_agr      commodity           gov         government       8 |
  | flow not represented   com_man      commodity           gov         government      12 |
  | flow not represented   com_ser      commodity           gov         government      10 |
  | flow not represented       hhd      household           gov         government      25 |
  +----------------------------------------------------------------------------------------+
```

Naming the government account in `saltax()` and `dirtax()` routes the taxes through internal accounts, and the mapping succeeds:

```
. cge_map, activities(act_agr act_man act_ser) commodities(com_agr com_man com_ser) ///
>     factors(lab cap) households(hhd) government(gov) savinv(sav) row(row)         ///
>     dirtax(gov) saltax(gov) imptax(tax_imp)
Exports recorded as sales by activities, moved to the paired commodities: 75
Direct taxes paid to government, routed through account _dirtax: 25
Sales taxes paid to government, routed through account _saltax: 30
Accounts mapped: 3 activities, 3 commodities, 2 factors, 1 households, 0 enterprises, 0 margin accou
> nts
```

**[CGE]** `cge_calibrate` gives the same model for this layout as for the standard one.

### Example 3: a template

A template lists the roles in a file, one row per account, which is easier to check and to keep than a long command. Written after a mapping, it holds that mapping, and `cge_map using` maps the SAM from it and displays the equivalent command:

```
. cge_template using roles.csv, replace
Template of 13 accounts, with roles from the current mapping
Template saved to roles.csv

. import delimited using roles.csv, clear
(7 vars, 13 obs)

. list account role pair, noobs

  +-----------------------------------------------+
  | account                        role      pair |
  |-----------------------------------------------|
  | act_agr                  activities   com_agr |
  | act_man                  activities   com_man |
  | act_ser                  activities   com_ser |
  | com_agr                 commodities           |
  | com_man                 commodities           |
  |-----------------------------------------------|
  | com_ser                 commodities           |
  |     lab                     factors           |
  |     cap                     factors           |
  |     hhd                  households           |
  |     gov   government dirtax saltax            |
  |-----------------------------------------------|
  | tax_imp                      imptax           |
  |     sav                      savinv           |
  |     row                         row           |
  +-----------------------------------------------+

. cge_map using roles.csv
Mapping from the template:
cge_map, activities(act_agr act_man act_ser) commodities(com_agr com_man com_ser) factors(lab cap) h
> ouseholds(hhd) government(gov) savinv(sav) row(row) dirtax(gov) saltax(gov) imptax(tax_imp)
Exports recorded as sales by activities, moved to the paired commodities: 75
Direct taxes paid to government, routed through account _dirtax: 25
Sales taxes paid to government, routed through account _saltax: 30
Accounts mapped: 3 activities, 3 commodities, 2 factors, 1 households, 0 enterprises, 0 margin accou
> nts
```

The government account has three roles, `government dirtax saltax`, so the taxes are routed through internal accounts as in example 2. Because the model is kept in Mata, the data in memory, here the template, can be replaced at any time.

In a template, the `role` column names the options that list the account, separated by spaces or commas, in either case; the singular forms `activity`, `commodity`, `factor`, `household`, `enterprise`, `margin`, `marginaccount`, `stock`, `vat`, `domesticmargin`, `importmargin` and `exportmargin` are also accepted. The accounts of each option are listed in the order of the SAM, and the correction commodity of `territorial` before its activity. The `pair` column pairs an activity with a commodity; where it is blank, the activity is paired with the commodity whose column pays it most. A tie, an activity with no sales to commodities, and a commodity paired with two activities are errors. For an account of taxes on factor use, the `pair` column names the factor it taxes. The template may be a `.csv`, `.xls`, `.xlsx` or `.dta` file with a string variable `account`.

If the SAM was aggregated with the same template (**[CGE]** `cge_aggregate`), its accounts are the template's groups: each group takes the role of its members, which must agree, and is paired with the group of its members' pairs.

### Example 4: a supply-use layout

The supply-use layout of the synthetic SAM has two margin accounts, a commodity that is only imported, `com_fuel`, listed after the paired commodities, and a stock-change account:

```
. cge_synthetic, clear sut
(20 vars, 19 obs)

Synthetic SAM loaded (supply-use layout): 19 accounts, total   1952.00
Largest row-column imbalance:  0.00e+00

. cge_sam, labels(account)
Account label s-i is not a valid Stata name; using si
SAM read: 19 accounts, total 1952
Largest row-column imbalance:  0.00e+00
```

```
. cge_map, activities(act_agr act_man act_ser)                         ///
>     commodities(com_agr com_man com_ser com_fuel) factors(lab cap)    ///
>     households(hhd) government(gov) savinv(si) row(row) dirtax(tax_dir) ///
>     saltax(tax_sal) imptax(tax_imp) marginaccounts(trd trp) stocks(dstk)
Stock-change account(s) dstk merged into savings-investment: 3
Accounts mapped: 3 activities, 4 commodities, 2 factors, 1 households, 0 enterprises, 2 margin accou
> nts
```

The stock-change account received 3 from savings-investment; its purchases are now investment demand.

## Stored results

`cge_map` stores the following in `r()`:

| *Scalars* | |
|---|---|
| `r(N_negative_sales)` | number of negative sales of commodities by activities |
| `r(exports_moved)` | exports recorded as sales by activities, moved to commodities |
| `r(margins_moved)` | margins moved to the output of margin activities |
| `r(territorial_imports)` | purchases abroad by residents, made household payments abroad |
| `r(territorial_exports)` | purchases by non-residents, made transfers from abroad to households |
| `r(stocks_merged)` | payments of savings-investment to stock-change accounts |
| `r(taxes_rerouted)` | direct, sales, import, export and activity taxes routed through internal tax accounts |
| `r(vat_moved)` | VAT moved out of the sales tax by `vatshare()` |
| *Macros* | |
| `r(command)` | with `using`, the equivalent `cge_map` command |

## Methods and formulas

Let $SAM(r,c)$ be the payment from column account $c$ to row account $r$ before a rewrite and $SAM'(r,c)$ after it, $c(a)$ the commodity paired with activity $a$, and $row$, $sav$ and $gov$ the rest-of-world, savings-investment and government accounts. Cells not shown are unchanged. The rewrites are made in the following order.

Exports recorded as sales by activity $a$:

$$SAM'(c(a),row) = SAM(c(a),row) + SAM(a,row), \qquad SAM'(a,c(a)) = SAM(a,c(a)) + SAM(a,row),$$
$$SAM'(a,row) = 0.$$

Margins netted out of activity $a$, with $SAM(c(a),a) < 0$:

$$SAM'(a,c(a)) = SAM(a,c(a)) - SAM(c(a),a), \qquad SAM'(c(a),a) = 0.$$

Territorial correction $k$: the exports of its activity, if any, are first moved to $k$ as above. With $IM = SAM(row,k)$, $EX = SAM(k,row)$ and the shares $s_h = SAM(k,h) / \sum_{h'} SAM(k,h')$ of the households,

$$SAM'(row,h) = SAM(row,h) + s_h\,IM, \qquad SAM'(h,row) = SAM(h,row) + s_h\,EX,$$

and the cells of $k$ are set to zero. Stock-change account $k$: $SAM'(r,sav) = SAM(r,sav) + SAM(r,k)$ for every account $r$, and the row and column of $k$ are set to zero.

A tax paid to the government by the accounts $j \in P$ (the payers of that tax) is routed through a new account $t$:

$$SAM'(t,j) = SAM(gov,j), \qquad SAM'(gov,j) = 0, \qquad SAM'(gov,t) = \sum_{j \in P} SAM(gov,j).$$

With VAT shares $v_c$ of the sales tax account $sal$, the new account $t$, `_vatshare`, has

$$SAM'(t,c) = v_c\,SAM(sal,c), \qquad SAM'(sal,c) = (1-v_c)\,SAM(sal,c),$$
$$SAM'(gov,t) = \sum_c v_c\,SAM(sal,c), \qquad SAM'(gov,sal) = SAM(gov,sal) - SAM'(gov,t).$$

The checks use the tolerance $\tau = 10^{-8}\max\bigl(1, \max_{r,c}|SAM(r,c)|\bigr)$ on the rewritten SAM. A cell is non-zero if $|SAM(r,c)| > \tau$, and account $i$ is not balanced if

$$\left|\sum_c SAM(i,c) - \sum_r SAM(r,i)\right| > \tau.$$

The share of a group of problem cells $G$ in the report is $\sum_{(r,c)\in G}|SAM(r,c)| / \sum_{r,c}|SAM(r,c)|$.

Section 9.1 lists the flows of the model and the cells that may be negative, and section 9.2 gives the rewrites and the totals they change.

## Also see

**[CGE]** `cge_sam` — Read a social accounting matrix
**[CGE]** `cge_template` — Write a template that maps SAM accounts to roles and groups
**[CGE]** `cge_aggregate` — Merge SAM accounts into groups
**[CGE]** `cge_samexport` — Write the SAM to a file or to memory
**[CGE]** `cge_calibrate` — Calibrate the model to the SAM
**Chapter 9** — Calibration

# `cge_microsim` — Pass simulations to household survey data

## Description

`cge_microsim` passes the results of one or more simulations to the household survey in memory, top-down and without feedback to the model. Each survey household belongs to a household group of the model and keeps its own income composition and, if they are given, its own budget shares. Each income component changes as the model says that its source changes for the household's group; nominal consumption follows the household's income and its group's direct tax and savings rates; and consumption is deflated by a price index at the prices households pay, including VAT. The command reports real consumption per person, the poverty headcount and poverty gap, and the Gini coefficient, before and after the simulation, for all households and for each group, and can add the household's new consumption and price index to the data. From version 0.8.0 it also gives, for several simulations at once, poverty for several lines, including lines that differ by household, the number of poor, eight more inequality measures, incidence by quantile of base consumption and the growth incidence curve, and all results as a dataset. From version 0.8.1 it gives each household's equivalent and compensating variations, and divides every change among its channels; from version 0.8.2, fiscal incidence as the Commitment to Equity method measures it, before and after the simulation; from version 0.8.3, it can reweight the survey to the model's employment, give standard errors from the survey design, and give the range of the results across draws of the elasticities.

## Quick start

Pass simulation `notariff` to the survey in memory, in which `grp` holds each household's model group and `cons` its consumption; every household's consumption changes as its group's does

```
cge_microsim, sim(notariff) group(grp) consumption(cons)
```

As above, with sampling weights `w`, consumption per person and a poverty line of 1,200 per person

```
cge_microsim [pweight = w], sim(notariff) group(grp) consumption(cons) size(hhsize) line(1200)
```

Households' own incomes from wages, profits, remittances and grants, and their own budget shares for food, goods and services

```
cge_microsim [pweight = w], sim(notariff) group(grp) consumption(cons) size(hhsize) ///
    income(wage:lab profit:cap remit:row grants:gov)                             ///
    budget(food:com_agr goods:com_man services:com_ser) line(1200)
```

As above, and add the variables `nt_nom`, `nt_price`, `nt_real`, `nt_pc` and `nt_q`, replacing them if they exist

```
cge_microsim [pweight = w], sim(notariff) group(grp) consumption(cons) size(hhsize) ///
    income(wage:lab profit:cap remit:row grants:gov) line(1200) generate(nt) replace
```

Two simulations, a national line and regional lines in the variable `zreg`, quintiles, and every table

```
cge_microsim [pweight = w], sim(notariff vat18) group(grp) consumption(cons) size(hhsize) ///
    income(wage:lab profit:cap remit:row grants:gov) line(1200 zreg) quantiles(5) detail
```

Welfare by household, and the contribution of each channel to every change

```
cge_microsim [pweight = w], sim(notariff) group(grp) consumption(cons) size(hhsize) ///
    income(wage:lab profit:cap remit:row grants:gov) line(1200) generate(w1) decompose
```

Fiscal incidence of direct taxes `dtx`, transfers `grants`, indirect taxes `itx` and health services `hlt`, which follow government purchases of `com_hlt`

```
cge_microsim [pweight = w], sim(hfund) group(grp) consumption(cons) size(hhsize) ///
    income(wage:lab profit:cap remit:row grants:gov) line(1200) market(ym)        ///
    dtax(dtx) transfers(grants) itax(itx) inkind(hlt:com_hlt)
```

Weights adjusted so that the households' workers in agriculture, `agri`, follow the model's employment of labour there; standard errors from the survey design; and bands across the draws kept by **[CGE]** `cge_sensitivity`

```
cge_microsim [pweight = w], sim(notariff) group(grp) consumption(cons) size(hhsize) ///
    line(1200) reweight(agri:lab.act_agr)
svyset psu [pweight = w], strata(region)
cge_microsim, sim(notariff) group(grp) consumption(cons) size(hhsize) line(1200) svy
cge_microsim [pweight = w], sim(notariff) group(grp) consumption(cons) size(hhsize) ///
    line(1200) bands(dr)
```

As above, with the results as a dataset, saved to `incidence.dta`

```
cge_microsim [pweight = w], sim(notariff vat18) group(grp) consumption(cons) size(hhsize) ///
    income(wage:lab profit:cap remit:row grants:gov) line(1200 zreg) saving(incidence)
```

## Syntax

`cge_microsim` [*if*] [*in*] [*weight*], `sim(`*names*`)` `group(`*varname*`)` `consumption(`*varname*`)` [*options*]

| *options* | Description |
|---|---|
| *Main* | |
| `sim(`*names*`)` | one or more simulations; required |
| `group(`*varname*`)` | each household's model group; required |
| `consumption(`*varname*`)` | household consumption; required |
| `size(`*varname*`)` | household size; default 1 |
| `income(`*var* :*source* ...`)` | income components and their sources |
| `budget(`*var* :*commodity* ...`)` | budget items and the commodities whose prices apply |
| `line(`*lines*`)` | poverty lines per person, numbers or variables |
| `quantiles(#)` | number of quantile groups for incidence; default `10` |
| *Results* | |
| `generate(`*prefix*`)` | add new consumption and the price index as variables |
| `replace` | allow the variables of `generate()` to be replaced |
| `detail` | display poverty for every line, inequality, welfare and incidence |
| `decompose` | divide every change among its channels |
| *Fiscal incidence* | |
| `market(`*varname*`)` | market income |
| `dtax(`*varlist*`)` | direct taxes and contributions |
| `transfers(`*varlist*`)` | direct transfers |
| `itax(`*varlist*`)` | indirect taxes |
| `subsidies(`*varlist*`)` | indirect subsidies |
| `inkind(`*var* :*target* ...`)` | in-kind benefits, with the model's spending they follow |
| *Weights, sampling and uncertainty* | |
| `reweight(`*var* :*target* ...`)` | reweight the survey to the model's targets |
| `svy` | standard errors from the survey design |
| `bands(`*name*`)` | range of the results across draws of the elasticities |
| `clear` | replace the survey in memory with the results |
| `saving(`*filename*[, `replace`]`)` | save the results as a dataset |
| `nodisplay` | suppress the tables of results |
| *Graphs* | |
| `graph(`*graphs*`)` | draw graphs of the results |
| `gname(`*stub*`)` | names of the graphs; default `microsim` |
| `gexport(`*filename*`)` | save the graphs as files |

| *options* | Description |
|---|---|
| `glabels(`*labels*`)` | labels of the simulations in the legends |
| `gcolor` | series in colour rather than grey |
| `gopts(`*options*`)` | further graph options |

`pweights`, `aweights`, `fweights` and `iweights` are allowed; all are used as proportional household weights.

## Options

`sim(`*names*`)` names the simulations whose results are passed to the survey: simulations solved by **[CGE]** `cge_solve`, periods of a run of **[CGE]** `cge_dynamic`, or solutions imported by **[CGE]** `cge_gams`. Changes are measured from the calibrated base. Several simulations may be given from version 0.8.0; `r()` then holds the results of the first, and `clear` or `saving()` give those of every one.

`group(`*varname*`)` gives each household's group: a string variable whose values are labels of the model's households, or a numeric variable whose value labels, or failing those whose values, are. A value that is not a household of the model is an error.

`consumption(`*varname*`)` is the household's total consumption, the welfare measure, in the survey's units.

`size(`*varname*`)` is the household's size, which must be positive. Real consumption, poverty and inequality are then measured per person, over persons. The default is a size of 1.

`income(`*var* `:`*source* `...)` lists the household's income components, numeric variables, each with its source:

- a factor of the model: the group's income from that factor, or the factor's income if the group has none in the base;
- `gov`: transfers from the government, set in real terms and paid at the consumer price index;
- `row`: transfers from abroad, such as remittances;
- `dom`: transfers from other domestic institutions, such as dividends;
- `cpi`: income indexed to the consumer price index;
- `fixed`: nominal income that does not change.

Without `income()`, every household's nominal consumption changes as its group's does.

`budget(`*var* `:`*commodity* `...)` lists spending on budget items, numeric and not negative, and the commodity whose price applies to each. An item may instead name an activity, for home consumption of its output at the activity's price. A household's price index then uses its own budget shares; without `budget()`, or for a household whose items sum to zero, it uses its group's base budget shares, including home consumption.

`line(`*lines*`)` gives one or more poverty lines per person, in the units of consumption: positive numbers, or (from version 0.8.0) numeric variables that give each household its own line, such as a regional line or a line that depends on the household's composition. A household whose line is missing is left out. The first line is the one shown in the main table and in `r(table)`. Without `line()`, no poverty measures are computed.

`quantiles(#)` sets the number of groups of persons, ranked by base consumption per person, for incidence: 10, the default, for deciles, 5 for quintiles, and so on, from 2 to 100.

`generate(`*prefix*`)` adds the variables *prefix*`_nom` (new nominal consumption), *prefix*`_price` (the household's price index, relative to the base), *prefix*`_real` (real consumption, at base prices), *prefix*`_pc` (real consumption per person) and *prefix*`_ev` and *prefix*`_cv` (the household's equivalent and compensating variations, from version 0.8.1), and *prefix*`_q` (the household's quantile group of base consumption per person). With several simulations, all but *prefix*`_q` are numbered in the order of `sim()`: *prefix*`_nom1`, *prefix*`_nom2`, and so on. They are missing for observations outside the sample. *prefix* may have at most 26 characters, fewer the more simulations there are.

`replace` allows the variables of `generate()` to be replaced. Without it, a variable that exists is an error.

`detail` also displays, for each simulation, poverty for every line and group, the inequality measures for all households, the equivalent and compensating variations by group, and incidence by quantile.

`decompose` (from version 0.8.1) divides the change in every measure for all households among its channels, by the Shapley method: each source of income named in `income()`; `taxsav`, the direct tax and savings rates of the household's group; `group`, for households without income in the components given, which follow their group's consumption; and `prices`. Without `income()`, the channels are `income`, the group's consumption, and `prices`. The contributions add up to the change. The change in each poverty measure is also divided into growth and redistribution. With $K$ channels, every measure is computed for $2^K$ distributions; $K$ may be at most 12.

`market(`*varname*`)` (from version 0.8.2) is the household's market income, before taxes and transfers, in the units of consumption. It ranks persons, by market income per person, for fiscal incidence, and is required with the instruments below, which are amounts that the household pays or receives and cannot be negative. Market income changes in the simulation as the household's income components do, or as its group's income without `income()`.

`dtax(`*varlist*`)` lists direct taxes and contributions paid. Each changes with market income and the group's direct tax rate.

`transfers(`*varlist*`)` lists direct transfers received, which change as the group's transfers from the government.

`itax(`*varlist*`)` lists indirect taxes paid, which change with the household's nominal spending and with the share of sales tax and VAT in the prices it pays, at its own or its group's budget shares.

`subsidies(`*varlist*`)` lists indirect subsidies received, which change with nominal spending.

`inkind(`*var*`:`*target* ...`)` lists in-kind benefits, such as health or education services, valued at cost, each with the model's spending it follows: a commodity, whose purchases by the government give the change, or `shi`, the purchases of the health insurance fund.

`reweight(`*var*`:`*target* ...`)` (from version 0.8.3) adjusts the survey weights in the simulation so that the weighted total of each household characteristic *var*, a non-negative variable such as the household's number of workers in a sector, changes as its target does: a factor of the model, for its employment in all activities; *factor*`.`*activity*, for its employment in that activity; or a positive number, a ratio given by the user, such as the growth of the population. The new weights are as close to the survey's as the targets allow, in relative entropy. The simulated results use them, the base results the survey's weights, and `decompose` has a channel `weights`.

`svy` (from version 0.8.3) computes linearized standard errors of mean consumption per person and of FGT0, FGT1 and FGT2 for each line, in the base, the simulation and of the change, for all households and each group, as a subpopulation, from the design declared by `svyset`: one stage, with strata and clusters, without a finite population correction. The weights are those of `svyset`, and a weight may not be given as well. The standard errors measure the sampling error of the survey, taking the model's changes as given.

`bands(`*name*`)` (from version 0.8.3) gives the range of the results for all households across the draws of a sensitivity analysis kept as the simulations *name*1, *name*2, ... by **[CGE]** `cge_sensitivity` with `keep(`*name*`)`: for each measure, its 5th, 50th and 95th percentiles and standard deviation across the draws, beside its value in the first simulation of `sim()`.

`clear` replaces the survey in memory with the results, as a dataset (see *Results as a dataset* below). It cannot be combined with `generate()`, whose variables would be lost.

`saving(`*filename*`[, replace])` saves the results as a dataset in *filename* and leaves the survey in memory; `replace` allows an existing file to be replaced.

`nodisplay` suppresses the tables of results.

`graph(`*graphs*`)` (from version 0.9.6) draws graphs of the results, one or more of `gic`, `incidence`, `poverty`, `groups`, `shapley`, `fiscal` and `bands`, or `all` for every graph the other options allow; see *Graphs* below.

`gname(`*stub*`)` names the graphs *stub_graph*, such as `microsim_gic`; the default stub is `microsim`.

`gexport(`*filename*`)` saves each graph as a file named after the graph: with `gexport(fig.pdf)`, the growth incidence curve is saved as `fig_gic.pdf`, and so on. The extension gives the format: `.pdf` or `.eps` for a journal, `.png` (the default, 2,000 pixels wide) for slides and documents.

`glabels(`*labels*`)` gives the simulations, in the order of `sim()`, the labels shown in the legends, such as `glabels("Tariffs removed" "Tariffs doubled")`; by default they are the names of the simulations.

`gcolor` draws the series in the colours of Okabe and Ito (2008), which readers with colour blindness can tell apart, instead of shades of grey.

`gopts(`*options*`)` adds graph options to every graph, such as `gopts(title("Tariff reform"))` or `gopts(xsize(5) ysize(3.5))`.

## Remarks and examples

A model has a few household groups; poverty and inequality are measured on household surveys. `cge_microsim` links the two in the top-down way described by Bourguignon, Bussolo and Pereira da Silva (2008): the model gives, for each group, the change in each source of income and in each price; the survey gives each household's income composition, budget shares, size and weight. The survey's own income and consumption data drive the results, so a household that lives on wages gains more than its group when wages rise relative to other income. There is no feedback from the survey to the model. Every substantial source of income should be mapped, and the survey and the SAM should refer to the same year where possible.

An observation is used if it satisfies *if* and *in* and has no missing value in the group, consumption, weight, size or the variables of `income()` and `budget()`.

### Example 1: every household follows its group

The synthetic SAM is calibrated as in **[CGE]** `cge_solve`, and every import tariff is removed, with the direct tax rates of households adjusting to keep government savings fixed:

```
. cge_closure, government(fixgsav direct)
Closure
  external account     flexexr
  government           fixgsav
  savings-investment   savings
  numeraire            cpi
  factor lab           mobile
  factor cap           mobile

. cge_shock tm, value(0)
Parameter tm changed for 3 element(s)

. cge_solve, name(notariff)
Simulation notariff solved in 7 iteration(s) (newton)
Largest scaled residual  7.70e-14; WALRAS = -3.37e-13
```

The survey is synthetic, generated with a fixed seed: 1,000 households of one to six persons in the model's single group, `hhd`, with weights, incomes from wages, profits and government transfers, and spending on food, goods and services:

```
. clear

. set seed 2026

. set obs 1000
obs was 0, now 1000

. generate str3 group = "hhd"

. generate double hhsize = 1 + floor(6 * runiform())

. generate double weight = 100 + 50 * runiform()

. generate double wage = exp(rnormal(5, 0.7)) * hhsize
. generate double profit = cond(runiform() < 0.3, exp(rnormal(5, 1)), 0)

. generate double transfer = cond(runiform() < 0.2, 50 * runiform(), 0)

. generate double cons = 0.9 * (wage + profit + transfer)

. generate double food = cons * (0.6 - 0.3 * runiform())

. generate double services = (cons - food) / 2

. generate double goods = cons - food - services

. summarize hhsize weight wage profit transfer cons food goods services

    Variable |        Obs        Mean    Std. Dev.       Min        Max
-------------+---------------------------------------------------------
      hhsize |       1000       3.533    1.698763          1          6
      weight |       1000    125.3665    14.48972   100.0673    149.834
        wage |       1000    656.0342    616.9286   31.80859   5619.254
      profit |       1000    64.56535    183.1834          0    2612.67
    transfer |       1000    4.574924    11.33942          0   49.75252
-------------+---------------------------------------------------------
        cons |       1000     652.657    577.8481   28.62773   5413.999
        food |       1000    298.2126    286.0872   11.90586   2977.315
       goods |       1000    177.2222    155.8523   6.217155    1384.16
    services |       1000    177.2222    155.8523   6.217155    1384.16
```

Without `income()` and `budget()`, each household's nominal consumption changes as its group's, and its price index is the group's:

```
. cge_microsim [pweight = weight], sim(notariff) group(group) consumption(cons) size(hhsize) ///
>     line(100)

Household survey results of simulation notariff: 1000 households
Real consumption per person (base prices); poverty and inequality over persons
--------------------------------------------------------------------------------------------------
                     base   simulated    % chg      headcount (%)    poverty gap (%)          Gini (%)
--------------------------------------------------------------------------------------------------
all                 184.3       183.5   -0.431    26.75 ->  26.94     8.72 ->   8.80    36.32 ->  36.32
hhd                 184.3       183.5   -0.431    26.75 ->  26.94     8.72 ->   8.80    36.32 ->  36.32
--------------------------------------------------------------------------------------------------
Poverty line 100 per person; severity (FGT2) is in r(table)
```

The table shows, for all households and for each group in the survey, mean real consumption per person, weighted by persons, in the base and in the simulation, and its per cent change; the poverty headcount and poverty gap, in per cent; and the Gini coefficient of consumption per person, in per cent. Every household of a group changes in the same proportion, so with one group the Gini coefficient cannot change; the poverty measures change because households near the line cross it.

### Example 2: households' own incomes and budgets

With `income()`, each household's income components change as their sources do for its group; with `budget()`, each household has its own price index:

```
. cge_microsim [pweight = weight], sim(notariff) group(group) consumption(cons) size(hhsize) ///
>     income(wage:lab profit:cap transfer:gov)                                          ///
>     budget(food:com_agr goods:com_man services:com_ser) line(100) generate(nt)

Household survey results of simulation notariff: 1000 households
Real consumption per person (base prices); poverty and inequality over persons
--------------------------------------------------------------------------------------------------
                     base   simulated    % chg      headcount (%)    poverty gap (%)          Gini (%)
--------------------------------------------------------------------------------------------------
all                 184.3       183.6   -0.389    26.75 ->  26.91     8.72 ->   8.80    36.32 ->  36.33
hhd                 184.3       183.6   -0.389    26.75 ->  26.91     8.72 ->   8.80    36.32 ->  36.33
--------------------------------------------------------------------------------------------------
Poverty line 100 per person; severity (FGT2) is in r(table)
```

Real consumption per person falls by less, 0.389 against 0.431 per cent, and the Gini coefficient now changes, from 36.32 to 36.33 per cent, because households with different shares of wages, profits and transfers, and of food, goods and services, are affected differently. The survey and the SAM are synthetic: the results illustrate the command and are not findings about any country.

### Example 3: stored results and new variables

The measures for all households are returned as scalars, and the whole table, with the severity of poverty (FGT2), in `r(table)`; poverty measures and Gini coefficients are stored as proportions, not per cent:

```
. return list

scalars:
       r(N_quantiles) =  10
           r(N_lines) =  1
            r(N_sims) =  1
          r(gini_sim) =  .3632654507489588
         r(gini_base) =  .3632179401351559
          r(fgt0_sim) =  .2690883392367442
         r(fgt0_base) =  .2674509902763488
                 r(N) =  1000

macros:
             r(lines) : "100"
              r(sims) : "notariff"

matrices:
               r(gic) :  99 x 3
           r(welfare) :  2 x 4
         r(quantiles) :  10 x 10
        r(inequality) :  2 x 18
           r(poverty) :  2 x 8
             r(table) :  2 x 11

. matrix list r(table), format(%9.4f)

r(table)[2,11]
     mean_base   mean_sim   mean_pct  fgt0_base   fgt0_sim  fgt1_base   fgt1_sim  fgt2_base
all   184.2888   183.5727    -0.3886     0.2675     0.2691     0.0872     0.0880     0.0392
hhd   184.2888   183.5727    -0.3886     0.2675     0.2691     0.0872     0.0880     0.0392

      fgt2_sim  gini_base   gini_sim
all     0.0396     0.3632     0.3633
hhd     0.0396     0.3632     0.3633
```

`generate(nt)` added each household's new nominal consumption, price index, real consumption and real consumption per person:

```
. summarize nt_nom nt_price nt_real nt_pc

    Variable |       Obs        Mean    Std. Dev.       Min        Max
-------------+---------------------------------------------------------
      nt_nom |      1000    650.0388    575.6191   28.51639   5393.112
    nt_price |      1000    .9998786    .0000725   .9997569   1.000003
     nt_real |      1000    650.1204    575.7009   28.52252   5394.202
       nt_pc |      1000    193.5642    157.5233   19.73313   1446.804
```

These variables can be analysed with any Stata command, for example to compute other poverty or inequality measures, or measures by region.

### Example 4: several simulations and several poverty lines

A second scenario doubles the tariffs. Both scenarios are passed to the survey at once, with the national line of 100 and a regional line of 90 or 110 per person, held in a variable:

```
. cge_shock, reset
All shocks removed

. cge_shock tm, scale(2)
Parameter tm changed for 3 element(s)

. cge_solve, name(tariff2)
Simulation tariff2 solved in 7 iteration(s) (newton)
Largest scaled residual  1.95e-14; WALRAS = -2.39e-13

. generate double zreg = cond(runiform() < 0.5, 90, 110)
. cge_microsim [pweight = weight], sim(notariff tariff2) group(group) consumption(cons) size(hhsize) ///
>     income(wage:lab profit:cap transfer:gov)                                                    ///
>     budget(food:com_agr goods:com_man services:com_ser) line(100 zreg) quantiles(5) detail

Household survey results of simulation notariff: 1000 households
Real consumption per person (base prices); poverty and inequality over persons
-------------------------------------------------------------------------------------------------
                      base   simulated    % chg      headcount (%)    poverty gap (%)          Gini (%)
-------------------------------------------------------------------------------------------------
all                  184.3       183.6   -0.389    26.75 ->   26.91     8.72 ->    8.80    36.32 ->   36.33
hhd                  184.3       183.6   -0.389    26.75 ->   26.91     8.72 ->    8.80    36.32 ->   36.33
-------------------------------------------------------------------------------------------------
Poverty line 100 per person; severity (FGT2) is in r(table), the other lines in r(poverty)

Poverty by line, base -> simulated: headcount, poverty gap and severity in per cent of persons;
number of poor persons in units of the weights
-------------------------------------------------------------------------------------------------
line                      headcount       poverty gap          severity              number poor
-------------------------------------------------------------------------------------------------
100       all       26.75 ->  26.91    8.72 ->   8.80    3.92 ->   3.96     118,660 ->    119,387
          hhd       26.75 ->  26.91    8.72 ->   8.80    3.92 ->   3.96     118,660 ->    119,387
zreg      all       25.82 ->  26.40    8.95 ->   9.02    4.16 ->   4.20     114,562 ->    117,145
          hhd       25.82 ->  26.40    8.95 ->   9.02    4.16 ->   4.20     114,562 ->    117,145
-------------------------------------------------------------------------------------------------

Inequality of consumption per person over persons, all households
--------------------------------------------------------
                             base  simulated     change
--------------------------------------------------------
Gini                      0.36322    0.36327    0.00005
Theil, GE(1)              0.22526    0.22532    0.00006
Mean log deviation        0.22488    0.22494    0.00006
GE(2)                     0.28873    0.28883    0.00010
Atkinson, e = 0.5         0.10631    0.10634    0.00003
Atkinson, e = 1           0.20139    0.20143    0.00004
Atkinson, e = 2           0.36302    0.36307    0.00005
Palma                     1.49633    1.49674    0.00040
S80/S20                   6.72631    6.72794    0.00163
--------------------------------------------------------

Equivalent and compensating variations at base prices: per person, and per cent of base consumption
------------------------------------------------------------------
                         EV       EV (%)           CV       CV (%)
------------------------------------------------------------------
all                  -.6939      -0.3765       -.6937      -0.3764
hhd                  -.6939      -0.3765       -.6937      -0.3764
------------------------------------------------------------------

Incidence by quantile of base consumption per person (same persons before and after)
-------------------------------------------------------------------------------------------------
quantile    persons    mean base    simulated    % chg   share of consumption  share of gain
-------------------------------------------------------------------------------------------------
1            20.04%        59.09        58.86   -0.404      6.43% ->    6.42%          6.68%
2            19.99%          106        105.6   -0.402     11.50% ->   11.50%         11.88%
3            20.05%        148.4        147.8   -0.396     16.15% ->   16.15%         16.45%
4            20.03%        211.5        210.7   -0.389     22.98% ->   22.98%         23.00%
5            19.89%        397.9        396.3   -0.380     42.94% ->   42.95%         41.99%
-------------------------------------------------------------------------------------------------

Household survey results of simulation tariff2: 1000 households
Real consumption per person (base prices); poverty and inequality over persons
-------------------------------------------------------------------------------------------------
                      base   simulated    % chg      headcount (%)    poverty gap (%)          Gini (%)
-------------------------------------------------------------------------------------------------
all                  184.3       184.5    0.095    26.75 ->   26.68     8.72 ->    8.70    36.32 ->   36.32
hhd                  184.3       184.5    0.095    26.75 ->   26.68     8.72 ->    8.70    36.32 ->   36.32
-------------------------------------------------------------------------------------------------
Poverty line 100 per person; severity (FGT2) is in r(table), the other lines in r(poverty)

Poverty by line, base -> simulated: headcount, poverty gap and severity in per cent of persons;
number of poor persons in units of the weights
-------------------------------------------------------------------------------------------------
line                      headcount       poverty gap          severity              number poor
-------------------------------------------------------------------------------------------------
100       all       26.75 ->  26.68    8.72 ->   8.70    3.92 ->   3.91     118,660 ->    118,381
          hhd       26.75 ->  26.68    8.72 ->   8.70    3.92 ->   3.91     118,660 ->    118,381
zreg      all       25.82 ->  25.82    8.95 ->   8.93    4.16 ->   4.15     114,562 ->    114,562
          hhd       25.82 ->  25.82    8.95 ->   8.93    4.16 ->   4.15     114,562 ->    114,562
-------------------------------------------------------------------------------------------------

Inequality of consumption per person over persons, all households
--------------------------------------------------------
                             base  simulated     change
--------------------------------------------------------
Gini                      0.36322    0.36318   -0.00004
Theil, GE(1)              0.22526    0.22521   -0.00005
Mean log deviation        0.22488    0.22484   -0.00004
GE(2)                     0.28873    0.28865   -0.00008
```

```
Atkinson, e = 0.5          0.10631    0.10629   -0.00002
Atkinson, e = 1            0.20139    0.20136   -0.00003
Atkinson, e = 2            0.36302    0.36298   -0.00004
Palma                      1.49633    1.49602   -0.00031
S80/S20                    6.72631    6.72507   -0.00123
--------------------------------------------------------

Equivalent and compensating variations at base prices: per person, and per cent of base consumption
------------------------------------------------------------------
                        EV      EV (%)          CV      CV (%)
------------------------------------------------------------------
all                  .1926      0.1045       .1926      0.1045
hhd                  .1926      0.1045       .1926      0.1045
------------------------------------------------------------------

Incidence by quantile of base consumption per person (same persons before and after)
-----------------------------------------------------------------------------------------------------
quantile    persons    mean base    simulated    % chg   share of consumption  share of gain
-----------------------------------------------------------------------------------------------------
1            20.04%        59.09        59.16    0.107      6.43% ->   6.43%           7.23%
2            19.99%          106        106.1    0.106     11.50% ->  11.50%          12.80%
3            20.05%        148.4        148.6    0.101     16.15% ->  16.15%          17.18%
4            20.03%        211.5        211.7    0.095     22.98% ->  22.98%          23.01%
5            19.89%        397.9        398.2    0.088     42.94% ->  42.94%          39.78%
-----------------------------------------------------------------------------------------------------
```

The table shows each simulation in turn. With `detail`, each simulation also has poverty for every line, the inequality measures, and incidence by quintile (here, with `quantiles(5)`): the persons of each fifth of the base distribution of consumption per person, their mean consumption before and after, their shares of total consumption, and their share of the total change. At the regional lines, 25.82 per cent of persons are poor in the base, against 26.75 per cent at the national line, and removing the tariffs raises the headcount by 0.58 points, against 0.16. Doubling the tariffs raises mean consumption by 0.095 per cent, and the poorest fifth receives 7.2 per cent of the gain while holding 6.4 per cent of consumption.

### Example 5: the results as a dataset

With `clear`, the results replace the survey in memory; `saving()` would save them and keep the survey. Each observation is one result:

```
. cge_microsim [pweight = weight], sim(notariff tariff2) group(group) consumption(cons) size(hhsize) ///
>     income(wage:lab profit:cap transfer:gov)                                                    ///
>     budget(food:com_agr goods:com_man services:com_ser) line(100 zreg) clear nodisplay

. describe, short

Contains data
  obs:           378
 vars:            11
 size:        28,350
Sorted by:
     Note:  dataset has changed since last saved

. list sim measure line base value change pct if table == "summary" & group == "all" & ///
>     inlist(measure, "mean", "fgt0", "npoor", "theil", "palma"), noobs sepby(sim)

  +-----------------------------------------------------------------------------+
  |      sim   measure   line        base       value       change          pct |
  |-----------------------------------------------------------------------------|
  | notariff      mean          184.28883   183.57269   -.71614257   -.38859791 |
  | notariff      fgt0    100   .26745099   .26908834    .00163735    .61220523 |
  | notariff     npoor    100   118660.08   119386.52    726.44319    .61220523 |
  | notariff      fgt0   zreg    .2582147   .26403556    .00582086     2.254272 |
  | notariff     npoor   zreg   114562.21   117144.75    2582.5437     2.254272 |
  | notariff     theil          .22525936   .22532191    .00006255    .02776857 |
  | notariff     palma          1.4963349   1.4967363    .00040139    .02682493 |
  |-----------------------------------------------------------------------------|
  |  tariff2      mean          184.28883   184.46422    .17538989     .0951712 |
  |  tariff2      fgt0    100   .26745099   .26682288   -.00062811     -.234851 |
  |  tariff2     npoor    100   118660.08    118381.4   -278.67437     -.234851 |
  |  tariff2      fgt0   zreg    .2582147    .2582147            0            0 |
  |  tariff2     npoor   zreg   114562.21   114562.21            0            0 |
  |  tariff2     theil          .22525936    .2252111   -.00004826   -.02142531 |
  |  tariff2     palma          1.4963349   1.4960225   -.00031239    -.0208768 |
  +-----------------------------------------------------------------------------+
```

The growth incidence curve is in the table `gic`: the per cent change of each percentile of consumption per person, from the base to the simulation. It compares the distributions, not the same households; the table `quantile` follows the same persons:

```
. list sim q base value pct if table == "gic" & inlist(q, 10, 50, 90), noobs sepby(sim)

  +----------------------------------------------------+
  |      sim    q        base       value          pct |
  |----------------------------------------------------|
  | notariff   10   60.153662   59.929553   -.37256058 |
  | notariff   50   150.19273   149.64448   -.36502854 |
  | notariff   90   341.61609   339.80555   -.52999198 |
  |----------------------------------------------------|
  |  tariff2   10   60.153662   60.201621    .07972802 |
  |  tariff2   50   150.19273   150.30709    .07614246 |
  |  tariff2   90   341.61609   341.89953    .08297064 |
  +----------------------------------------------------+

. list sim q base value pct if table == "quantile" & measure == "mean" & q <= 3, noobs sepby(sim)

  +---------------------------------------------------+
  |      sim   q        base       value          pct |
  |---------------------------------------------------|
  | notariff   1   45.808231     45.6273   -.39497573 |
  | notariff   2   72.473935   72.176882   -.40987659 |
  | notariff   3    95.71359    95.32399   -.40704725 |
  |---------------------------------------------------|
  |  tariff2   1   45.808231   45.853345    .09848529 |
  |  tariff2   2   72.473935   72.555491    .11253122 |
  |  tariff2   3    95.71359   95.819687    .11084901 |
  +---------------------------------------------------+
```

### Example 6: welfare, and the channels of the changes

`generate()` adds each household's equivalent and compensating variations, and `decompose` divides each change among its channels:

```
. cge_microsim [pweight = weight], sim(notariff) group(group) consumption(cons) size(hhsize) ///
>     income(wage:lab profit:cap transfer:gov)                                          ///
>     budget(food:com_agr goods:com_man services:com_ser) line(100) generate(w1) decompose

Household survey results of simulation notariff: 1000 households
Real consumption per person (base prices); poverty and inequality over persons
--------------------------------------------------------------------------------------------------
                      base   simulated    % chg      headcount (%)    poverty gap (%)           Gini (%)
--------------------------------------------------------------------------------------------------
all                  184.3       183.6   -0.389    26.75 ->   26.91     8.72 ->    8.80    36.32 ->   36.33
hhd                  184.3       183.6   -0.389    26.75 ->   26.91     8.72 ->    8.80    36.32 ->   36.33
--------------------------------------------------------------------------------------------------
Poverty line 100 per person; severity (FGT2) is in r(table)

Contributions to the changes, all households (Shapley decomposition by channel)
------------------------------------------------------------------------------
channel            mean (% chg)  headcount (pts)   pov. gap (pts)   Gini (pts)
------------------------------------------------------------------------------
lab                      2.4056          -1.2478          -0.4560      -0.0308
cap                      0.2341          -0.0709          -0.0237       0.0352
gov                      0.0000           0.0000           0.0000       0.0000
taxsav                  -3.0408           1.4824           0.5550       0.0000
prices                   0.0125           0.0000          -0.0020       0.0003
total                   -0.3886           0.1637           0.0733       0.0048
------------------------------------------------------------------------------
Poverty at line 100: growth  0.1637 and redistribution  0.0000 points of the headcount
Every measure and line is in r(shapley) and r(growthred)
```

```
. generate double dreal = w1_real - cons

. summarize w1_ev w1_cv dreal

    Variable |       Obs        Mean    Std. Dev.       Min        Max
-------------+--------------------------------------------------------
       w1_ev |      1000   -2.457871    2.099621  -19.25553  -.1024495
       w1_cv |      1000   -2.457267    2.099087  -19.24971  -.1024175
       dreal |      1000   -2.536581    2.166768  -19.79686  -.1052118
```

```
. count if w1_ev < dreal
    0
```

Removing the tariffs raises wages, which add 2.41 per cent to mean consumption per person, and profits, which add 0.23 per cent; the direct tax rate rises to keep government savings fixed, which takes away 3.04 per cent; prices add 0.01 per cent, and government transfers, fixed in real terms, nothing. The contributions add up to the change, $-0.389$ per cent, and those to the headcount, to its rise of 0.16 points. Growth and redistribution divide the change in the headcount in the same way: here it is all growth, since the change in the distribution, at either mean, moves no one across the line. The equivalent variation, the change in consumption at base prices worth as much as the simulation, is at least the change in real consumption for every household, since the household can substitute among goods.

### Example 7: fiscal incidence

The survey gives market income, wages and profits; direct taxes of 10 per cent of it; indirect taxes of 5 per cent of consumption; and free schooling, valued at cost, for some households, which follows government purchases of services:

```
. generate double market = wage + profit

. generate double dtx = 0.1 * market

. generate double itx = 0.05 * cons

. generate double school = cond(runiform() < 0.4, 20 * runiform(), 0)


. cge_microsim [pweight = weight], sim(notariff) group(group) consumption(cons) size(hhsize)       ///
>     income(wage:lab profit:cap transfer:gov) budget(food:com_agr goods:com_man services:com_ser) ///
>     line(100) market(market) dtax(dtx) transfers(transfer) itax(itx) inkind(school:com_ser)

Household survey results of simulation notariff: 1000 households
Real consumption per person (base prices); poverty and inequality over persons
-------------------------------------------------------------------------------------------------
                     base   simulated    % chg      headcount (%)    poverty gap (%)          Gini (%)
-------------------------------------------------------------------------------------------------
all                 184.3       183.6   -0.389    26.75 ->   26.91     8.72 ->    8.80    36.32 ->   36.33
hhd                 184.3       183.6   -0.389    26.75 ->   26.91     8.72 ->    8.80    36.32 ->   36.33
-------------------------------------------------------------------------------------------------
Poverty line 100 per person; severity (FGT2) is in r(table)

Fiscal incidence: income concepts per person, persons ranked by market income per person
-------------------------------------------------------------------------------------------------
income        mean, base -> simulated                  Gini         concentration   headcount (%)
-------------------------------------------------------------------------------------------------
market            203.5 ->        209   0.3649 ->    0.3649   0.3649 ->    0.3649  23.22 ->  22.13
net market        183.1 ->      181.5   0.3649 ->    0.3649   0.3649 ->    0.3649  27.33 ->  27.78
disposable        184.4 ->      182.8   0.3631 ->    0.3631   0.3628 ->    0.3628  26.75 ->  27.17
consumable        175.2 ->      173.6   0.3631 ->    0.3631   0.3628 ->    0.3628  29.39 ->  29.68
final             176.4 ->      174.8   0.3610 ->    0.3610   0.3607 ->    0.3606  28.72 ->  29.17
-------------------------------------------------------------------------------------------------
instrument                   % of market income                Kakwani  coverage (%)    MC to Gini
-------------------------------------------------------------------------------------------------
dtx          direct tax     10.000 ->    13.144  -0.0000 ->    -0.0000        100.00       0.00036
transfer     transfer         0.628 ->    0.612   0.3004 ->     0.3018         18.29       0.00189
itx          indirect tax     4.528 ->    4.390  -0.0019 ->    -0.0019        100.00       0.00010
school       in kind          0.581 ->    0.578   0.3177 ->     0.3178         41.31       0.00202
-------------------------------------------------------------------------------------------------
Redistributive effect  0.0038 ->  0.0039; Reynolds-Smolensky  0.0042 ->  0.0043; reranking  0.0004 ->  0.0004
Fiscal impoverishment at line 100:  29.00 ->  29.45 per cent of persons
Kakwani: positive when progressive. MC: Gini of final income without the instrument less with it, base
```

Direct taxes, 10 per cent of market income, are proportional: their Kakwani index is 0. Transfers and schooling, of amounts unrelated to income, are worth relatively more to poorer persons and are progressive, with Kakwani indexes of 0.30 and 0.32; indirect taxes, 5 per cent of consumption, take a slightly larger share of the market income of poorer persons, and their index is $-0.002$. Each instrument's marginal contribution is the change in the Gini coefficient of final income that it makes. The simulation changes each instrument with the model: the direct tax rate rises to keep government savings fixed, so direct taxes rise from 10.0 to 13.1 per cent of market income, and fiscal impoverishment from 29.00 to 29.45 per cent of persons.

**Example 8: reweighting, standard errors and bands**

Two-fifths of the synthetic households, `agri`, work in agriculture. With the tariffs removed, the model's employment of labour in agriculture falls by 1.3 per cent; reweighting the survey so that the agricultural households fall in the same proportion lowers their weights by that much and leaves the others' unchanged, the closest weights to the survey's that meet the target:

```
. generate double agri = runiform() < 0.4


. cge_microsim [pweight = weight], sim(notariff) group(group) consumption(cons) size(hhsize) ///
>     income(wage:lab profit:cap transfer:gov) line(100) reweight(agri:lab.act_agr) generate(rw)

Household survey results of simulation notariff: 1000 households
Real consumption per person (base prices); poverty and inequality over persons
--------------------------------------------------------------------------------------------------
                      base   simulated    % chg      headcount (%)    poverty gap (%)           Gini (%)
--------------------------------------------------------------------------------------------------
all                  184.3       183.5   -0.403    26.75 ->   26.90     8.72 ->    8.79    36.32 ->   36.32
hhd                  184.3       183.5   -0.403    26.75 ->   26.90     8.72 ->    8.79    36.32 ->   36.32
--------------------------------------------------------------------------------------------------
Poverty line 100 per person; severity (FGT2) is in r(table)
Weights reweighted to 1 targets in 3 iterations (largest relative deviation  2.9e-16)

. matrix list r(reweight)

r(reweight)[1,4]
           base     target   achieved      ratio
agri  50289.323  49641.184  49641.184  .98711179
```

With the survey design declared, in clusters of ten households in four strata, the mean and the poverty measures have standard errors, and so do their changes. The standard error of the change in the headcount, 0.0016, is much smaller than that of the headcount itself, 0.0137, since the same households are compared before and after:

```
. generate long psu = ceil(_n / 10)

. generate long stratum = mod(psu, 4)

. svyset psu [pweight = weight], strata(stratum)

      pweight: weight
          VCE: linearized
  Single unit: missing
     Strata 1: stratum
         SU 1: psu
        FPC 1: <zero>

. cge_microsim, sim(notariff) group(group) consumption(cons) size(hhsize) ///
>     income(wage:lab profit:cap transfer:gov) line(100) svy nodisplay

. matrix list r(se), format(%9.5f)

r(se)[4,3]
              se_base     se_sim  se_change
      mean    4.59262    4.57479    0.01800
line1:fgt0    0.01366    0.01381    0.00164
line1:fgt1    0.00554    0.00556    0.00004
line1:fgt2    0.00320    0.00322    0.00002

. svyset, clear
```

A sensitivity analysis with 20 draws of the elasticities, each kept as a simulation, gives the range of the results. Here poverty and inequality hardly depend on the elasticities, while the equivalent variation per person ranges from −0.81 to −0.63 between the 5th and 95th percentiles:

```
. capture program drop notariff_scen

. program define notariff_scen
  1.     cge_shock tm, value(0)
  2. end

. cge_shock, reset
All shocks removed
```

```
. cge_sensitivity, scenario(notariff_scen) draws(20) seed(2026) keep(dr) nodisplay
Sensitivity analysis: 20 draws of the elasticities (uniform within +/-50 per cent)
  10 draws done
  20 draws done
Kept 21 draws as simulations dr0 to dr20


. cge_microsim [pweight = weight], sim(notariff) group(group) consumption(cons) size(hhsize) ///
>     income(wage:lab profit:cap transfer:gov) line(100) bands(dr)

Household survey results of simulation notariff: 1000 households
Real consumption per person (base prices); poverty and inequality over persons
---------------------------------------------------------------------------------------------------
                     base   simulated    % chg      headcount (%)    poverty gap (%)           Gini (%)
---------------------------------------------------------------------------------------------------
all                 184.3       183.5   -0.401    26.75 ->   26.91     8.72 ->    8.80    36.32 ->   36.33
hhd                 184.3       183.5   -0.401    26.75 ->   26.91     8.72 ->    8.80    36.32 ->   36.33
---------------------------------------------------------------------------------------------------
Poverty line 100 per person; severity (FGT2) is in r(table)

Bands across 20 draws of the elasticities, simulation notariff, all households
---------------------------------------------------------------------------------
                                base    central        5th     median       95th
---------------------------------------------------------------------------------
mean consumption               184.3      183.5      183.5      183.6      183.6
headcount, line 100            .2675      .2691      .2691      .2691      .2694
poverty gap, line 100         .08725       .088     .08793     .08799      .0881
Gini                           .3632      .3633      .3633      .3633      .3633
Theil                          .2253      .2253      .2253      .2253      .2253
EV per person                      0     -.7107     -.8084     -.6989     -.6329
---------------------------------------------------------------------------------
Every measure is in r(bands), with the standard deviation across the draws
```

### Example 9: graphs for a paper

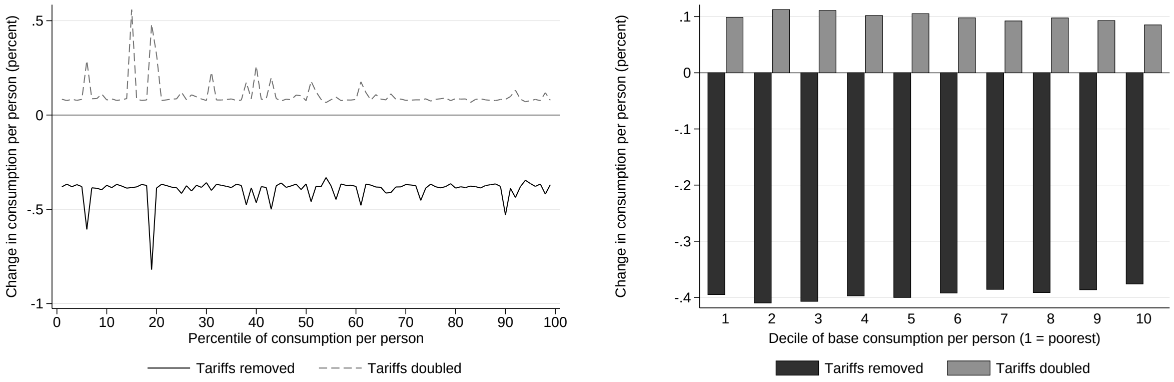
From version 0.9.6, `graph()` draws the results in the style of figures in economics journals, and `gexport()` saves them as files. Both simulations, with labels for the legends, as PDF files:

```
. cge_microsim [pweight = weight], sim(notariff tariff2) group(group)        ///
>     consumption(cons) size(hhsize) income(wage:lab profit:cap transfer:gov) ///
>     budget(food:com_agr goods:com_man services:com_ser) line(100 zreg)       ///
>     decompose graph(gic incidence poverty shapley)                           ///
>     glabels("Tariffs removed" "Tariffs doubled") gexport(fig_ms.pdf) nodisplay

. display "`r(graphs)'"
microsim_gic microsim_incidence microsim_poverty microsim_shapley
```

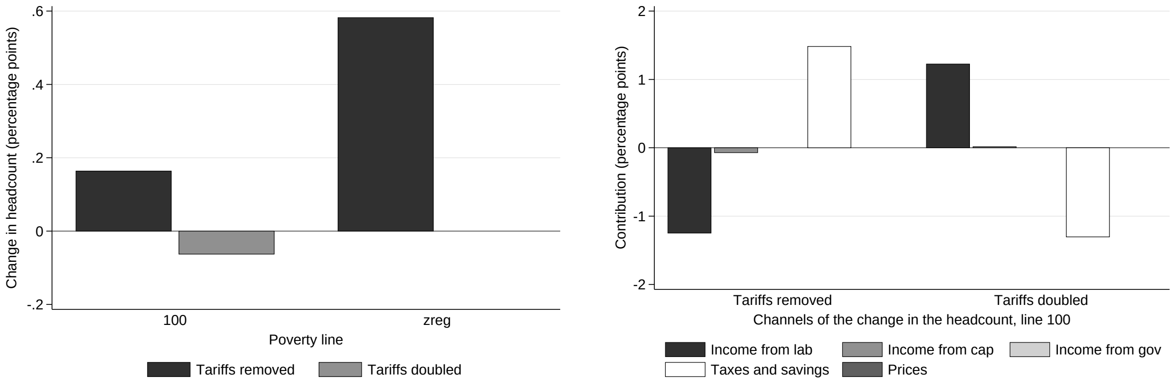
Figure 26.1 shows four of them. The graphs have a white background, axes on the left and below, light horizontal grid lines, a line at zero and the legend below, without a frame; they have no title, which a journal sets in the caption. Shades of grey and line patterns tell the series apart, so the graphs print in black and white; `gcolor` uses colours instead.

| Graph | Content |
|---|---|
| `gic` | the per cent change of each percentile of consumption per person, one line for each simulation |
| `incidence` | the per cent change in mean consumption per person of each quantile group of base consumption |
| `poverty` | the change in the poverty headcount, in percentage points, at each line; needs `line()` |
| `groups` | the per cent change in mean consumption per person of each household group |
| `shapley` | the contribution of each channel to the change in the headcount at the first line, in percentage points, or to the change in mean consumption without a line; needs `decompose` |
| `fiscal` | the Gini coefficient of each income concept, market to final, in the base and each simulation; needs `market()` |
| `bands` | the median and the 5th to 95th percentiles across the draws of the per cent change in mean consumption, the headcount at each line and the Gini coefficient; needs `bands()` |



(a) `gic`: growth incidence curve

(b) `incidence`: by decile



(c) `poverty`: headcount at each line

(d) `shapley`: channels of the change in the headcount

**Figure 26.1:** Graphs of `cge_microsim`: removing the tariffs and doubling them, for the SYNTHETIC survey

### Results as a dataset

With clear or saving(), the results form a dataset with one observation per simulation, table, group, measure, line and quantile:

| Variable | Content |
|---|---|
| sim | the simulation |
| table | summary, quantile, gic, shapley, growthred, fiscal, fiscalq, reweight or bands |
| group | all, every household, or a household group (summary) |
| measure | the measure, below |
| line | the poverty line, for fgt0, fgt1, fgt2 and npoor |
| channel | the channel (shapley), component (growthred), income concept or instrument (fiscal, fiscalq), from version 0.8.1 |
| q | the quantile group (quantile) or the percentile (gic) |
| base, value | the base and simulated values |
| change, pct | the change and the per cent change |
| se_base, se_value, se_change | with svy, the standard errors of the base value, the simulated value and the change |

The measures of summary are mean (mean consumption per person); ev and cv, the equivalent and compensating variations per person, with pct their per cent of base consumption; for each line fgt0, fgt1, fgt2 and npoor (the number of poor persons, in units of the weights); and gini, theil, mld, ge2, atkinson_05, atkinson_1, atkinson_2, palma and s80s20. Those of quantile are popshare, mean, share (the share of total consumption), inc_abs (the share of the total change, in value) and ev; that of gic is quantile, the percentile. With decompose, the table shapley has, for every measure of summary for all households and each channel, the base value and the base value plus the channel's contribution, so that change is the contribution; growthred has the growth and redistribution components of each poverty measure. With market(), the table fiscal has, for each income concept (channel market, net_market, disposable, consumable or final), mean, gini, conc and the poverty measures; for each instrument, total, pct_market, conc, kakwani, coverage, coverage_poor, mc_gini and mc_fgt0; and re, rs, rerank and fi; the table fiscalq has each instrument's share and pct_market by quantile of market income per person. With reweight(), the table reweight has each characteristic's weighted total, base and reweighted; with bands(), the table bands has, for every measure for all households, its percentiles (channel p5, p50, p95) and standard deviation (sd) across the draws, in value. Poverty measures and inequality indexes are proportions; pct is the per cent change of the measure itself.

## Stored results

cge_microsim stores the following in r():

*Scalars*

| | |
|---|---|
| r(N) | number of households used |
| r(N_sims) | number of simulations |
| r(N_lines) | number of poverty lines |
| r(N_quantiles) | number of quantile groups |
| r(rw_iterations) | with reweight(): iterations of the reweighting |
| r(rw_deviation) | with reweight(): largest relative deviation from the targets |
| r(N_draws) | with bands(): number of draws |
| r(fgt0_base) | poverty headcount at the first line in the base, all households (missing without line()) |
| r(fgt0_sim) | poverty headcount in the simulation, all households |
| r(gini_base) | Gini coefficient in the base, all households |
| r(gini_sim) | Gini coefficient in the simulation, all households |

*Macros*

| | |
|---|---|
| r(sims) | the simulations |
| r(lines) | the poverty lines |
| r(channels) | the channels of decompose |

| | |
|---|---|
| `r(instlist)` | the fiscal instruments |
| `r(graphs)` | with `graph()`: the names of the graphs drawn |
| *Matrices* | |
| `r(table)` | one row for all households (`all`) and one for each group in the survey; columns `mean_base`, `mean_sim`, `mean_pct`, `fgt0_base`, `fgt0_sim`, `fgt1_base`, `fgt1_sim`, `fgt2_base`, `fgt2_sim` (first line), `gini_base` and `gini_sim` |
| `r(poverty)` | for each line (equations `line1`, `line2`, . . . ) and group: FGT0, FGT1, FGT2 and the number of poor, base and simulated |
| `r(inequality)` | for each group: the nine inequality measures, base and simulated |
| `r(quantiles)` | for each quantile group: `popshare`, `mean_base`, `mean_sim`, `mean_pct`, `share_base`, `share_sim` and `inc_abs` |
| `r(gic)` | for percentiles 1 to 99: the base and simulated values and their per cent change |
| `r(welfare)` | for each group: `ev` and `cv` per person, `ev_pct` and `cv_pct`, as per cent of base consumption |
| `r(shapley)` | with `decompose`: for each measure (poverty measures with equations `line1`, . . . ), the base, the contribution of each channel, and the simulated value |
| `r(growthred)` | with `decompose`: for each line and poverty measure, the base, growth, redistribution and the simulated value |
| `r(concepts)` | with `market()`: for each income concept, the mean per person, Gini and concentration coefficients and, with a line, FGT0 to FGT2 and the number of poor, base and simulated |
| `r(instruments)` | with `market()`: for each instrument, the measures of the table `fiscal`, base and simulated |
| `r(redistribution)` | with `market()`: `re`, `rs`, `rerank` and `fi`, base and simulated |
| `r(fshare)` | with `market()`: each instrument's base share by quantile of market income |
| `r(reweight)` | with `reweight()`: for each characteristic, the base total, the target, the total achieved and the ratio |
| `r(se)` | with `svy`: for all households, the standard errors of mean consumption and of FGT0 to FGT2 for each line, base, simulated and of the change |
| `r(bands)` | with `bands()`: for each measure, the base, the value in the first simulation, the 5th, 50th and 95th percentiles and the standard deviation across the draws |

The results in `r()` are those of the first simulation of `sim()`.

## Methods and formulas

Household $i$ belongs to group $h = h(i)$ and has income components $y_{ik}$ with sources $k$. The model gives, for each group, the ratio of the simulated to the base value of each source,

| Source | Ratio $\rho_{hk}$ | If the group's base value is zero |
|---|---|---|
| factor $f$ | $YIF_{hf}/YIF^0_{hf}$ | $YF_f/YF^0_f$ |
| `gov` | $trnsfr_{h,gov}\,CPI/(trnsfr^0_{h,gov}\,CPI^0)$ | $CPI/CPI^0$ |
| `row` | $trnsfr_{h,row}\,EXR/(trnsfr^0_{h,row}\,EXR^0)$ | $EXR/EXR^0$ |
| `dom` | $\sum_d shii_{hd}\,DY_d/\sum_d shii^0_{hd}\,DY^0_d$ | $CPI/CPI^0$ |
| `cpi` | $CPI/CPI^0$ | |
| `fixed` | 1 | |

where $DY_d = (1 - MPS_d)(1 - TINS_d)\,YI_d$ is the income that institution $d$ passes on or spends. The household's nominal consumption becomes

$$C^1_i = C^0_i\,\frac{\sum_k \rho_{hk}\,y_{ik}}{\sum_k y_{ik}}\,\kappa_h, \qquad \kappa_h = \frac{(1 - MPS_h)(1 - TINS_h)}{(1 - MPS^0_h)(1 - TINS^0_h)},$$

and $C^1_i = C^0_i\,EH_h/EH^0_h$ for a household whose components sum to zero, and for every household without `income()`. The household's price index is

$$P_i = \sum_j s_{ij}\,\frac{p_j}{p_j^0},$$

over commodities and home-consumed activity outputs $j$, where $p_j$ is the price households pay for commodity $j$, $PH_j$, including VAT, or the activity price $PA_j$ for home consumption; $s_{ij}$ are the household's budget shares from `budget()` or else the group's base shares, $p_j^0\,QH_{jh}^0/EH_h^0$ (with $QHA_{jh}^0$ for home consumption). Real consumption is $R_i = C_i^1/P_i$, a first-order (Laspeyres) approximation for households with LES demand, and with household size $n_i$ and weight $w_i$, consumption per person is $r_i^0 = C_i^0/n_i$ in the base and $r_i^1 = R_i/n_i$ in the simulation.

Over the households of a row of the table, with person weights $v_i = w_i n_i$, the mean is $\sum_i v_i r_i/\sum_i v_i$, and for the poverty line $z$

$$\mathrm{FGT}_\alpha = \frac{\sum_i v_i\,\mathbf{1}[r_i < z]\,\big((z - r_i)/z\big)^\alpha}{\sum_i v_i}, \qquad \alpha = 0, 1, 2.$$

The Gini coefficient is computed from the Lorenz curve over persons: with households sorted by $r_i$, $F_i$ the cumulative share of person weights and $L_i$ the cumulative share of $v_i r_i$, and $F_0 = L_0 = 0$,

$$G = 1 - \sum_i (F_i - F_{i-1})(L_i + L_{i-1}).$$

A poverty line given as a variable replaces $z$ by the household's line $z_i$. The number of poor is $\sum_i v_i\,\mathbf{1}[r_i < z_i]$.

**Other inequality measures (from version 0.8.0).** With $\mu = \sum_i v_i r_i/\sum_i v_i$ and $s_i = r_i/\mu$, over the persons with $v_i > 0$,

$$\mathrm{GE}(0) = -\frac{\sum_i v_i \ln s_i}{\sum_i v_i}, \qquad \mathrm{GE}(1) = \frac{\sum_i v_i\, s_i \ln s_i}{\sum_i v_i}, \qquad \mathrm{GE}(2) = \frac{\sum_i v_i (s_i^2 - 1)}{2\sum_i v_i},$$

$$A(\varepsilon) = 1 - \frac{1}{\mu}\left(\frac{\sum_i v_i\, r_i^{1-\varepsilon}}{\sum_i v_i}\right)^{1/(1-\varepsilon)}, \qquad A(1) = 1 - \frac{1}{\mu}\exp\left(\frac{\sum_i v_i \ln r_i}{\sum_i v_i}\right),$$

the mean log deviation, the Theil index, half the squared coefficient of variation, and the Atkinson index for $\varepsilon = 0.5$, 1 and 2. The measures that need $\ln r_i$, or $r_i^{1-\varepsilon}$ with $\varepsilon > 1$, are missing when some person's consumption is 0 or less. With $L(p)$ the Lorenz curve, linear between the points $(F_i, L_i)$, so that a household at a boundary is shared between the groups on either side,

$$\mathrm{Palma} = \frac{1 - L(0.9)}{L(0.4)}, \qquad \mathrm{S80/S20} = \frac{1 - L(0.8)}{L(0.2)}.$$

**Quantiles and incidence.** With the values $r_{(1)} \le \cdots \le r_{(n)}$, $W(i)$ the cumulative person weight and $P = W(n)\,p/100$, the $p$th percentile is $r_{(i)}$ for the first $i$ with $W(i) > P$, or $(r_{(i-1)} + r_{(i)})/2$ when $W(i-1) = P$: the definition of `_pctile` with weights. Persons fall into quantile group $k$ of $K$ when their base consumption per person is above percentile $100(k-1)/K$ and up to percentile $100k/K$, as `xtile` forms groups; tied values stay together, so groups may hold somewhat more or less than $1/K$ of persons. For group $k$, with $S_k^t = \sum_{i\in k} v_i r_i^t$, the table gives the population share $\sum_{i\in k} v_i/\sum_i v_i$, mean consumption $S_k^t/\sum_{i\in k} v_i$ in the base ($t = 0$) and the simulation ($t = 1$), the shares of total consumption $S_k^t/\sum_j S_j^t$, and the share of the total change, $(S_k^1 - S_k^0)/\sum_j (S_j^1 - S_j^0)$. The growth incidence curve (Ravallion and Chen, 2003) is $g(p) = 100\left(r^1(p)/r^0(p) - 1\right)$ at $p = 1, \ldots, 99$, with $r^t(p)$ the $p$th percentile of consumption per person in the base and the simulation.

**Welfare (from version 0.8.1).** With $p_j$ the prices households pay, $\gamma_{jh}$ and $\beta_{jh}$ the subsistence quantities and marginal budget shares of group $h$, and $\lambda_i = C_i^0/EH_h^0$,

$$EV_i = \lambda_i \sum_j p_j^0 \gamma_{jh} + \Big(C_i^1 - \lambda_i \sum_j p_j^1 \gamma_{jh}\Big) \prod_j \Big(\frac{p_j^0}{p_j^1}\Big)^{\beta_{jh}} - C_i^0,$$

$$CV_i = C_i^1 - \lambda_i \sum_j p_j^1 \gamma_{jh} - \Big(C_i^0 - \lambda_i \sum_j p_j^0 \gamma_{jh}\Big) \prod_j \Big(\frac{p_j^1}{p_j^0}\Big)^{\beta_{jh}},$$

the measures of the group's expenditure function with subsistence quantities $\lambda_i \gamma_{jh}$; a household with its group's change in consumption has $\lambda_i$ times its group's $EV_h$ and $CV_h$. With `budget()`, a household with budget items has Cobb–Douglas preferences in its own shares $s_{ik}$: with $G_i = \prod_k (p^1_{j(k)} / p^0_{j(k)})^{s_{ik}}$, $EV_i = C^1_i / G_i - C^0_i$ and $CV_i = C^1_i - C^0_i\, G_i$. Since $G_i \leq P_i$, $EV_i \geq R_i - C^0_i$. The tables give $\sum_i w_i EV_i / \sum_i v_i$ per person and $100 \sum_i w_i EV_i / \sum_i w_i C^0_i$.

**Decomposition (from version 0.8.1).** Consumption per person in the simulation is $r^1_i = r^0_i\, f_i / P_i$, where the nominal factor $f_i$ is the product of the changes in the household's income components and in its group's direct tax and savings rates. For a subset $S$ of the $K$ channels, $r^S_i$ takes the channels in $S$ at their simulated values and the others at their base values; $r^{\varnothing} = r^0$ and $r^{\{1,\ldots,K\}} = r^1$. The contribution of channel $j$ to a measure $I$ is its Shapley value (Shorrocks, 2013),

$$\phi_j = \sum_{S \not\ni j} \frac{|S|!\,(K - |S| - 1)!}{K!} \left[ I(r^{S \cup \{j\}}) - I(r^S) \right], \qquad \sum_j \phi_j = I(r^1) - I(r^0),$$

the change that channel $j$ makes, averaged over the orders in which the channels change. A channel that changes nothing contributes nothing. With $P(\mu, L)$ a poverty measure of the distribution with mean $\mu$ and Lorenz curve $L$, the growth and redistribution components (Datt and Ravallion, 1992), in Shapley form, are

$$G = \tfrac{1}{2} \left[ P(\mu^1, L^0) - P(\mu^0, L^0) + P(\mu^1, L^1) - P(\mu^0, L^1) \right],$$
$$R = \tfrac{1}{2} \left[ P(\mu^0, L^1) - P(\mu^0, L^0) + P(\mu^1, L^1) - P(\mu^1, L^0) \right],$$

with $G + R = P(\mu^1, L^1) - P(\mu^0, L^0)$, computed from the distributions $r^0 \mu^1 / \mu^0$ and $r^1 \mu^0 / \mu^1$.

**Fiscal incidence (from version 0.8.2).** Following the Commitment to Equity method (Lustig, 2018), with market income $y^m_i$, direct taxes $D_i$, direct transfers $B_i$, indirect taxes $I_i$, indirect subsidies $S_i$ and in-kind benefits $K_i$, the income concepts are

$$y^n = y^m - D, \quad y^d = y^n + B, \quad y^c = y^d - I + S, \quad y^f = y^c + K,$$

net market, disposable, consumable and final income, measured per person over persons. Persons are ranked by market income per person; with $F_i$ the mid-point of person $i$'s cumulative share of the weights in that order (tied values sharing their mid-point), the concentration coefficient of $t$ is

$$C_t = \frac{2 \sum_i v_i\, t_i\, (F_i - 1/2)}{\sum_i v_i\, t_i},$$

the Gini coefficient when $t$ is market income itself. The Kakwani index (Kakwani, 1977) is $C_T - G_m$ for a tax and $G_m - C_T$ for a benefit; the redistributive effect is $G_m - G_f$, the Reynolds–Smolensky index (Reynolds and Smolensky, 1977) $G_m - C_f$, and reranking $G_f - C_f$. The marginal contribution of an instrument to a measure is the measure of final income without it less the measure with it. Fiscal impoverishment (Higgins and Lustig, 2016) is the share of persons with $y^c_i < y^m_i$ who are poor in consumable income. In the simulation, market income changes as the household's income components, or as its group's income $YI_h$; direct taxes as market income times the group's direct tax rate $TINS_h$; transfers as the group's transfers from the government; indirect taxes as nominal spending times the household's tax wedge, $\sum_j s_{ij} w_j$ with $w_j = 1 - (1 - tq_j\, TQADJ)/(1 + tv_j\, TVADJ)$ the share of sales tax and VAT in the price of commodity $j$, at the shares of the household's budget at the new prices; subsidies as nominal spending; and in-kind benefits as the model's spending on their target, $PQ_c\, QG_c$ for a commodity or the health insurance fund's purchases $\sum_c PQ_c\, QSHI_c$. All are then deflated by the household's price index.

**Reweighting (from version 0.8.3).** With the household characteristics $x_{ik} \geq 0$ and targets $T_k = \rho_k \sum_i w_i x_{ik}$, where $\rho_k$ is the model's ratio of employment, $\sum_a QF_{fa} / \sum_a QF^0_{fa}$ for factor $f$ or $QF_{fa}/QF^0_{fa}$ for factor $f$ in activity $a$, or a ratio given, the weights of the simulation solve

$$\min_{w^1} \sum_i w^1_i \ln \frac{w^1_i}{w_i} \quad \text{subject to} \quad \sum_i w^1_i x_{ik} = T_k,$$

so that $w^1_i = w_i \exp(x_i' \lambda)$, with $\lambda$ found by Newton's method until the largest relative deviation from the targets is below $10^{-12}$ (Wittenberg, 2010). This is the reweighting step of the macro-micro approach of Corral et al. (2022), with the targets taken from the same model as the price and income changes.

**Standard errors (from version 0.8.3).** A measure $\theta = \sum_i w_i d_i y_i / \sum_i w_i d_i n_i$, over the households of a domain ($d_i = 1$), with $y_i = n_i r_i$ for the mean and $y_i = n_i \mathbf{1}[r_i < z_i]((z_i - r_i)/z_i)^\alpha$ for FGT$_\alpha$, has the linearized values $u_i = w_i d_i (y_i - \theta n_i) / \sum_j w_j d_j n_j$ and the variance

$$\widehat{V}(\theta) = \sum_h \frac{n_h}{n_h - 1} \sum_{j \in h} (u_{hj} - \bar{u}_h)^2,$$

where $u_{hj}$ is the sum of $u_i$ over cluster $j$ of stratum $h$, $n_h$ the number of clusters in the stratum, and $\bar{u}_h$ their mean: the variance of `svy: ratio`. The change uses $u_i^1 - u_i^0$, with the simulation's weights in $u^1$. A stratum with a single cluster makes the variance missing.

**Bands (from version 0.8.3).** For each draw $d$ kept by **[CGE]** `cge_sensitivity`, every measure for all households is computed as for a simulation; the bands are the 5th, 50th and 95th percentiles of the draws, by the definition of `_pctile`, and their standard deviation.

Chapter 20 describes the link in full.

## References

Bourguignon, F., M. Bussolo and L. A. Pereira da Silva, eds. 2008. *The Impact of Macroeconomic Policies on Poverty and Income Distribution: Macro-Micro Evaluation Techniques and Tools*. World Bank and Palgrave Macmillan.

Datt, G., and M. Ravallion. 1992. Growth and redistribution components of changes in poverty measures: A decomposition with applications to Brazil and India in the 1980s. *Journal of Development Economics* 38: 275–295.

Corral, P., E. Muñoz and I. Osorio-Rodarte. 2022. Modeling the distributional impact of macroeconomic shocks: `ms_reweight`. Presentation at the Stata Conference, Washington, DC.

Higgins, S., and N. Lustig. 2016. Can a poverty-reducing and progressive tax and transfer system hurt the poor? *Journal of Development Economics* 122: 63–75.

Kakwani, N. C. 1977. Measurement of tax progressivity: An international comparison. *Economic Journal* 87: 71–80.

Lustig, N., ed. 2018. *Commitment to Equity Handbook: Estimating the Impact of Fiscal Policy on Inequality and Poverty*. Washington, DC: Brookings Institution Press and CEQ Institute, Tulane University.

Ravallion, M., and S. Chen. 2003. Measuring pro-poor growth. *Economics Letters* 78: 93–99.

Reynolds, M., and E. Smolensky. 1977. *Public Expenditures, Taxes, and the Distribution of Income: The United States, 1950, 1961, 1970*. New York: Academic Press.

Wittenberg, M. 2010. An introduction to maximum entropy and minimum cross-entropy estimation using Stata. *Stata Journal* 10: 315–330.

Shorrocks, A. F. 2013. Decomposition procedures for distributional analysis: A unified framework based on the Shapley value. *Journal of Economic Inequality* 11: 99–126.

## Also see

**[CGE]** `cge_solve` — Solve the model
**[CGE]** `cge_report` — Standard report tables for one or more simulations
**[CGE]** `cge_dynamic` — Solve a recursive dynamic run
**[CGE]** `cge_gams` — Write the model as a GAMS program, and solve it in GAMS
**Chapter 20** — The link to household surveys

# cge_multiplier — SAM multipliers and their decomposition

## Description

`cge_multiplier` computes the accounting multipliers of the SAM mapped by **[CGE]** `cge_map` (Pyatt and Round 1979): the effects of an injection into one account on the incomes of all the endogenous accounts, the accounts whose spending the SAM explains, with prices fixed. For each injected account it displays the total multiplier and its parts in gross output, value added and household income. `decompose` splits the multipliers into transfer, open-loop and closed-loop effects, and `inject()` gives the effects of a vector of injections.

The multipliers use the SAM after the layout rewrites of **[CGE]** `cge_map` and need no calibration. The model in memory is not changed.

## Quick start

Multipliers of every endogenous account

```
cge_multiplier
```

As above, with the decomposition into transfer, open-loop and closed-loop effects

```
cge_multiplier, decompose
```

Effects of 10 more spending on health services

```
cge_multiplier, inject(com_hlt:10)
```

Multipliers with the government endogenous, loaded as data

```
cge_multiplier, endogenous(gov) clear
```

## Syntax

`cge_multiplier` [, *options*]

| *options* | Description |
|---|---|
| *Accounts* | |
| `endogenous(`*accounts*`)` | make exogenous accounts endogenous, in the institutions block |
| `exogenous(`*accounts*`)` | make endogenous accounts exogenous |
| *Results* | |
| `decompose` | add the decomposition into transfer, open-loop and closed-loop effects |
| `inject(`*account* :*value* …`)` | display the effects of injections into endogenous accounts |
| `accounts(`*accounts*`)` | injected accounts to display |
| `clear` | replace the data in memory with the multipliers or the effects |

## Options

`endogenous(`*accounts*`)` adds accounts that are exogenous by default, for example the government, to the endogenous accounts, in the institutions block.

`exogenous(`*accounts*`)` makes accounts that are endogenous by default exogenous, for example an enterprise.

`decompose` adds the decomposition of each multiplier into transfer, open-loop and closed-loop effects. It requires the SAM to have no flows among the blocks other than those of the circular flow; the command lists the flows that break the structure if it does. With `inject()`, `decompose` is ignored, and the effects of the injection are shown without it.

`inject(`*account*`:`*value* ...`)` gives injections into endogenous accounts, separated by spaces, for example `inject(com_hlt:100)` for 100 more spending on health services by an exogenous account such as the government. The command then displays, instead of the multipliers, the change in each endogenous account's income, its income in the SAM and the percentage change, and the totals.

`accounts(`*accounts*`)` chooses the injected accounts whose multipliers are displayed. The default is every endogenous account, or the commodities and households if there are more than 40.

`clear` replaces the data in memory with the multiplier matrix, one observation per pair of endogenous accounts, with the variables `row`, `column`, `row_block`, `column_block` and `multiplier`, and, with `decompose`, `transfer`, `open_loop` and `closed_loop`. With `inject()`, it loads instead the effects of the injection, one observation per endogenous account, with the variables `account`, `block`, `base`, `change` and `pct`.

## Remarks and examples

The accounts of the SAM are divided into endogenous accounts, whose incomes are determined by the spending of the SAM's accounts, and exogenous accounts, whose spending injects income into them. By default the endogenous accounts form three blocks:

| Block | Accounts |
|---|---|
| production | activities, commodities and margin accounts |
| factors | factors |
| institutions | households and enterprises, and the accounts in `endogenous()` |

The government, savings–investment, the rest of the world, the tax accounts and the health insurance fund are exogenous. Each endogenous account spends its income in the shares of its column of the SAM; spending on exogenous accounts (taxes, savings and imports) leaks out of the circular flow.

Accounting multipliers assume fixed prices, unused capacity in every sector, and average propensities to spend equal to marginal ones. They are upper bounds on what a CGE model with supply constraints and price changes would find, and complement it.

### Example 1: the multipliers

The health layout of the synthetic SAM is mapped:

```
. cge_synthetic, clear health
(19 vars, 18 obs)

Synthetic SAM loaded (health layout): 18 accounts, total   1746.00
Largest row-column imbalance:  0.00e+00

. cge_sam, labels(account)
SAM read: 18 accounts, total 1746
Largest row-column imbalance:  0.00e+00

. cge_map, activities(act_agr act_man act_ser act_hlt)                    ///
>     commodities(com_agr com_man com_ser com_hlt) factors(lab cap)       ///
>     households(hhd) government(gov) savinv(sav) row(row) dirtax(tax_dir) ///
>     saltax(tax_sal) imptax(tax_imp) insurance(shi)
Accounts mapped: 4 activities, 4 commodities, 2 factors, 1 households, 0 enterprises, 0 margin accou
> nts
Health insurance fund: shi
```

The multipliers have one row per injected account:

```
. cge_multiplier

Accounting multipliers: effects of a unit injection into each account (fixed prices)
Injection     Block            Total    Output Value add. Hh income
-------------------------------------------------------------------
act_agr       production       8.520     2.622     1.873     1.831
act_man       production       8.372     2.708     1.689     1.652
act_ser       production       8.538     2.657     1.842     1.801
act_hlt       production       8.408     2.581     1.864     1.813
com_agr       production       7.554     2.017     1.441     1.408
com_man       production       6.377     1.739     1.085     1.061
com_ser       production       7.830     2.126     1.473     1.441
com_hlt       production       9.408     2.581     1.864     1.813
lab           factors          7.470     1.492     2.015     1.947
cap           factors          7.771     1.562     2.062     2.038
hhd           institutions     6.771     1.562     1.062     2.038
-------------------------------------------------------------------
Total: all endogenous accounts; output: activities; value added: factors; household income: households
```

A unit injection into health services, `com_hlt`, raises the incomes of all endogenous accounts by 9.408 in total, including the injection itself: gross output by 2.581, value added by 1.864 and household income by 1.813. Health services are neither imported nor taxed, so the whole of the injection passes to the health activity, and the multipliers of `com_hlt` are those of `act_hlt`, plus the injection itself in the total.

### Example 2: the decomposition

With `decompose`, each total multiplier is split into 1, the injection itself, and three effects:

```
. cge_multiplier, decompose

Accounting multipliers: effects of a unit injection into each account (fixed prices)
Injection     Block            Total    Output Value add. Hh income  Transfer Open loop    Closed
-------------------------------------------------------------------------------------------------
act_agr       production       8.520     2.622     1.873     1.831     0.518     1.818     5.184
act_man       production       8.372     2.708     1.689     1.652     1.056     1.639     4.678
act_ser       production       8.538     2.657     1.842     1.801     0.651     1.787     5.100
act_hlt       production       8.408     2.581     1.864     1.813     0.464     1.809     5.135
com_agr       production       7.554     2.017     1.441     1.408     1.168     1.398     3.988
com_man       production       6.377     1.739     1.085     1.061     1.320     1.052     3.004
com_ser       production       7.830     2.126     1.473     1.441     1.321     1.430     4.080
com_hlt       production       9.408     2.581     1.864     1.813     1.464     1.809     5.135
lab           factors          7.470     1.492     2.015     1.947     0.000     2.677     3.793
cap           factors          7.771     1.562     2.062     2.038     0.000     2.801     3.970
hhd           institutions     6.771     1.562     1.062     2.038     0.000     2.323     3.448
-------------------------------------------------------------------------------------------------
Total: all endogenous accounts; output: activities; value added: factors; household income: households
Total = 1 + transfer + open loop + closed loop (Pyatt and Round 1979)
```

The transfer effects are those within the injected account's block, such as the intermediate purchases among activities and commodities; the open-loop effects are those on the other blocks, before any income returns to the injected block; and the closed-loop effects are those of the circular flow production → factors → institutions → production back to it. The factors and the household have no transfer effects: no factor pays another, and the household pays no other institution.

### Example 3: an injection

An injection of 10 into health services, such as more spending on them by the government or another exogenous account:

```
. cge_multiplier, inject(com_hlt:10)

Effects of the injection (fixed prices)
Account       Block         Base income       Change  % change
---------------------------------------------------------------
act_agr       production            100        4.593     4.593
act_man       production            140        5.524     3.946
act_ser       production            120         4.92     4.100
act_hlt       production             30        10.77    35.915
com_agr       production            130        5.971     4.593
com_man       production            218        8.602     3.946
com_ser       production            150         6.15     4.100
com_hlt       production             30        10.77    35.915
```

```
lab          factors                135         11.4     8.448
cap          factors                127        7.237     5.698
hhd          institutions           281        18.13     6.454
---------------------------------------------------------------
Injection 10; gross output 25.8113343; value added 18.6413813; household income 18.1345
```

Gross output rises by 25.8 and household income by 18.1. The base income of an account is its column total in the SAM.

### Example 4: other endogenous accounts

`endogenous()` and `exogenous()` change the accounts whose incomes the multipliers determine. With the government endogenous:

```
. cge_multiplier, endogenous(gov) accounts(com_hlt hhd gov)

Accounting multipliers: effects of a unit injection into each account (fixed prices)
Injection     Block            Total    Output Value add. Hh income
------------------------------------------------------------------
com_hlt       production        9.408     2.581     1.864     1.813
hhd           institutions      6.771     1.562     1.062     2.038
gov           institutions      8.340     1.932     1.336     1.543
------------------------------------------------------------------
Total: all endogenous accounts; output: activities; value added: factors; household income: households
3 of 12 injections shown; accounts() chooses them and clear loads the matrix
```

The multipliers of `com_hlt` and `hhd` do not change: the government's income comes only from the tax accounts, which remain exogenous, so no injection elsewhere reaches it. Making the government endogenous adds the effects of an injection into its income, through its purchases and its transfers to the household.

Accounts added to the institutions block may break the circular structure that the decomposition requires. A sales tax account paid by the commodities is a flow from production to institutions:

```
. cge_multiplier, endogenous(gov tax_sal) decompose
decompose: the SAM has flows outside the circular structure production -> factors -> institutions ->
>  production:
  com_agr (production) pays tax_sal (institutions): 8
  com_man (production) pays tax_sal (institutions): 12
  com_ser (production) pays tax_sal (institutions): 10

r(459);
```

### Example 5: the multipliers as data

With `clear`, the matrix of multipliers replaces the data in memory. The column of `com_hlt` gives the effects of a unit injection into it on each account:

```
. quietly cge_multiplier, decompose clear

. list row multiplier transfer open_loop closed_loop if column == "com_hlt", noobs sep(0)

  +----------------------------------------------------------+
  |     row   multipl~r    transfer   open_loop   closed_~p |
  |----------------------------------------------------------|
  | act_agr     .459302   .01499124           0   .44431076 |
  | act_man   .55239482   .10506608           0   .44732873 |
  | act_ser    .4919939   .07152119           0   .42047271 |
  | act_hlt   1.0774427           1           0    .0774427 |
  | com_agr    .5970926   .01948861           0   .57760399 |
  | com_man   .86015764    .1636029           0   .69655474 |
  | com_ser   .61499238   .08940149           0   .52559089 |
  | com_hlt   1.0774427           0           0    .0774427 |
  |     lab    1.140483           0   .65729543   .48318756 |
  |     cap   .72365514           0   .26179008   .46186506 |
  |     hhd     1.81345           0   .88987238   .92357762 |
  +----------------------------------------------------------+
```

Off the diagonal, each multiplier is the sum of its transfer, open-loop and closed-loop parts; on the diagonal, the sum is one less than the multiplier.

## Stored results

`cge_multiplier` stores the following in `r()`:

*Scalars*

| | |
|---|---|
| `r(N_endogenous)` | number of endogenous accounts |
| `r(injection)` | total injection (with `inject()`) |
| `r(output)` | change in gross output (with `inject()`) |
| `r(value_added)` | change in value added (with `inject()`) |
| `r(household_income)` | change in household income (with `inject()`) |
| `r(total)` | change in the incomes of all endogenous accounts (with `inject()`) |

*Macros*

| | |
|---|---|
| `r(endogenous)` | endogenous accounts |

## Methods and formulas

Let $N$ be the set of endogenous accounts and $t_j$ the column total of account $j$ in the SAM after the layout rewrites. The average expenditure propensities among the endogenous accounts are

$$A_{ij} = \frac{SAM_{ij}}{t_j}, \qquad i, j \in N,$$

with $A_{ij} = 0$ when $t_j = 0$. The incomes $y$ of the endogenous accounts satisfy $y = Ay + x$, where $x$ is their income from the exogenous accounts, so that

$$y = Mx, \qquad M = (I - A)^{-1}.$$

Column $j$ of $M$ is the change in the income of every endogenous account for a unit injection into account $j$. $I - A$ is invertible when every chain of payments among the endogenous accounts eventually leaks to an exogenous account; if it is singular, `cge_multiplier` stops and asks for one of the accounts to be made exogenous. For injected account $j$ the command displays the sums of column $j$ of $M$ over all endogenous accounts (`Total`), over the activities (`Output`), over the factors (`Value add.`) and over the households (`Hh income`). With `inject()`, the changes in income are $\Delta y = Mx$ for the vector $x$ of injections, and the percentage change of account $i$ is $100\,\Delta y_i / |t_i|$.

*Decomposition.* With the blocks ordered as production (1), factors (2) and institutions (3), the circular flow is made of the payments of production to factors ($A_{21}$), of factors to institutions ($A_{32}$) and of institutions to production ($A_{13}$), with transfers within each block ($A_{11}$, $A_{22}$, $A_{33}$). The decomposition requires the other blocks, $A_{12}$, $A_{23}$ and $A_{31}$, to be zero. With $\tilde{A}$ the block-diagonal part of $A$ and $A^* = (I - \tilde{A})^{-1}(A - \tilde{A})$,

$$M = M_3 M_2 M_1, \qquad M_1 = (I - \tilde{A})^{-1}, \quad M_2 = I + A^* + A^{*2}, \quad M_3 = \left(I - A^{*3}\right)^{-1},$$

the transfer, open-loop and closed-loop multipliers of Pyatt and Round (1979), and additively

$$M = I + (M_1 - I) + (M_2 - I)M_1 + (M_3 - I)M_2 M_1.$$

The columns `Transfer`, `Open loop` and `Closed` are the column sums of the last three terms, so that the total multiplier is one plus their sum. Multipliers and their decomposition are described in section 24.6 and section 24.6.1.

## References

Pyatt, G., and J. I. Round. 1979. Accounting and fixed price multipliers in a social accounting matrix framework. *Economic Journal* 89: 850–873.

## Also see

**[CGE]** `cge_spa` — Structural path analysis of a SAM multiplier
**[CGE]** `cge_map` — Assign SAM accounts to model roles
**[CGE]** `cge_sam` — Read a social accounting matrix
**[CGE]** `cge` — Computable general equilibrium modelling in Stata
**Chapter 24** — SAM tools

# `cge_optimize` — Choose policy instruments to maximise an outcome or reach targets

## Description

`cge_optimize` chooses the values of a few policy instruments to maximise or minimise a report measure, or to reach targets for report measures, solving the model at every trial value. From version 0.9.4, the objective and the targets may also be expressions of the user language (**[CGE]** `cge_equation`), and inequality constraints may be added, as in a GAMS model solved by an NLP solver. An instrument is one element of a parameter that **[CGE]** `cge_shock` changes, such as a tariff rate, or one element of a variable that the closure fixes, such as a tax adjustment factor. The optimisation starts from the instruments' current values, with any shocks already applied, under the current closure. The optimum is stored as a simulation, and the instruments are left at their optimal values as current shocks, so that **[CGE]** `cge_report` and **[CGE]** `cge_results` show it.

## Quick start

The tariff on manufactures, between −0.5 and 1, that maximises the equivalent variation of household `hhd`

```
cge_optimize, instruments(tm(com_man)) maximize(welfare equivalent_variation hhd) ///
    lower(tm(com_man):-0.5) upper(tm(com_man):1)
```

Both tariffs that maximise it while tariff revenue stays at 8, storing the optimum as simulation `two`

```
cge_optimize, instruments(tm(com_agr) tm(com_man)) maximize(welfare equivalent_variation hhd) ///
    targets(government tax_import = 8) name(two)
```

The direct tax adjustment factor that keeps government savings at 3, with the iterations displayed

```
cge_optimize, instruments(TINSADJ) targets(government savings = 3) trace
```

Government purchases of health services and the direct tax factor that reach targets for real GDP and government savings

```
cge_optimize, instruments(qg(com_hlt) TINSADJ) targets(macro gdp_real = 310, government savings = 3)
```

The tariff that maximises government revenue while imports of manufactures stay within 2 per cent of the base, written as expressions (from version 0.9.4)

```
cge_optimize, instruments(tm(com_man)) maximize(YG) ///
    constraints(QM(com_man) >= 0.98 * base(QM(com_man))) lower(tm(com_man):0)
```

## Syntax

**cge_optimize**, <u>instr</u>uments(*list*) {<u>max</u>imize(*objective*) | <u>min</u>imize(*objective*) | <u>targ</u>ets(*spec*)} [*options*]

| *options* | Description |
|---|---|
| *Main* | |
| <u>instr</u>uments(*list*) | the instruments; required |
| <u>max</u>imize(*objective*) | report measure or expression to maximise |
| <u>min</u>imize(*objective*) | report measure or expression to minimise |
| <u>targ</u>ets(*spec*) | targets for report measures or expressions |
| <u>cons</u>traints(*spec*) | inequality constraints |

| *options* | Description |
|---|---|
| `lower(`*spec*`)` | lower bounds of instruments |
| `upper(`*spec*`)` | upper bounds of instruments |
| `name(`*name*`)` | name of the simulation that holds the optimum; default `optimum` |
| *Solver* | |
| `tolerance(#)` | tolerance of each solve of the model; default `1e-12` |
| `iterate(#)` | maximum number of Newton iterations of each solve; default `100` |
| `trace` | display the iterations of the optimiser |

A *measure* is a measure of **[CGE]** `cge_report`, written *table measure* [*element*], for example `welfare equivalent_variation hhd` or `macro gdp_real`. From version 0.9.4, an objective, a target or a constraint may instead be an expression of the user language (**[CGE]** `cge_equation`), such as `YG - EG` or `sum(c, PQ(c) * QG(c))`. An objective, targets or both must be given; `maximize()` and `minimize()` may not be combined.

## Options

`instruments(`*list*`)` lists the instruments, separated by spaces. An instrument is

- one element of a parameter that **[CGE]** `cge_shock` changes, written *parameter*`(`*element*`)`, such as `tm(com_man)` or `qg(com_hlt)`; the element may be left out when the parameter has only one;
- a scalar parameter, such as `trgr`;
- one element of a user parameter (**[CGE]** `cge_parameter`), such as `w(com_agr)` or `sh(com_hlt,hhd)`, or a scalar one; or
- one element of a variable that the closure fixes, such as `GADJ`, `TINSADJ` or `QFS(lab)`. A variable that the closure does not fix is an error; it can be fixed by a swap (**[CGE]** `cge_closure`, `swap()`).

An instrument may be given only once.

`maximize(`*objective*`)` gives the measure or expression to maximise.

`minimize(`*objective*`)` gives the measure or expression to minimise.

`targets(`*spec*`)` gives targets, separated by commas, as *table measure* [*element*] = *value*, for example `targets(government savings = 3, macro investment_real = 50)`. Without an objective there must be one target for each instrument. With an objective, the targets are equality constraints. From version 0.9.4, either side may be an expression, as in `targets(sum(c, PQ(c) * QG(c)) = 50)`.

`constraints(`*spec*`)` (from version 0.9.4) gives inequality constraints, separated by commas, as a measure or an expression `<=` or `>=` (or `=l=`, `=g=`) a value or an expression, for example `constraints(QM(com_man) >= 0.98 * base(QM(com_man)), government tax_import <= 9)`. Constraints need an objective. The display marks those that bind at the optimum.

`lower(`*spec*`)` and `upper(`*spec*`)` give bounds, as *instrument*`:`*value* pairs with the instrument written as in `instruments()`, for example `lower(tm(com_man):0) upper(tm(com_man):0.2)`. An instrument without a bound is unbounded on that side. A lower bound must be below the upper bound, and the current value of each instrument must lie strictly within its bounds.

`name(`*name*`)` names the simulation that holds the optimum; the default is `optimum`. A simulation of that name is replaced.

`tolerance(#)` sets the convergence tolerance of each solve of the model; the default is `1e-12`.

`iterate(#)` sets the maximum number of Newton iterations of each solve of the model; the default is 100.

`trace` displays the objective and the largest gradient of the free instruments at each iteration, or, with targets alone, the largest scaled gap from the targets.

## Remarks and examples

`cge_optimize` is meant for a few instruments: a tax rate, the volume of a public service, a transfer. At each trial value of the instruments the model is solved by Newton's method, from the solution of the trial before, and the measures are computed at the solution. The objective and the targets are values of the measures, such as amounts in the units of the SAM, not percentage changes from the base; the equivalent and compensating variations are amounts of base-price spending, not per cent of base consumption. Chapter 22 states the problem and the algorithm.

### Example 1: a welfare-maximising tariff

The synthetic SAM is calibrated as in **[CGE]** `cge_solve`. The base values of government revenue and its components, and a closure under which government savings are fixed and the direct tax rates adjust, are

```
. cge_solve, name(base)
Simulation base solved in 0 iteration(s) (newton)
Largest scaled residual  3.55e-16; WALRAS =  0.00e+00

. cge_report, sims(base) tables(government)

Government
-------------------------------------------------
                                  base       base
revenue                             63      0.000
spending                            60      0.000
savings                              3      0.000
tax_direct                          25      0.000
tax_sales                           30      0.000
tax_import                           8      0.000
tax_export                           0          .
tax_activity                         0          .
tax_vat                              0          .
tax_factor_use                       0          .
tax_factor_income                    0          .
-------------------------------------------------
Percentage changes from the base (rates: changes in points)

. cge_closure, government(fixgsav)
Closure
  external account     flexexr
  government           fixgsav
  savings-investment   savings
  numeraire            cpi
  factor lab           mobile
  factor cap           mobile
```

The tariff on manufactures that maximises the equivalent variation of the household, within $[-0.5, 1]$, with the iterations displayed:

```
. cge_optimize, instruments(tm(com_man)) maximize(welfare equivalent_variation hhd) ///
>     lower(tm(com_man):-0.5) upper(tm(com_man):1) trace
Iteration 0: objective             0, largest free gradient  8.38e-01
Iteration 1: objective    .7280912943, largest free gradient  1.63e-01
Iteration 2: objective    .7677599778, largest free gradient  1.04e-02
Iteration 3: objective    .7679361175, largest free gradient  4.96e-05
Iteration 4: objective    .7679361216, largest free gradient  1.28e-08
Iteration 5: objective    .7679361216, largest free gradient  7.11e-09

Optimisation optimum: converged after 32 solve(s) of the model
----------------------------------------------------------------
Instrument                                start          optimum
----------------------------------------------------------------
tm(com_man)                                  .1      .3032241215
----------------------------------------------------------------
Maximised welfare equivalent_variation hhd:      .7679361216
```

The objective is the equivalent variation in the units of the SAM; it is zero at the start, the base. The gradient falls to $7.11 \times 10^{-9}$ in five Newton iterations, and the 32 solves of the model include those of the finite differences. On the synthetic SAM the optimal tariff is about 30 per cent, above the base rate of

10 per cent. The result depends on the SAM and the elasticities, which here are synthetic and illustrative, and is not a finding about any economy.

The optimum is stored as simulation `optimum` and left as the current shock:

```
. display r(converged), r(N_evaluations), r(iterations), r(objective)
1 32 5 .76793612

. matrix list r(instruments)

symmetric r(instruments)[1,1]
    tm_com_man
r1   .30322412

. cge_report, sims(optimum) tables(government welfare)

Government
-------------------------------------------------
                                  base    optimum
revenue                             63     -1.145
spending                            60     -1.202
savings                              3      0.000
tax_direct                          25    -37.901
tax_sales                           30      0.193
tax_import                           8    108.697
tax_export                           0          .
tax_activity                         0          .
tax_vat                              0          .
tax_factor_use                       0          .
tax_factor_income                    0          .
-------------------------------------------------

Welfare (per cent of base consumption)
---------------------------------------------------
                                            optimum
equivalent_variation hhd                      0.334
compensating_variation hhd                    0.334
---------------------------------------------------
Percentage changes from the base (rates: changes in points)
```

The equivalent variation of 0.768 is 0.334 per cent of base consumption, as **[CGE]** `cge_report` shows it. Government savings are fixed by the closure: tariff revenue more than doubles and direct taxes fall by 38 per cent.

### Example 2: an objective with a target

Both tariffs are chosen to maximise welfare while tariff revenue stays at its base value of 8. The target is an equality constraint, met by the augmented Lagrangian method:

```
. cge_shock, reset
All shocks removed

. cge_optimize, instruments(tm(com_agr) tm(com_man)) maximize(welfare equivalent_variation hhd) ///
>     targets(government tax_import = 8) name(two_tariffs)

Optimisation two_tariffs: converged after 419 solve(s) of the model
----------------------------------------------------------------
Instrument                               start         optimum
----------------------------------------------------------------
tm(com_agr)                                 .1     -.0335061325
tm(com_man)                                 .1      .1577235277
----------------------------------------------------------------
Maximised welfare equivalent_variation hhd:      .3775370166
Target government tax_import = 8:               8
```

The optimum subsidises imports of agriculture slightly and taxes those of manufactures at 16 per cent. Each round of the augmented Lagrangian method maximises again, so the optimisation took 419 solves of the model.

#### Example 3: targets alone

With as many targets as instruments and no objective, the instruments are found by Newton's method on the targets. After the removal of every tariff, under a closure in which government savings adjust, the direct tax adjustment factor that keeps government savings at their base value of 3, found in two Newton iterations, is

```
. cge_shock, reset
All shocks removed

. cge_closure, government(flexgsav)
Closure
  external account     flexexr
  government           flexgsav
  savings-investment   savings
  numeraire            cpi
  factor lab           mobile
  factor cap           mobile

. cge_shock tm, value(0)
Parameter tm changed for 3 element(s)

. cge_optimize, instruments(TINSADJ) targets(government savings = 3) name(tins) trace
Newton iteration 0: largest scaled target gap  2.71e+00
Newton iteration 1: largest scaled target gap  6.57e-05
Newton iteration 2: largest scaled target gap  1.27e-14

Optimisation tins: converged after 8 solve(s) of the model
----------------------------------------------------------------
Instrument                                start          optimum
----------------------------------------------------------------
TINSADJ                                       1      1.314441344
----------------------------------------------------------------
Target government savings = 3:                3
```

Holding government savings fixed with the direct tax rates adjusting is what the closure `fixgsav` does. Solving the same shock under that closure gives the same factor:

```
. cge_shock, reset
All shocks removed

. cge_closure, government(fixgsav)
Closure
  external account     flexexr
  government           fixgsav
  savings-investment   savings
  numeraire            cpi
  factor lab           mobile
  factor cap           mobile

. cge_shock tm, value(0)
Parameter tm changed for 3 element(s)

. cge_solve, name(fixgsav)
Simulation fixgsav solved in 7 iteration(s) (newton)
Largest scaled residual  7.70e-14; WALRAS = -3.37e-13

. cge_results, sim(fixgsav) clear nodisplay

. list variable value if variable == "TINSADJ", noobs

  +----------------------+
  | variable       value |
  |----------------------|
  |  TINSADJ   1.3144413 |
  +----------------------+
```

### Example 4: expressions and constraints

From version 0.9.4, the objective, targets and constraints may be written in the user language. The tariff on manufactures that maximises government revenue, `YG`, while imports of manufactures fall by no more than 2 per cent:

```
. cge_shock, reset
All shocks removed

. cge_closure, government(flexgsav)
Closure
  external account     flexexr
  government           flexgsav
  savings-investment   savings
  numeraire            cpi
  factor lab           mobile
  factor cap           mobile

. cge_optimize, instruments(tm(com_man)) maximize(YG)        ///
>     constraints(QM(com_man) >= 0.98 * base(QM(com_man)))   ///
>     lower(tm(com_man):0) upper(tm(com_man):2) name(revenue)

Optimisation revenue: converged after 234 solve(s) of the model
----------------------------------------------------------------
Instrument                                start          optimum
----------------------------------------------------------------
tm(com_man)                                  .1      .1296864671
----------------------------------------------------------------
Maximised YG:      64.46989099
Constraint QM(com_man) >= 0.98 * base(QM(com_man)):             58.8 (binding)

. display r(N_binding)
1
```

The constraint binds: the tariff is the one at which imports are 2 per cent below the base, the value a target `QM(com_man) = 0.98 * base(QM(com_man))` would give. Without it, the revenue-maximising tariff is far higher.

### Bounds, failures and local optima

An instrument at a bound whose gradient points outwards is held there, and the display marks it `(at its bound)`. A trial value at which the model does not solve counts in `r(N_failed)`; the next trial starts from the last solution, and the optimiser steps back. The model must solve at the starting values. Derivatives are found by central differences of the solved model, and a smooth objective is flat at its optimum, so the optimum is located to about $10^{-6}$ relative to the instruments' scale. An optimum may be local: other starting values, set with **[CGE]** `cge_shock`, can be tried.

## Stored results

`cge_optimize` stores the following in `r()`:

*Scalars*

| | |
|---|---|
| `r(converged)` | 1 if the optimisation converged, 0 otherwise |
| `r(objective)` | the objective at the optimum, with an objective |
| `r(N_evaluations)` | number of solves of the model |
| `r(N_failed)` | number of trial values at which the model did not solve |
| `r(iterations)` | number of iterations of the optimiser |
| `r(N_binding)` | number of constraints that bind at the optimum, with constraints |

*Macros*

| | |
|---|---|
| `r(name)` | name of the simulation that holds the optimum |
| `r(instruments_list)` | the instruments |

*Matrices*

| | |
|---|---|
| `r(instruments)` | the instruments at the optimum |
| `r(start)` | their starting values |
| `r(targets)` | the target measures or expressions at the optimum, with targets |

| | |
|---|---|
| `r(constraints)` | the constrained measures or expressions at the optimum, with constraints |

## Methods and formulas

With instruments $u$, the model's solution $x(u)$ under the current closure, an objective measure $f$ and target measures $g_j$,

$$\max_u \; f\big(x(u), u\big) \quad \text{subject to} \quad g_j\big(x(u), u\big) = \bar{g}_j, \qquad \underline{u} \le u \le \overline{u},$$

with $-f$ for `minimize()`. The instruments are scaled as $u = u^0 + s \odot z$, with $u^0$ the starting values and $s_k = |u^0_k|$, or 1 when $|u^0_k| \le 10^{-12}$. The target gaps are scaled as $r_j = (g_j - \bar{g}_j)/\tau_j$, with $\tau_j = |\bar{g}_j|$, or 1 when $|\bar{g}_j| \le 10^{-12}$.

*Targets alone.* Newton's method solves $r(z) = 0$. The Jacobian comes from central differences with steps $h_k = 10^{-5}\max(1, |z_k|)$; the step $\Delta z = -J^{-1}r$ is projected on the bounds and halved, up to 30 times, until $\max_j |r_j|$ falls. The iterations stop, converged, when $\max_j |r_j| < 10^{-11}$; they stop without converging when a trial value does not solve, when 30 halvings do not reduce the gap, and after 50 iterations.

*An objective.* A projected Newton method maximises

$$F(z) = \frac{f}{\max(1, |f^0|)} - \lambda' r - \frac{\rho}{2} r' r,$$

with $f^0$ the objective at the start; without targets, $F$ is the scaled objective alone. The gradient comes from central differences with step $10^{-5}$ (one-sided at a bound); an instrument at a bound whose gradient points outwards is held there. The iterations converge when the largest gradient of the free instruments is below $10^{-8}$. The Hessian of the free instruments comes from central second differences with step $10^{-3}$. The step is $-H^{-1}\nabla F$ when $H$ is negative definite, and the gradient, divided by $\max(1, \max_k |\partial F/\partial z_k|)$, otherwise; it is projected on the bounds and halved, up to 40 times, until $F$ rises. If no halving raises $F$, the iterations stop, and they are counted as converged if the largest free gradient is below $10^{-6}$; they also stop, converged, when a step changes $z$ by less than $10^{-12}$, and after at most 100 iterations.

*Targets with an objective.* By the augmented Lagrangian method, with $\lambda = 0$ and $\rho = 100$ at the start, each round maximises $F$ and then sets $\lambda \leftarrow \lambda + \rho\, r$; $\rho$ is multiplied by 10 when $\max_j |r_j|$ has fallen by less than three quarters since the round before. The rounds stop, converged, when $\max_j |r_j| < 10^{-10}$, and after at most 40 rounds.

*Constraints* (from version 0.9.4). A constraint $h_k \le \bar{h}_k$ or $h_k \ge \bar{h}_k$, with $\bar{h}_k$ a number or an expression, has the scaled slack $s_k = (\bar{h}_k - h_k)/\sigma_k$ or $(h_k - \bar{h}_k)/\sigma_k$, which is not negative where it holds, with $\sigma_k$ the scale of $\bar{h}_k$ at the base (1 when it is near 0). By the method of Powell, Hestenes and Rockafellar the maximiser takes

$$F(z) - \frac{1}{2\rho} \sum_k \Big( \max(0, \mu_k - \rho\, s_k)^2 - \mu_k^2 \Big),$$

and after each round $\mu_k \leftarrow \max(0, \mu_k - \rho\, s_k)$, starting from $\mu = 0$, with $\rho$ raised as for the targets. The rounds stop, converged, when the gaps in the targets and $\max_k |\min(s_k, \mu_k/\rho)|$, which measures both the violation of the constraints and of complementarity, are below $10^{-10}$. A constraint binds when $|h_k - \bar{h}_k| \le 10^{-6}\sigma_k$. An expression, as an objective, a target or a constraint, is compiled as a user equation is (chapter 23) and evaluated at the solution of each trial.

Chapter 22 gives the details.

## Also see

**[CGE]** `cge_shock` — Change a parameter or a fixed variable
**[CGE]** `cge_closure` — Choose the model closure
**[CGE]** `cge_solve` — Solve the model
**[CGE]** `cge_report` — Standard report tables for one or more simulations
**Chapter 22** — Policy optimisation

# `cge_parameter` — User parameters for user variables and equations

## Description

`cge_parameter` adds a parameter whose values the user gives, with no index, one or two, over model sets or user sets (**[CGE]** `cge_set`). User variables (**[CGE]** `cge_variable`) and equations (**[CGE]** `cge_equation`) refer to it by name. The values given are its base values; **[CGE]** `cge_shock` changes them in a scenario, **[CGE]** `cge_optimize` may take one of its elements as an instrument, continuation and dynamic runs carry it like the model's own parameters, and **[CGE]** `cge_gams` writes it into the GAMS program.

Do not confuse `cge_parameter`, which defines a user parameter, with **[CGE]** `cge_parameters`, which lists every parameter of the model, user parameters included, as a dataset.

## Quick start

A scalar target, 1.1

```
cge_parameter hgoal = 1.1
```

Weights by commodity, the elements not given 0

```
cge_parameter w(c), values(com_agr:0.4 com_man:0.3 com_ser:0.2 com_hlt:0.1)
```

Shares by commodity and household, from the data in memory

```
cge_parameter sh(c,h), from(share) labels(commodity household)
```

A scenario with a higher target

```
cge_shock hgoal, value(1.2)
```

## Syntax

A parameter with one value for all its elements

**`cge_parameter`** *name*[`(`*i*[`,`*j*]`)`] `= #`

Values by element

**`cge_parameter`** *name*`(`*i*[`,`*j*]`)` `,` `values(`*spec*`)` [`default(#)`]

Values from the data in memory

**`cge_parameter`** *name*`(`*i*[`,`*j*]`)` `,` `from(`*varname*`)` `labels(`*varlist*`)` [`default(#)`]

Display or remove the user parameters

**`cge_parameter`** `, list`
**`cge_parameter`** `, clear`

*i* and *j* are indexes of model sets (`a c f h d i n`) or of user sets. *name* follows the rules of **[CGE]** `cge_set`.

| *options* | Description |
|---|---|
| *Main* | |

| *options* | Description |
|---|---|
| `values(`*spec*`)` | values by element, *element*`:#` or *e1*`.`*e2*`:#` |
| `from(`*varname*`)` | a variable of the data in memory with the values |
| `labels(`*varlist*`)` | the string variables with the element labels, one for each index |
| `default(#)` | value of the elements not given; default `0` |

## Options

`values(`*spec*`)` gives values by element, as *element*`:#` pairs separated by spaces, for example `values(com_agr:0.3 com_man:0.5)`; for a parameter with two indexes, *e1*`.`*e2*`:#`.

`from(`*varname*`)` takes the values from the numeric variable *varname* of the data in memory, one observation per element. Observations with a missing value or an empty label are skipped.

`labels(`*varlist*`)` names the string variables that hold the labels of the elements, one for each index, with `from()`.

`default(#)` gives the value of the elements that `values()` or `from()` leaves out; the default is 0.

`list` displays the user parameters, with the range of their current and base values.

`clear` removes every user parameter. A parameter that a user variable or equation uses cannot be removed until they are.

## Remarks and examples

The SAM with a health sector is mapped and calibrated:

```
. cge_synthetic, clear health
(19 vars, 18 obs)

Synthetic SAM loaded (health layout): 18 accounts, total   1746.00
Largest row-column imbalance:  0.00e+00

. cge_sam, labels(account)
SAM read: 18 accounts, total 1746
Largest row-column imbalance:  0.00e+00

. cge_map, activities(act_agr act_man act_ser act_hlt)                    ///
>     commodities(com_agr com_man com_ser com_hlt) factors(lab cap)          ///
>     households(hhd) government(gov) savinv(sav) row(row) dirtax(tax_dir)   ///
>     saltax(tax_sal) imptax(tax_imp) insurance(shi)
Accounts mapped: 4 activities, 4 commodities, 2 factors, 1 households, 0 enterprises, 0 margin accou
> nts
Health insurance fund: shi

. cge_calibrate, sigmava(0.8) sigmaq(2) sigmat(2)
Calibrated: 117 equations, 117 free variables
Benchmark replication: largest scaled residual  3.55e-16
```

### Example 1: a target set by a parameter

A target for government purchases of health services, 1.1 times the base, which a user equation enforces with the scaling factor of government consumption adjusting:

```
. cge_parameter hgoal = 1.1
User parameter hgoal: 1 value(s), from 1.1 to 1.1

. cge_variable LAM, value(1)
User variable LAM: 1 element(s), base value 1
Add the equations that determine it with cge_equation

. cge_equation lam: QG(com_hlt) = LAM * hgoal * base(QG(com_hlt))
User equation lam: 1 element(s)
Note: the base does not satisfy it (largest relative gap  9.09e-02); it acts as a shock
```

```
. cge_closure, swap(LAM GADJ)
Closure
  external account     flexexr
  government           flexgsav
  savings-investment   savings
  numeraire            cpi
  factor lab           mobile
  factor cap           mobile
  swap                 LAM fixed, GADJ free

. cge_solve, name(goal)
Simulation goal solved in 5 iteration(s) (newton)
Largest scaled residual  4.60e-14; WALRAS =  2.03e-13
```

The base does not satisfy the equation, which therefore acts as a shock.

### Example 2: a shock to a parameter

[CGE] `cge_shock` changes a user parameter like a parameter of the model:

```
. cge_shock hgoal, value(1.2)
User parameter hgoal changed for 1 element(s)

. cge_solve, name(goal12)
Simulation goal12 solved in 7 iteration(s) (newton)
Largest scaled residual  1.62e-12; WALRAS =  4.06e-11

. cge_report, sims(goal goal12) tables(macro)

Macroeconomic aggregates
------------------------------------------------------------
                                  base       goal     goal12
gdp_nominal                        300     -0.078     -0.155
gdp_real                           300     -0.087     -0.174
gdp_deflator                         1      0.009      0.019
consumption_real                   222      0.005      0.010
government_real                     45     10.000     20.000
investment_real                     50     -9.550    -19.106
insurance_fund_real                  8      0.051      0.103
exports_real                        75     -1.210     -2.419
imports_real                       100     -0.907     -1.814
value_added_real                   262     -0.000     -0.001
cpi                                  1      0.000      0.000
dpi                                  1      0.027      0.054
exchange_rate                        1     -0.077     -0.155
real_exchange_rate                   1     -0.103     -0.208
private_savings                     34      0.005      0.010
investment_value                    50     -9.563    -19.129
foreign_savings                     14      0.000      0.000
------------------------------------------------------------
Percentage changes from the base (rates: changes in points)
```

The scaling factor of government consumption raises every government purchase in proportion, so real government consumption is 10 and then 20 per cent above the base; investment falls by about as much.

### Example 3: values by element

Weights by commodity, for a price index of the user's own:

```
. cge_parameter w(c), values(com_agr:0.4 com_man:0.3 com_ser:0.2 com_hlt:0.1)
User parameter w(c): 4 value(s), from .1 to .4

. cge_variable PIDX = sum(c, w(c) * PQ(c))
User variable PIDX: 1 element(s), base value 1
and its definition, equation PIDX
```

From the data in memory, with the labels in a string variable; the elements without an observation take `default()`, 0:

```
. clear

. input str8 commodity double share

     commodity       share
  1. "com_agr" 0.6
  2. "com_man" 0.4
  3. end

. cge_parameter sw(c), from(share) labels(commodity)
User parameter sw(c): 4 value(s), from 0 to .6
```

With two indexes, here commodity and household:

```
. cge_parameter sh(c,h), values(com_hlt.hhd:0.5) default(0.1)
User parameter sh(c,h): 4 value(s), from .1 to .5

. cge_parameter, list
parameter hgoal: 1 value(s), current from 1.2 to 1.2 (base from 1.1 to 1.1)
parameter w(c): 4 value(s), current from .1 to .4 (base from .1 to .4)
parameter sw(c): 4 value(s), current from 0 to .6 (base from 0 to .6)
parameter sh(c,h): 4 value(s), current from .1 to .5 (base from .1 to .5)
```

### Example 4: the parameters as data

**[CGE]** `cge_parameters` lists user parameters after those of the model:

```
. cge_parameters, clear

. list if inlist(parameter, "hgoal", "w", "sw", "sh"), noobs sep(0)

  +----------------------------------------+
  | parame~r   element1   element2   value |
  |----------------------------------------|
  |    hgoal                           1.1 |
  |        w    com_agr                 .4 |
  |        w    com_man                 .3 |
  |        w    com_ser                 .2 |
  |        w    com_hlt                 .1 |
  |       sw    com_agr                 .6 |
  |       sw    com_man                 .4 |
  |       sw    com_ser                  0 |
  |       sw    com_hlt                  0 |
  |       sh    com_agr        hhd      .1 |
  |       sh    com_man        hhd      .1 |
  |       sh    com_ser        hhd      .1 |
  |       sh    com_hlt        hhd      .5 |
  +----------------------------------------+
```

### Example 5: a parameter as an instrument

The health purchase target that leaves government savings at 1.5, against 1 in the base:

```
. cge_shock, reset
All shocks removed

. cge_optimize, instruments(hgoal) targets(government savings = 1.5) name(hgopt)

Optimisation hgopt: converged after 11 solve(s) of the model
----------------------------------------------------------------
Instrument                                  start        optimum
----------------------------------------------------------------
hgoal                                         1.1   .9895248078
----------------------------------------------------------------
Target government savings  = 1.5:              1.5
```

## Stored results

`cge_parameter` stores the following in `r()`:

| | |
|---|---|
| *Scalars* | |
| `r(N_elements)` | number of values of the parameter defined |
| `r(N_parameters)` | number of user parameters (`list`) |
| *Macros* | |
| `r(name)` | name of the parameter defined |

## Methods and formulas

The values of all user parameters are held, row by row, in one vector of the model's parameters, so every simulation stores them with its other parameters, `cge_shock, reset` returns them to their base values, and continuation in $n$ steps moves them as $\theta_k = \theta^0 + (k/n)(\theta^1 - \theta^0)$ like the others. In an expression, a reference to a parameter element compiles to its position in that vector; its derivative is zero. When the model is calibrated again, the parameters keep their base values. In the GAMS program, `cge_user.inc` declares each parameter as `UP_`*name*, with its base values, and the scenario file changes it by the ratio of its value to the base, as for the model's parameters.

## Also see

**[CGE]** `cge_set` — User sets of elements, for indexes and sums
**[CGE]** `cge_variable` — User variables, defined by an expression or determined by user equations
**[CGE]** `cge_equation` — User equations: new relations among the model's variables
**[CGE]** `cge_shock` — Change a parameter or a fixed variable
**[CGE]** `cge_parameters` — The calibrated parameters as a dataset
**Chapter 23** — User variables and equations

# `cge_parameters` — The calibrated parameters as a dataset

## Description

`cge_parameters` replaces the data in memory with the parameters of the model, one observation per parameter element, with the string variables `parameter`, `element1` and `element2` and the numeric variable `value`. It gives the base (calibrated) parameters, the parameters of a stored simulation, or the current parameters, including shocks not yet solved. Use it to inspect, document or compare a calibration.

## Quick start

Load the calibrated parameters and list the import tariff rates

```
cge_parameters, clear
list if parameter == "tm"
```

Load the parameters of the simulation `notariff`, with its shocks

```
cge_parameters, clear sim(notariff)
```

Load the current parameters, with the shocks set since the last solve

```
cge_parameters, clear current
```

## Syntax

**`cge_parameters`**, `clear` [*options*]

| *options* | Description |
|---|---|
| *Main* | |
| `clear` | replace the data in memory; required |
| `sim(`*name*`)` | parameters of a stored simulation |
| `current` | current parameters, including shocks not yet solved |

## Options

`clear` is required: the data in memory are replaced. The model itself, which is kept in Mata, is not changed.

`sim(`*name*`)` gives the parameters of a stored simulation, including its shocks. By default the base (calibrated) parameters are given.

`current` gives the current parameters, including the shocks set with **[CGE]** `cge_shock` since the last solve. `sim()` and `current` may not be combined.

## Remarks and examples

Each parameter is stored by element: a parameter of commodities has one observation per commodity, with the commodity in `element1`; a parameter of two sets, such as the intermediate input coefficients, has one observation per pair, with the second element in `element2`; a scalar parameter has one observation with both elements empty. Every element of the set is listed, including elements to which the parameter does not apply; these hold a value that the model does not use, such as an export tax rate of zero for a commodity without exports. The names of the parameters are those of the table below.

### Example 1: tax rates and shares

After the synthetic SAM is mapped and calibrated as in Example 1 of **[CGE]** `cge_calibrate`:

```
. cge_parameters, clear

. display r(N)
252

. list if inlist(parameter, "tm", "tq"), noobs sepby(parameter) abbreviate(9)

  +--------------------------------------------+
  | parameter   element1   element2      value |
  |--------------------------------------------|
  |        tm    com_agr                    .1 |
  |        tm    com_man                    .1 |
  |        tm    com_ser                     0 |
  |--------------------------------------------|
  |        tq    com_agr              .07272727 |
  |        tq    com_man              .06741573 |
  |        tq    com_ser              .06060606 |
  +--------------------------------------------+
```

The import tariff rate of `com_agr` is the tariff over the value of imports, 2/20. The sales tax rate is the tax over composite supply, which includes the tax: for `com_agr`, with domestic sales of 80, imports of 20 at the price 1.1, which includes the tariff, and the tax of 8, $\texttt{tq} = 8/(80 + 1.1 \times 20 + 8) = 8/110$. A parameter of two sets lists every pair:

```
. list element1 element2 value if parameter == "dva", noobs

  +--------------------------------+
  | element1   element2      value |
  |--------------------------------|
  |      lab    act_agr   .52434372 |
  |      lab    act_man   .45836805 |
  |      lab    act_ser   .55673016 |
  |      cap    act_agr   .47565628 |
  |      cap    act_man   .54163195 |
  |--------------------------------|
  |      cap    act_ser   .44326984 |
  +--------------------------------+
```

These are the shares $\delta^{va}_{fa}$ of the CES value-added functions; with `sigmava(0.8)`, they differ from the factors' shares of value added in the SAM (section 9.5).

### Example 2: the parameters of a simulation

A simulation is stored with its parameters, shocks included:

```
. quietly cge_shock tm, value(0)

. quietly cge_solve, name(notariff)

. cge_parameters, clear sim(notariff)

. list if parameter == "tm", noobs abbreviate(9)

  +----------------------------------------+
  | parameter   element1   element2   value |
  |----------------------------------------|
  |        tm    com_agr                  0 |
  |        tm    com_man                  0 |
  |        tm    com_ser                  0 |
  +----------------------------------------+
```

A shock set after the last solve is in the current parameters, not yet in any simulation:

```
. quietly cge_shock tq, elements(com_ser) scale(1.5)
. cge_parameters, clear current

. list if parameter == "tq", noobs abbreviate(9)

  +--------------------------------------------+
  | parameter   element1   element2      value |
  |--------------------------------------------|
  |        tq    com_agr              .07272727 |
  |        tq    com_man              .06741573 |
  |        tq    com_ser              .09090909 |
  +--------------------------------------------+
```

### The parameters

The parameters follow the notation of section 6.4. In the table, $C$, $A$, $F$ and $H$ are the commodities, activities, factors and households, $D$ the households and enterprises, $I$ the institutions that receive factor income ($D$, the government, the rest of the world and, when mapped, the health insurance fund) and $N$ the nests of value added.

| `parameter` | Elements | Description |
|---|---|---|
| *Trade and taxes on commodities* | | |
| `pwm`, `pwe` | $C$ | world import and export prices |
| `tm`, `te`, `tq` | $C$ | import tariff, export tax and sales tax rates |
| `tv` | $C$ | VAT rate |
| `at`, `dt`, `dtc`, `rt`, `st` | $C$ | CET: efficiency, share of exports, its complement, exponent, elasticity |
| `aq`, `dq`, `dqc`, `rq`, `sq` | $C$ | Armington: efficiency, share of imports, its complement, exponent, elasticity |
| `qer` | $C$ | base exports drawn from composite supply |
| `etae`, `qe0` | $C$ | elasticity of export demand (0 for a fixed world price) and base exports |
| `icm` | $C \times C$ | margin services per unit of composite supply |
| `icd`, `ici`, `ice` | $C \times C$ | margin services per unit of domestic sales, imports and exports |
| *Final demand and price indices* | | |
| `qg`, `qinv`, `qshi` | $C$ | base government consumption, investment and purchases of the health insurance fund |
| `cwts`, `dwts` | $C$ | weights of the consumer and producer price indices |
| `beta`, `gamma` | $C \times H$ | marginal budget shares and subsistence quantities |
| `betah`, `gammah` | $A \times H$ | the same, for home consumption |
| *Production* | | |
| `theta` | $A \times C$ | yields: output of each commodity per unit of marketed activity output |
| `ta` | $A$ | activity tax rate |
| `iva`, `inta` | $A$ | value added and intermediate input per unit of activity (Leontief top nest) |
| `aa`, `dla`, `dlac`, `rha`, `sa` | $A$ | top nest: efficiency, share of value added, its complement, exponent, elasticity |
| `ica` | $C \times A$ | intermediate input per unit of aggregate intermediate input |
| `ad`, `rva`, `sva` | $A$ | value added: efficiency, exponent, elasticity |
| `dva` | $F \times A$ | value added: shares of the factors |
| `kappa` | $A$ | share of the VAT on inputs recovered as a credit |
| `tf` | $F \times A$ | tax rate on factor use |
| `tfy` | $F$ | tax rate on factor income |
| `dvn`, `an` | $N \times A$ | nests: share in value added, efficiency (with nests only) |
| `dce` | $C \times A$ | nests: shares of the commodities (with nests only) |
| `sgn`, `rhn` | $N$ | nests: elasticity, exponent (with nests only) |
| `sac`, `rac`, `aac` | $C$ | output from several activities: elasticity, exponent, efficiency (with `sigmaac()` only) |
| `dac` | $A \times C$ | output from several activities: shares (with `sigmaac()` only) |
| *Institutions and transfers* | | |
| `shif` | $I \times F$ | shares of factor income, after contributions and taxes, received by each institution |
| `tshi` | $F$ | contribution rate to the health insurance fund |
| `tins0`, `mps0` | $D$ | base direct tax rate and savings rate |

| | | |
|---|---|---|
| `shii` | $D \times D$ | share of the income of the second institution, after taxes and savings, transferred to the first |
| `shir` | $D$ | share of income after taxes and savings transferred abroad |
| `hc` | $H$ | share of income after taxes and savings spent on consumption |
| `trdg`, `trdr` | $D$ | transfers from the government (real terms) and from the rest of the world (foreign currency) |
| `trfr` | $F$ | factor income from the rest of the world (foreign currency) |
| `trgr`, `trrg`, `trgg` | scalar | transfers from the rest of the world to the government, from the government abroad, and within government |
| `sshi`, `trshi`, `trshir` | scalar | health insurance fund: savings rate, transfers from the government and from the rest of the world |
| `debt`, `ddebt`, `fdebt` | scalar | 1 if the model has public debt; debt at home and abroad |
| `rd`, `rf`, `phi` | scalar | interest rates at home and abroad; share of new borrowing from abroad |
| `hold` | $D$ | share of domestic interest received |
| *Factors, labour market and health* | | |
| `af` | $F$ | exogenous efficiency |
| `etah`, `hs0` | $F$; scalar | elasticity of efficiency with respect to health services; base volume of health services |
| `inlf`, `hasmin` | $F$ | 1 if the factor has a labour force; 1 if it has a minimum wage |
| `qlf`, `u0` | $F$ | labour force; base unemployment rate (for factors with a labour force) |
| `epsw`, `wref`, `wmin` | $F$ | wage-curve elasticity, reference real wage, real minimum wage |
| `mu` | scalar | smoothing parameter of the minimum-wage condition |
| `ins` | $F$ | 1 if the factor loses working time to illness |
| `sick0`, `sick`, `epss` | $F$ | base share of working time lost, its current value, its elasticity |
| `hlink` | scalar | 1 if health services enter the model through efficiency or health outcomes |
| `hasmort`, `mort0`, `mort`, `epsm` | scalar | 1 if the model has a death rate; its base value, current value and elasticity |
| `pop` | scalar | population index |

## Stored results

`cge_parameters` stores the following in `r()`:

*Scalars*

| | |
|---|---|
| `r(N)` | number of parameter elements |

## Also see

**[CGE]** `cge_calibrate` — Calibrate the model to the SAM
**[CGE]** `cge_shock` — Change a parameter or a fixed variable
**[CGE]** `cge_results` — Report simulation results
**[CGE]** `cge_report` — Standard report tables for one or more simulations
**Chapter 9** — Calibration

# cge_report — Standard report tables for one or more simulations

## Description

`cge_report` computes the standard report measures for one or more simulations, at the base and at each solution, and displays them side by side as percentage changes from the base: macroeconomic aggregates, the government and public debt, institutions, factors, activities, commodities, health and welfare. The report can be loaded as data or saved as a Stata dataset or an Excel workbook.

## Quick start

Report the last simulation

```
cge_report
```

Compare two simulations in the macroeconomic, government and welfare tables

```
cge_report, sims(notariff nt_sales) tables(macro government welfare)
```

Save every table of every simulation to an Excel workbook

```
cge_report, sims(_all) tables(all) saving(results.xlsx, replace) nodisplay
```

Report every period of the dynamic run `base`

```
cge_report, run(base) tables(macro)
```

Load the report as data

```
cge_report, sims(_all) clear nodisplay
```

## Syntax

`cge_report` [, *options*]

| *options* | Description |
|---|---|
| *Main* | |
| `sims`(*names*) | simulations, or `_all`; default the last one stored |
| `run`(*name*) | every period of dynamic run *name*, in period order |
| `tables`(*list*) | tables to display, or `all` |
| *Data* | |
| `clear` | replace the data in memory with the report |
| `saving`(*filename*[, `replace`]) | save the report as a Stata dataset or an Excel workbook |
| *Reporting* | |
| `nodisplay` | do not display the tables |

## Options

`sims`(*names*) lists the simulations to report; `_all` selects every stored simulation. The default is the last simulation stored. At most six simulations are displayed; all are loaded and saved.

`run`(*name*) selects every period of a run solved by **[CGE]** `cge_dynamic`, in period order, instead of `sims()`; the two options may not be combined.

`tables(`*list*`)` lists the tables to display: `macro`, `government`, `debt`, `institutions`, `factors`, `activities`, `commodities`, `health` and `welfare`, or `all`. The default is `macro government debt institutions factors health welfare`. A table with no measures in the model, such as `debt` in a model without public debt, is not displayed. Every table is loaded and saved, whatever `tables()`.

`clear` replaces the data in memory with the report, one observation per simulation and measure, with the variables `sim`, `table`, `measure`, `element`, `base`, `value` and `pct`.

`saving(`*filename*`[, replace])` saves the report, leaving the data in memory unchanged; `replace` allows an existing file to be overwritten. A file ending in `.xlsx` or `.xls` is written as an Excel workbook, with one sheet per table, one row per measure, and the base followed by a value and a percentage change for each simulation; any other file is saved as a Stata dataset in the format of `clear`.

`nodisplay` suppresses the tables.

## Remarks and examples

`cge_report` turns solutions into the measures that policy reports use: real GDP and its components, revenue by tax, real wages, sectoral output and welfare. Real measures are valued at base prices, and trade at base world prices, so their changes are changes in volume; nominal measures use current prices. With VAT, final demand is valued at the prices that final users pay, including VAT. Consumption and GDP include home consumption at activity prices, which the `macro` table then also shows separately.

### The tables

| Table | Measures |
|---|---|
| `macro` | nominal and real GDP and the GDP deflator; real consumption, home consumption, government consumption, investment, purchases of the health insurance fund, exports, imports and value added; the CPI and DPI; the exchange rate and the real exchange rate; private savings, investment in value and foreign savings |
| `government` | revenue, spending and savings; revenue from direct taxes, sales taxes, import tariffs, export taxes, activity taxes, VAT, taxes on factor use and taxes on factor income |
| `debt` | with public debt: interest paid at home and abroad, the primary balance, and debt at the end of the period at home, abroad (in local and in foreign currency), in total and in per cent of nominal GDP |
| `institutions` | income of each household and enterprise; consumption spending of each household |
| `factors` | wage, real wage, income and supply of each factor; the labour force and the unemployment rate (per cent) of factors with a labour force |
| `activities` | output, price and real value added of each activity |
| `commodities` | composite supply, price, output, domestic sales, exports and imports of each commodity; the world export price of commodities with finite export demand |
| `health` | when the model has them: health services and services per head, and the efficiency of factors linked to health services; the share of working time lost to illness (per cent) and days worked; the death rate (per 1,000) and the population index; the revenue and the volume of purchases of the health insurance fund |
| `welfare` | equivalent and compensating variations of each household, in per cent of base consumption |

Measures are displayed as percentage changes from the base, missing where the base is zero. Rates—the unemployment rate, public debt in per cent of GDP, the share of working time lost and the death rate—are shown as changes in percentage points, and the welfare measures in per cent of base consumption.

### Example 1: two simulations side by side

In the synthetic SAM, mapped and calibrated as in **[CGE]** `cge_solve`, import tariffs are removed under the default closure, with government savings adjusting, and again with government savings fixed and sales tax rates adjusting (**[CGE]** `cge_closure`):

```
. cge_shock tm, value(0)
Parameter tm changed for 3 element(s)
```

```
. quietly cge_solve, name(notariff)

. cge_closure, government(fixgsav sales)
Closure
  external account     flexexr
  government           fixgsav:sales
  savings-investment   savings
  numeraire            cpi
  factor lab           mobile
  factor cap           mobile

. quietly cge_solve, name(nt_sales)
```

```
. cge_report, sims(notariff nt_sales) tables(macro government welfare)

Macroeconomic aggregates
------------------------------------------------------------
                                 base   notariff   nt_sales
gdp_nominal                       300     -0.305     -0.122
gdp_real                          300      0.081      0.120
gdp_deflator                        1     -0.386     -0.242
consumption_real                  230      2.684     -0.247
government_real                    45      0.000      0.000
investment_real                    50    -11.861      1.857
insurance_fund_real                 0          .          .
exports_real                       75      5.644      6.348
imports_real                      100      4.233      4.761
value_added_real                  262     -0.000     -0.000
cpi                                 1      0.000      0.000
dpi                                 1      1.013     -1.114
exchange_rate                       1      4.759      2.854
real_exchange_rate                  1      3.709      4.013
private_savings                    32      2.652     -0.267
investment_value                   50    -13.157      0.686
foreign_savings                    15      0.000      0.000
------------------------------------------------------------

Government
------------------------------------------------------------
                                 base   notariff   nt_sales
revenue                            63    -11.727      0.981
spending                           60      1.255      1.030
savings                             3   -271.374      0.000
tax_direct                         25      2.652     -0.267
tax_sales                          30     -0.171     28.949
tax_import                          8   -100.000   -100.000
tax_export                          0          .          .
tax_activity                        0          .          .
tax_vat                             0          .          .
tax_factor_use                      0          .          .
tax_factor_income                   0          .          .
------------------------------------------------------------

Welfare (per cent of base consumption)
--------------------------------------------------------------
                                          notariff   nt_sales
equivalent_variation hhd                     2.668     -0.257
compensating_variation hhd                   2.668     -0.257
--------------------------------------------------------------
Percentage changes from the base (rates: changes in points)
```

Without replacement, the tariff revenue is lost, government savings fall by 271 per cent of their base value, and with savings-driven investment real investment falls by 11.9 per cent; the household's consumption rises and its equivalent variation is 2.7 per cent of base consumption. With the revenue replaced by sales taxes, government savings are unchanged, sales tax revenue rises by 28.9 per cent, real investment rises, and the household loses 0.26 per cent of base consumption. Real government consumption is fixed under both closures. The rows with a missing change, such as the export tax, have a base of zero.

### Example 2: stored results and the report as data

```
. return list

scalars:
             r(N_sims) =  2
         r(N_measures) =  67
```

`r(N_measures)` counts the measures of every table, 67 here, including those not displayed. With `clear` they become data, one observation per simulation and measure:

```
. cge_report, sims(_all) clear nodisplay

. describe, short

Contains data
  obs:           134
 vars:             7
 size:        18,224
Sorted by:
     Note:  dataset has changed since last saved

. list sim measure element value pct if table == "welfare", noobs sepby(sim)

  +-----------------------------------------------------------------------+
  |      sim                  measure   element        value          pct |
  |-----------------------------------------------------------------------|
  | notariff     equivalent_variation       hhd    6.1373219    2.6684008 |
  | notariff   compensating_variation       hhd    6.1363604    2.6679828 |
  |-----------------------------------------------------------------------|
  | nt_sales     equivalent_variation       hhd   -.59016788   -.25659473 |
  | nt_sales   compensating_variation       hhd   -.59010877   -.25656903 |
  +-----------------------------------------------------------------------+
```

`pct` holds the displayed change: the percentage change, the change in points for rates, and, for the welfare measures, whose base is zero, the per cent of base consumption.

### Example 3: an Excel workbook

```
. cge_report, sims(_all) tables(all) saving(report.xlsx, replace) nodisplay
Report saved to report.xlsx (one sheet per table)
```

Each sheet holds one table, with the columns `table`, `measure`, `element` and `base`, then the values of the simulations and their percentage changes, labelled with the names of the simulations. A table with no measures in the model, such as `debt` without public debt, has no sheet.

### Dynamic runs

The periods of a dynamic run are stored as simulations named after the run and the period. `run()` reports them in period order, and **[CGE]** `cge_dynresults` gives every variable of a run as a dataset.

## Stored results

`cge_report` stores the following in `r()`:

*Scalars*

| | |
|---|---|
| `r(N_measures)` | number of measures |
| `r(N_sims)` | number of simulations |

## Methods and formulas

Superscript 0 marks base values. Final demand is valued at $PH_c = PQ_c\,(1 + tv_c\,TVADJ)$, and home consumption at the activity prices $PA_a$. Nominal GDP is

$$\begin{aligned} GDP = {} & \sum_c PH_c \Big( \sum_h QH_{ch} + QG_c + QINV_c + QSHI_c \Big) + \sum_c PQ_c\,QER_c + \sum_a PA_a \sum_h QHA_{ah} \\ & + EXR \sum_c PWE_c\,QE_c - EXR \sum_c pwm_c\,QM_c - R_{row}, \end{aligned}$$

where $R_{row}$ is the premiums of constraints paid abroad, and real GDP is the same sum at $PH^0_c$, $PQ^0_c$, $PA^0_a$, $EXR^0$ and the base world prices, without $R_{row}$. The other real measures are

$$\begin{aligned} \text{consumption} &= \sum_c PH^0_c \sum_h QH_{ch} + \sum_a PA^0_a \sum_h QHA_{ah}, & \text{government} &= \sum_c PH^0_c\, QG_c, \\ \text{exports} &= EXR^0 \sum_c pwe^0_c\, QE_c + \sum_c PQ^0_c\, QER_c, & \text{imports} &= EXR^0 \sum_c pwm^0_c\, QM_c, \end{aligned}$$

with investment and the fund's purchases as for the government, and real value added $\sum_a PVA^0_a\, QVA_a$, less the commodities in nests of value added at base prices. The real exchange rate is $EXR/DPI$, private savings $\sum_d MPS_d\,(1 - TINS_d)\, YI_d$, the real wage $W_f/CPI$ and the unemployment rate $100\,(1 - QFS_f/QLF_f)$. Revenue by tax is the corresponding term of government revenue (I8).

The welfare measures of household $h$ use its expenditure function, $E_h(P,u) = \sum_c P_c\, \gamma_{ch} + u \prod_c P_c^{\beta_{ch}}$, over marketed goods at $PH_c$ and home-consumed goods at $PA_a$:

$$\begin{aligned} EV_h &= \sum_c P^0_c\, \gamma_{ch} + \Big(EH_h - \sum_c P_c\, \gamma_{ch}\Big) \prod_c \Big(\frac{P^0_c}{P_c}\Big)^{\beta_{ch}} - EH^0_h, \\ CV_h &= EH_h - \sum_c P_c\, \gamma_{ch} - \Big(EH^0_h - \sum_c P^0_c\, \gamma_{ch}\Big) \prod_c \Big(\frac{P_c}{P^0_c}\Big)^{\beta_{ch}}. \end{aligned}$$

The equivalent variation is the change in spending at base prices that gives the new utility; the compensating variation is the change in spending at the new prices that would restore base utility. Both are positive for a gain and are reported as $100\, EV_h/EH^0_h$ and $100\, CV_h/EH^0_h$. With Cobb–Douglas demand, $\gamma_{ch} = 0$.

A measure $x$ is reported as $100\,(x - x^0)/|x^0|$, missing when $x^0 = 0$; a rate as $x - x^0$. Chapter 12 gives every measure (section 12.1) and the welfare measures (section 12.2).

## Also see

**[CGE]** `cge_results` — Report simulation results
**[CGE]** `cge_solve` — Solve the model
**[CGE]** `cge_dynresults` — Results of a dynamic run as a dataset
**[CGE]** `cge_save` — Save the model and its simulations to a file
**Chapter 12** — Reporting measures and welfare

# `cge_results` — Report simulation results

## Description

`cge_results` displays a summary of a simulation: real value added, the exchange rate, the CPI, investment, imports, exports, government savings, factor prices and activity output, as percentage changes from the base; the equivalent variation of each household; and the checks of Walras's law and of the GDP identity. With `clear` or `saving()`, it writes every variable of the solution as a dataset, with one observation per variable element.

## Quick start

Display the summary of simulation `notariff`

```
cge_results, sim(notariff)
```

Load every variable of the simulation as data, without the summary

```
cge_results, sim(notariff) clear nodisplay
```

Save the results of the last simulation as a Stata dataset

```
cge_results, saving(results, replace)
```

## Syntax

`cge_results` [, *options*]

| *options* | Description |
|---|---|
| *Main* | |
| `sim(`*name*`)` | simulation; default the last one stored |
| *Data* | |
| `clear` | replace the data in memory with the results |
| `saving(`*filename*[`, replace`]`)` | save the results as a Stata dataset |
| *Reporting* | |
| `nodisplay` | do not display the summary |

## Options

`sim(`*name*`)` selects the simulation. The default is the last simulation stored, which is the one most recently added: solving again under an existing name replaces that simulation where it stands.

`clear` replaces the data in memory with the results of the simulation. The data in memory need not have been saved.

`saving(`*filename*[`, replace`]`)` saves the results as a Stata dataset, leaving the data in memory unchanged; `replace` allows an existing file to be overwritten.

`nodisplay` suppresses the summary. The summary also stores the results in `r()`, so nothing is stored with `nodisplay`.

## Remarks and examples

`cge_results` is the quick look at one simulation, and the way to every variable of its solution. **[CGE]** `cge_report` computes the standard report measures, such as real GDP and revenue by tax, for several simulations side by side.

### Example 1: the summary

In the synthetic SAM, mapped and calibrated as in **[CGE]** `cge_solve`, every import tariff is removed and the model solved as simulation `notariff`:

```
. cge_results, sim(notariff)

Simulation notariff (closure: flexexr flexgsav savings cpi lab:mobile cap:mobile)
----------------------------------------------------
                                       base % change
----------------------------------------------------
Real value added (base prices)          262   -0.000
Exchange rate                             1    4.759
Consumer price index                      1    0.000
Investment volume (IADJ)                  1  -11.861
Imports (world prices)                  100    4.233
Exports (world prices)                   75    5.644
Government savings                        3 -271.374
Price of lab                              1    2.811
Price of cap                              1    2.631
Output of act_agr                       100   -0.409
Output of act_man                       140   -1.399
Output of act_ser                       150    1.240
----------------------------------------------------
Equivalent variation, hhd              6.137 ( 2.67% of consumption)
WALRAS -1.56e-14; GDP identity gap -6.82e-13
```

The heading gives the closure under which the simulation was solved. The percentage changes are relative to the absolute value of the base, so a fall is negative even when the base is negative: here government savings, 3 in the base, fall by 271 per cent to a deficit of about 5. The household gains, with an equivalent variation of 6.1, 2.7 per cent of its base consumption. The two checks are zero to the precision of the solution: the slack of the savings–investment balance, `WALRAS`, and the gap between nominal GDP measured from expenditure and from income.

The equivalent variations are returned in a row vector, one column per household:

```
. return list

macros:
                r(sim) : "notariff"

matrices:
                 r(ev) :  1 x 1

. matrix list r(ev)

symmetric r(ev)[1,1]
           c1
r1  6.1373219
```

### Example 2: the results as data

With `clear`, every variable element of the solution becomes an observation, with its base and simulated values:

```
. cge_results, sim(notariff) clear nodisplay

. describe, short

Contains data
  obs:          118
 vars:            5
 size:         13,216
Sorted by:
     Note:  dataset has changed since last saved

. list if variable == "QA", noobs

  +----------------------------------------------------+
  | variable   element1   element2   base        value |
  |----------------------------------------------------|
  |       QA    act_agr               100    99.590677 |
  |       QA    act_man               140    138.04142 |
  |       QA    act_ser               150    151.86047 |
  +----------------------------------------------------+
```

The variables are `variable`, the name of the model variable as in section 6.3; `element1` and `element2`, its set elements, empty for a variable without index; `base`, the base value; and `value`, the value in the simulation. Every variable is included, fixed or free, with the rents of constraints, the slack `WALRAS` and the user variables of **[CGE]** `cge_variable`; elements that do not exist in the model, such as imports of a commodity that is not imported, are left out. Percentage changes are then a line away:

```
. generate double pct = 100 * (value - base) / abs(base)
(15 missing values generated)

. list variable element1 base value pct if inlist(variable, "PQ", "QM"), noobs

  +------------------------------------------------------+
  | variable   element1   base       value           pct |
  |------------------------------------------------------|
  |       PQ    com_agr      1   .99948068   -.05193189 |
  |       PQ    com_man      1   .97900528   -2.0994718 |
  |       PQ    com_ser      1   1.0214491    2.1449122 |
  |       QM    com_agr     20   22.236866     11.18433 |
  |       QM    com_man     60   63.002825    5.0047091 |
  |------------------------------------------------------|
  |       QM    com_ser     20   18.993677   -5.0316161 |
  +------------------------------------------------------+
```

The 15 missing values are the elements whose base value is zero.

### Example 3: saving the results

`saving()` writes the same dataset to a file and leaves the data in memory unchanged:

```
. cge_results, sim(notariff) saving(notariff_results, replace) nodisplay
(note: file notariff_results.dta not found)
file notariff_results.dta saved
```

## Stored results

`cge_results` stores the following in `r()`, unless `nodisplay` is specified:

| | |
|---|---|
| *Macros* | |
| `r(sim)` | name of the simulation |
| *Matrices* | |
| `r(ev)` | equivalent variation of each household, $1 \times H$ |

## Methods and formulas

Let $x^0$ be a variable at the base and $x$ in the simulation. The summary shows $100\,(x - x^0)/|x^0|$, missing when $x^0 = 0$. Its rows are real value added, $\sum_a PVA^0_a\, QVA_a$ less the commodities in nests of value added at base prices (section 18.2); $EXR$; $CPI$; $IADJ$; imports, $\sum_c pwm_c\, QM_c$, and exports, $\sum_c pwe_c\,(QE_c + QER_c)$, both at the world prices of the simulation for the base and for the solution alike; $GSAV$; $SHIADJ$, when the model has a health insurance fund; $W_f$; and $QA_a$.

The equivalent variation of household $h$ is the change in spending at base prices that gives its new utility,

$$EV_h = \sum_c P_c^0\,\gamma_{ch} + \Big(EH_h - \sum_c P_c\,\gamma_{ch}\Big)\prod_c \Big(\frac{P_c^0}{P_c}\Big)^{\beta_{ch}} - EH_h^0,$$

where the sums and the product run over marketed commodities, at the prices that final users pay, $PH_c$, and over home-consumed goods, at the activity prices $PA_a$, with the shares and subsistence quantities of home consumption; $\gamma_{ch} = 0$ with Cobb–Douglas demand. It is shown in per cent of base consumption, $EH_h^0$. The GDP identity gap is nominal GDP measured from expenditure less nominal GDP measured from income, zero at any solution. Chapter 12 gives the measures, the compensating variation and the two sides of GDP (section 12.2).

## Also see

**[CGE]** `cge_report` — Standard report tables for one or more simulations
**[CGE]** `cge_solve` — Solve the model
**[CGE]** `cge_check` — Run model validation checks
**[CGE]** `cge_parameters` — The calibrated parameters as a dataset
**Chapter 12** — Reporting measures and welfare

# `cge_sam` — Read a social accounting matrix

## Description

`cge_sam` reads a social accounting matrix (SAM) and starts a new model from it; any earlier mapping, calibration and simulations are discarded. The SAM is taken from the data in memory or, with `using`, from a `.csv`, `.dta`, `.xls` or `.xlsx` file. It may be in wide form, with one observation and one variable per account, or in long form, with one observation per cell. A wide CSV file can also be read directly, without loading it as data, so that a SAM may have more accounts than Stata can hold as variables.

`cge_sam` checks that the SAM is square, with row and column labels that agree, and reports the largest difference between an account's row and column totals. The SAM is then mapped to the model with **[CGE]** `cge_map`, after any aggregation by **[CGE]** `cge_aggregate` and balancing by **[CGE]** `cge_balance`.

## Quick start

Read the SAM in memory, with the account labels in the string variable `account`

```
cge_sam, labels(account)
```

Read a SAM from the worksheet `SAM` of an Excel file, replacing the data in memory

```
cge_sam using mysam.xlsx, sheet("SAM") labels(account) clear
```

Read a wide CSV file directly, leaving the data in memory unchanged

```
cge_sam using large_sam.csv, direct
```

Read long data, one observation per cell, from a Stata dataset

```
cge_sam using sam_cells.dta, long(row column value) clear
```

## Syntax

Wide data: one observation and one variable per account

**`cge_sam`** [`using` *filename*] [`,` `labels(`*varname*`)` `direct` `sheet(`*string*`)` `clear`]

Long data: one observation per cell

**`cge_sam`** [`using` *filename*]`,` `long(`*rowvar colvar valuevar*`)` [`sheet(`*string*`)` `clear`]

*filename* ends in `.csv`, `.dta`, `.xls` or `.xlsx`.

| *options* | Description |
|---|---|
| *Main* | |
| `labels(`*varname*`)` | string variable with the account labels in wide data; default the first string variable |
| `long(`*rowvar colvar valuevar*`)` | read long data, one observation per cell |
| `direct` | read a wide CSV file directly, without loading it as data |
| *Files* | |
| `sheet(`*string*`)` | Excel worksheet to read |
| `clear` | allow the data in memory to be replaced by the file |

## Options

`labels(`*varname*`)` names the string variable that holds the account labels in wide data. The default is the first string variable. Every other variable is an account and must be numeric.

`long(`*rowvar colvar valuevar*`)` reads long data: *rowvar* and *colvar* are string variables with the row and column accounts of each cell, and *valuevar* is a numeric variable with its value. `long` may not be combined with `direct`.

`direct` reads a wide CSV file directly in Mata, without loading it as data; the data in memory are not changed. It applies only to a `.csv` file named with `using`. A wide CSV file with more accounts than fit as variables (`c(maxvar)`) is read directly without the option.

`sheet(`*string*`)` names the worksheet of an Excel file to read. The default is the first worksheet.

`clear` allows the data in memory to be replaced when a file is read as data. It has no effect with `direct`.

## Remarks and examples

Cell $(r,c)$ of a SAM is a payment from the column account $c$ to the row account $r$: an account's row records its receipts and its column its payments. The SAM is balanced when every account's row total equals its column total. `cge_sam` reads the SAM in one of three forms, described below, and reports its size, its total and its largest imbalance. It does not require the SAM to be balanced; **[CGE]** `cge_map` does, and **[CGE]** `cge_balance` balances a SAM that is not. Blank cells are read as zero and counted.

### Example 1: wide data in memory

In wide data, each observation is an account, a string variable holds the account labels, and there is one numeric variable per account, named as the account and in the same order as the observations. The synthetic SAM of **[CGE]** `cge_synthetic` is loaded in this form:

```
. cge_synthetic, clear
(16 vars, 15 obs)

Synthetic SAM loaded (standard layout): 15 accounts, total   1743.00
Largest row-column imbalance:  0.00e+00

. cge_sam, labels(account)
SAM read: 15 accounts, total 1743
Largest row-column imbalance:  0.00e+00
```

A file is read as data with `import delimited` (a `.csv` file, with the first column read as strings and the others as doubles), `use` (a `.dta` file) or `import excel` (a `.xls` or `.xlsx` file, with the first row as variable names), and then read in the same way. The file replaces the data in memory, which `clear` allows.

### Example 2: labels that are not Stata names

Account labels are used as names by **[CGE]** `cge_map`, so they must be valid Stata names. When Stata loads a wide file, it renames a variable whose label is not a valid name, and `cge_sam` then uses the variable name and displays the change. In the supply–use layout of the synthetic SAM, savings–investment is labelled `s-i`:

```
. cge_synthetic, clear sut
(20 vars, 19 obs)

Synthetic SAM loaded (supply-use layout): 19 accounts, total   1952.00
Largest row-column imbalance:  0.00e+00

. cge_sam, labels(account)
Account label s-i is not a valid Stata name; using si
SAM read: 19 accounts, total 1952
Largest row-column imbalance:  0.00e+00
```

A row label that is a valid name but differs from its variable is an error.

### Example 3: long data

In long data, each observation is a cell, with string variables for the row and column accounts and a numeric variable for the value. Cells that are not listed are zero, so the data need only list the non-zero cells, and the number of accounts is limited only by memory. A small SAM of five accounts, with one cell whose value is missing:

```
. clear

. input str8 row str8 column double value

           row     column       value
  1. "act"  "com"  100
  2. "com"  "act"   40
  3. "com"  "hhd"   50
  4. "com"  "s-i"   10
  5. "lab"  "act"   60
  6. "hhd"  "lab"   60
  7. "s-i"  "hhd"   10
  8. "hhd"  "com"    .
  9. end

. cge_sam, long(row column value)
Account label s-i is not a valid Stata name; using si
SAM read: 5 accounts, total 330
Blank cells treated as zero: 1
Largest row-column imbalance:  0.00e+00
```

The accounts are ordered as they first appear as row accounts, followed by any accounts that appear only as column accounts, so that a SAM written in long form by **[CGE]** `cge_samexport` and read again keeps its order. A cell given twice is an error. Labels that are not valid Stata names are converted by the rule given in Methods and formulas, here `s-i` to `si`, and each change is displayed.

### Example 4: a wide CSV file read directly

The direct reader parses a wide CSV file in Mata. The first line holds the column labels, the first field of each other line the row label and the other fields the cells; blank lines are skipped, and fields may be quoted. The row labels must agree with the column labels, in the same order, and are converted to Stata names as in long data. The data in memory are left as they are:

```
. cge_synthetic, clear
(16 vars, 15 obs)

Synthetic SAM loaded (standard layout): 15 accounts, total   1743.00
Largest row-column imbalance:  0.00e+00

. cge_sam using sam.csv, direct
SAM read: 15 accounts, total 1743
Largest row-column imbalance:  0.00e+00

. describe, short

Contains data
  obs:            15
 vars:            16
 size:           390
Sorted by:
     Note:  dataset has changed since last saved
```

This is the only form that can read a wide SAM with more accounts than Stata can hold as variables (`c(maxvar)`, 2,048 in Stata/IC): `cge_sam` counts the fields of the first line of a CSV file and reads it directly when they do not fit, displaying a note.

A blank field, or a field that holds only a period, is a zero, counted as a blank cell. A field that is not a number is an error, never a zero. A thousands separator is the most common case:

```
. type bad.csv
account,act,com
act,0,"1,234"
```

```
com,"1,234",0

. cge_sam using bad.csv, direct
row act, column com: 1,234 is not a number

r(459);
```

## Stored results

`cge_sam` stores the following in `r()`:

| | |
|---|---|
| *Scalars* | |
| `r(N_accounts)` | number of accounts |
| `r(total)` | sum of all cells |
| `r(maxdiff)` | largest difference between an account's row and column totals |
| `r(N_blank)` | number of blank cells read as zero |
| *Macros* | |
| `r(source)` | `wide`, `direct` or `long` |
| `r(labels)` | variable with the account labels (wide data) |

## Methods and formulas

With $x_{ij}$ the cell in row $i$ and column $j$ of a SAM of $n$ accounts, `cge_sam` reports

$$\texttt{r(total)} = \sum_{i=1}^{n}\sum_{j=1}^{n} x_{ij}, \qquad \texttt{r(maxdiff)} = \max_{k}\left|\sum_{j=1}^{n} x_{kj} - \sum_{i=1}^{n} x_{ik}\right|,$$

the largest absolute difference between an account's receipts and its payments.

In long data and in the direct reader, a label that is not a valid Stata name is converted: characters other than letters, digits and the underscore are removed, then any leading digits, and the result is cut to 32 characters. A label for which this gives no valid name (an empty string or a reserved word), or the same name as another label, is an error that names the label. In wide data loaded from a file, Stata's own renaming applies, and the variable names are used. The three readers are described in section 24.1.

## Also see

**[CGE]** `cge_synthetic` — Load the synthetic social accounting matrix
**[CGE]** `cge_template` — Write a template that maps SAM accounts to roles and groups
**[CGE]** `cge_aggregate` — Merge SAM accounts into groups
**[CGE]** `cge_balance` — Balance a SAM by RAS or cross entropy
**[CGE]** `cge_samexport` — Write the SAM to a file or to memory
**[CGE]** `cge_map` — Assign SAM accounts to model roles
**Chapter 24** — SAM tools

# `cge_samexport` — Write the SAM to a file or to memory

## Description

`cge_samexport` writes the SAM of the current model, as read, aggregated or balanced, to a `.csv`, `.dta`, `.xls` or `.xlsx` file, or with `clear` loads it as data in memory. With `mapped`, it writes instead the SAM to which the model is calibrated: the SAM after the layout rewrites of **[CGE]** `cge_map`, with its internal tax accounts. The SAM is written in wide form, one row and one column per account, or in long form, one observation per non-zero cell.

## Quick start

Write the SAM to a CSV file, replacing the file if it exists

```
cge_samexport using sam_balanced.csv, replace
```

Write the SAM to the worksheet `balanced` of an Excel file

```
cge_samexport using sam.xlsx, sheet("balanced") replace
```

Replace the data in memory with the SAM in long form

```
cge_samexport, clear long
```

Replace the data in memory with the SAM after the layout rewrites of the mapping

```
cge_samexport, clear mapped
```

## Syntax

**`cge_samexport`** [`using` *filename*] [, *options*]

*filename* ends in `.csv`, `.dta`, `.xls` or `.xlsx`. `using` *filename*, `clear` or both must be given.

| *options* | Description |
|---|---|
| *Main* | |
| `long` | write long output, one observation per non-zero cell |
| `mapped` | write the SAM after the layout rewrites of `cge_map` |
| `clear` | replace the data in memory with the SAM |
| *Files* | |
| `sheet(`*string*`)` | Excel worksheet; default `SAM` |
| `replace` | allow an existing file to be replaced |

## Options

`long` writes long output: string variables `row` and `column` and a double `value`, one observation per non-zero cell. The default is wide output: a string variable `account` and one double variable per account.

`mapped` writes the SAM after the layout rewrites of **[CGE]** `cge_map` (section 9.2), including the internal tax accounts that it adds, such as `_saltax` for sales taxes paid directly to the government. The model must be mapped.

`clear` replaces the data in memory with the SAM. It may be combined with `using`, to write the file and load the data.

`sheet(`*string*`)` names the worksheet of an Excel file. The default is `SAM`.

`replace` allows an existing file to be replaced.

## Remarks and examples

A `.csv` file is written directly in Mata and has no limit on the number of accounts. A `.dta` or Excel file in wide form, and wide data in memory, need the accounts to fit as variables (`c(maxvar)`, 2,048 in Stata/IC); long output has no such limit. Writing a file leaves the data in memory unchanged, unless `clear` is also given.

### Example 1: a SAM written and read again

The unbalanced synthetic SAM is balanced and written to a CSV file, which **[CGE]** `cge_sam` then reads:

```
. cge_synthetic, clear unbalanced
(18 vars, 17 obs)

Synthetic SAM loaded (institutions layout, unbalanced for the balancing tests): 17 accounts, total
>  1860.46
Largest row-column imbalance:  1.21e+01

. cge_sam, labels(account)
SAM read: 17 accounts, total 1860.4569
Largest row-column imbalance:  1.21e+01

. quietly cge_balance

. cge_samexport using balanced.csv, replace
SAM saved to balanced.csv: 17 accounts

. cge_sam using balanced.csv, clear
(18 vars, 17 obs)
SAM read: 17 accounts, total 1860.4569
Largest row-column imbalance:  1.53e-08
```

The SAM read back has the total and the largest imbalance of the balanced SAM. Numbers are written with as many digits as they need to be read back as the same values, so that a SAM written and read again is unchanged.

### Example 2: long data

With `clear` and `long`, the SAM replaces the data in memory, one observation per non-zero cell, by row and, within each row, by column:

```
. cge_samexport, clear long
SAM loaded as data: 68 observations

. list in 1/6, noobs sep(0)

  +-------------------------------+
  |     row    column       value |
  |-------------------------------|
  | act_agr   com_agr    96.19955 |
  | act_man   com_man    138.6652 |
  | act_ser   com_ser   152.68605 |
  | com_agr   act_agr   9.6749097 |
  | com_agr   act_man   20.354716 |
  | com_agr   act_ser   5.4718925 |
  +-------------------------------+
```

**[CGE]** `cge_sam` reads such data with `long(row column value)`.

### Example 3: the mapped SAM

In the national-accounts layout of the synthetic SAM, sales and direct taxes are paid directly to the government account. **[CGE]** `cge_map` routes them through the internal accounts `_saltax` and `_dirtax`, which the model uses as tax accounts. The mapped SAM shows them:

```
. cge_synthetic, clear national
(14 vars, 13 obs)
```

```
Synthetic SAM loaded (national-accounts layout): 13 accounts, total   1613.00
Largest row-column imbalance:  0.00e+00

. cge_sam, labels(account)
SAM read: 13 accounts, total 1613
Largest row-column imbalance:  0.00e+00

. quietly cge_map, activities(act_agr act_man act_ser) commodities(com_agr com_man com_ser) ///
>     factors(lab cap) households(hhd) government(gov) savinv(sav) row(row)             ///
>     dirtax(gov) saltax(gov) imptax(tax_imp)

. cge_samexport, clear mapped long
SAM loaded as data: 48 observations

. list if inlist("_dirtax", row, column) | inlist("_saltax", row, column), noobs sep(0)

  +---------------------------+
  |     row    column   value |
  |---------------------------|
  |     gov   _dirtax      25 |
  |     gov   _saltax      30 |
  | _dirtax       hhd      25 |
  | _saltax   com_agr       8 |
  | _saltax   com_man      12 |
  | _saltax   com_ser      10 |
  +---------------------------+
```

The SAM as read is not changed: without `mapped`, `cge_samexport` writes it, with its 13 accounts.

## Methods and formulas

Each non-zero number in a CSV file is written with the format `%21.0g` or, if that does not read back as the same double, with `%24.17g`; zeros are written as `0`. A wide CSV file has the line `account,`*label*`,`*label*... followed by one line per account, its label and its row of cells; a long CSV file has the line `row,column,value` followed by one line per non-zero cell. Chapter 24 describes the output in section 24.2.

## Also see

**[CGE]** `cge_sam` — Read a social accounting matrix
**[CGE]** `cge_balance` — Balance a SAM by RAS or cross entropy
**[CGE]** `cge_aggregate` — Merge SAM accounts into groups
**[CGE]** `cge_map` — Assign SAM accounts to model roles
**Chapter 24** — SAM tools

# `cge_save` — Save the model and its simulations to a file

## Description

`cge_save` writes the model in memory to a file: the SAM, the mapping, the calibrated parameters, the closure, the current shocks and every stored simulation. **[CGE]** `cge_use` loads it again, so that a calibrated model need not be rebuilt in a later session and its simulations can be reported without solving them again.

## Quick start

Save the model as `mymodel.cgem`, replacing the file if it exists

```
cge_save using mymodel, replace
```

Load it again

```
cge_use using mymodel
```

## Syntax

`cge_save using` *filename* [`, replace`]

If *filename* has no extension, `.cgem` is added. Only the file name counts: a period in the name of a folder is not an extension.

## Options

`replace` allows an existing file to be overwritten.

## Remarks and examples

A model may be saved at any stage after **[CGE]** `cge_sam`: with the SAM only, after **[CGE]** `cge_map`, or calibrated. A calibrated model is saved with everything that later commands set: the closure and its swaps (**[CGE]** `cge_closure`), the current shocks (**[CGE]** `cge_shock`), the constraints (**[CGE]** `cge_constraint`), the user variables and equations (**[CGE]** `cge_variable`, **[CGE]** `cge_equation`), and every simulation stored by **[CGE]** `cge_solve`, **[CGE]** `cge_dynamic` or **[CGE]** `cge_optimize`, with the parameters and closure under which each was solved. The dataset in memory is not saved.

### Example 1: saving a calibrated model

The synthetic SAM is mapped and calibrated as in **[CGE]** `cge_solve`, and the removal of import tariffs is solved as simulation `notariff`:

```
. cge_save using mymodel, replace
Model saved to mymodel.cgem (calibrated, 1 simulation(s))

. return list

macros:
           r(filename) : "mymodel.cgem"
```

The message gives the stage of the model and the number of simulations saved. An existing file is not overwritten without `replace`:

```
. cge_save using mymodel
file mymodel.cgem already exists; use replace

r(602);
```

### Example 2: saving a SAM

A SAM read by **[CGE]** `cge_sam` can be saved before it is mapped:

```
. cge_synthetic, clear
(16 vars, 15 obs)

Synthetic SAM loaded (standard layout): 15 accounts, total   1743.00
Largest row-column imbalance:  0.00e+00

. cge_sam, labels(account)
SAM read: 15 accounts, total 1743
Largest row-column imbalance:  0.00e+00

. cge_save using samonly, replace
Model saved to samonly.cgem (SAM only, 0 simulation(s))
```

### Versions of Stata and of cge-stata

Files are written in Mata's file format, which changed after Stata 13: later versions of Stata read files saved in Stata 13, but Stata 13 cannot read files saved in later versions (tested with StataNow 19.5). Save a model in Stata 13 when it is to be shared with users of Stata 13.

Each item of the model is stored with its name, so a newer version of cge-stata can read files saved by an older one; **[CGE]** `cge_use` upgrades such a model, and ignores, with a note, items that the current version no longer uses.

## Stored results

`cge_save` stores the following in `r()`:

*Macros*

| | |
|---|---|
| `r(filename)` | name of the file written |

## Methods and formulas

The file is a sequence of Mata matrices written by `fputmatrix()`: the identifier `cge-stata model`, the version of cge-stata that wrote it, and then pairs of a name and a value, one pair for each item of the model, ending with the name `__end__`. The parameters are stored item by item for the base, for the current shocks and for each simulation.

## Also see

**[CGE]** `cge_use` — Load a model saved with cge_save
**[CGE]** `cge_report` — Standard report tables for one or more simulations
**[CGE]** `cge` — Computable general equilibrium modelling in Stata

# `cge_sensitivity` — Sensitivity of a scenario's results to the elasticities

## Description

`cge_sensitivity` shows how much the results of a scenario depend on the elasticities. For each of a number of draws, it multiplies the elasticities by random factors around one, calibrates the model again from the SAM, restores the closure, runs a program that applies the scenario's shocks, and solves. It then summarises every report measure of **[CGE]** `cge_report` across the draws that converged: the result with the central elasticities, the mean, the standard deviation and the 5th and 95th percentiles. The model in memory, with its simulations, is restored exactly at the end.

## Quick start

200 draws of every value-added, Armington and CET elasticity within 50 per cent of its central value, for the shocks applied by program `tariff_cut`

```
cge_sensitivity, scenario(tariff_cut) draws(200)
```

As above, with factors drawn within 25 per cent from a triangular distribution, and every report table displayed

```
cge_sensitivity, scenario(tariff_cut) draws(200) spread(0.25) distribution(triangular) tables(all)
```

Vary only the Armington and CET elasticities, with one factor for each type applied to every commodity, and load every draw as data

```
cge_sensitivity, scenario(tariff_cut) draws(200) vary(q t) common clear
```

Save every draw to `sens.dta` without displaying the summary

```
cge_sensitivity, scenario(tariff_cut) draws(200) saving(sens, replace) nodisplay
```

## Syntax

**`cge_sensitivity`**, `scenario(`*program*`)` `draws(#)` [*options*]

| *options* | Description |
|---|---|
| *Main* | |
| `scenario(`*program*`)` | program that applies the scenario's shocks; required |
| `draws(#)` | number of draws; required |
| `seed(#)` | random-number seed; default `12345` |
| *Draws* | |
| `spread(#)` | factors drawn between $1 - \#$ and $1 + \#$; default `0.5` |
| `distribution(`*name*`)` | `uniform` or `triangular`; default `uniform` |
| `common` | one factor for each type of elasticity, applied to every sector |
| `vary(`*list*`)` | elasticities to vary: `va`, `q`, `t`, `a`; default `va q t` |
| *Reporting* | |
| `tables(`*list*`)` | report tables to display; default `macro welfare` |
| `nodisplay` | suppress the summary |
| `clear` | replace the data in memory with every draw |
| `saving(`*filename*[`, ` *`replace`*]`)` | save every draw to a Stata dataset |
| `keep(`*name*`)` | keep every draw as a simulation |

## Options

`scenario(program)` names a program, without arguments, that applies the scenario's shocks with **[CGE]** `cge_shock` to the freshly calibrated model. It is run quietly for every draw. If it fails, the model is restored and the command stops.

`draws(#)` sets the number of draws, at least 1. Draw 0, with the central elasticities, is always added, so the model is calibrated and solved # + 1 times.

`seed(#)` sets the random-number seed; the default is 12345. It sets the seed of Stata's random-number generator. The same seed gives the same draws, whichever elasticities are varied.

`spread(#)` sets the range of the factors, from $1 - \#$ to $1 + \#$; the default is 0.5. # must be at least 0 and less than 1, so that every drawn elasticity keeps its sign.

`distribution(name)` draws the factors from a `uniform` distribution on that range, the default, or from a symmetric `triangular` distribution on the same range, which puts more weight near the central values.

`common` draws one factor for each type of elasticity and each draw, applied to every sector, so that the elasticities of a type move together. By default, each sector's elasticity is drawn separately.

`vary(list)` lists the elasticities to vary: `va`, the elasticity of substitution in value added of each activity; `q`, the Armington elasticity of each commodity; `t`, the CET elasticity of each commodity, which also applies to exports drawn from composite supply; and `a`, the elasticity of the top nest of each activity. The default is `va q t`. A Leontief top nest, with elasticity 0, stays Leontief. Other elasticities and calibration settings keep their values.

`tables(list)` lists the tables of **[CGE]** `cge_report` whose measures are displayed in the summary, or `all`; the default is `macro welfare`. The dataset of `clear` and `saving()` holds every table.

`nodisplay` suppresses the summary. The progress of the draws is still displayed.

`clear` replaces the data in memory with the results described under Remarks and examples below.

`saving(filename[, replace])` saves the same dataset as a Stata dataset, leaving the data in memory unchanged unless `clear` is also given; `replace` allows an existing file to be replaced.

`keep(name)` (from version 0.8.3) keeps the solution of every draw that converged as a simulation of the model: *name*`0`, with the central elasticities, *name*`1`, *name*`2`, and so on, each with the parameters of its draw. **[CGE]** `cge_microsim` with `bands(name)` then gives the range of poverty and inequality across the draws, and any command that takes a simulation, such as **[CGE]** `cge_results`, can use them. Simulations of the same names are replaced.

## Remarks and examples

Elasticities are rarely estimated for the country and period of a SAM, and a scenario's results can depend on them. `cge_sensitivity` solves the scenario again for many sets of elasticities around those used. Each draw calibrates the model from the SAM with the drawn elasticities and the other settings of the last **[CGE]** `cge_calibrate`, so that every draw replicates the SAM; it then restores the closure, including swaps, and any constraints, runs the scenario program and solves. A model calibrated by a version of cge-stata that did not record its calibration settings must be calibrated again first (error 459). The solution of each draw is summarised by the report measures of **[CGE]** `cge_report`: percentage changes from the base, changes in points for rates, and per cent of base consumption for the welfare measures.

The draws do not estimate uncertainty about the elasticities. They show how the results change over a range chosen by the user, so the range, the distribution and whether sectors move together should be stated with the results (chapter 21). Standard errors of published estimates, such as those of **[CGE]** `cge_elasticities`, can guide the choice of range.

### Example 1: the elasticities of every sector drawn separately

The synthetic SAM is calibrated as in **[CGE]** `cge_solve`, with `sigmava(0.8) sigmaq(2) sigmat(2)`. The scenario removes every import tariff; its program applies the shock and nothing else:

```
. program define tariff_cut
  1.     cge_shock tm, value(0)
  2. end
```

Twenty draws, each elasticity of each sector drawn uniformly within 50 per cent of its central value:

```
. cge_sensitivity, scenario(tariff_cut) draws(20) seed(2026)
Sensitivity analysis: 20 draws of the elasticities (uniform within +/-50 per cent)
  10 draws done
  20 draws done

Sensitivity of the scenario to the elasticities: 20 of 20 draws converged
Per cent changes from the base (welfare: per cent of base consumption)

macro
--------------------------------------------------------------------------------
                             central       mean         sd         p5        p95
gdp_nominal                   -0.305     -0.299      0.042     -0.386     -0.246
gdp_real                       0.081      0.079      0.014      0.056      0.104
gdp_deflator                  -0.386     -0.378      0.034     -0.453     -0.333
consumption_real               2.684      2.687      0.022      2.638      2.717
government_real                0.000      0.000      0.000      0.000      0.000
investment_real              -11.861    -11.883      0.072    -11.980    -11.749
insurance_fund_real                .          .          .          .          .
exports_real                   5.644      5.590      0.751      4.436      6.940
imports_real                   4.233      4.192      0.563      3.327      5.205
value_added_real              -0.000     -0.000      0.000     -0.000     -0.000
cpi                            0.000      0.000      0.000      0.000      0.000
dpi                            1.013      1.041      0.137      0.725      1.233
exchange_rate                  4.759      4.668      0.432      4.010      5.271
real_exchange_rate             3.709      3.591      0.569      2.718      4.381
private_savings                2.652      2.654      0.021      2.608      2.683
investment_value             -13.157    -13.180      0.089    -13.312    -13.031
foreign_savings                0.000      0.000      0.000      0.000      0.000
--------------------------------------------------------------------------------

welfare
----------------------------------------------------------------------------------
                               central       mean         sd         p5        p95
equivalent_variation hhd         2.668      2.670      0.021      2.623      2.700
compensating_variation hhd       2.668      2.670      0.021      2.623      2.700
----------------------------------------------------------------------------------
```

The column `central` is the result of the scenario with the central elasticities, the result of **[CGE]** `cge_solve`; the other columns summarise the 20 draws. Real consumption and household welfare vary little across the draws, by a standard deviation of about 0.02 points around 2.7 per cent; real exports and imports, and the exchange rate, vary much more. With 20 draws, p5 is the smallest result and p95 the second largest; with more draws the percentiles are more precise. The results come from a synthetic SAM and illustrative elasticities and are not findings about any economy.

### Example 2: common factors, and the draws as data

With `common`, the Armington and CET elasticities of every commodity are multiplied by the same factor in each draw, here drawn from a triangular distribution; only the welfare table is displayed, and every draw is loaded:

```
. cge_sensitivity, scenario(tariff_cut) draws(20) seed(2026) vary(q t) common ///
>     distribution(triangular) tables(welfare) clear
Sensitivity analysis: 20 draws of the elasticities (triangular within +/-50 per cent)
  10 draws done
  20 draws done

Sensitivity of the scenario to the elasticities: 20 of 20 draws converged
Per cent changes from the base (welfare: per cent of base consumption)

welfare
----------------------------------------------------------------------------------
                               central       mean         sd         p5        p95
equivalent_variation hhd         2.668      2.660      0.021      2.618      2.685
compensating_variation hhd       2.668      2.659      0.021      2.617      2.684
----------------------------------------------------------------------------------
```

The dataset holds the report measures of each draw that converged, draw 0 included, and the elasticities of every draw (table `elasticity`). With `common`, the Armington elasticities are the same for every commodity in each draw:

```
. tabulate table

Report table, or |
      elasticity |      Freq.     Percent        Cum.
-----------------+-----------------------------------
      activities |        189       11.39       11.39
     commodities |        378       22.78       34.18
      elasticity |        252       15.19       49.37
         factors |        168       10.13       59.49
      government |        231       13.92       73.42
    institutions |         42        2.53       75.95
           macro |        357       21.52       97.47
         welfare |         42        2.53      100.00
-----------------+-----------------------------------
           Total |      1,659      100.00

. list draw measure element value if table == "elasticity" & draw <= 2 & measure == "sigmaq", ///
>     noobs sepby(draw)

  +---------------------------------------+
  | draw   measure   element        value |
  |---------------------------------------|
  |    0    sigmaq   com_agr            2 |
  |    0    sigmaq   com_man            2 |
  |    0    sigmaq   com_ser            2 |
  |---------------------------------------|
  |    1    sigmaq   com_agr    1.9453829 |
  |    1    sigmaq   com_man    1.9453829 |
  |    1    sigmaq   com_ser    1.9453829 |
  |---------------------------------------|
  |    2    sigmaq   com_agr    2.4448375 |
  |    2    sigmaq   com_man    2.4448375 |
  |    2    sigmaq   com_ser    2.4448375 |
  +---------------------------------------+

. summarize pct if measure == "equivalent_variation" & draw > 0

    Variable |        Obs        Mean    Std. Dev.       Min        Max
-------------+---------------------------------------------------------
         pct |         20    2.659612    .0210667   2.617638   2.708059
```

The mean and standard deviation of the equivalent variation over draws 1 to 20 are those of the summary.

### The dataset

The dataset of `clear` and `saving()` has the variables

| Variable | Content |
|---|---|
| `draw` | the draw; 0 for the central elasticities |
| `table` | the report table, or `elasticity` |
| `measure` | the report measure, or `sigmava`, `sigmaq`, `sigmat` or `sigmaa` |
| `element` | the set element |
| `base` | the base value of the measure |
| `value` | its value in the draw, or the value of the elasticity |
| `pct` | the per cent change from the base (rates: points; welfare: per cent of base consumption) |

Report measures are included for the draws that converged; elasticities for every draw, with `base` and `pct` missing. A draw that does not calibrate or does not solve is counted as not converged and left out of the summary. If the scenario does not solve with the central elasticities, the command stops with error 430 after restoring the model.

### The model in memory

The model is saved before the first draw and loaded again after the last, so the elasticities, closure, shocks and simulations in memory are those before the command. The simulation that each draw solves is not kept. The time taken is about # + 1 times that of a calibration and a solve.

## Stored results

`cge_sensitivity` stores the following in `r()`:

*Scalars*

| | |
|---|---|
| `r(N_draws)` | number of draws |
| `r(N_converged)` | number of draws that converged |

## Methods and formulas

Let $\sigma_k^0$ be the central value of elasticity $k$ of a varied type, and $s$ the value of `spread()`. In draw $d = 1, \ldots, D$,

$$\sigma_k^d = \sigma_k^0\,(1 + s\,u_{kd}), \qquad u_{kd} = \begin{cases} 2\,U_{kd} - 1 & \text{uniform,} \\ U_{kd} + U'_{kd} - 1 & \text{triangular,} \end{cases}$$

where $U_{kd}$ and $U'_{kd}$ are independent uniform draws on $(0,1)$, so that $u_{kd}$ is uniform, or symmetric triangular with density $1 - |u|$, on $(-1,1)$. With `common`, $u_{kd} = u_d$ for every element $k$ of a type. The draws are made after the seed is set, for the four types in a fixed order (value added, Armington, CET, top nest), whether or not a type is varied, so that a seed gives the same factors for a type whichever types are varied. $\sigma_k^0 = 0$ gives $\sigma_k^d = 0$.

For a report measure with results $y_d$ in the set $S$ of draws $d \geq 1$ that converged, $n = |S|$, the summary gives the central result $y_0$,

$$\bar{y} = \frac{1}{n} \sum_{d \in S} y_d, \qquad s_y = \left[ \frac{1}{n-1} \sum_{d \in S} (y_d - \bar{y})^2 \right]^{1/2},$$

and the order statistics $y_{(\lceil 0.05\,n \rceil)}$ and $y_{(\lceil 0.95\,n \rceil)}$ as the 5th and 95th percentiles, with $y_{(1)} \leq \cdots \leq y_{(n)}$. Chapter 21 discusses the method and its interpretation.

## Also see

**[CGE]** `cge_calibrate` — Calibrate the model to the SAM
**[CGE]** `cge_report` — Standard report tables for one or more simulations
**[CGE]** `cge_elasticities` — Published elasticities, with their sources
**[CGE]** `cge_shock` — Change a parameter or a fixed variable
**Chapter 21** — Sensitivity analysis

# `cge_set` — User sets of elements, for indexes and sums

## Description

`cge_set` defines a set of elements of the calibrated model, such as the goods that are traded, the health commodities or the rural households. Its name then serves as an index wherever the letter of a model set does: in the definitions of user variables (**[CGE]** `cge_variable`), in user equations (**[CGE]** `cge_equation`), in sums, and in the indexes of user parameters (**[CGE]** `cge_parameter`). From version 0.9.1, a set of pairs records a mapping between two sets, such as the activities and the commodities they make, for sums and conditions.

## Quick start

The commodities other than services and health

```
cge_set goods = c, except(com_ser com_hlt)
```

Government spending on them, as a user variable

```
cge_variable GGOODS = sum(goods, PQ(goods) * QG(goods)), nominal
```

Activities paired with the commodities they make, and the value of their output

```
cge_set map(a,c) = act_agr.com_agr act_hlt.com_hlt
cge_variable VMAP = sum(map(a,c), PA(a) * QA(a)), nominal
```

Display the user sets

```
cge_set, list
```

## Syntax

Define a set

**`cge_set`** *name* = *elements* [, `except`(*elements*)]

Define a set of pairs (from version 0.9.1)

**`cge_set`** *name*(*i*,*j*) = *e1*.*f1* [*e2*.*f2* ...]

Display or remove the user sets

**`cge_set`** , `list`

**`cge_set`** , `clear`

*elements* are labels of elements of the model (accounts, such as `com_hlt`, or nests), letters of model sets (`a` activities, `c` commodities, `f` factors, `h` households, `d` households and enterprises, `i` institutions receiving factor income, `n` nests of value added), and names of user sets, separated by spaces. *name* has at most 24 characters, letters, digits and underscores; it may not be the name of a variable, a parameter, an element, a user equation or another user set, nor an index letter of a model set. In a set of pairs, *i* and *j* are two different indexes, model set letters or user sets, and each pair *e*.*f* joins an element of the set of *i* to one of the set of *j*.

| *options* | Description |
|---|---|
| *Main* | |
| `except`(*elements*) | elements left out |

## Options

`except(`*elements*`)` leaves out the elements listed, given as for the set.

`list` displays the user sets and their elements.

`clear` removes every user set. A set that a user variable, equation or parameter uses cannot be removed until they are.

## Remarks and examples

The SAM with a health sector is mapped and calibrated:

```
. cge_synthetic, clear health
(19 vars, 18 obs)

Synthetic SAM loaded (health layout): 18 accounts, total   1746.00
Largest row-column imbalance:  0.00e+00

. cge_sam, labels(account)
SAM read: 18 accounts, total 1746
Largest row-column imbalance:  0.00e+00

. cge_map, activities(act_agr act_man act_ser act_hlt)                       ///
>     commodities(com_agr com_man com_ser com_hlt) factors(lab cap)          ///
>     households(hhd) government(gov) savinv(sav) row(row) dirtax(tax_dir)   ///
>     saltax(tax_sal) imptax(tax_imp) insurance(shi)
Accounts mapped: 4 activities, 4 commodities, 2 factors, 1 households, 0 enterprises, 0 margin accou
> nts
Health insurance fund: shi

. cge_calibrate, sigmava(0.8) sigmaq(2) sigmat(2)
Calibrated: 117 equations, 117 free variables
Benchmark replication: largest scaled residual  3.55e-16
```

### Example 1: sets of commodities

The elements of a set are written out in the order given, each once, without those in `except()`; a model set letter stands for all its elements:

```
. cge_set goods = c, except(com_ser com_hlt)
User set goods: 2 element(s): com_agr com_man

. cge_set services = com_ser com_hlt
User set services: 2 element(s): com_ser com_hlt

. return list

scalars:
         r(N_elements) =  2

macros:
           r(elements) : "com_ser com_hlt"
               r(name) : "services"
```

### Example 2: sums and variables over a set

A user set's name is an index, in sums and in the definitions of variables and equations. Government purchases of goods at market prices, and the prices of services relative to the consumer price index, after a tariff removal:

```
. cge_variable GGOODS = sum(goods, PQ(goods) * QG(goods)), nominal
User variable GGOODS: 1 element(s), base value 5
and its definition, equation GGOODS

. cge_variable PREL(services) = PQ(services) / CPI
User variable PREL(services): 2 element(s), base values 1 1
and its definition, equation PREL
```

```
. cge_shock tm, value(0)
Parameter tm changed for 4 element(s)

. cge_solve, name(notariff)
Simulation notariff solved in 7 iteration(s) (newton)
Largest scaled residual  3.20e-15; WALRAS =  1.27e-14

. cge_results, sim(notariff) clear nodisplay

. list variable element1 base value if inlist(variable, "GGOODS", "PREL"), noobs

  +---------------------------------------+
  | variable   element1   base      value |
  |---------------------------------------|
  |   GGOODS                 5   4.8988872 |
  |     PREL    com_ser      1   1.0224253 |
  |     PREL    com_hlt      1   1.0213229 |
  +---------------------------------------+
```

A second index over the same set adds digits to its name, as in `sum(goods, sum(goods2, ...))`.

### Example 3: listing the sets

```
. cge_set, list
set goods (2): com_agr com_man
set services (2): com_ser com_hlt
```

User sets are saved with the model (**[CGE]** `cge_save`) and kept when it is calibrated again.

An index that is neither a model set letter nor a user set is an error:

```
. cge_variable X = sum(goodz, PQ(goodz))
index goodz is neither a model set (a, c, f, h, d, i, n) nor a user set

r(198);
```

### Example 4: sets of pairs

From version 0.9.1, a set of pairs maps the elements of one set to those of another, as a GAMS set over two indexes does: here the activities and the commodities they make. It is not an index. A sum runs over its pairs, and in a condition it is 1 for a pair of the set, so that a variable with two indexes can be defined for the pairs only:

```
. cge_set map(a,c) = act_agr.com_agr act_man.com_man act_hlt.com_hlt
User set map(a,c): 3 pair(s): act_agr.com_agr act_man.com_man act_hlt.com_hlt

. cge_variable VMAP = sum(map(a,c), PA(a) * QA(a)), nominal
User variable VMAP: 1 element(s), base value 270
and its definition, equation VMAP

. cge_variable PR(a,c)$(map(a,c)) = PA(a) / PQ(c)
User variable PR(a,c)$(map(a,c)): 3 element(s), base values 1 1 1
and its definition, equation PR

. cge_solve, name(notariff2)
Simulation notariff2 solved in 7 iteration(s) (newton)
Largest scaled residual  5.57e-15; WALRAS =  1.74e-14

. quietly cge_results, sim(notariff2) clear nodisplay

. list variable element1 element2 base value if inlist(variable, "VMAP", "PR"), ///
>     noobs sepby(variable)

  +--------------------------------------------------+
  | variable   element1   element2   base      value |
  |--------------------------------------------------|
  |     VMAP                          270   272.23147 |
  |--------------------------------------------------|
  |       PR    act_agr    com_agr      1   1.0208505 |
  |       PR    act_man    com_man      1    1.0339964 |
  |       PR    act_hlt    com_hlt      1            1 |
  +----------------------------------------------------+

. cge_set, list
set goods (2): com_agr com_man
set services (2): com_ser com_hlt
set map(a,c) (3 pairs): act_agr.com_agr act_man.com_man act_hlt.com_hlt
```

## Stored results

`cge_set` stores the following in `r()`:

| | |
|---|---|
| *Scalars* | |
| `r(N_elements)` | number of elements, or pairs, of the set defined |
| `r(N_sets)` | number of user sets (`list`) |
| *Macros* | |
| `r(name)` | name of the set defined |
| `r(elements)` | its elements |

## Methods and formulas

A user set is an ordered list of labels. An expression is written out for each element of the set of its index, and each sum over the elements of its set (chapter 23), so a user set changes only which elements are written out; it adds no variables or equations of its own. A set of pairs $P$ of the indexes $i$ and $j$ is a list of pairs of labels; a sum over it is written out as

$$\texttt{sum(}P(i,j)\texttt{, }g\texttt{)} = \sum_{k\in I} \sum_{l\in J:\ (k,l)\in P} g\,[\,i=k,\ j=l\,],$$

and in a condition $P(k,l)$ is 1 if $(k,l) \in P$ and 0 if not. In the GAMS program the equations are written out element by element, so user sets need no declaration there.

## Also see

**[CGE]** `cge_parameter` — User parameters for user variables and equations
**[CGE]** `cge_variable` — User variables, defined by an expression or determined by user equations
**[CGE]** `cge_equation` — User equations: new relations among the model's variables
**Chapter 23** — User variables and equations

# `cge_shock` — Change a parameter or a fixed variable

## Description

`cge_shock` changes a parameter of the calibrated model, or a variable that the current closure fixes, before the next **[CGE]** `cge_solve`. Shocks accumulate until `cge_shock, reset` restores every base value.

## Quick start

Remove every import tariff

```
cge_shock tm, value(0)
```

Raise the world import price of manufactures by 10 per cent

```
cge_shock pwm, elements(com_man) scale(1.1)
```

Raise government purchases of health services by 20 per cent

```
cge_shock qg, elements(com_hlt) scale(1.2)
```

Raise the supply of labour, which the default closure fixes, by 5 per cent

```
cge_shock QFS, elements(lab) scale(1.05)
```

Remove every shock

```
cge_shock, reset
```

## Syntax

Change a parameter or a fixed variable

**cge_shock** *name* [, elements(*list*)] {scale(#) | value(#)}

Remove every shock

**cge_shock**, reset

*name* is a parameter, in lower case, from the table under *Remarks and examples*, a user parameter (**[CGE]** `cge_parameter`), or a model variable, in upper case, that the current closure fixes. The elements of a user parameter are those of its first index, each with every element of the second, or pairs *e1*.*e2*.

| *options* | Description |
|---|---|
| *Main* | |
| `elements(`*list*`)` | elements to change; default all |
| `scale(#)` | multiply the current values by # |
| `value(#)` | set the values to # |
| `reset` | remove every shock |

Exactly one of `scale()` and `value()` is required, except with `reset`, which may not be combined with a shock.

## Options

`elements(`*list*`)` lists the elements to change: commodities, activities, factors, or households and enterprises, as the parameter or variable requires; for a variable added by **[CGE]** `cge_variable`, the elements of the set of its index, such as institutions for an index `i`. The default is every element, as is `elements(all)`. For a variable indexed by factor and activity, $QF_{fa}$ or $WFDIST_{fa}$, the list names factors, and every activity of each factor is changed. A parameter or variable without index takes no elements.

`scale(#)` multiplies the current values by #. The current values include earlier shocks, so two shocks of `scale(1.1)` give a rise of 21 per cent.

`value(#)` sets the values to #.

`reset` removes every shock: all parameters and fixed variables return to their base values. The constraints of **[CGE]** `cge_constraint`, and the recipients of their rents, are kept.

## Remarks and examples

A simulation compares an equilibrium under changed parameters or fixed values with the base. `cge_shock` makes the changes; **[CGE]** `cge_solve` then solves the model under them and stores the simulation, with the parameters it used. The shocks stay in memory after the solve, so that further shocks add to them, until `cge_shock, reset`. **[CGE]** `cge_calibrate` also removes them.

### Parameters

The parameters that `cge_shock` changes are listed below with the set over which each is defined; those marked “no elements” are scalars. Values in foreign currency are converted at the exchange rate, and values “in real terms” are indexed to the CPI. The symbols are those of section 6.4.

| Name | Elements | Description |
|---|---|---|
| *Taxes* | | |
| `tm` | commodities | import tariff rates, $tm_c$ |
| `te` | commodities | export tax rates, $te_c$ |
| `tq` | commodities | sales tax rates, $tq_c$ |
| `ta` | activities | activity tax rates, $ta_a$ |
| `tv` | commodities | VAT rates, $tv_c$; needs a VAT account |
| `kappa` | activities | VAT credit shares, $\kappa_a$, between 0 and 1; needs a VAT account |
| `tf` | factors | rates of tax on the use of factors, $tf_{fa}$, in every activity that uses them |
| `tfy` | factors | rates of tax on factor income, $tfy_f$ |
| *World prices* | | |
| `pwm` | commodities | world import prices, $pwm_c$, in foreign currency |
| `pwe` | commodities | world export prices, $pwe_c$; with finite export demand, a shift in world demand |
| *Productivity* | | |
| `ad` | activities | efficiency of value added, $ad_a$, that is productivity |
| `af` | factors | efficiency of each factor in every activity, $af_f$; one in the base |
| *Spending and transfers* | | |
| `qg` | commodities | base quantities of government consumption, $\overline{qg}_c$, which $GADJ$ scales |
| `qinv` | commodities | base quantities of investment, $\overline{qinv}_c$, which $IADJ$ scales |
| `trdg` | households and enterprises | government transfers, $trnsfr_{d,gov}$, in real terms |
| `trdr` | households and enterprises | transfers from the rest of the world, $trnsfr_{d,row}$, in foreign currency |
| `trfr` | factors | factor income from abroad, $trnsfr_{f,row}$, in foreign currency |
| `trgr` | no elements | transfers from the rest of the world to the government, $trnsfr_{gov,row}$, for example budget support, in foreign currency |
| `trrg` | no elements | government transfers to the rest of the world, $trnsfr_{row,gov}$, in foreign currency |
| *Health insurance fund (needs a fund, mapped with `cge_map, insurance()`)* | | |
| `tshi` | factors | contribution rates on factor income, $tshi_f$ |

| Name | Elements | Description |
|---|---|---|
| `qshi` | commodities | base purchases of the fund, $\overline{qshi}_c$, which *SHIADJ* scales |
| `trshi` | no elements | government transfers to the fund, $trnsfr_{shi,gov}$, in real terms |
| `trshir` | no elements | transfers from the rest of the world to the fund, $trnsfr_{shi,row}$, in foreign currency |
| *Public debt (needs debt, from `cge_calibrate, debt() interest()`)* | | |
| `rd`, `rf` | no elements | interest rates on domestic and foreign public debt, $r^d$ and $r^f$ |
| `ddebt` | no elements | domestic public debt at the start of the period, *DD*, in local currency |
| `fdebt` | no elements | foreign public debt at the start of the period, *DF*, in foreign currency; for example debt relief |
| *Labour markets (factors with a labour force, from `cge_calibrate, unemployment()` or `minwage()`)* | | |
| `qlf` | factors | labour force, $QLF_f$ |
| `wmin` | factors | real minimum wage, $\underline{w}_f$ |
| `wref` | factors | reference real wage of the wage curve, $\overline{w}_f$ |
| *Health outcomes (from `cge_calibrate, sickdays()` or `mortality()`)* | | |
| `sick` | factors | share of working time lost to illness before the response to health services, $\sigma_f$; for example an epidemic |
| `mort` | no elements | crude death rate before the response to health services, $\mu^m$ |

A shock to a parameter that the model lacks is an error: `tv` and `kappa` need a VAT account, mapped with `cge_map, vattax()` or `vatshare()` (`vatshare(0)` creates an empty one, so that VAT can be introduced); `rd`, `rf`, `ddebt` and `fdebt` need public debt; `qlf`, `wmin` and `wref` need factors with a labour force; `sick` needs days lost to illness for the factors given, and `mort` a death rate. The values must stay in range: `kappa` between 0 and 1, `sick` and `mort` at least 0 and below 1, and `qlf`, `wmin` and `wref`, when set with `value()`, positive.

Some parameters call for a remark. `qg` and `qinv` are the base quantities that *GADJ* and *IADJ* scale, so changing some of them changes the composition of government consumption or investment. In the default closure government savings adjust to a change in government spending or transfers. `af` multiplies the efficiency of a factor: factor quantities and wages stay in physical units, so with mobile factors a fall of 2 per cent in labour efficiency has the same real effects as 2 per cent less labour. When **[CGE]** `cge_calibrate` links efficiency to health services, more health services also raise the efficiency variable $EFF_f$. For `tf`, `scale()` multiplies each rate of the factors given, and `value()` sets one rate for every activity that uses them, which introduces a tax where the SAM has none. `tshi`, `qshi`, `trshi` and `trshir` change the revenue of the health insurance fund, which spends it less a fixed share saved. Under the closures `wagecurve` and `minwage` of **[CGE]** `cge_closure`, a larger labour force raises unemployment unless the demand for labour grows with it. Doubling `sick` doubles the share of working time lost at base health services per head. A higher death rate changes no result within a period; in a dynamic run it slows the growth of population and labour from the next period.

### Fixed variables

A variable can be changed while the current closure fixes it; changing a variable that the closure leaves free is an error. Only the elements that exist in the model are changed. Any variable element fixed by a swap of **[CGE]** `cge_closure` can be changed, including a user variable defined by **[CGE]** `cge_variable`. The closure options fix the following variables:

| Variable | Fixed under |
|---|---|
| `FSAV`, `EXR` | `external(flexexr)`, `external(fixexr)` |
| `GSAV` | `government(fixgsav)` |
| `TINSADJ`, `TQADJ`, `TMADJ`, `TVADJ` | every government closure, except the factor that `fixgsav` frees |
| `GADJ` | every closure |
| `MPSADJ`, `IADJ` | `savinv(savings)`, `savinv(investment)` |
| `CPI`, `DPI` | `numeraire(cpi)`, `numeraire(dpi)` |
| `QFS` | factors that are `mobile` |
| `W` | factors that are `specific` or `unemployed` |
| `QF` | factors that are `specific` |

| Variable | Fixed under |
|---|---|
| `WFDIST` | factors that are `mobile`, `unemployed`, `wagecurve` or `minwage` |
| `RM`, `RE`, `RA` | every closure, unless a constraint frees them |

The tax scaling factors can therefore be set while the closure fixes them. A premium on imports or exports, or a rent on output, set by hand acts as a tax whose revenue goes to the recipient of the rent, the government unless **[CGE]** `cge_constraint` names another. A shock to the adjusting variable of a constraint changes the level at which it is fixed while the constraint does not bind. Of the variables with two indexes, only $QF_{fa}$ and $WFDIST_{fa}$ can be shocked.

### Example 1: parameters

The synthetic SAM is mapped and calibrated as in **[CGE]** `cge_solve`. Each shock reports the number of elements changed:

```
. cge_shock tm, value(0)
Parameter tm changed for 3 element(s)

. cge_shock pwm, elements(com_man) scale(1.1)
Parameter pwm changed for 1 element(s)

. cge_shock ad, elements(act_agr) scale(1.05)
Parameter ad changed for 1 element(s)
```

### Example 2: shocks accumulate

Shocks build on one another. Two rises of 10 per cent in government purchases of services raise them by 21 per cent, from 40 to 48.4, as the current parameters of **[CGE]** `cge_parameters` show:

```
. cge_shock qg, elements(com_ser) scale(1.1)
Parameter qg changed for 1 element(s)

. cge_shock qg, elements(com_ser) scale(1.1)
Parameter qg changed for 1 element(s)

. quietly cge_parameters, clear current

. list parameter element1 value if parameter == "qg", noobs abbreviate(9)

  +------------------------------+
  | parameter   element1   value |
  |------------------------------|
  |        qg    com_agr       0 |
  |        qg    com_man       5 |
  |        qg    com_ser    48.4 |
  +------------------------------+
```

`cge_shock, reset` removes every shock, those of Example 1 included:

```
. cge_shock, reset
All shocks removed
```

### Example 3: fixed variables

Under the default closure labour supply and foreign savings are fixed, and can be shocked; the exchange rate is free, and cannot:

```
. cge_shock QFS, elements(lab) scale(1.05)
Variable QFS changed for 1 element(s)

. cge_shock FSAV, scale(0.8)
Variable FSAV changed for 1 element(s)

. cge_shock EXR, scale(1.1)
EXR is not fixed under the current closure, so it cannot be set

r(459);
```

The solution keeps the fixed variables at their new values:

```
. cge_solve, name(lab_up)
Simulation lab_up solved in 7 iteration(s) (newton)
Largest scaled residual  1.79e-14; WALRAS =  6.32e-13

. quietly cge_results, sim(lab_up) clear nodisplay

. list variable element1 base value if inlist(variable, "QFS", "FSAV", "EXR"), noobs

  +---------------------------------------+
  | variable   element1   base      value |
  |---------------------------------------|
  |      EXR                 1   1.007757 |
  |      QFS        lab    135     141.75 |
  |      QFS        cap    127        127 |
  |     FSAV                15         12 |
  +---------------------------------------+
```

#### Example 4: parameters the model does not have

The standard synthetic SAM has no health insurance fund, and `trgr` is a scalar:

```
. cge_shock, reset
All shocks removed

. cge_shock tshi, scale(1.1)
tshi needs a health insurance fund: name it with cge_map, insurance()

r(459);

. cge_shock trgr, elements(hhd) scale(2)
trgr has no elements

r(198);
```

#### Example 5: health shocks

In the health layout of **[CGE]** `cge_synthetic`, with a social health insurance fund mapped by `cge_map, insurance(shi)`, government purchases of health services rise by 20 per cent, the contribution rate on labour income by half, and labour efficiency falls by 2 per cent:

```
. cge_shock qg, elements(com_hlt) scale(1.2)
Parameter qg changed for 1 element(s)

. cge_shock tshi, elements(lab) scale(1.5)
Parameter tshi changed for 1 element(s)

. cge_shock af, elements(lab) scale(0.98)
Parameter af changed for 1 element(s)

. cge_solve, name(health)
Simulation health solved in 8 iteration(s) (newton)
Largest scaled residual  6.33e-15; WALRAS =  2.99e-14

. cge_report, sims(health) tables(health)

Health
-------------------------------------------------
                                  base     health
fund_revenue                         9     33.518
fund_volume                          1     32.359
-------------------------------------------------
Percentage changes from the base (rates: changes in points)
```

The higher contribution rate raises the fund's revenue, and the fund buys more health services with it.

## Methods and formulas

Let $\theta_e$ be the current value of element $e$ of a parameter, or of a fixed variable, and $E$ the elements given. `cge_shock` sets

$$\theta_e \leftarrow s\,\theta_e \quad (\texttt{scale}(s)), \qquad \theta_e \leftarrow v \quad (\texttt{value}(v)), \qquad e \in E,$$

except that `value(`$v$`)` for `tf` sets $tf_{fa} = v$ for every activity $a$ that uses factor $f$ in the SAM and leaves the others at zero. `cge_shock, reset` sets every parameter and fixed variable to its base value, $\theta = \theta^0$. **[CGE]** `cge_solve` holds the fixed variables at their current values and uses the current parameters; continuation in $n$ steps solves at $\theta^0 + (k/n)(\theta - \theta^0)$, $k = 1, \ldots, n$.

The parameters enter the equations of chapter 7 and of the extensions: VAT and public debt in chapter 13, export demand in chapter 14, the labour force and wages in chapter 15, health outcomes in chapter 16, taxes on factors in section 18.1, and the rents in section 17.1.

## Also see

**[CGE]** `cge_closure` — Choose the model closure
**[CGE]** `cge_solve` — Solve the model
**[CGE]** `cge_constraint` — Limits that bind in some scenarios and not in others
**[CGE]** `cge_parameters` — The calibrated parameters as a dataset
**Chapter 7** — The equations of the standard model
**Chapter 8** — Closures

# `cge_solve` — Solve the model

## Description

`cge_solve` solves the model under the current closure and shocks, starting from the base, and stores the solution as a named simulation, for **[CGE]** `cge_results`, **[CGE]** `cge_report` and **[CGE]** `cge_check`. Solving again under an existing name replaces that simulation.

## Quick start

Solve the model with the shocks in memory and store the solution as `notariff`

```
cge_solve, name(notariff)
```

As above, with a tighter tolerance, and display the residual at each iteration

```
cge_solve, name(notariff) tolerance(1e-12) trace
```

Apply the shocks in 10 equal steps

```
cge_solve, name(qg_up) steps(10)
```

Display the diagnostics of the Jacobian at the solution

```
cge_solve, name(notariff) diagnose
```

## Syntax

`cge_solve` [, *options*]

| *options* | Description |
|---|---|
| *Main* | |
| `name(`*name*`)` | name of the simulation; default `sim#` |
| `tolerance(#)` | tolerance on the largest scaled residual; default `1e-10` |
| `iterate(#)` | maximum number of Newton iterations; default `100` |
| `steps(#)` | apply the shocks in # equal steps; default `0`, only after a direct solve fails |
| *Reporting* | |
| `trace` | display the largest scaled residual at each iteration |
| `diagnose` | display the diagnostics after a solve that converges |
| *Advanced* | |
| `full` | solve the full system of equations instead of the reduced one |

## Options

`name(`*name*`)` names the simulation. The default is `sim#`, with # one more than the number of simulations stored.

`tolerance(#)` sets the convergence tolerance on the largest scaled residual; the default is `1e-10`. Each equation is scaled by its base magnitude, so the tolerance applies alike to every block of the model.

`iterate(#)` sets the maximum number of Newton iterations; the default is 100.

`steps(#)` applies the shocks in # equal steps, each solution the start of the next. The default, `steps(0)`, does so only after a direct solve fails, first in 4 steps and then in 16; `steps(1)` never does.

`trace` displays the largest scaled residual, the number of Jacobians computed and the number of Broyden updates at each iteration.

`diagnose` displays the diagnostics described below after a solve that converges. They are always displayed after a solve that fails.

`full` solves the full system, with every equation, instead of the reduced system, from which explicit definitions are left out. Both give the same solution; `full` is for checks, and is much slower for large models.

## Remarks and examples

A simulation is an equilibrium of the model under the closure set by **[CGE]** `cge_closure` and the shocks set by **[CGE]** `cge_shock`. `cge_solve` finds it by Newton's method, starting from the base, and stores every variable of the solution with the closure, the shocks and the solution's residual. Chapter 10 describes the method; the remarks below show how it is used.

### Example 1: a tariff removal

The synthetic SAM is mapped and calibrated with uniform elasticities:

```
. cge_synthetic, clear
(16 vars, 15 obs)

Synthetic SAM loaded (standard layout): 15 accounts, total   1743.00
Largest row-column imbalance:  0.00e+00

. cge_sam, labels(account)
SAM read: 15 accounts, total 1743
Largest row-column imbalance:  0.00e+00

. cge_map, activities(act_agr act_man act_ser) commodities(com_agr com_man com_ser) ///
>     factors(lab cap) households(hhd) government(gov) savinv(sav) row(row)       ///
>     dirtax(tax_dir) saltax(tax_sal) imptax(tax_imp)
Accounts mapped: 3 activities, 3 commodities, 2 factors, 1 households, 0 enterprises, 0 margin accou
> nts

. cge_calibrate, sigmava(0.8) sigmaq(2) sigmat(2)
Calibrated: 93 equations, 93 free variables
Benchmark replication: largest scaled residual  3.55e-16
```

Removing every import tariff and solving takes seven iterations:

```
. cge_shock tm, value(0)
Parameter tm changed for 3 element(s)

. cge_solve, name(notariff)
Simulation notariff solved in 7 iteration(s) (newton)
Largest scaled residual  4.49e-14; WALRAS = -1.56e-14
```

The largest scaled residual, over every equation of the model, is below the default tolerance of `1e-10`. `WALRAS` is the slack of the savings–investment balance, the equation that Walras's law makes redundant (section 7.5); it is zero, to the precision of the solution, at every solution. The simulation can now be reported:

```
. cge_results, sim(notariff)
. cge_report, sims(notariff) tables(macro welfare)
```

### Example 2: convergence

With `trace`, the residual is displayed at each iteration. One Jacobian was computed, exactly, and every later step used its LU factors with Broyden's updates, because each step at least halved the residual:

```
. cge_solve, name(notariff) tolerance(1e-12) trace
Iteration 0: largest scaled residual =  2.10e-01 (0 Jacobian(s), 0 Broyden update(s))
Iteration 1: largest scaled residual =  1.19e-02 (1 Jacobian(s), 0 Broyden update(s))
Iteration 2: largest scaled residual =  1.10e-03 (1 Jacobian(s), 1 Broyden update(s))
Iteration 3: largest scaled residual =  7.36e-05 (1 Jacobian(s), 2 Broyden update(s))
Iteration 4: largest scaled residual =  1.66e-06 (1 Jacobian(s), 3 Broyden update(s))
Iteration 5: largest scaled residual =  1.18e-07 (1 Jacobian(s), 4 Broyden update(s))
Iteration 6: largest scaled residual =  5.04e-10 (1 Jacobian(s), 5 Broyden update(s))
Iteration 7: largest scaled residual =  4.77e-11 (1 Jacobian(s), 6 Broyden update(s))
Iteration 8: largest scaled residual =  4.32e-12 (1 Jacobian(s), 7 Broyden update(s))
Iteration 9: largest scaled residual =  4.49e-14 (1 Jacobian(s), 8 Broyden update(s))
Simulation notariff solved in 9 iteration(s) (newton)
Largest scaled residual  1.02e-15; WALRAS =  1.44e-15
```

Newton's method converges quadratically near a solution when the Jacobian is recomputed at each step. Broyden's updates correct the Jacobian along each step at the cost of a few inner products, and converge superlinearly; each step costs one evaluation of the equations and two triangular solves, against a Jacobian and a factorisation for a Newton step, so for large models they are much faster. A new Jacobian is computed whenever a step fails to halve the residual (section 10.1).

The results are returned in `r()`:

```
. return list

scalars:
           r(N_solved) =  69
             r(walras) =  1.44286592302e-15
       r(max_residual) =  1.01506105109e-15
         r(iterations) =  9
          r(converged) =  1
              r(steps) =  0
        r(N_collapsed) =  0
            r(broyden) =  9
          r(jacobians) =  1

macros:
            r(closure) : "flexexr flexgsav savings cpi lab:mobile cap:mobile"
             r(method) : "newton"
               r(name) : "notariff"
```

`r(N_solved)` is the number of equations in the system that Newton's method solved, 69 of the model's 93: the others are explicit definitions, computed from the variables of the reduced system (section 10.2).

### Example 3: a solution that makes no economic sense

A tenfold rise in government purchases is far outside the range of the base. With savings-driven investment the equations can still be solved:

```
. cge_shock, reset
All shocks removed

. cge_shock qg, scale(10)
Parameter qg changed for 3 element(s)

. cge_solve, name(qg10)
Simulation qg10 solved in 52 iteration(s) (newton)
Largest scaled residual  5.50e-11; WALRAS = -2.26e-09
Note: 30 value(s) of the solution are negative or below one millionth of their base value:
  IADJ                             -26.0597  (base 1)
  QINV(com_man)                    -1042.39  (base 40)
  QINV(com_agr)                    -130.299  (base 5)
  QINV(com_ser)                    -130.299  (base 5)
  QM(com_man)                   1.77691e-11  (base 60)
  and 25 more
The equations hold, but such a solution rarely makes economic sense.
```

The solution satisfies every equation, but the economy has absorbed the purchases by negative investment, and the agricultural and manufacturing activities have all but stopped. `cge_solve` notes such values, negative or below one millionth of their base, and stores their number in `r(N_collapsed)`; a solution with collapsed values should be checked before it is reported.

### Example 4: a shock the model cannot absorb

With investment fixed, the same rise cannot be supplied:

```
. cge_closure, savinv(investment)
Closure
  external account     flexexr
  government           flexgsav
  savings-investment   investment
  numeraire            cpi
  factor lab           mobile
  factor cap           mobile

. cge_solve, name(qg10fixed)
Not solved directly; trying again in 4 steps
Not solved directly; trying again in 16 steps
Simulation qg10fixed did not converge (largest scaled residual  2.26e+00)
Largest scaled residual  2.26e+00; WALRAS =  4.32e-02
Diagnostics
  Largest scaled residuals
    I7   government demand                     com_ser               -2.257e+00
    I7   government demand                     com_man               -2.257e+00
    Q4   factor demand                         lab,act_agr           -6.693e-02
    Q4   factor demand                         cap,act_agr           -2.364e-02
    Q4   factor demand                         lab,act_man           -9.633e-03
  Smallest pivot of the Jacobian, relative to the largest:  9.57e-06 (GSAV)

r(430);
```

The direct solve failed, and so did continuation in 4 and in 16 steps. `cge_solve` then stops with error 430 and displays the diagnostics: the equations with the largest scaled residuals at the last point, by block and element, and the smallest pivot of the Jacobian relative to the largest, with the variable it belongs to. A small pivot means that the Jacobian is close to singular at the last point. Such a failure is usually a property of the shock, the closure or the SAM, not of the solver: a shock that the economy cannot absorb, a closure with an equation that no free variable can satisfy, or a SAM with a very small flow that the shock must expand many times. Equations with no free variable, and free variables in no equation, are listed when there are any.

Before it solves, `cge_solve` checks that the system is square, with as many free variables as equations. If it is not, for example after a variable added by `cge_variable, value()` without the equation that determines it, the command stops with error 459; **[CGE]** `cge_variable` shows an example.

### Constraints

With constraints set by **[CGE]** `cge_constraint`, `cge_solve` first solves the complementarity conditions in a smoothed form, which finds whether each constraint binds, and then solves the model again with the exact conditions (section 17.3). The regime of each constraint is displayed and stored with the simulation, in `r(constraints)`.

### The reduced and the full systems

Explicit definitions, such as intermediate and household demand, are left out of the Newton system and computed from the other variables. For a SAM with 250 activities and 20 households, 4,254 of 40,190 equations remain. With `full`, every equation is solved by Newton's method; the solutions agree to the tolerance, and the test suite checks that they do.

## Stored results

`cge_solve` stores the following in `r()`:

*Scalars*

| | |
|---|---|
| `r(converged)` | 1 if converged, 0 otherwise |
| `r(iterations)` | number of iterations |
| `r(max_residual)` | largest scaled residual, over every equation |
| `r(N_solved)` | number of equations in the system solved |

| | |
|---|---|
| `r(jacobians)` | number of Jacobians computed |
| `r(broyden)` | number of steps with Broyden's updates |
| `r(N_collapsed)` | number of values of a solution that are negative or below one millionth of their base value |
| `r(walras)` | value of the slack variable `WALRAS` |
| `r(steps)` | steps of continuation (0 for a direct solve) |
| `r(N_constraints)` | number of constraints, if any |
| `r(N_binding)` | number of constraints that bind |
| *Macros* | |
| `r(name)` | name of the simulation |
| `r(method)` | solution method: `newton` or `broydenpowell` |
| `r(closure)` | closure |
| `r(constraints)` | the constraints and their regimes, if any |
| *Diagnostics (after a failure, or with* `diagnose`*; [CGE]* `cge_listing` *lists every equation element)* | |
| `r(diag_block)` | block of the largest residual |
| `r(diag_element)` | its element |
| `r(diag_residual)` | that residual |
| `r(diag_N_empty_rows)` | number of equations with no free variable |
| `r(diag_N_empty_cols)` | number of free variables in no equation |
| `r(diag_pivot_ratio)` | smallest pivot of the Jacobian relative to the largest |
| `r(diag_singular)` | 1 if the Jacobian is singular or nearly so |

## Methods and formulas

Let $x$ be the free variables of the system and $r(x) = 0$ its equations. The solver works on scaled variables $z_j = x_j/s_j$ and scaled residuals $F_i(z) = r_i(x)/e_i$, where $s_j$ and $e_i$ are the base magnitudes of the variable and of the equation (one where the base value is zero). A Newton step solves

$$J(z)\,\Delta z = -F(z),$$

and the new point is $z + \lambda\,\Delta z$, with $\lambda$ the largest of $1, \frac{1}{2}, \frac{1}{4}, \ldots$ such that

$$\|F(z + \lambda\,\Delta z)\| \le (1 - 10^{-4}\lambda)\,\|F(z)\|.$$

After each Newton step, later steps use the LU factors of the last Jacobian with Broyden's updates (Broyden, 1965; Kelley, 1995): after a step $s$, with $y$ the change in $F$, the inverse $H$ of the Jacobian becomes

$$H_+ = (I + a\,s')\,H, \qquad a = \frac{s - Hy}{s'Hy},$$

and the next direction is $-H_+F$, with a line search of at most two halvings. A new Jacobian is computed when that fails, after a step that reduces $\|F\|$ by less than a tenth, and after 30 updates. The solve has converged when $\max_i |F_i| <$ `tolerance()`, over every equation of the model, including those left out of the reduced system; up to three more steps then bring the residual toward a thousandth of the tolerance.

The Jacobian is exact: the derivatives of every equation are written out, user equations are differentiated in forward mode, and in the reduced system the variables of the left-out definitions $D_k(d_k, y) = 0$ are eliminated by the chain rule, with $\partial d_k/\partial y_j = -(\partial D_k/\partial y_j)/(\partial D_k/\partial d_k)$. If Newton fails, it is tried again with central differences over groups of columns that share no row,

$$J_{ij} \approx \frac{F_i(z + h \odot e_G) - F_i(z - h \odot e_G)}{2h_j}, \qquad j \in G, \qquad h_j = 10^{-6}\max(1, |z_j|),$$

where $e_G$ has ones in the columns of $G$ and $\odot$ is the elementwise product, then Powell's hybrid method (`solvenl()`, technique `broydenpowell`), followed by Newton steps to polish the result.

Continuation in $n$ steps solves the model for $k = 1, \ldots, n$ with parameters and fixed values

$$\theta_k = \theta^0 + \frac{k}{n}\,(\theta^1 - \theta^0),$$

between their base values $\theta^0$ and their shocked values $\theta^1$, each step starting from the solution of the step before.

Chapter 10 gives the details: the reduced system, the exact Jacobian, Broyden's updates, the grouping of columns for the differences and the treatment of constraints.

## Also see

**[CGE]** `cge_closure` — Choose the model closure
**[CGE]** `cge_shock` — Change a parameter or a fixed variable
**[CGE]** `cge_constraint` — Limits that bind in some scenarios and not in others
**[CGE]** `cge_results` — Report simulation results
**[CGE]** `cge_report` — Standard report tables for one or more simulations
**[CGE]** `cge_check` — Run model validation checks
**Chapter 10** — The solution method

# `cge_spa` — Structural path analysis of a SAM multiplier

## Description

`cge_spa` decomposes the accounting multiplier of account `to()` with respect to an injection into account `from()` into the paths along which the injection travels through the SAM (Defourny and Thorbecke 1984). A path is a chain of payments that passes through no account twice, for example `com_hlt > act_hlt > lab > hhd`: spending on health services is received by the health activity, which pays labour, which pays households. For each path the command reports its direct influence, its path multiplier and its total influence, and the share of the multiplier that it carries.

The SAM is the one mapped by **[CGE]** `cge_map`, and the endogenous accounts are those of **[CGE]** `cge_multiplier`. The model in memory is not changed.

## Quick start

The paths from spending on health services to household income

```
cge_spa, from(com_hlt) to(hhd)
```

As above, with paths of up to 8 arcs and a lower threshold, loading every path found as data

```
cge_spa, from(com_hlt) to(hhd) maxlength(8) threshold(0.0001) clear
```

The paths to labour income, with the government endogenous

```
cge_spa, from(com_hlt) to(lab) endogenous(gov)
```

## Syntax

**`cge_spa`**`, from(`*account*`) to(`*account*`)` [*options*]

| *options* | Description |
|---|---|
| *Main* | |
| `from(`*account*`)` | origin: the account into which the injection enters |
| `to(`*account*`)` | destination: the account whose income is decomposed |
| *Search* | |
| `maxlength(#)` | largest number of arcs in a path; default 6 |
| `threshold(#)` | smallest direct influence kept, as a share of the global influence; default `0.001` |
| *Accounts* | |
| `endogenous(`*accounts*`)` | make exogenous accounts endogenous, as in `cge_multiplier` |
| `exogenous(`*accounts*`)` | make endogenous accounts exogenous, as in `cge_multiplier` |
| *Reporting* | |
| `top(#)` | number of paths displayed; default 10 |
| `clear` | replace the data in memory with every path found |

`from()` and `to()` are required.

## Options

`from(`*account*`)` and `to(`*account*`)` name the origin, where the injection enters, and the destination. Both must be endogenous accounts, and they must differ.

`maxlength(#)` sets the largest number of arcs in a path; the default is 6, and # must be at least 1.

`threshold(#)` sets the smallest direct influence of a path, as a share of the global influence in absolute value, below which the path and its extensions are dropped. The default is `0.001`. `threshold(0)` with a large `maxlength()` finds every path, which can take long in a large SAM; the search stops with an error beyond 100,000 paths.

`endogenous(`*accounts*`)` and `exogenous(`*accounts*`)` choose the endogenous accounts, as in **[CGE]** `cge_multiplier`.

`top(#)` sets the number of paths displayed, largest total influence (in absolute value) first; the default is 10.

`clear` replaces the data in memory with every path found, in the order of the display, with the variables `path` (the accounts of the path, separated by ` > `), `length` (its number of arcs), `direct`, `multiplier`, `total` and `share` (the total influence as a percentage of the global influence).

## Remarks and examples

Structural path analysis treats the endogenous accounts of the SAM as the poles of a graph. There is an arc from pole $i$ to pole $k$ when $k$ receives part of $i$'s spending, with intensity $A_{ki}$, the share of $i$'s spending that $k$ receives. For each path from the origin to the destination, `cge_spa` reports

| Measure | Meaning |
|---|---|
| direct influence | the product of the intensities of its arcs: the effect along the path with every other account held fixed |
| path multiplier | the feedback of the loops that start and end at the accounts of the path |
| total influence | the direct influence times the path multiplier |
| % global | the total influence as a percentage of the global influence, the multiplier itself |

The total influences of all the paths from the origin to the destination add up to the global influence. Like the multipliers, the paths assume fixed prices and unused capacity.

### Example 1: the paths from health spending to household income

The health layout of the synthetic SAM is mapped as in **[CGE]** `cge_multiplier`:

```
. cge_synthetic, clear health
(19 vars, 18 obs)

Synthetic SAM loaded (health layout): 18 accounts, total   1746.00
Largest row-column imbalance:  0.00e+00

. cge_sam, labels(account)
SAM read: 18 accounts, total 1746
Largest row-column imbalance:  0.00e+00

. cge_map, activities(act_agr act_man act_ser act_hlt)                    ///
>     commodities(com_agr com_man com_ser com_hlt) factors(lab cap)        ///
>     households(hhd) government(gov) savinv(sav) row(row) dirtax(tax_dir) ///
>     saltax(tax_sal) imptax(tax_imp) insurance(shi)
Accounts mapped: 4 activities, 4 commodities, 2 factors, 1 households, 0 enterprises, 0 margin accou
> nts
Health insurance fund: shi
```

A unit injection into health services raises household income by 1.813, the multiplier of `hhd` with respect to `com_hlt`:

```
. cge_spa, from(com_hlt) to(hhd)
```

```
Structural paths from com_hlt to hhd (Defourny and Thorbecke 1984)
Global influence (the multiplier):     1.813
6 paths with at most 6 arcs and a direct influence of at least .001 times the global influence
------------------------------------------------------------------------------
     Total     Direct Path mult. % global  Path
------------------------------------------------------------------------------
     1.168      .5733     2.0379    64.43  com_hlt > act_hlt > lab > hhd
     .4076         .2     2.0379    22.48  com_hlt > act_hlt > cap > hhd
    .05865     .02446     2.3974     3.23  com_hlt > act_hlt > com_man > act_man > cap > hhd
    .04904     .02046     2.3974     2.70  com_hlt > act_hlt > com_man > act_man > lab > hhd
    .04569     .01956     2.3365     2.52  com_hlt > act_hlt > com_ser > act_ser > cap > hhd
    .04168     .01784     2.3365     2.30  com_hlt > act_hlt > com_ser > act_ser > lab > hhd
------------------------------------------------------------------------------
The 6 paths explain  97.66 per cent of the global influence
```

The path through labour carries 64 per cent of the effect and the path through capital 22 per cent; paths through the intermediate inputs of the health activity carry most of the rest. The direct influence of the first path, 0.5733, is the product of the shares along it: all of the spending on health services goes to the health activity, which pays 18 of its 30 to labour, which pays 129 of its 135 to households. Its path multiplier, 2.0379, raises it to a total influence of 1.168. The six paths found explain 97.66 per cent of the global influence; the others have a direct influence below 0.001 times the global influence or more than six arcs.

The results are stored in `r()`:

```
. return list

scalars:
             r(total1) =  1.168382516579885
              r(share) =  97.66013626611974
          r(explained) =  1.771017738921657
            r(N_paths) =  6
             r(global) =  1.813449997751087

macros:
              r(path1) : "com_hlt > act_hlt > lab > hhd"
```

### Example 2: every path

With `threshold(0)` and paths of up to ten arcs, as many as the eleven endogenous accounts allow, every elementary path is found, and the total influences add up to the global influence:

```
. cge_spa, from(com_hlt) to(hhd) maxlength(10) threshold(0) top(3) clear

Structural paths from com_hlt to hhd (Defourny and Thorbecke 1984)
Global influence (the multiplier):     1.813
22 paths with at most 10 arcs and a direct influence of at least 0 times the global influence
------------------------------------------------------------------------------
     Total     Direct Path mult. % global  Path
------------------------------------------------------------------------------
     1.168      .5733     2.0379    64.43  com_hlt > act_hlt > lab > hhd
     .4076         .2     2.0379    22.48  com_hlt > act_hlt > cap > hhd
    .05865     .02446     2.3974     3.23  com_hlt > act_hlt > com_man > act_man > cap > hhd
------------------------------------------------------------------------------
The 3 largest of 22 paths are shown; clear loads them all
The 22 paths explain 100.00 per cent of the global influence

. summarize total

    Variable |       Obs        Mean    Std. Dev.       Min        Max
-------------+---------------------------------------------------------
       total |        22    .0824295    .2574331    .000062   1.168383

. display r(sum)
1.81345

. tabulate length

  Number of |
       arcs |      Freq.     Percent        Cum.
```

```
------------+-----------------------------------
          3 |          2        9.09        9.09
          5 |          4       18.18       27.27
          7 |          8       36.36       63.64
          9 |          8       36.36      100.00
------------+-----------------------------------
      Total |         22      100.00
```

In this SAM a path alternates between commodities and activities until it reaches a factor, which pays the household, so every path has an odd number of arcs.

## Stored results

`cge_spa` stores the following in `r()`:

| | |
|---|---|
| *Scalars* | |
| `r(global)` | global influence (the multiplier) |
| `r(N_paths)` | number of paths found |
| `r(explained)` | sum of their total influences |
| `r(share)` | that sum as a percentage of the global influence |
| `r(total1)` | total influence of the largest path, if any |
| *Macros* | |
| `r(path1)` | the largest path, if any |

## Methods and formulas

Let $A$ be the matrix of average expenditure propensities among the endogenous accounts and $M = (I-A)^{-1}$ the matrix of multipliers, as in **[CGE]** `cge_multiplier`. An elementary path $p$ from origin $i$ to destination $j$ is a sequence of poles $i = k_0, k_1, \ldots, k_m = j$, none repeated, with $A_{k_l k_{l-1}} \neq 0$ for each arc. With $P$ the set of its poles,

$$I_p^D = \prod_{l=1}^{m} A_{k_l k_{l-1}}, \qquad M_p = \frac{\det\big((I-A)_{-P,-P}\big)}{\det(I-A)} = \det\big(M_{P,P}\big), \qquad I_p^T = I_p^D\, M_p,$$

the direct influence, the path multiplier and the total influence, where $(I-A)_{-P,-P}$ leaves out the rows and columns of $P$ and $M_{P,P}$ is the block of $M$ on $P$; the two expressions of $M_p$ are equal by Jacobi's identity for complementary minors, and the command computes the second. The global influence of $i$ on $j$ is

$$I_{ji}^G = M_{ji} = \sum_p I_p^T,$$

the sum over every elementary path from $i$ to $j$.

The paths are found by a depth-first search from the origin. A path is extended along each arc to a pole not already in it; it is dropped, with every extension of it, once $|I_p^D| <$ `threshold()` $\times\, |M_{ji}|$, and it has at most `maxlength()` arcs. When every intensity is at most one in absolute value, as when the SAM has no negative cells, an extension cannot raise the direct influence, so no path above the threshold is missed. The share of the global influence explained is $100 \sum_p I_p^T / M_{ji}$ over the paths found. Structural path analysis is described in section 24.7.

## References

Defourny, J., and E. Thorbecke. 1984. Structural path analysis and multiplier decomposition within a social accounting matrix framework. *Economic Journal* 94: 111–136.

## Also see

**[CGE]** `cge_multiplier` — SAM multipliers and their decomposition
**[CGE]** `cge_map` — Assign SAM accounts to model roles
**[CGE]** `cge` — Computable general equilibrium modelling in Stata
**Chapter 24** — SAM tools

# `cge_synthetic` — Load the synthetic social accounting matrix

## Description

`cge_synthetic` loads one of the synthetic social accounting matrices (SAMs) bundled with cge-stata as data in memory, in the wide form that **[CGE]** `cge_sam` reads. It checks that the SAM is square and that its row and column labels match, and reports its total and the largest difference between an account's row and column totals.

The SAMs are SYNTHETIC: they were constructed for testing and teaching and describe no country. The examples of this manual use them.

## Quick start

Load the standard synthetic SAM, replacing the data in memory, and read it

```
cge_synthetic, clear
cge_sam, labels(account)
```

Load the SAM with a health sector and a social health insurance fund

```
cge_synthetic, clear health
```

Load the unbalanced SAM used by the examples of `cge_balance`

```
cge_synthetic, clear unbalanced
```

## Syntax

**`cge_synthetic`** [, `clear` *layout*]

| *layout* | Description |
|---|---|
| `national` | national-accounts layout: exports sold by activities, taxes paid to the government (13 accounts) |
| `institutions` | an enterprise, a non-profit institution and transfers among institutions (17) |
| `margins` | margins netted out of the service activity (15) |
| `territorial` | a territorial correction of household consumption (17) |
| `multiproduct` | secondary production of a commodity (15) |
| `nonimported` | a commodity that is not imported (15) |
| `sut` | supply–use layout: margin accounts, stock changes and re-exports (19) |
| `health` | health services and a social health insurance fund (18) |
| `unbalanced` | the institutions layout made unbalanced (17) |
| `vat` | a value added tax account (16) |
| `home` | home consumption (15) |
| `trade` | an export-only cash crop and margins by market (19) |
| `factortax` | taxes on factor use and on factor income (17) |
| `nests` | energy and two types of labour, for nests of value added (19) |

At most one *layout* may be given; the default is the standard layout of 15 accounts.

## Options

`clear` allows the data in memory to be replaced.

`national` loads the economy of the standard layout as many national SAMs record it: exports are recorded as sales by the activities to the rest of the world, and sales and direct taxes are paid directly to the government account, so that the `tax_sal` and `tax_dir` accounts are absent (13 accounts). Map it with `saltax(gov)` and `dirtax(gov)`; it gives the same model as the standard layout.

`institutions` loads a variant of 17 accounts with an enterprise (`ent`) and a non-profit institution (`npi`). It includes transfers among institutions, transfers to and from the rest of the world, factor income from and to abroad, transfers within the government, and negative savings. Map it with `households(hhd npi)` and `enterprises(ent)`.

`margins` loads the standard SAM with margins of 30 netted out of the service activity: its output is recorded as 120 and its use of its own product as −10. Map it with `margins(act_ser)`.

`territorial` loads a variant of 17 accounts with a territorial correction (`act_tc`, `com_tc`): residents' purchases abroad of 4, non-residents' purchases of 9, and household consumption of the correction of −5. It also has a fall of 2 in the inventories of `com_agr`. Map it with `territorial(com_tc act_tc)`.

`multiproduct` loads the standard SAM with secondary production: `act_agr` also produces 10 of `com_man`.

`nonimported` loads the standard SAM with `com_ser`, which pays sales tax, not imported: its imports move to `com_man`.

`sut` loads a supply–use layout of 19 accounts: margin accounts `trd` and `trp` that collect margins from the commodities and buy margin services, a commodity that is only imported (`com_fuel`), a stock-change account (`dstk`), savings–investment labelled `s-i`, which **[CGE]** `cge_sam` renames `si`, and exports at least as large as domestic output: `com_fuel` re-exports 2 of its imports, and `com_agr` exports 105 against output of 100, with a fall in its stocks. Map it with `commodities(com_agr com_man com_ser com_fuel)`, `savinv(si)`, `marginaccounts(trd trp)` and `stocks(dstk)`.

`health` loads a layout of 18 accounts with health services (`act_hlt`, `com_hlt`, neither imported, exported nor taxed) and a social health insurance fund (`shi`) that receives contributions of 6 on labour income, 2 from the government and 1 from abroad, buys 8 of health services and saves 1. Map it with the four activities and commodities and `insurance(shi)`.

`unbalanced` loads the institutions layout made unbalanced. Every non-zero cell is changed by up to 6 per cent, by a fixed rule, and rounded to four decimals; the negative savings of `npi` and `gov` keep their signs. Accounts are out of balance by up to 12. The examples and tests of **[CGE]** `cge_balance` use it.

`vat` loads the standard layout with a VAT account (`tax_vat`) that collects 18 from the commodities and passes it to the government, which returns it to households as transfers. Map it with `vattax(tax_vat)`.

`home` loads the standard layout with home consumption: the household consumes 20 of `act_agr`'s output at home and buys 20 less of `com_agr`.

`trade` loads the standard layout with a cash crop, `com_cash`, that `act_agr` produces and the economy exports entirely, and margin accounts on `com_man`'s domestic sales (`mtrd`), imports (`mtrm`) and exports (`mtre`). Map it with the four commodities and `domesticmargins(mtrd)`, `importmargins(mtrm)` and `exportmargins(mtre)`, and calibrate with `exportonly(com_cash)`.

`factortax` loads the standard layout with a payroll tax on labour of 4, 3 and 6 paid by the three activities (`tax_pay`) and a tax of 7 on capital income (`tax_fy`); direct taxes are 20 lower, so government revenue is unchanged. Map it with `factortax(tax_pay:lab)` and `factorincometax(tax_fy)`.

`nests` loads a layout with four activities and commodities, including energy (`act_ene`, `com_ene`), two types of labour (`lab_hi`, `lab_lo`), capital, and a payroll tax on skilled labour (`tax_pay`). It is used to test nests of value added: map it with `factortax(tax_pay:lab_hi)` and calibrate with, for example, `nests(labour: lab_hi lab_lo, ke: cap com_ene)` (see **[CGE]** `cge_calibrate`).

## Remarks and examples

The standard SAM has 15 accounts: three activities (`act_agr`, `act_man`, `act_ser`), three commodities (`com_agr`, `com_man`, `com_ser`), two factors (`lab`, `cap`), one household (`hhd`), the government (`gov`), direct, sales and import-tariff tax accounts (`tax_dir`, `tax_sal`, `tax_imp`), savings–investment (`sav`) and the rest of the world (`row`). The other layouts change it to exercise one feature of the model each: a layout of national accounts that the mapping rewrites, an account role, or an extension of the model.

Each SAM is stored as a CSV file and loaded as wide data: the string variable `account` holds the row labels, and there is one numeric variable per column account. Cell $(r, c)$ is a payment from the column account $c$ to the row account $r$. The data are not yet a model: **[CGE]** `cge_sam` reads them.

### Example 1: the standard SAM

The columns of the activities record their purchases of intermediate inputs and their payments to the factors; the columns of the commodities record domestic output, sales and import taxes, and imports:

```
. cge_synthetic, clear
(16 vars, 15 obs)

Synthetic SAM loaded (standard layout): 15 accounts, total   1743.00
Largest row-column imbalance:  0.00e+00

. list account act_agr act_man act_ser com_agr com_man com_ser, noobs sep(0)

  +----------------------------------------------------------------------+
  | account   act_agr   act_man   act_ser   com_agr   com_man   com_ser |
  |----------------------------------------------------------------------|
  | act_agr         0         0         0       100         0         0 |
  | act_man         0         0         0         0       140         0 |
  | act_ser         0         0         0         0         0       150 |
  | com_agr        10        20         5         0         0         0 |
  | com_man         8        30        15         0         0         0 |
  | com_ser         5        15        20         0         0         0 |
  |     lab        40        35        60         0         0         0 |
  |     cap        37        40        50         0         0         0 |
  |     hhd         0         0         0         0         0         0 |
  |     gov         0         0         0         0         0         0 |
  | tax_dir         0         0         0         0         0         0 |
  | tax_sal         0         0         0         8        12        10 |
  | tax_imp         0         0         0         2         6         0 |
  |     sav         0         0         0         0         0         0 |
  |     row         0         0         0        20        60        20 |
  +----------------------------------------------------------------------+
```

### Example 2: the health layout

In the health layout, the health activity sells 30 of health services, which it produces with intermediate inputs, labour and capital; the insurance fund buys 8 of health services and saves 1:

```
. cge_synthetic, clear health
(19 vars, 18 obs)

Synthetic SAM loaded (health layout): 18 accounts, total   1746.00
Largest row-column imbalance:  0.00e+00

. list account act_hlt com_hlt shi if act_hlt != 0 | com_hlt != 0 | shi != 0, noobs sep(0)

  +------------------------------------+
  | account   act_hlt   com_hlt   shi |
  |------------------------------------|
  | act_hlt         0        30     0 |
  | com_man         4         0     0 |
  | com_ser         2         0     0 |
  | com_hlt         0         0     8 |
  |     lab        18         0     0 |
  |     cap         6         0     0 |
  |     sav         0         0     1 |
  +------------------------------------+
```

### Example 3: the unbalanced SAM

The unbalanced SAM is the institutions layout with every non-zero cell perturbed:

```
. cge_synthetic, clear unbalanced
(18 vars, 17 obs)

Synthetic SAM loaded (institutions layout, unbalanced for the balancing tests): 17 accounts, total
>  1860.46
Largest row-column imbalance:  1.21e+01

. display r(maxdiff)
12.0859
```

**[CGE]** `cge_balance` balances it.

## Stored results

`cge_synthetic` stores the following in `r()`:

| | |
|---|---|
| *Scalars* | |
| `r(N_accounts)` | number of accounts |
| `r(total)` | sum of all cells |
| `r(maxdiff)` | largest difference between an account's row and column totals |
| *Macros* | |
| `r(accounts)` | account labels, in the order of the SAM |
| `r(filename)` | path of the bundled SAM file |

## Methods and formulas

The SAM file is found on the ado-path and read with `import delimited`, with the first column as strings and the others as doubles. The SAM must have as many observations as account variables, and each row label must equal the name of the variable in the same position, unless the label is not a valid Stata name (as `s-i`, which Stata renames). With $x_{ij}$ the cell in row $i$ and column $j$ of a SAM of $n$ accounts,

$$\texttt{r(total)} = \sum_{i=1}^{n}\sum_{j=1}^{n} x_{ij}, \qquad \texttt{r(maxdiff)} = \max_k \left| \sum_{j=1}^{n} x_{kj} - \sum_{i=1}^{n} x_{ik} \right|.$$

The files are

| Layout | File | Layout | File |
|---|---|---|---|
| standard | `cge_synthetic_sam.csv` | `sut` | `cge_synthetic_sam_sut.csv` |
| `national` | `cge_synthetic_sam_na.csv` | `health` | `cge_synthetic_sam_health.csv` |
| `institutions` | `cge_synthetic_sam_inst.csv` | `unbalanced` | `cge_synthetic_sam_unbal.csv` |
| `margins` | `cge_synthetic_sam_margins.csv` | `vat` | `cge_synthetic_sam_vat.csv` |
| `territorial` | `cge_synthetic_sam_tc.csv` | `home` | `cge_synthetic_sam_home.csv` |
| `multiproduct` | `cge_synthetic_sam_mp.csv` | `trade` | `cge_synthetic_sam_trade.csv` |
| `nonimported` | `cge_synthetic_sam_cnm.csv` | `factortax` | `cge_synthetic_sam_ftax.csv` |
| | | `nests` | `cge_synthetic_sam_nest.csv` |

## Also see

**[CGE]** `cge_sam` — Read a social accounting matrix
**[CGE]** `cge_map` — Assign SAM accounts to model roles
**[CGE]** `cge` — Computable general equilibrium modelling in Stata
**Chapter 24** — SAM tools

# `cge_template` — Write a template that maps SAM accounts to roles and groups

## Description

`cge_template` writes a template with one row per account of the SAM of the current model, as read, aggregated or balanced. The template is filled in, in a spreadsheet or any editor, and then used by **[CGE]** `cge_map` `using` to map the SAM to the model's roles and by **[CGE]** `cge_aggregate` `using` to merge accounts into groups. A template is easier to check and to keep than a long `cge_map` command, and it has no limit on the number of accounts.

If the model is mapped, the roles and pairs of the template are filled in from the mapping. The data in memory are not changed.

## Quick start

Write a template of the SAM read by `cge_sam` to an Excel file

```
cge_template using sam_roles.xlsx
```

As above, to a CSV file, replacing the file if it exists

```
cge_template using sam_roles.csv, replace
```

Aggregate and map the SAM with the template, once it has been filled in

```
cge_aggregate using sam_roles.xlsx
cge_map using sam_roles.xlsx
```

## Syntax

**`cge_template`** `using` *filename* [`, replace`]

*filename* ends in `.xlsx`, `.xls` or `.csv`.

## Options

`replace` allows an existing file to be replaced.

## Remarks and examples

The template has the columns

| Column | Content |
|---|---|
| `account` | the account label |
| `role` | the `cge_map` options that list the account, separated by spaces |
| `pair` | for an activity, the commodity it is paired with; for a tax on the use of a factor, the factor it taxes |
| `group` | for aggregation, the account it is merged into |
| `row_total` | the account's row total (receipts) |
| `column_total` | the account's column total (payments) |
| `difference` | the row total less the column total |

An Excel template has the columns in the sheet `accounts`, and a second sheet, `roles`, that lists every entry of the `role` column with its meaning, and notes on the `pair` and `group` columns. Only the columns `account`, `role`, `pair` and `group` are read back; the totals are there to help assign the roles.

### Roles

A role is the name of a **[CGE]** `cge_map` option:

| Role | Account |
|---|---|
| `activities` | an activity (production sector) |
| `commodities` | a commodity |
| `factors` | a factor of production |
| `households` | a household group, or a non-profit institution serving households |
| `enterprises` | an enterprise |
| `government` | the government (one account) |
| `savinv` | savings–investment (one account) |
| `row` | the rest of the world (one account) |
| `dirtax` | the direct tax account, or the government account if direct taxes are paid to it |
| `saltax` | the sales tax account, or the government account |
| `imptax` | the import tariff account, or the government account |
| `exptax` | the export tax account, or the government account |
| `acttax` | the activity tax account, or the government account |
| `vattax` | the value added tax account |
| `margins` | an activity whose margins are netted out of its output (also an activity) |
| `territorial` | a territorial correction: its commodity and, if there is one, its activity |
| `marginaccounts` | a margin account paid per unit of composite supply |
| `domesticmargins` | a margin account paid per unit of domestic sales |
| `importmargins` | a margin account paid per unit of imports |
| `exportmargins` | a margin account paid per unit of exports |
| `stocks` | a stock-change account, merged into savings–investment |
| `insurance` | the social health insurance fund (one account) |
| `factortax` | an account of taxes on the use of a factor, named in the pair column |
| `factorincometax` | the account of taxes on factor income (one account) |

Roles may be typed in either case, and the singular forms `activity`, `commodity`, `factor`, `household`, `enterprise`, `margin`, `marginaccount`, `domesticmargin`, `importmargin`, `exportmargin`, `stock` and `vat` are also accepted. An account may have several roles, separated by spaces or commas, where `cge_map` allows it: `activities margins` for a margin activity, or `government saltax` when sales taxes are paid to the government account. A role that is not in the list is an error. Leave the role blank only for an account without flows, which `cge_map` accepts unmapped. A share of the sales tax that is VAT is not a role: it is given to `cge_map using` with its option `vatshare()`.

### Pairs

Each activity is paired with a commodity, its primary product. A blank `pair` pairs an activity with the commodity whose column pays it most. Give the pair when the largest sale is not the activity's main product, when two activities would be paired with the same commodity, or when an activity sells equal amounts to several commodities; the last two are errors otherwise.

### Groups

A blank `group` keeps the account. Accounts with the same group are merged by **[CGE]** `cge_aggregate`, and must have the same role. After aggregation, `cge_map using` the same template maps each group with its members' role, and pairs it with the group of its members' pairs.

### Example 1: a template of a SAM that is not mapped

The template of the standard synthetic SAM, written as a CSV file, has blank roles, pairs and groups:

```
. cge_synthetic, clear
(16 vars, 15 obs)

Synthetic SAM loaded (standard layout): 15 accounts, total   1743.00
Largest row-column imbalance:  0.00e+00
```

```
. cge_sam, labels(account)
SAM read: 15 accounts, total 1743
Largest row-column imbalance:  0.00e+00

. cge_template using roles.csv, replace
Template of 15 accounts
Template saved to roles.csv

. type roles.csv
account,role,pair,group,row_total,column_total,difference
act_agr,,,,100,100,0
act_man,,,,140,140,0
act_ser,,,,150,150,0
com_agr,,,,130,130,0
com_man,,,,218,218,0
com_ser,,,,180,180,0
lab,,,,135,135,0
cap,,,,127,127,0
hhd,,,,287,287,0
gov,,,,63,63,0
tax_dir,,,,25,25,0
tax_sal,,,,30,30,0
tax_imp,,,,8,8,0
sav,,,,50,50,0
row,,,,100,100,0
```

### Example 2: filling in the template

Roles and groups are filled in, here with Stata commands; the template is usually edited in a spreadsheet. The manufacturing and service activities and commodities are put in the groups `act_nag` and `com_nag`:

```
. import delimited using roles.csv, clear stringcols(_all)
(7 vars, 15 obs)

. quietly replace role = "activities"  if inlist(account, "act_agr", "act_man", "act_ser")

. quietly replace role = "commodities" if inlist(account, "com_agr", "com_man", "com_ser")

. quietly replace role = "factors"     if inlist(account, "lab", "cap")

. quietly replace role = "households"  if account == "hhd"

. quietly replace role = "government"  if account == "gov"

. quietly replace role = "savinv"      if account == "sav"

. quietly replace role = "row"         if account == "row"

. quietly replace role = "dirtax"      if account == "tax_dir"

. quietly replace role = "saltax"      if account == "tax_sal"

. quietly replace role = "imptax"      if account == "tax_imp"

. quietly replace group = "act_nag"    if inlist(account, "act_man", "act_ser")

. quietly replace group = "com_nag"    if inlist(account, "com_man", "com_ser")

. export delimited account role pair group using roles.csv, replace
file roles.csv saved

. type roles.csv
account,role,pair,group
act_agr,activities,,
act_man,activities,,act_nag
act_ser,activities,,act_nag
com_agr,commodities,,
com_man,commodities,,com_nag
com_ser,commodities,,com_nag
lab,factors,,
cap,factors,,
hhd,households,,
gov,government,,
tax_dir,dirtax,,
tax_sal,saltax,,
tax_imp,imptax,,
sav,savinv,,
row,row,,
```

The template then aggregates the SAM and maps the result. `cge_map` displays the command that the template stands for: each group takes the role of its members, and the activities are paired with the commodities that pay them most.

```
. cge_aggregate using roles.csv
Accounts merged: 4 accounts into 2 groups; 11 accounts left as they are
Aggregated SAM: 13 accounts, total 1743
Largest row-column imbalance:  0.00e+00

. cge_map using roles.csv
Mapping from the template:
cge_map, activities(act_agr act_nag) commodities(com_agr com_nag) factors(lab cap) households(hhd) g
> overnment(gov) savinv(sav) row(row) dirtax(tax_dir) saltax(tax_sal) imptax(tax_imp)
Accounts mapped: 2 activities, 2 commodities, 2 factors, 1 households, 0 enterprises, 0 margin accou
> nts
```

### Example 3: a template of a mapped SAM

When the model is mapped, the template records the mapping. In the national-accounts layout, direct and sales taxes are paid directly to the government account, which therefore has the roles `government`, `dirtax` and `saltax`:

```
. cge_synthetic, clear national
(14 vars, 13 obs)

Synthetic SAM loaded (national-accounts layout): 13 accounts, total   1613.00
Largest row-column imbalance:  0.00e+00

. cge_sam, labels(account)
SAM read: 13 accounts, total 1613
Largest row-column imbalance:  0.00e+00

. cge_map, activities(act_agr act_man act_ser) commodities(com_agr com_man com_ser) ///
>     factors(lab cap) households(hhd) government(gov) savinv(sav) row(row)       ///
>     dirtax(gov) saltax(gov) imptax(tax_imp)
Exports recorded as sales by activities, moved to the paired commodities: 75
Direct taxes paid to government, routed through account _dirtax: 25
Sales taxes paid to government, routed through account _saltax: 30
Accounts mapped: 3 activities, 3 commodities, 2 factors, 1 households, 0 enterprises, 0 margin accou
> nts

. cge_template using roles_na.csv, replace
Template of 13 accounts, with roles from the current mapping
Template saved to roles_na.csv

. type roles_na.csv
account,role,pair,group,row_total,column_total,difference
act_agr,activities,com_agr,,100,100,0
act_man,activities,com_man,,140,140,0
act_ser,activities,com_ser,,150,150,0
com_agr,commodities,,,110,110,0
com_man,commodities,,,178,178,0
com_ser,commodities,,,165,165,0
lab,factors,,,135,135,0
cap,factors,,,127,127,0
hhd,households,,,287,287,0
gov,government dirtax saltax,,,63,63,0
tax_imp,imptax,,,8,8,0
sav,savinv,,,50,50,0
row,row,,,100,100,0
```

A template written from a mapping can be extended with groups and used again.

## Stored results

`cge_template` stores the following in `r()`:

*Scalars*

| | |
|---|---|
| `r(N_accounts)` | number of accounts |

## Methods and formulas

With $x_{ij}$ the cell in row $i$ and column $j$ of the SAM, the totals of account $k$ are

$$\texttt{row\_total}_k = \sum_j x_{kj}, \qquad \texttt{column\_total}_k = \sum_i x_{ik}, \qquad \texttt{difference}_k = \texttt{row\_total}_k - \texttt{column\_total}_k.$$

When `cge_map using` pairs an activity $a$ whose pair is blank, it chooses the commodity

$$c(a) = \arg\max_{c \in C} x_{ac},$$

the largest payment of a commodity's column to the activity's row; a tie, or no positive payment, is an error. The `commodities()` list then starts with the paired commodities, in the order of the activities, followed by the other commodities in the order of the SAM. Every other option lists its accounts in the order of the SAM, except that for `territorial()` the commodity that households consume is listed before its activity. Chapter 24 describes the template in section 24.3.

## Also see

**[CGE]** `cge_map` — Assign SAM accounts to model roles
**[CGE]** `cge_aggregate` — Merge SAM accounts into groups
**[CGE]** `cge_sam` — Read a social accounting matrix
**Chapter 24** — SAM tools

# `cge_use` — Load a model saved with cge_save

## Description

`cge_use` replaces the model in memory with one saved by **[CGE]** `cge_save`, including its closure, current shocks and stored simulations. The dataset in memory is not changed.

## Quick start

Load the model saved in `mymodel.cgem`

```
cge_use using mymodel
```

Report every simulation saved with it

```
cge_report, sims(_all)
```

## Syntax

`cge_use` `using` *filename*

If *filename* has no extension, `.cgem` is added. Only the file name counts: a period in the name of a folder is not an extension.

## Remarks and examples

A loaded model is in the state in which it was saved: the SAM, the mapping and the calibration; the closure, with its swaps; the shocks in force when it was saved; its constraints, user variables and user equations; and every stored simulation, with the parameters and closure under which each was solved. It can be reported, shocked and solved again at once. Loading a model does not change the data in memory, so a model can be loaded while survey or other data are in use.

### Example 1: loading a model in a new session

A model is calibrated, two simulations are solved, the second with government savings fixed and sales tax rates adjusting, and the model is saved as `mymodel.cgem`, as in **[CGE]** `cge_save`. `clear all` then removes everything from memory, as a new session of Stata would:

```
. clear all

. cge_use using mymodel
Model loaded from mymodel.cgem (saved by cge-stata 0.7.1; calibrated, 2 simulation(s))

. return list

macros:
           r(filename) : "mymodel.cgem"
           r(saved_by) : "0.7.1"
```

The model's closure, simulations and shocks are those of the session that saved it:

```
. cge_closure
Closure
  external account     flexexr
  government           fixgsav:sales
  savings-investment   savings
  numeraire            cpi
  factor lab           mobile
  factor cap           mobile
```

```
. cge_report, sims(_all) tables(welfare)

Welfare (per cent of base consumption)
--------------------------------------------------------------
                                           notariff   nt_sales
equivalent_variation hhd                      2.668     -0.257
compensating_variation hhd                    2.668     -0.257
--------------------------------------------------------------
Percentage changes from the base (rates: changes in points)

. quietly cge_parameters, clear current

. list parameter element1 value if parameter == "tm", noobs abbreviate(9)

  +------------------------------+
  | parameter   element1   value |
  |------------------------------|
  |        tm    com_agr       0 |
  |        tm    com_man       0 |
  |        tm    com_ser       0 |
  +------------------------------+
```

The closure is the one in force when the model was saved, the two simulations can be reported without solving them again, and the removal of import tariffs is still the current shock.

### Example 2: a file that is not a model

`cge_use` checks the file's identifier before reading it:

```
. cge_synthetic, clear
(16 vars, 15 obs)

Synthetic SAM loaded (standard layout): 15 accounts, total   1743.00
Largest row-column imbalance:  0.00e+00

. save notamodel, replace
(note: file notamodel.dta not found)
file notamodel.dta saved

. cge_use using notamodel.dta
notamodel.dta is not a cge-stata model file

r(610);
```

The model in memory, if any, is unchanged after an error.

### Versions of Stata and of cge-stata

Later versions of Stata read files saved in Stata 13. Stata 13 cannot read files saved in later versions, whose Mata file format differs (tested with StataNow 19.5); `cge_use` then stops with a message and leaves the model in memory unchanged. Save a model in Stata 13 when it is to be shared with users of Stata 13.

A file saved by an earlier version of cge-stata is upgraded as it is loaded: the variables, parameters and equations added since are given the values that leave the saved model unchanged, for example tax scaling factors fixed at one and rents fixed at zero. Items saved by a version that the current one no longer uses are ignored, with a note. `r(saved_by)` gives the version that saved the file.

## Stored results

`cge_use` stores the following in `r()`:

*Macros*

| | |
|---|---|
| `r(filename)` | name of the file read |
| `r(saved_by)` | version of cge-stata that saved the file |

## Methods and formulas

`cge_use` first compares the opening bytes of the file with those that Mata writes for the identifier of a model file, and looks for the identifier `cge-stata model` in the first 512 bytes; a file that fails the test is not a model file. A byte of the header gives the version of the Mata file format; a file in a later format than the running Stata writes is rejected. The file is then read item by item, by name, as **[CGE]** `cge_save` wrote it, and the model replaces the one in memory only when the whole file has been read.

## Also see

**[CGE]** `cge_save` — Save the model and its simulations to a file
**[CGE]** `cge_report` — Standard report tables for one or more simulations
**[CGE]** `cge` — Computable general equilibrium modelling in Stata

# `cge_variable` — User variables, defined by an expression or determined by user equations

## Description

`cge_variable` adds a variable to the calibrated model. With an expression, it adds the variable and the equation that defines it; with `value()`, it adds a free variable, with that base value, that user equations added by **[CGE]** `cge_equation` determine. A variable may be indexed by a set of the model, and is then defined for every element of the set, or, from version 0.9.1, by two sets, and is then defined for every pair of their elements.

## Quick start

Government purchases at market prices

```
cge_variable GEXP = sum(c, PQ(c) * QG(c)), nominal
```

Fix them in value, with the volumes adjusting

```
cge_closure, swap(GEXP GADJ)
```

Prices relative to the consumer price index, for every commodity

```
cge_variable PREL(c) = PQ(c) * (1 + tv(c)) / CPI
```

A variable with base value 1, for an equation of **[CGE]** `cge_equation` to determine

```
cge_variable LAM, value(1)
```

## Syntax

A variable and its definition

**`cge_variable`** *name*[`(`*i*[`,`*j*]`)`[`$(`*condition*`)`]] = *expression* [`,` `nominal` `lower(#)` `upper(#)`]

A variable that user equations determine

**`cge_variable`** *name*[`(`*i*[`,`*j*]`)`[`$(`*condition*`)`]]`,` `value(#)` [`fixed` `nominal`]

An exogenous variable at a bound, which an inequality may move (from version 0.9.0)

**`cge_variable`** *name*[`(`*i*[`,`*j*]`)`[`$(`*condition*`)`]]`,` `lower(#)` | `upper(#)` [`nominal`]

*name* has at most 24 characters, letters, digits and underscores; it may not be the name of a model variable, a user variable or a parameter that expressions use, and a defined variable may not take the name of a user equation. *i* is an index: the letter of a set, or the name of a user set (**[CGE]** `cge_set`), possibly followed by digits, as in `c` or `c2`. From version 0.9.1, a variable may have two different indexes, *i* and *j*, as in `X(c,h)`; it is then defined for every pair of elements of their sets, or for the pairs that meet its condition, and an element is written *e1*`.`*e2* in **[CGE]** `cge_shock`. *expression* is written in the language described in **[CGE]** `cge_equation`.

| *options* | Description |
|---|---|
| *Main* | |
| `value(#)` | base value of a variable that user equations determine |
| `fixed` | the variable is exogenous, at its value |
| `lower(#)`, `upper(#)` | bounds; without a definition, the exogenous level |

| *options* | Description |
|---|---|
| `nominal` | the variable is measured in local currency |

## Options

`value(#)` gives the base value of a variable that user equations determine, and adds no equation. A variable has either a definition or a value, not both.

`fixed` (from version 0.9.0) makes a variable given a value exogenous, as the closure makes the model's exogenous variables: it adds no unknown, **[CGE]** `cge_shock` can change it, and an inequality of **[CGE]** `cge_equation` can be paired with it.

`lower(#)`, `upper(#)` (from version 0.9.0) bound a variable. A defined variable is held within its bounds: it equals its expression, or the bound the expression would cross. A variable without a definition takes one of them and is then exogenous at that bound, from which an inequality paired with it may move it: away from a lower bound, up, and from an upper bound, down.

`nominal` marks a variable measured in local currency, so that the homogeneity check of **[CGE]** `cge_check` expects it to scale with the numeraire.

## Remarks and examples

User variables extend the model without changing its code. A defined variable gives a name to an expression of the model's variables, which can then be reported, bounded by **[CGE]** `cge_constraint` or, through a swap, fixed in the closure. A variable given a value is a new unknown, and needs equations of its own.

User variables are free when they are added. A defined variable is determined by its definition. Swapped into the closure with `cge_closure, swap(`*name variable*`)`, it is fixed, which fixes the expression that defines it, and *variable*, which the closure fixed, adjusts instead; **[CGE]** `cge_shock` can then change its value. A variable given a value adds one free variable for each element and no equation, so the closure is not square until **[CGE]** `cge_equation` adds as many equations; until then **[CGE]** `cge_solve` stops with error 459.

Indexes name the sets of the model: `a` activities, `c` commodities, `f` factors, `h` households, `d` households and enterprises, `i` institutions that receive factor income and `n` nests of value added. *name*`(`*i*`)` defines the variable for every element of the set of *i*, and the index may then appear in the expression.

User variables are saved with the model by **[CGE]** `cge_save`, kept when the model is calibrated again, with base values from the new base, solved in every simulation, dynamic run and sensitivity draw, listed in the results with the other variables, and written into the GAMS program by **[CGE]** `cge_gams`. Simulations stored before a variable was added hold its base value until they are solved again. `cge_equation, list` lists the user variables, and `cge_equation, clear` removes them with the user equations.

### Example 1: government spending fixed in value

The synthetic SAM is mapped and calibrated as in **[CGE]** `cge_solve`. The value of government purchases at market prices is defined, swapped for the scaling factor of government consumption, and so fixed while world import prices rise by 10 per cent:

```
. cge_variable GEXP = sum(c, PQ(c) * QG(c)), nominal
User variable GEXP: 1 element(s), base value 45
and its definition, equation GEXP

. return list

scalars:
              r(N_elements) =  1

macros:
                    r(name) : "GEXP"
```

```
. cge_closure, swap(GEXP GADJ)
Closure
  external account     flexexr
  government           flexgsav
  savings-investment   savings
  numeraire            cpi
  factor lab           mobile
  factor cap           mobile
  swap                 GEXP fixed, GADJ free

. cge_shock pwm, scale(1.1)
Parameter pwm changed for 3 element(s)

. cge_solve, name(budget)
Simulation budget solved in 7 iteration(s) (newton)
Largest scaled residual  1.44e-14; WALRAS =  2.78e-13
```

The value of purchases stays at its base, 45, and the volumes adjust, all in the same proportion:

```
. quietly cge_results, sim(budget) clear nodisplay

. list variable element1 base value if inlist(variable, "GEXP", "GADJ", "QG"), noobs

  +---------------------------------------+
  | variable   element1   base      value |
  |---------------------------------------|
  |       QG    com_agr      0          0 |
  |       QG    com_man      5   5.0474646 |
  |       QG    com_ser     40   40.379716 |
  |     GADJ                 1   1.0094929 |
  |     GEXP                45          45 |
  +---------------------------------------+
```

### Example 2: an indexed variable

A variable indexed by commodity is defined for each one; `tv(c)`, the VAT rate, is zero in this SAM. Solving the simulation again gives the variable its values in the solution:

```
. cge_variable PREL(c) = PQ(c) * (1 + tv(c)) / CPI
User variable PREL(c): 3 element(s), base values 1 1 1
and its definition, equation PREL

. quietly cge_solve, name(budget)

. quietly cge_results, sim(budget) clear nodisplay

. list variable element1 base value if variable == "PREL", noobs

  +---------------------------------------+
  | variable   element1   base      value |
  |---------------------------------------|
  |     PREL    com_agr      1   .99455161 |
  |     PREL    com_man      1   1.0175388 |
  |     PREL    com_ser      1   .98722855 |
  +---------------------------------------+
```

### Example 3: a rigid real wage

Fixing the real wage of labour, and freeing its supply, gives unemployment at a fixed real wage. The shock and the swap of Example 1 are removed first:

```
. cge_shock, reset
All shocks removed

. cge_closure, swap(none)
Closure
  external account     flexexr
  government           flexgsav
  savings-investment   savings
  numeraire            cpi
  factor lab           mobile
  factor cap           mobile

. cge_variable RW = W(lab) / CPI
User variable RW: 1 element(s), base value 1
and its definition, equation RW

. cge_closure, swap(RW QFS(lab))
Closure
  external account     flexexr
  government           flexgsav
  savings-investment   savings
  numeraire            cpi
  factor lab           mobile
  factor cap           mobile
  swap                 RW fixed, QFS(lab) free

. cge_shock pwm, scale(1.1)
Parameter pwm changed for 3 element(s)

. quietly cge_solve, name(realwage)

. quietly cge_results, sim(realwage) clear nodisplay

. list variable element1 base value if inlist(variable, "RW", "W", "QFS") & element1 != "cap", noobs

  +----------------------------------------+
  | variable   element1   base       value |
  |----------------------------------------|
  |        W        lab      1           1 |
  |      QFS        lab    135   128.04806 |
  |       RW                 1           1 |
  +----------------------------------------+
```

Under the CPI numeraire the real wage and the wage are both fixed at the base, and the rise in import prices lowers employment by about 5 per cent.

### Example 4: a variable that equations determine

A variable given a value has no equation, and the closure is then not square, as **[CGE]** `cge_check` shows; **[CGE]** `cge_solve` stops with error 459 before it solves:

```
. cge_variable LAM, value(1)
User variable LAM: 1 element(s), base value 1
Add the equations that determine it with cge_equation

. cge_check
  [ok]   SAM balance (largest imbalance)                      0.00e+00
  [ok]   benchmark replication (largest scaled residual)      3.55e-16
  [fail] closure is square (free variables minus equations)   1.00e+00
Simulation realwage:
  [ok]     solution (largest scaled residual)                 6.14e-14
  [ok]     Walras's law (WALRAS)                              2.47e-13
  [ok]     GDP identity (expenditure minus income)            2.39e-12
1 check(s) failed

r(9);

. cge_solve, name(unpaired)
the system is not square: 75 free variables, 74 equations
a variable added with cge_variable, value() needs an equation from cge_equation,
and each user equation needs a free variable

r(459);
```

**[CGE]** `cge_equation` gives an example that adds the equation.

## Stored results

`cge_variable` stores the following in `r()`:

*Scalars*

| | |
|---|---|
| `r(N_elements)` | number of elements of the variable |

*Macros*

| | |
|---|---|
| `r(name)` | name of the variable |

## Methods and formulas

A defined variable $U$ with index $i$ adds, for every element $e$ of the set of $i$, the variable $U_e$ and the equation

$$U_e = g_e(x),$$

where $g_e$ is the expression with $i$ replaced by $e$ and $x$ the vector of the model's variables, parameters at their current values. Its base value is $U_e^0 = g_e(x^0)$, which must not be missing. The equation is a user equation named after the variable, and `cge_equation, list` lists it with the others. A variable given a value $v$ adds $U_e$ with $U_e^0 = v$ and no equation. User variables come after the model's variables in the stacked vector of variables, in the order in which they are added, so that a definition may use the variables added before it. Chapter 23 describes the compilation of the expressions and their derivatives.

## Also see

**[CGE]** `cge_equation` — User equations: new relations among the model's variables
**[CGE]** `cge_closure` — Choose the model closure
**[CGE]** `cge_shock` — Change a parameter or a fixed variable
**[CGE]** `cge_gams` — Write the model as a GAMS program, and solve it in GAMS
**Chapter 23** — User variables and equations

# References

Blanchflower, D. G., and A. J. Oswald. 1994. *The Wage Curve*. Cambridge, MA: MIT Press.

Blanchflower, D. G., and A. J. Oswald. 1995. An introduction to the wage curve. *Journal of Economic Perspectives* 9(3): 153–167.

Bokhari, F. A. S., Y. Gai, and P. Gottret. 2007. Government health expenditures and health outcomes. *Health Economics* 16(3): 257–273.

Bourguignon, F., M. Bussolo, and L. A. Pereira da Silva, eds. 2008. *The Impact of Macroeconomic Policies on Poverty and Income Distribution: Macro-Micro Evaluation Techniques and Tools*. Washington, DC: World Bank and Palgrave Macmillan.

Broyden, C. G. 1965. A class of methods for solving nonlinear simultaneous equations. *Mathematics of Computation* 19: 577–593.

Burniaux, J.-M., and T. P. Truong. 2002. GTAP-E: An energy-environmental version of the GTAP model. GTAP Technical Paper 16, Center for Global Trade Analysis, Purdue University.

Corral, P., E. Muñoz, and I. Osorio-Rodarte. 2022. Modeling the distributional impact of macroeconomic shocks: `ms_reweight`. Presentation at the Stata Conference, Washington, DC.

Curtis, A. R., M. J. D. Powell, and J. K. Reid. 1974. On the estimation of sparse Jacobian matrices. *IMA Journal of Applied Mathematics* 13(1): 117–119.

Datt, G., and M. Ravallion. 1992. Growth and redistribution components of changes in poverty measures: A decomposition with applications to Brazil and India in the 1980s. *Journal of Development Economics* 38: 275–295.

Decaluwé, B., A. Lemelin, V. Robichaud, and H. Maisonnave. 2013. *PEP-1-1: The PEP Standard Single-Country, Static CGE Model*. Partnership for Economic Policy.

Defourny, J., and E. Thorbecke. 1984. Structural path analysis and multiplier decomposition within a social accounting matrix framework. *Economic Journal* 94(373): 111–136.

Devarajan, S., D. S. Go, and S. Robinson. 2023. Trade elasticities in aggregate models: Estimates for 191 countries. Policy Research Working Paper 10490, World Bank.

Fischer, A. 1992. A special Newton-type optimization method. *Optimization* 24(3–4): 269–284.

Hosni, M., and H. Ouakil. 2024. Estimation de l’élasticité de substitution entre le capital et le travail de l’économie africaine : une analyse en données de panel. *Revue AME* 6(3): 247–269.

Junius, T., and J. Oosterhaven. 2003. The solution of updating or regionalizing a matrix with both positive and negative entries. *Economic Systems Research* 15(1): 87–96.

Kanzow, C. 1996. Some noninterior continuation methods for linear complementarity problems. *SIAM Journal on Matrix Analysis and Applications* 17(4): 851–868.

Kelley, C. T. 1995. *Iterative Methods for Linear and Nonlinear Equations*. Philadelphia: SIAM.

Kingdon, G. G., and J. Knight. 2006. How flexible are wages in response to local unemployment in South Africa? *ILR Review* 59(3): 471–495.

Lenzen, M., R. Wood, and B. Gallego. 2007. Some comments on the GRAS method. *Economic Systems Research* 19(4): 461–465.

Lofgren, H., R. L. Harris, and S. Robinson. 2002. *A Standard Computable General Equilibrium (CGE) Model in GAMS*. Microcomputers in Policy Research 5. Washington, DC: International Food Policy Research Institute.

McDougall, R. A. 1999. Entropy theory and RAS are friends. GTAP Working Paper 6, Center for Global Trade Analysis, Purdue University.

Higgins, S., and N. Lustig. 2016. Can a poverty-reducing and progressive tax and transfer system hurt the poor? *Journal of Development Economics* 122: 63–75.

Kakwani, N. C. 1977. Measurement of tax progressivity: An international comparison. *Economic Journal* 87: 71–80.

Lustig, N., ed. 2018. *Commitment to Equity Handbook: Estimating the Impact of Fiscal Policy on Inequality and Poverty*. Washington, DC: Brookings Institution Press and CEQ Institute, Tulane University.

Muhammad, A., J. L. Seale Jr., B. Meade, and A. Regmi. 2011. *International Evidence on Food Consumption Patterns: An Update Using 2005 International Comparison Program Data*. Technical Bulletin TB-1929, Economic Research Service, U.S. Department of Agriculture.

Okabe, M., and K. Ito. 2008. Color universal design (CUD): How to make figures and presentations that are friendly to colorblind people. J*FLY data depository for Drosophila researchers, `https://jfly.uni-koeln.de/color/`.

Pyatt, G., and J. I. Round. 1979. Accounting and fixed price multipliers in a social accounting matrix framework. *Economic Journal* 89(356): 850–873.

Ravallion, M., and S. Chen. 2003. Measuring pro-poor growth. *Economics Letters* 78: 93–99.

Reynolds, M., and E. Smolensky. 1977. *Public Expenditures, Taxes, and the Distribution of Income: The United States, 1950, 1961, 1970*. New York: Academic Press.

Robinson, S., A. Cattaneo, and M. El-Said. 2001. Updating and estimating a social accounting matrix using cross entropy methods. *Economic Systems Research* 13(1): 47–64.

Shorrocks, A. F. 2013. Decomposition procedures for distributional analysis: A unified framework based on the Shapley value. *Journal of Economic Inequality* 11: 99–126.

Thurlow, J. 2004. A dynamic computable general equilibrium (CGE) model for South Africa: Extending the static IFPRI model. Working Paper 1-2004, Trade and Industrial Policy Strategies.

Wittenberg, M. 2010. An introduction to maximum entropy and minimum cross-entropy estimation using Stata. *Stata Journal* 10: 315–330.